

Руководство по программированию в среде 4diac

Начало

С чего начать?

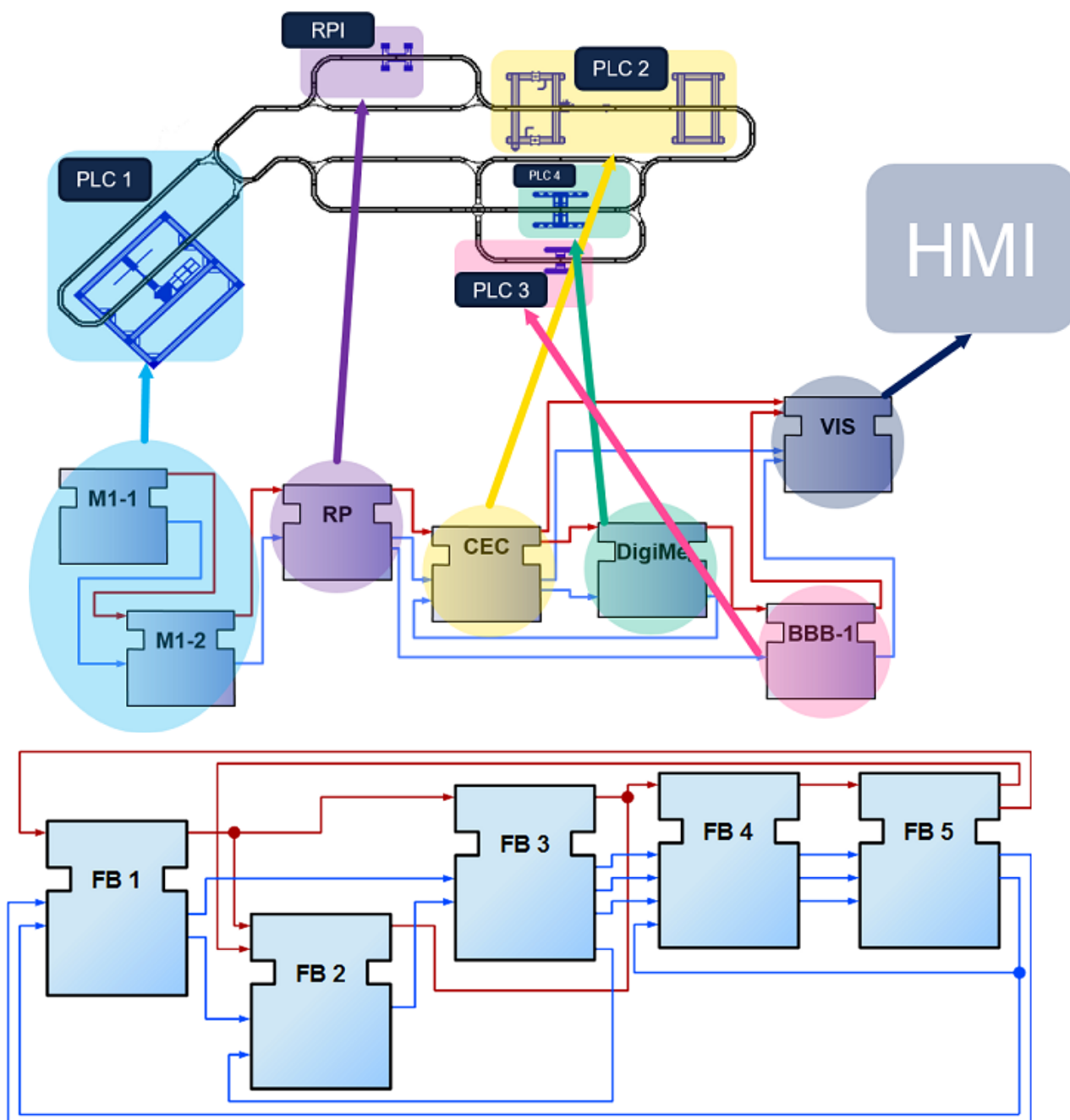
Эта страница предназначена для того, чтобы служить общим указателем документации по Eclipse 4diac. Если у вас возникли вопросы, загляните сюда, и, возможно, вы найдете нужную страницу. В зависимости от вашей области знаний (автоматизация, механика, информатика или просто любопытство), вам потребуется разная информация, чтобы понять, что такое Eclipse 4diac и его преимущества. Но поскольку это может быть первая страница, на которую вы попали, ниже списка ссылок приведено краткое введение.

Вот список основных тем в документации:

- [Узнайте о IEC 61499](#): Чтобы использовать Eclipse 4diac, вам нужно понять термины стандарта IEC 61499 и то, как работают функциональные блоки. Для начала ознакомьтесь с этим объяснением стандартов и технологий, связанных с Eclipse 4diac.
- [Поймите фреймворк 4diac](#): Если вы знаете о IEC 61499 и что такое ПЛК, но не понимаете, что такое 4diac IDE или как он связан с 4diac FORTE, то вам стоит ознакомиться с этой темой о фреймворке 4diac IDE.
- [Пошаговые руководства](#): Чтобы понять основы 4diac IDE, лучше всего начать с его использования.
- [Установка Eclipse 4diac](#): Если вы понимаете всё вышесказанное и хотите начать использовать Eclipse 4diac, вам сначала нужно его установить.
- [Примеры 4diac IDE](#): Если вы хотите увидеть примеры использования 4diac IDE, эта ссылка для вас.
- [Конфигурация ввода-вывода для различных платформ](#): Если у вас есть конкретная платформа с входами и выходами, поддерживаемая 4diac FORTE, например, Raspberry Pi или ПЛК, и вы хотите их использовать, обратитесь к этой теме.
- [Протоколы связи](#): Если вы хотите использовать конкретный протокол связи, поддерживаемый 4diac FORTE, например, MQTT, OPC UA, Modbus и другие, перейдите по ссылке.
- [Разработка 4diac FORTE и 4diac IDE](#): Если вы разработчик или хотите узнать больше о внутренней работе 4diac FORTE или 4diac IDE, вам сюда.
- [Сообщить об ошибке в 4diac FORTE или 4diac IDE](#): Нашли ошибку? Сообщите о ней здесь.
- [Часто задаваемые вопросы](#): Если у вас возникли трудности, посетите страницу FAQ.
- [Форум 4diac](#): Если у вас остались вопросы или есть что обсудить, не забудьте поучаствовать в форуме.

Краткое введение

4diac IDE — это открытая среда разработки, предназначенная для распределенных промышленных систем. Она также применима в других областях, таких как домашняя автоматизация, электрические сети или везде, где требуется автоматизированная система. Вы можете "программировать" приложение с помощью функциональных блоков, и ваша программа будет выглядеть, как на изображении ниже.



Типичное приложение в IEC61499

Одно и то же приложение может работать на разных устройствах, таких как Raspberry Pi, ПЛК или ваш компьютер, поскольку приложение не зависит от устройства, на котором оно запущено. Обратите внимание, что это приложение можно запускать не только на одном устройстве, но и распределять его на множество устройств, как показано на рисунке ниже.

Распределение приложения

Краткая справка: каждый функциональный блок (FB) похож на обычную функцию, которая выполняется при поступлении события (красная входящая линия слева от FB). Входящие синие линии — это входные данные (параметры функции), а исходящие синие линии — параметры, передаваемые другим FB (аналогично возвращаемым значениям, но они не всегда "возвращаются", а передаются дальше).

Это приложение может быть предназначено не только для одного устройства, но и для целой системы с множеством устройств. В этом случае некоторые FB будут выполняться на одном устройстве, а другие — на других, создавая распределенную систему. С помощью 4diac IDE вы можете проектировать свои распределенные приложения и развертывать их.

Вклад в документацию Eclipse 4diac

Спасибо за ваш интерес к этому проекту.

Описание проекта

Eclipse 4diac — это платформа с открытым исходным кодом для промышленной автоматизации и управления. Это эталонная реализация стандарта IEC 61499. IEC 61499 определяет специализированный язык моделирования для разработки распределённых решений для промышленного управления. IEC 61499 расширяет IEC 61131-1, улучшая инкапсуляцию программных компонентов для повышения их повторного использования, предоставляя независимый от поставщика формат и упрощая поддержку связи между контроллерами. Его функции распределения и встроенная поддержка динамической реконфигурации обеспечивают необходимую инфраструктуру для Industrie 4.0 и промышленных IoT-приложений.

Ресурсы для разработчиков

Проблемы проекта и исходный код хранятся в нескольких репозиториях GitHub. Прежде чем создавать новую проблему, обязательно поищите существующие. Помните, что вклады всегда приветствуются!

Настройка вашей учетной записи Eclipse и GitHub

Создайте [учетную запись](#) Eclipse, если у вас ее еще нет. Смотрите раздел "[Учетная запись Eclipse Foundation](#)" в руководстве по коммиттингу Eclipse.

Все участники, которые не являются коммиттерами в этом проекте Eclipse, должны подписать Соглашение с участниками [Eclipse \(ECA\)](#) в электронном виде через свою [учетную запись](#) Eclipse. Для получения дополнительной информации, пожалуйста, ознакомьтесь с [Руководством по коммиттингу Eclipse](#).

Чтобы подтвердить соглашение с участниками Eclipse, для публикации материалов на GitHub необходимо использовать тот же адрес электронной почты, что и в вашей [учетной записи Eclipse](#). Если ваша учетная запись на GitHub была зарегистрирована с другим адресом, вы можете [добавить свой адрес электронной почты Eclipse в учетную запись](#).

Также добавьте имя своей учетной записи на GitHub в свою [учетную запись Eclipse](#), чтобы включить автоматическое управление правами доступа для участников проекта Eclipse. Вы можете сделать это, войдя в свою учетную запись Eclipse, выбрав "[Редактировать профиль](#)" и добавив имя своей учетной записи на GitHub в разделе "Ссылки на социальные сети".

Если вы являетесь участником разработки какого-либо из проектов Eclipse, в течение 2 часов вы должны получить электронное письмо с приглашением присоединиться к организации GitHub, связанной с этим проектом. Вы должны принять приглашение, чтобы сохранить статус участника разработки в организации GitHub.

Также рекомендуется [добавить открытые SSH-ключи в вашу учетную запись на GitHub](#).

Создание среды разработки

Документация Eclipse 4diac написана с использованием [AsciiDoc](#), который представляет собой разновидность текстового формата Markdown, предназначенного для документации. Вы можете редактировать документацию в любом текстовом редакторе.

Использование Eclipse IDE для редактирования документации Eclipse 4diac

Расширение [AsciiDoctor для Eclipse](#) позволяет удобно редактировать файлы AsciiDoc в Eclipse IDE. Для этого вам необходимо выполнить следующие действия по настройке:

- [скачать Eclipse](#) для разработчиков на вашу платформу.
- через Eclipse Marketplace в Eclipse установить редактор AsciiDoctor

Рекомендуемый рабочий процесс

Рекомендуемый способ внесения изменений — создать ответвление от основного репозитория проекта и вносить изменения только в это ответвление (см. раздел «[Модель ответвления и вытягивания](#)» в документации GitHub). Затем вы можете создать запрос на вытягивание (PR), чтобы ваши изменения были объединены из ответвления в основной репозиторий проекта.

Рабочий процесс можно разделить на две части:

- [Создание ответвления и управление им](#)

- [Создание запроса на вытягивание из ответвления](#) и особенности этого процесса в организации Eclipse-platform

Если вы знаете, как создавать форк и управлять им, вы можете [пропустить эту часть](#).

Создание форка и управление им

Нажмите кнопку «Форк» (в правом верхнем углу) на странице репозитория GitHub и выберите «Создать новый форк», чтобы создать форк-репозиторий в своей учётной записи GitHub. Всякий раз, когда в основной репозиторий проекта вносятся изменения, вы можете нажать «Синхронизировать форк» в форк-репозитории, чтобы получить последнюю версию исходного кода из основного репозитория проекта.

Затем у вас есть два варианта:

1. Клонировать основной репозиторий проекта и добавить ответвление репозитория в качестве дополнительного удалённого репозитория.

С помощью этой опции вы можете легко получить последние изменения из основного репозитория проекта, просто потянув основную ветку, так как удалённым репозиторием по умолчанию будет основной репозиторий проекта.

Клонировать основной репозиторий в рабочую область Eclipse или, если вы используете установщик Eclipse, как описано в разделе [«Создание среды разработки»](#), репозиторий уже клонирован таким образом. URL-адрес репозитория можно найти, нажав кнопку <> «Код» в правом верхнем углу вкладки <> «Код» на странице GitHub для основного репозитория проекта. При клонировании репозитория он автоматически добавляется в качестве удалённого репозитория «origin» в ваш локальный клон.

Затем вам нужно добавить свой форк в качестве дополнительного удалённого репозитория, чтобы отправить изменения в ветку этого форка. Для этого добавьте форк-репозиторий в качестве дополнительного удалённого репозитория под названием «fork» (или любым другим названием, которое вам нравится):

- Перейдите на свой форк на GitHub.
- Нажмите кнопку <> «Код» в правом верхнем углу вкладки <> «Код» и скопируйте URL-адрес.
- Добавьте URL-адрес в качестве удалённого «ответвления» в свой локальный клон.

Всякий раз, когда основная ветка основного репозитория проекта изменяется, обновление основной ветки вашего локального клона даст вам актуальное состояние. Если вы создали ветку и внесли в неё изменения, вы можете перебазировать её на обновлённую основную ветку.

2. Клонировать ответвление репозитория и добавить основной репозиторий в качестве дополнительного удалённого

С помощью этой опции вы можете легко создавать новые ветки для зафиксированных изменений в вашей вилке, так как удалённым репозиторием по умолчанию будет ваша вилочная версия репозитория.

Клонировать вилочную версию репозитория в рабочую область Eclipse или, если вы используете установщик Eclipse, как описано в разделе [«Создание среды разработки»](#), вам нужно будет изменить URL-адрес репозитория во время настройки. URL-адрес репозитория можно найти, нажав кнопку <> «Код» в правом верхнем углу вкладки <> «Код» на странице GitHub для вилочной версии репозитория. Когда вы клонируете разветвленный репозиторий, он автоматически добавляется в качестве исходного к вашему локальному клону.

Затем вам необходимо добавить основной репозиторий проекта в качестве дополнительного удалённого хранилища. Для этого добавьте основной репозиторий проекта в качестве дополнительного удалённого хранилища под названием upstream:

- Перейдите в основной репозиторий проекта на GitHub.
- Нажмите кнопку <> Код в правом верхнем углу вкладки <> Код, скопируйте URL-адрес.
- Добавьте URL-адрес в качестве "исходного" для вашего локального клона.

Всякий раз, когда основная ветка основного репозитория проекта изменяется, вам нужно синхронизировать основную ветку вашего форка. Перейдите на страницу GitHub вашего форка и нажмите «Синхронизировать форк». Затем вы можете обновить основную ветку вашего локального

клона, чтобы получить текущее состояние. Если вы создали ветку и внесли в неё изменения, вы можете перенести её на обновлённую основную ветку.

Создание запроса на вытягивание

Если вы создали свою вилку репозитория, в который хотите внести изменения, и подготовили для него локальный клон, вы можете вносить изменения, фиксировать их и предлагать их с помощью запроса на вытягивание следующим образом.

1. Создайте ветку для изменений, которые вы хотите внести. Настройте эту ветку, начиная с локальной основной ветки, и настройте удалённый доступ к вашей вилке репозитория.
2. Измените код по мере необходимости, чтобы устранить проблему.
3. Зафиксируйте изменения кода в локальной ветке вашей вилки. При необходимости сделайте несколько коммитов, чтобы сохранить историю, но учтите, что обычно вас попросят объединить ваши изменения в один коммит для каждого запроса на извлечение. Поэтому делайте изменения как можно меньше и старайтесь разделять их.
4. Как только исправление будет готово, переместите ветвь в свой расширенный репозиторий (не в исходный репозиторий вышестоящего проекта).
5. Откройте страницу расширенного репозитория в своем браузере, перейдите в ветку, в которой разрабатывается исправление. Нажмите кнопку "Добавить", чтобы запустить рабочий процесс открытого запроса на извлечение.
6. На новой странице убедитесь, что целевая ветвь является правильной для объединения ваших коммитов. Это должна быть основная ветвь основного репозитория проекта. Также убедитесь, что список коммитов содержит изменения, которые вы собираетесь объединить. Если все в порядке, нажмите на кнопку "Создать запрос на удаление" в правом нижнем углу.
7. Если вы являетесь пользователем, вы можете добавить рецензентов, если хотите, чтобы определенные люди проверяли изменения. Если вы оставите это поле пустым, его проверит любой из проектной команды.
8. Если вы отправляете больше коммитов в ту же ветку в своей вилке, они автоматически добавляются в запрос на вытягивание (и запускают новый цикл сборки и проверки). Вы также можете изменить свои предыдущие коммиты и принудительно отправить их в свою ветку.
- Обратите внимание, что перед объединением запроса на вытягивание вас обычно попросят объединить коммиты в один (есть одно возможное исключение, о котором читайте далее).
- Также обратите внимание, что при обновлении версий в файлах Manifest/POM вы можете предоставить 2 отдельных коммита в одном запросе на вытягивание: один с функциональными изменениями (т. е. в коде), а другой с обновлением версий (т. е. в файлах Manifest/POM). Это не правило, но при необходимости это поможет выбрать нужные коммиты при объединении запроса на вытягивание.
9. Рецензенты могут просмотреть запрос на вытягивание на сайте GitHub или получить запрос на вытягивание с помощью пункта меню «Получить запрос на вытягивание GitHub» в разделе «Репозитории Git» в Eclipse IDE.
10. Каждый PR автоматически проверяется на наличие действительного Соглашения участника Eclipse (ECA).
11. После одобрения PR его может объединить разработчик. Выберите наиболее подходящий вариант из следующих:
 - Перебазирование и слияние (сохраняет все коммиты по отдельности; полезно при разработке функции, состоящей из нескольких независимых коммитов)
 - Сжатие и слияние (все коммиты в PR объединяются в один коммит; полезно, чтобы избежать отдельных коммитов с комментариями по исправлению)
12. После слияния PR исходная ветка, использованная для PR, может быть удалена.
13. Теперь ваша ветка устарела по сравнению с основным (верхним) репозиторием проекта. Если «исходным» репозиторием вашего локального клона является разветвлённый репозиторий, вам следует обновить его. Вы можете либо использовать функцию «Синхронизировать ветку» на GitHub, чтобы обновить ветку и перенести основную ветку в ваш локальный клон, либо обновить ветку локально с помощью git:

- Загрузите последние изменения из «верхнего источника» (основного репозитория проекта) в свой локальный репозиторий.
- Отправьте эти изменения из своего локального репозитория в «исходный» (ваш форк). Если вы вернётесь в свой форк на GitHub, то увидите, что основная ветка синхронизирована с основным репозиторием проекта.

14. Теперь ваш форк готов к следующему вкладу.

Рекомендации по сообщениям о коммите

<issue title> #<issue number>

Short description/summary as to why this change is being made

Fixes <https://github.com/<Full URL to issues/issue #>>

Пример:

Улучшить документацию по использованию интерфейсов адаптера #321

Интерфейсы адаптера являются мощной функцией в IEC 61499.

Eclipse 4diac IDE была одной из первых IDE, поддерживающих интерфейсы адаптеров во всех типах FB и подприложениях.

Это изменение расширяет существующую документацию по интерфейсам адаптеров, показывая, как разрабатываются интерфейсы адаптеров в 4diac IDE и как они используются в различных типах FB и в подприложениях.

Исправления <https://github.com/eclipse-4diac/4diac-documentation/issues/321>

Контакты

Свяжитесь с разработчиками проекта через список рассылки "[разработчики](#)" проекта.

Также рекомендуется добавить открытые ключи SSH в свою учетную запись GitHub.

Публичная лицензия Eclipse — версия 2.0

ПРИЛАГАЕМАЯ ПРОГРАММА ПРЕДОСТАВЛЯЕТСЯ НА УСЛОВИЯХ НАСТОЯЩЕЙ ПУБЛИЧНОЙ ЛИЦЕНЗИИ ECLIPSE («СОГЛАШЕНИЕ»). ЛЮБОЕ ИСПОЛЬЗОВАНИЕ, ВОСПРОИЗВЕДЕНИЕ ИЛИ РАСПРОСТРАНЕНИЕ

ПРОГРАММЫ СОСТАВЛЯЕТ ПРИНЯТИЕ ПОЛУЧАТЕЛЕМ НАСТОЯЩЕГО СОГЛАШЕНИЯ.

ОПРЕДЕЛЕНИЯ

"Вклад" означает:

а) в случае первоначального участника — первоначальный контент, распространяемый в соответствии с настоящим Соглашением, и

б) в случае каждого последующего участника: i) изменения в Программе и ii) дополнения к Программе; при этом такие изменения и/или дополнения к Программе исходят от конкретного участника и распространяются им. Вклад «исходит» от участника, если он был добавлен в Программу самим участником или кем-либо, действующим от его имени. Вклады не включают изменения или дополнения к Программе, которые не являются изменёнными работами.

«Участник» означает любое физическое или юридическое лицо, которое распространяет Программу.

«Лицензированные патенты» означают патентные заявки, которые участник может лицензировать и которые обязательно нарушаются при использовании или продаже его вклада отдельно или в сочетании с Программой.

«Программа» означает вклады, распространяемые в соответствии с настоящим Соглашением.

«Получатель» означает любого, кто получает Программу в соответствии с настоящим Соглашением или любой дополнительной лицензией (в зависимости от обстоятельств), включая участников.

«Производные работы» — это любая работа, будь то в виде исходного кода или в другой форме, которая основана на (или является производной от) Программы и для которой редакционные правки, аннотации, доработки или другие изменения в совокупности представляют собой оригинальное авторское произведение.

«Модифицированные работы» — это любая работа в виде исходного кода или в другой форме, которая является результатом добавления, удаления или изменения содержимого Программы, включая, для ясности, любой новый файл в виде исходного кода, содержащий любое содержимое Программы. Модифицированные работы не должны включать работы, которые содержат только объявления, интерфейсы, типы, классы, структуры или файлы Программы исключительно для того, чтобы ссылаться на них, связывать их по имени или создавать подклассы Программы или модифицированных работ на ее основе.

"Распространение" означает действия по а) распространению или б) предоставлению доступа любым способом, позволяющим передать копию.

"Исходный код" означает форму Программы, предпочтительную для внесения изменений, включая, помимо прочего, исходный код программного обеспечения, исходную документацию и файлы конфигурации.

«Вторичная лицензия» означает либо Стандартную общественную лицензию GNU версии 2.0, либо любые более поздние версии этой лицензии, включая любые исключения или дополнительные разрешения, указанные первоначальным участником.

ПРЕДОСТАВЛЕНИЕ ПРАВ

а) В соответствии с условиями настоящего Соглашения каждый участник настоящим предоставляет Получателю неисключительную всемирную лицензию на воспроизведение, подготовку производных работ, публичное представление, публичное исполнение, распространение и сублицензирование вклада такого участника, если таковой имеется, и таких производных работ.

б) В соответствии с условиями настоящего Соглашения каждый Участник настоящим предоставляет Получателю неисключительную, всемирную, безвозмездную патентную лицензию на использование Лицензированных патентов для создания, использования, продажи, предложения к продаже, импорта и иной передачи Контента такого Участника, если таковой имеется, в исходном коде или в иной форме. Настоящая патентная лицензия распространяется на комбинацию Контента и Программы,

если на момент добавления Контента Участником такое добавление Контента приводит к тому, что такая комбинация подпадает под действие Лицензированных патентов. Патентная лицензия не распространяется на какие-либо другие комбинации, включающие в себя Вклад. Никакое оборудование само по себе не лицензируется в соответствии с настоящим документом.

с) Получатель понимает, что, хотя каждый Участник предоставляет лицензии на свои Вклады, указанные в настоящем документе, ни один Участник не гарантирует, что Программа не нарушает патентные или другие права интеллектуальной собственности какой-либо другой организации. Каждый Участник отказывается от какой-либо ответственности перед Получателем за претензии, выдвинутые какой-либо другой организацией на основании нарушения прав интеллектуальной собственности или иным образом. В качестве условия осуществления прав и лицензий, предоставленных по настоящему соглашению, каждый Получатель настоящим принимает на себя единоличную ответственность за обеспечение любых других необходимых прав интеллектуальной собственности, если таковые имеются. Например, если для того, чтобы Получатель мог распространять Программу, требуется патентная лицензия третьей стороны, он обязан приобрести эту лицензию до распространения Программы.

d) Каждый Участник заявляет, что, насколько ему известно, он обладает достаточными авторскими правами на свой вклад, если таковые имеются, для предоставления лицензии на авторское право, указанной в настоящем Соглашении.

e) Несмотря на условия любой дополнительной лицензии, ни один из Участников не предоставляет дополнительные гранты какому-либо Получателю (кроме тех, которые указаны в настоящем Соглашении) в результате получения таким Получателем Программы на условиях дополнительной лицензии (если это разрешено условиями раздела 3).

ТРЕБОВАНИЯ

3.1 Если участник распространяет Программу в какой-либо форме, то:

а) Программа также должна быть доступна в виде исходного кода в соответствии с разделом 3.2, и участник должен сопровождать Программу заявлением о том, что исходный код Программы доступен в соответствии с настоящим Соглашением, и информировать Получателей о том, как получить его разумным способом на носителе, обычно используемом для обмена программным обеспечением; и

б) Участник может распространять Программу по лицензии, отличной от настоящего Соглашения, при условии, что такая лицензия:

i) фактически снимает с себя и с других Участников все гарантии и условия, явные и подразумеваемые, включая гарантии или условия, касающиеся права собственности и отсутствия нарушений, а также подразумеваемые гарантии или условия, касающиеся пригодности для продажи и использования по назначению;

ii) фактически снимает с себя и с других Участников всю ответственность за ущерб, включая прямой, косвенный, особый, случайный и последующий ущерб, такой как упущенная выгода;

iii) не пытается ограничить или изменить права получателей в отношении исходного кода в соответствии с разделом 3.2; и

iv) требует, чтобы любое последующее распространение Программы какой-либо стороной осуществлялось по лицензии, удовлетворяющей требованиям настоящего раздела 3.

3.2 При распространении Программы в виде исходного кода:

а) она должна быть доступна в соответствии с настоящим Соглашением, или если Программа (i) объединена с другими материалами в отдельном файле или файлах, доступных по вторичной лицензии, и (ii) первоначальный участник добавил к исходному коду уведомление, описанное в Приложении А к настоящему Соглашению, то Программа может быть доступна на условиях таких вторичных лицензий, и

б) копия настоящего Соглашения должна прилагаться к каждой копии Программы.

3.3 Участники не могут удалять или изменять какие-либо уведомления об авторских правах, патентах, товарных знаках, указания авторства, отказ от гарантийных обязательств или ограничения ответственности («уведомления»), содержащиеся в Программе, в любой распространяемой ими копии Программы, при условии, что Участники могут добавлять свои собственные соответствующие уведомления.

КОММЕРЧЕСКОЕ РАСПРОСТРАНЕНИЕ

Коммерческие распространители программного обеспечения могут брать на себя определенные обязательства в отношении конечных пользователей, деловых партнеров и т. п. Хотя эта лицензия предназначена для упрощения коммерческого использования Программы, участник, который включает Программу в коммерческий продукт, должен делать это таким образом, чтобы не создавать потенциальную ответственность для других участников. Таким образом, если участник включает Программу в коммерческое предложение, такой участник («Коммерческий участник») настоящим соглашается защищать и возместить убытки, ущерб и расходы (далее «Убытки») всем остальным участникам («Возмещающий участник») в связи с претензиями, судебными исками и другими юридическими действиями, поданными третьей стороной против Возмещающего участника в связи с действиями или бездействием такого Коммерческого участника в связи с распространением Программы в рамках коммерческого предложения. Обязательства, изложенные в этом разделе, не распространяются на какие-либо претензии или убытки, связанные с каким-либо фактическим или предполагаемым нарушением прав интеллектуальной собственности. Чтобы соответствовать требованиям, Получивший компенсацию Вкладчик должен: а) незамедлительно уведомить Коммерческого вкладчика в письменной форме о такой претензии и б) позволить Коммерческому вкладчику контролировать и сотрудничать с Коммерческим вкладчиком в процессе защиты и любых связанных с этим переговоров об урегулировании. Получивший компенсацию Вкладчик может участвовать в любом таком иске за свой счет.

Например, участник может включить Программу в коммерческое предложение по продукту X. В этом случае участник является коммерческим участником. Если коммерческий участник заявляет о характеристиках продукта X или предоставляет гарантии, связанные с продуктом X, то ответственность за эти характеристики и гарантии лежит только на коммерческом участнике. В соответствии с этим разделом коммерческий участник должен будет защищать свои интересы в отношении других участников, связанных с этими характеристиками и гарантиями, и если суд обяжет любого другого участника возместить ущерб, то коммерческий участник должен будет возместить этот ущерб.

5. ОТСУТСТВИЕ ГАРАНТИЙ

КРОМЕ СЛУЧАЕВ, ЯВНО УКАЗАННЫХ В НАСТОЯЩЕМ СОГЛАШЕНИИ, И В ТОЙ СТЕПЕНИ, В КОТОРОЙ ЭТО ДОПУСКАЕТСЯ ПРИМЕНИМЫМ ЗАКОНОДАТЕЛЬСТВОМ, ПРОГРАММА ПРЕДОСТАВЛЯЕТСЯ НА УСЛОВИЯХ «КАК ЕСТЬ», БЕЗ КАКИХ-ЛИБО ГАРАНТИЙ ИЛИ УСЛОВИЙ ЛЮБОГО РОДА, ЯВНЫХ ИЛИ ПОДРАЗУМЕВАЕМЫХ, ВКЛЮЧАЯ, БЕЗ ОГРАНИЧЕНИЙ, ЛЮБЫЕ ГАРАНТИИ ИЛИ УСЛОВИЯ, КАСАЮЩИЕСЯ ПРАВА СОБСТВЕННОСТИ, НЕНАРУШЕНИЯ ПРАВ, ПРИГОДНОСТИ ДЛЯ ОПРЕДЕЛЁННЫХ ЦЕЛЕЙ. Каждый Получатель несет единоличную ответственность за определение целесообразности использования и распространения Программы и принимает на себя все риски, связанные с реализацией своих прав по настоящему Соглашению, включая, помимо прочего, риски и расходы, связанные с ошибками в программе, соблюдением применимых законов, повреждением или потерей данных, программ или оборудования, а также недоступностью или прерыванием работы.

ОТКАЗ ОТ ОТВЕТСТВЕННОСТИ

ЗА ИСКЛЮЧЕНИЕМ СЛУЧАЕВ, ПРЯМО ОГОВОРЕННЫХ В НАСТОЯЩЕМ СОГЛАШЕНИИ, И В ТОЙ МЕРЕ, В КАКОЙ ЭТО РАЗРЕШЕНО ПРИМЕНИМЫМ ЗАКОНОДАТЕЛЬСТВОМ, НИ ПОЛУЧАТЕЛЬ, НИ КАКИЕ-ЛИБО ВКЛАДЧИКИ НЕ НЕСУТ НИКАКОЙ ОТВЕТСТВЕННОСТИ ЗА ЛЮБЫЕ ПРЯМЫЕ, КОСВЕННЫЕ, СЛУЧАЙНЫЕ, ОСОБЫЕ, ПОКАЗАТЕЛЬНЫЕ ИЛИ КОСВЕННОПОСЛЕДОВАТЕЛЬНЫЕ УБЫТКИ (ВКЛЮЧАЯ, ПОМИМО ПРОЧЕГО, УПУЩЕННУЮ ВЫГОДУ), НЕЗАВИСИМО ОТ ТОГО, БЫЛИ ЛИ ОНИ ВЫЗВАНЫ И ПО КАКОЙ ТЕОРИИ ОТВЕТСТВЕННОСТИ, БУДЬ ТО В КОНТРАКТЕ, СТРОГОЙ ОТВЕТСТВЕННОСТИ ИЛИ ДЕЛИКТЕ (ВКЛЮЧАЯ ХАЛАТНОСТЬ ИЛИ ИНОЕ ПРАВОНАРУШЕНИЕ), ВОЗНИКАЮЩЕМ КАКИМ-ЛИБО ОБРАЗОМ В РЕЗУЛЬТАТЕ ИСПОЛЬЗОВАНИЯ ИЛИ РАСПРОСТРАНЕНИЯ ПРОГРАММЫ ИЛИ ОСУЩЕСТВЛЕНИЯ ЛЮБЫХ ПРАВ, ПРЕДОСТАВЛЕННЫХ ПО НАСТОЯЩЕМУ СОГЛАШЕНИЮ, ДАЖЕ ЕСЛИ ВЫ БЫЛИ ПРЕДУПРЕЖДЕНЫ О ВОЗМОЖНОСТИ ТАКОГО УЩЕРБА.

ОБЩИЕ ПОЛОЖЕНИЯ

Если какое-либо положение настоящего Соглашения является недействительным или не имеющим законной силы в соответствии с применимым законодательством, это не влияет на действительность

или законность остальных положений настоящего Соглашения, и без дальнейших действий сторон такое положение должно быть изменено в минимальной степени, необходимой для того, чтобы такое положение стало действительным и имеющим законную силу.

Если Получатель возбуждает патентный иск против какой-либо организации (включая встречный иск или контр-иск в рамках судебного разбирательства), утверждая, что сама Программа (за исключением комбинаций Программы с другим программным или аппаратным обеспечением) нарушает патент(ы) Получателя, то права Получателя, предоставленные в соответствии с разделом 2(b), прекращаются с даты подачи такого иска.

Все права Получателя по настоящему Соглашению прекращаются, если он не соблюдает какие-либо существенные положения или условия настоящего Соглашения и не устраняет такие нарушения в течение разумного периода времени после того, как ему стало известно о таком несоблюдении. Если все права Получателя по настоящему Соглашению прекращаются, Получатель соглашается прекратить использование и распространение Программы как можно скорее. Однако обязательства Получателя по настоящему Соглашению и любые лицензии, предоставленные Получателем в отношении Программы, остаются в силе.

Всем разрешено копировать и распространять копии настоящего Соглашения, но во избежание противоречий настоящее Соглашение защищено авторским правом и может быть изменено только следующим образом. Администратор Соглашения оставляет за собой право время от времени публиковать новые версии (включая исправления) настоящего Соглашения. Никто, кроме Администратора Соглашения, не имеет права изменять настоящее Соглашение. Фонд Eclipse является первоначальным Администратором Соглашения. Фонд Eclipse может передать полномочия Администратора Соглашения подходящей отдельной организации. Каждая новая версия Соглашения будет иметь отличительный номер версии. Программа (включая материалы) всегда может распространяться в соответствии с версией Соглашения, на основании которой она была получена. Кроме того, после публикации новой версии Соглашения участник может принять решение о распространении Программы (включая материалы) в соответствии с новой версией.

За исключением случаев, прямо указанных в разделах 2(a) и 2(b) выше, Получатель не получает никаких прав или лицензий на интеллектуальную собственность любого Участника в соответствии с настоящим Соглашением, будь то прямо, косвенно, в силу отказа от права или иным образом. Все права на Программу, прямо не предоставленные в соответствии с настоящим Соглашением, сохраняются за Участником. Ничто в настоящем Соглашении не может быть истолковано как имеющее юридическую силу для какой-либо организации, не являющейся Участником или Получателем. Настоящее Соглашение не создает прав третьих лиц.

Приложение А — Форма уведомления о вторичных лицензиях

«Этот исходный код также может быть предоставлен по следующим дополнительным лицензиям при соблюдении условий, изложенных в Eclipse Public License, версия 2.0: {укажите название лицензии (лицензий), версию (версии) и исключения или дополнительные разрешения здесь}.

Простого включения копии настоящего Соглашения, включая Приложение А, недостаточно для предоставления исходного кода по дополнительным лицензиям.

Если нет возможности или желания разместить уведомление в определённом файле, вы можете разместить его в таком месте (например, в файле LICENSE в соответствующем каталоге), где получатель, скорее всего, будет искать такое уведомление.

Вы можете добавить дополнительные точные уведомления о правах собственности.

Уведомления для Eclipse 4DIAC

Этот контент создается и поддерживается проектом Eclipse 4DIAC.

Домашняя страница проекта: <https://projects.eclipse.org/projects/iot.4diac>

Товарные знаки

Eclipse 4DIAC является товарным знаком Eclipse Foundation.

Авторские права

Весь контент является собственностью соответствующих авторов или их работодателей. Для получения дополнительной информации об авторстве контента, пожалуйста, ознакомьтесь с указанными журналами репозитория исходного кода.

Заявленные лицензии проекта

Эта программа и прилагаемые к ней материалы предоставляются на условиях публичной лицензии Eclipse версии 2.0, которая доступна по адресу <https://www.eclipse.org/legal/epl-2.0>.

SPDX-License-Identifier: EPL-2.0

Исходный код

Проект поддерживает следующие репозитории исходного кода:

<https://git.eclipse.org/r/plugins/gitiles/4diac/org.eclipse.4diac.examples>

<https://git.eclipse.org/r/plugins/gitiles/4diac/org.eclipse.4diac.forte>

<https://git.eclipse.org/r/plugins/gitiles/4diac/org.eclipse.4diac.ide>

Стороннее содержимое

Этот проект использует следующее стороннее содержимое.

Нет

Криптография

Контент может содержать программное обеспечение для шифрования. В стране, в которой вы находитесь в данный момент, могут действовать ограничения на ввоз, хранение и использование программного обеспечения для шифрования и/или его реэкспорт в другую страну. ПЕРЕД использованием любого программного обеспечения для шифрования, пожалуйста, ознакомьтесь с законами, правилами и политикой страны в отношении ввоза, хранения или использования, а также реэкспорта программного обеспечения для шифрования, чтобы узнать, разрешено ли это.

Документация Eclipse 4diac

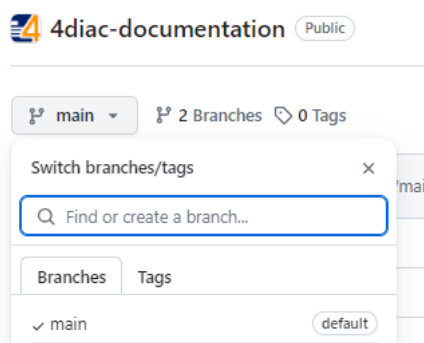
Этот репозиторий содержит файлы справки для [проекта Eclipse 4diac](#) в формате [Asciidoc](#).
[Как внести свой вклад](#)

Мы будем рады любым предложениям. Есть много способов внести свой вклад: от отправки качественных отчётов об ошибках до внесения кода или изменений в документацию. Полное руководство см. в [руководстве по внесению вклада](#).

Для внесения быстрых изменений в документацию вам не нужна среда разработки. Изменения можно вносить быстро и напрямую на странице Github. Вам нужно создать учётные записи Eclipse и Github, как описано в руководстве по участию в проекте. Затем выполните следующие действия:

Онлайн-участие

1. При необходимости выберите основную ветку. Выберите раскрывающийся список на странице репозитория Github.



2. Откройте один из файлов, входящих в репозиторий.
3. В правой верхней части окна просмотра нажмите кнопку «Изменить». Вы можете редактировать текст/код, а также перетаскивать изображения из локальной файловой системы в редактор. Изменить кнопку «Github»
4. Нажмите кнопку «зафиксировать изменения...» в правом верхнем углу
5. Во всплывающем окне введите сообщение о фиксации, описание и выберите «Создать новую ветку для этой фиксации и отправить запрос на вытягивание»
6. На следующей странице выберите «Отправить запрос на вытягивание». Кто-то из команды проекта проверит ваш патч.

Лицензия

[Eclipse Public License \(EPL\) 2.0](#)

Стандарт ISO 27005 определяет уязвимость следующим образом: "Слабое место актива или группы активов, которое может быть использовано одной или несколькими угрозами".

Команда безопасности Eclipse

Команда безопасности Eclipse предоставляет помощь и рекомендации проектам Eclipse по вопросам уязвимостей и является первым контактным лицом для устранения уязвимостей в системе безопасности. Члены команды безопасности являются ответственными за проекты Eclipse и членами Архитектурного совета Eclipse.

Свяжитесь с [командой безопасности Eclipse](#).

Обратите внимание, что в соответствии с политикой команда по обеспечению безопасности не открывает вложения.

Сообщение об уязвимости в системе безопасности

Об уязвимостях можно сообщать по электронной почте в команду по обеспечению безопасности Eclipse или напрямую в рамках проекта через экземпляр Bugzilla Фонда Eclipse.

Адрес общего списка рассылки по вопросам безопасности: <mailto:security@eclipse.org>. Члены команды по обеспечению безопасности Eclipse будут получать сообщения, отправленные на этот адрес. Этот адрес следует использовать только для сообщений о нераскрытых уязвимостях; регулярные сообщения о проблемах и вопросы, не связанные с уязвимостями в программном обеспечении Eclipse, будут игнорироваться. Обратите внимание, что этот адрес электронной почты не зашифрован.

Сообществу также рекомендуется сообщать об уязвимостях, используя экземпляр [Bugzilla от Eclipse Foundation](#). Обратите внимание, что для создания отчета о проблеме вам потребуется учетная запись Eclipse Foundation, но, сделав это, вы сможете непосредственно участвовать в решении проблемы.

Отчёты об ошибках, связанные с уязвимостями, должны быть помечены как «только для коммиттеров» либо автоматически при нажатии на предоставленную ссылку, либо репортёром, либо коммиттером в процессе сортировки. Обратите внимание, что ошибки, помеченные как «только для коммиттеров», видны всем коммиттерам Eclipse. По умолчанию ошибка, помеченная как «только для коммиттеров», также доступна репортёру и лицам, явно указанным в списке «сс».

Раскрытие информации

Изначально информация раскрывается только автору сообщения и всем участникам Eclipse, но затем она распространяется на других лиц и широкую общественность. Сроки и способы раскрытия информации регулируются [Политикой безопасности Eclipse](#).

Раскрытые в открытом доступе уязвимости перечислены на странице «[Раскрытые уязвимости](#)».

Настройки

////

This is a file containing common settings and attributes
for all AsciiDoc documents in this repo.

It is considered `_internal_` by AsciiDoctor because of the name starting with
an underscore and therefore not processed as separate document
by the AsciiDoctor Maven plugin.

All common attributes like "enable section numbering" should go into
this document, not into the separate content documents.

That way the styles are automatically consistent.

////

// Enable user interface macros like "kbd".

// <https://asciidoctor.org/docs/user-manual/#user-interface-macros>

:experimental:

//

// Normalize source listings to have no indent

// <https://docs.asciidoctor.org/asciidoc/latest/attributes/document-attributes-ref/#source-highlighting-and-formatting-attributes>

:source-indent: 0

//

// tabs with 4 spaces (default are 8), like in eclipse

// <https://asciidoctor.org/docs/user-manual/#builtin-attributes>

:tabsize: 4

//

// Use font characters for admonition icons

// <https://asciidoctor.org/docs/user-manual/#admonition-icons>

:icons: font

//

// explicitly use the "fontawesome-solid" icon set, since the default icon set is deprecated

:icon-set: fas

//

// hide URI scheme from plain text links.

// <https://docs.asciidoctor.org/asciidoc/latest/macros/links/#hide-uri-scheme>

:hide-uri-scheme:

//

// enable section numbering up to level 4

:sectnums:

:sectnumlevels: 4

//

// strip the leading underscore from generated ids (HTML anchors)

:idprefix:

//

// Enable section anchors in front of headlines.

// <https://asciidoctor.org/docs/user-manual/#anchors> -->

:sectanchors:

//

// enable source highlighting

// <https://docs.asciidoctor.org/asciidoc/latest/verbatim/source-highlighter/>

:source-highlighter: coderay

//

```
// define common URLs as attributes so they can be used like macros
//
:4diac-documentation-repo:      https://github.com/eclipse-4diac/4diac-documentation
:issues:                        {4diac-documentation-repo}/issues
:discussions:                   {4diac-documentation-repo}/discussions
:pulls:                          {4diac-documentation-repo}/pulls
```

Pom.xml

////

This is a file containing common settings and attributes
for all AsciiDoc documents in this repo.

It is considered `_internal_` by AsciiDoctor because of the name starting with
an underscore and therefore not processed as separate document
by the AsciiDoctor Maven plugin.

All common attributes like "enable section numbering" should go into
this document, not into the separate content documents.
That way the styles are automatically consistent.

////

// Enable user interface macros like "kbd".

// <https://asciidoctor.org/docs/user-manual/#user-interface-macros>

:experimental:

//

// Normalize source listings to have no indent

// <https://docs.asciidoctor.org/asciidoc/latest/attributes/document-attributes-ref/#source-highlighting-and-formatting-attributes>

:source-indent: 0

//

// tabs with 4 spaces (default are 8), like in eclipse

// <https://asciidoctor.org/docs/user-manual/#builtin-attributes>

:tabsize: 4

//

// Use font characters for admonition icons

// <https://asciidoctor.org/docs/user-manual/#admonition-icons>

:icons: font

//

// explicitly use the "fontawesome-solid" icon set, since the default icon set is deprecated

:icon-set: fas

//

// hide URI scheme from plain text links.

// <https://docs.asciidoctor.org/asciidoc/latest/macros/links/#hide-uri-scheme>

:hide-uri-scheme:

//

// enable section numbering up to level 4

:sectnums:

:sectnumlevels: 4

//

// strip the leading underscore from generated ids (HTML anchors)

:idprefix:

//

// Enable section anchors in front of headlines.

// <https://asciidoctor.org/docs/user-manual/#anchors> -->

:sectanchors:

//

// enable source highlighting

// <https://docs.asciidoctor.org/asciidoc/latest/verbatim/source-highlighter/>

:source-highlighter: coderay

//

```
// define common URLs as attributes so they can be used like macros
//
:4diac-documentation-repo:      https://github.com/eclipse-4diac/4diac-documentation
:issues:                        {4diac-documentation-repo}/issues
:discussions:                   {4diac-documentation-repo}/discussions
:pulls:                          {4diac-documentation-repo}/pulls
```

Как использовать библиотеку Arrowhead от 4diac

Введение

4diac FORTE поддерживает взаимодействие с основными системами ServiceRegistry, Orchestrator, Authorization и EventHandler фреймворка Arrowhead версии 4.1.2 с помощью HTTP-JSON и OPCUA-JSON.

Фреймворк [Arrowhead \(AF\)](#) — это фреймворк, обеспечивающий взаимодействие между сервисами в сервис-ориентированной архитектуре (SOA). На первый взгляд, нелегко понять саму область применения и ее определение, а документация к ней не всегда помогает, поэтому в следующем параграфе мы попытаемся объяснить основы AF.

AF - это набор интерфейсов, правил и системы документации, которые позволяют устройствам предоставлять и использовать услуги. AF абстрактно определяет типы сервисов, облака, системы и других понятий.

В AF также указано, что в локальном облаке должно быть как минимум 3 основные системы (также называемые базовыми системами): система ServiceRegistry (позволяет регистрировать, отменять регистрацию и запрашивать сервисы), система Orchestrator (позволяет находить предоставляемые сервисы, назначенные для использования устройствами) и система авторизации (управляет всеми необходимыми авторизациями между устройствами и сервисами). Эти системы, а также другие, не являющиеся обязательными, предоставляют набор определённых сервисов, которые, разумеется, определяются с помощью интерфейсов и типов AF. Документацию по каждой системе можно найти в соответствующей папке в [репозитории GitHub](#). На этом этапе важно отметить, что все сервисы, определённые для основных систем, не зависят от реализации, но HTTP/JSON является первой и фактически официальной реализацией, хотя сервисы могут использовать и другие протоколы связи и моделирования.

Сервис, который будет создавать или использовать ваша система, не определён в AF, но то, как вы должны его документировать, определено.

Базовая последовательность действий для создания и использования сервиса в AF выглядит следующим образом:

1. Настройте локальное облако: сначала вам нужно сообщить системе авторизации, какие сервисы разрешено использовать той или иной системе, а системе оркестрации — где находятся поставщики сервисов для потребителей (IP-адрес и порт).
2. Когда поставщик сервисов подключается к локальному облаку, он должен зарегистрироваться в реестре сервисов.
3. Потребитель сервисов сделает то же самое. Кроме того, он свяжется с Orchestrator и запросит конечную точку поставщика услуг.
4. Orchestrator проверит в своей частной базе данных (настроенной в пункте 1) конечную точку поставщика услуг и с помощью системы авторизации проверит, разрешено ли потребителю получать услуги от поставщика.
5. Если да, то Orchestrator ответит потребителю (пункт 3) конечной точкой поставщика услуг.
6. Потребитель услуг подключается к поставщику услуг с помощью конечной точки, указанной в пункте 5, и использует услуги.

Как уже говорилось ранее, интерфейс услуг между потребителем и поставщиком не является фиксированным, и AF не имеет к этому никакого отношения. AF обеспечивает поддержку регистрации услуг и их поиска.

Важно! Разработанная библиотека Arrowhead от 4diac предоставляет типы FB и AF для взаимодействия с официальной [платформой Arrowhead](#) версии 4.1.2. Новая версия 4.1.3 платформы AF находится в другой репозитории, и API изменился, поэтому она несовместима с библиотекой 4diac. Предоставленную библиотеку можно использовать с HTTP по предоставленной ссылке или с OPC UA, но для этого вам нужно использовать эту специальную версию, которая расширяет платформу AF интерфейсами OPC UA.

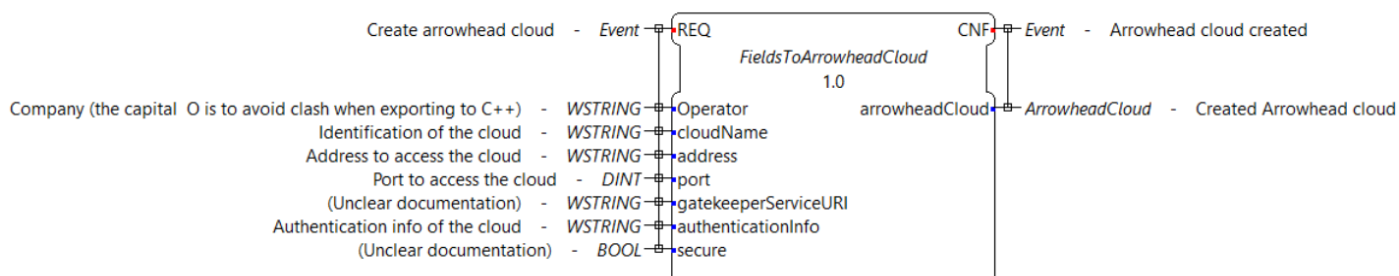
Включение модуля Arrowhead в 4diac FORTE

Первое, что необходимо, — это версия 4diac FORTE с включенным модулем Arrowhead. Для этого вам нужно скомпилировать 4diac FORTE самостоятельно. Для этого выполните следующие [действия](#) и

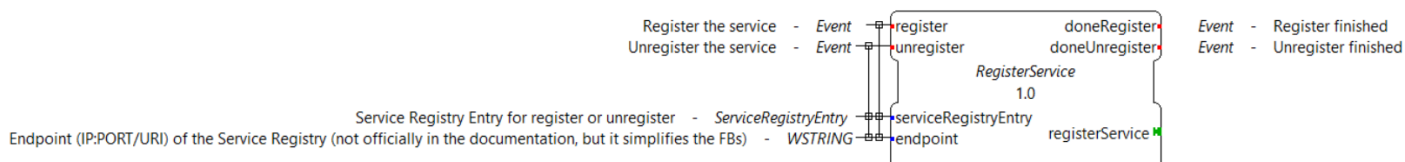
в CMake установите для переменных FORTE_MODULE_Arrowhead и FORTE_COM_HTTP или/и FORTE_COM_OPC_UA значение TRUE.

После компиляции 4diac FORTE вы будете готовы использовать библиотеку FBs в 4diac IDE.
FBs в 4diac IDE

В 4diac IDE была определена библиотека функциональных блоков, и это было сделано на трёх уровнях. На самом низком уровне вспомогательные функциональные блоки позволяют пользователю создавать типы Arrowhead Framework, используя стандартные типы из IEC 61499. На следующем рисунке показан функциональный блок для создания облака со стрелкой, которое впоследствии можно подключить к верхним уровням.



На втором уровне были реализованы фактические сервисы Arrowhead. Эти функциональные блоки предлагают адаптер в соответствии со стандартом IEC 61499, чтобы отделить абстрактное определение сервисов от их фактической реализации. В качестве примера на следующем рисунке показан функциональный блок для регистрации и отмены регистрации сервиса в реестре сервисов. Входные данные — это тип записи в реестре сервисов (независимый от протокола) и конечная точка для подключения. Адаптер «registerService» в нижней правой части предлагает подключение к фактической реализации связи, передавая все необходимые данные.



Сокет для этого адаптера предоставляется каждой реализацией. На следующем рисунке показан функциональный блок, реализующий фактическую HTTP-связь с реестром служб (существует блок с таким же интерфейсом для OPC UA, но другой внутренней реализацией). Его единственным входом является адаптер сокета, противоположная часть адаптера подключения с предыдущего рисунка. При их соединении информация передаётся в этот функциональный блок, который обрабатывает связь.



Эта отдельная архитектура позволяет быстро реализовать другой тип связи. Например, чтобы подключиться к реестру служб с помощью интерфейса OPC UA, нужно было только переработать функциональный блок на последнем рисунке с учётом особенностей OPC UA.

На самом верхнем уровне были реализованы вспомогательные приложения, чтобы упростить использование библиотеки для пользователя. Например, для регистрации сервиса вместо создания приложения с нуля, что требует множества функциональных блоков и соединений, предоставляется инкапсулированное подприложение со всеми необходимыми параметрами. На следующем рисунке показано

подприложение со всеми параметрами для регистрации и отмены регистрации сервиса с помощью HTTP.

Register the service	- Event	register	doneRegister	Event	- Register finished
Unregister the service	- Event	unregister	doneUnregister	Event	- Unregister finished
RegisterServiceFullHTTP 1.0					
Identification of the system	- WSTRING	systemName			
Address of the offered services	- WSTRING	address			
Port of the of the offered services	- DINT	port			
Authentication info of the system	- WSTRING	authenticationInfo			
Identification of the service	- WSTRING	serviceDefinition			
Interfaces implemented by the service	- WSTRING[10]	interfaces			
Metadata of the service in key=value format	- WSTRING[10]	serviceMetadata			
The URL subpath of the Service within the address:port of the Provider	- WSTRING	serviceURI			
The version numbering of the Service provided	- DINT	version			
True, if the service is provided via UDP. By default, it is false	- BOOL	udp			
Service validity time period expressed in seconds. This gets converted to a date-time, and stored in the database	- DINT	ttl			
Metadata belonging to a service/provider pair	- WSTRING	metadata			
Endpoint (IP:PORT/URI) of the Service Registry	- WSTRING	SREndpoint			

Назначение доступных FB в библиотеке Arrowhead

Ниже приведена информация о назначении всех FB и подприложений в библиотеке. Параметры каждого из них описаны в самих FB и подприложениях.

Те, которые отмечены этим текстом, являются основными и, как правило, единственными, которые вам нужны для быстрого использования библиотеки.

- Общие
 - FieldsToArrowheadCloud: преобразует отдельные поля в тип ArrowheadCloud
 - FieldsToArrowheadService: Преобразует отдельные поля в тип ArrowheadService
 - FieldsToArrowheadSystem: Преобразует отдельные поля в тип ArrowheadSystem
 - JSON
 - ANYToJSON: Преобразует 61499 типов в формат JSON
 - GetArrayResponseFromJSON: Преобразует HTTP-ответ в массив
- Реестр служб
 - FieldsToServiceQueryForm: Преобразует отдельные поля в форму запроса на обслуживание
 - FieldsToServiceRegistryEntry: Преобразование отдельных полей в тип реестра служб.
 - GetEndpointFromServiceRegistry: получение конечной точки IP:PORT/URI из записи реестра служб
 - QueryService: запрос блока функций службы
 - QueryServicesAdp: запрос адаптера служб
 - RegisterService: регистрация блока функций службы
 - RegisterServiceAdp: регистрация адаптера службы
 - ServiceRegistryEntry2ServiceRegistryEntry: вспомогательный FB для установки подключения к типу записи реестра служб
 - http
 - QueryServiceHTTP: Запрос служб с использованием HTTP
 - QueryServiceHTTPSub: Запрос служб с использованием HTTP с определением службы
 - QueryServiceHTTPSubFull: Запрос служб с использованием HTTP с определением всех полей службы
 - [.specificText]#RegisterMultipleServicesHTTP: Зарегистрировать множество служб с разными ServiceDefinition и serviceURI #
 - RegisterServiceFullHTTP: Зарегистрировать службу, используя HTTP. Доступны все возможные параметры для настройки
 - RegisterServiceHTTP: Зарегистрируйте службу, используя HTTP
 - RegisterServicePartialHTTP: Зарегистрируйте службу, используя HTTP. Системная информация инкапсулирована
 - OpсUa
 - QueryServiceOpсUa: Запрос служб с использованием OpсUa
 - QueryServiceOpсUaSub: Запрос служб с использованием OpсUa с определенной службой

- QueryServiceOpcUaSubFull: Запрашивает службы с использованием OpcUa, при этом все поля службы должны быть определены
 - [.specificText]#RegisterMultipleServicesOpcUa: Регистрирует множество служб с разными ServiceDefinition и serviceURI. #
 - RegisterServiceFullOpcUa: Зарегистрируйте услугу с помощью OpcUa. Доступны все возможные параметры для настройки
 - RegisterServiceOpcUa: Зарегистрируйте услугу с помощью OpcUa
 - RegisterServicePartialOpcUa: Зарегистрируйте службу, используя OpcUa. Системная информация инкапсулирована
- Orchestrator
 - FieldsPreferredProvider: Преобразуйте отдельные поля в тип PreferredProvider.
 - FieldsToServiceRequestForm: Преобразует отдельные поля в тип ServiceRequestForm
 - GetEndpointFromOrchestration: получает конечную точку IP:ПОРТ/URI из формы согласования.
 - OrchestrationForm2OrchestrationForm: Вспомогательный FB для установки соединения с типом формы оркестровки
 - OrchestratorRequestAdp: Запрос адаптера оркестровки
 - RequestOrchestrationForm: Запрос функционального блока оркестровки
 - http
 - GetEndpointFromOrchestrationHTTPPartial: Получить конечную точку по ИНДЕКСУ из ответа на согласование запроса
 - GetEndpointFromOrchestrationHTTPFull: Получить конечную точку по индексу из ответа на согласование запроса
 - RequestOrchestrationHTTP: Функциональный блок согласования запросов с использованием HTTP
 - Requestorchestrationhttp: Частичный блок согласования запросов с использованием HTTP, в котором должны быть установлены все поля из служб
 - RequestOrchestrationHTTPFull: Согласование запросов с использованием HTTP, при этом должны быть установлены все поля из служб, системы и облака
 - OpcUa
 - GetEndpointFromOrchestrationOpcUaPartial: Получить конечную точку по ИНДЕКСУ из ответа на согласование запроса
 - GetEndpointFromOrchestrationOpcUaFull: получить конечную точку по индексу из ответа на согласование запроса
 - RequestOrchestrationOpcUa: Функциональный блок запроса на согласование с использованием OpcUa
 - RequestOrchestrationOpcUaPartial: Запрос на согласование с использованием OpcUa, в котором должны быть заданы все поля из служб
 - RequestOrchestrationOpcUaFull: Запросить согласование с использованием OpcUa, при этом должны быть установлены все поля из служб, системы и облака
- Обработчик событий
 - ArrowheadPublish: Функциональный блок публикации событий
 - ArrowheadPublishAdp: Адаптер для публикации событий
 - FieldsToArrowheadEvent: Преобразует отдельные поля в тип события в виде стрелки
 - FieldsToEventFilter: Преобразует отдельные поля в тип фильтра событий в виде стрелки
 - FieldsToPublishEvent: Преобразует отдельные поля в событие публикации в виде стрелки
 - SubscribeEvent: Функциональный блок события подписки
 - SubscribeEventAdp: Адаптер события подписки
 - http
 - PublishEventHTTP: Публикация события с использованием HTTP
 - PublishEventHTTPFull: Публикация события с использованием HTTP. Доступны все возможные параметры для настройки

- PublishEventHTTPPartial: Опубликовать событие, используя HTTP. Информация о системе и событии инкапсулирована
- SubscribeEventHTTP: Подписаться на событие, используя HTTP
- SubscribeEventHTTPFull: Опубликовать событие, используя HTTP. Пользователь инкапсулирован
- SubscribeEventHTTPPartial: Публикуем событие, используя HTTP. Пользователь инкапсулирован
- OpcUa
 - PublishEventOpcUa: Публикуем событие, используя OpcUa
 - PublishEventOpcUaFull: Публикуем событие, используя OpcUa. Доступны все возможные параметры для установки
 - PublishEventOpcUaPartial: Публикация события с использованием OpcUa. Информация о системе и событии инкапсулирована
 - SubscribeEventOpcUa: Событие подписки с использованием OpcUa
 - SubscribeEventOpcUaFull: Событие публикации с использованием OpcUa. Пользователь инкапсулирован
 - SubscribeEventOpcUaPartial: Событие публикации с использованием OpcUa. Потребитель инкапсулирован

Примеры и функциональные блоки

Чтобы увидеть несколько примеров использования функциональных блоков, вы можете обратиться к [репозиторию примеров Eclipse 4diac](#). Функциональные блоки нельзя найти непосредственно в IDE 4diac, но вы найдёте их в репозитории примеров.

Куда двигаться дальше?

Вернитесь к индексу протоколов: [Индекс связи](#)

Если вы хотите вернуться на страницу «Начало работы», мы оставляем вам ссылку для быстрого перехода
Страница «[Начало работы](#)»

FBDK/IP

В этом разделе показано, как заставить приложения взаимодействовать с помощью протокола FBDK/ip по умолчанию, как определено в профиле соответствия [IEC 61499 для демонстрационных проектов](#). Поскольку мы будем использовать протокол по умолчанию, нам не нужно указывать его в качестве параметра в начале ввода идентификатора блоков коммуникационных функций. Нам нужно только указать IP-адрес и порт в следующем формате:

```
address:port
```

Таким образом, идентификатор будет выглядеть так:

```
192.168.0.2:61499
```

будет автоматически преобразовано в следующее:

```
fbdk[] .ip[192.168.0.2:61499]
```

Публикация/подписка — UDP/IP

Для связи с помощью блоков функций «Публикация/подписка» используется соединение UDP/IP, поэтому мы должны предоставить блокам функций «Публикация/подписка» многоадресный IP-адрес и порт. FB-клиенты для публикации будут отправлять данные по сети на этот многоадресный IP-адрес/порт, а FB-клиенты для подписки будут прослушивать тот же многоадресный IP-адрес/порт, получая любые данные, отправленные FB-клиентами для публикации. Многоадресные IP-адреса можно выбрать из диапазона 224.0.0.0 — 239.255.255.255, а порт может быть любым. Таким образом, пример параметра ID для FB-клиентов для публикации/подписки будет следующим:

```
225.0.0.1:61499
```

В отличие от пар «клиент/сервер» (см. [следующий раздел](#)), FB «публикация/подписка» не обязательно должны работать в паре. Любой FB «подписки» на любом устройстве, прослушивающем этот многоадресный IP-адрес, будет получать данные, отправленные любым FB «публикации» на этом многоадресном IP-адресе.

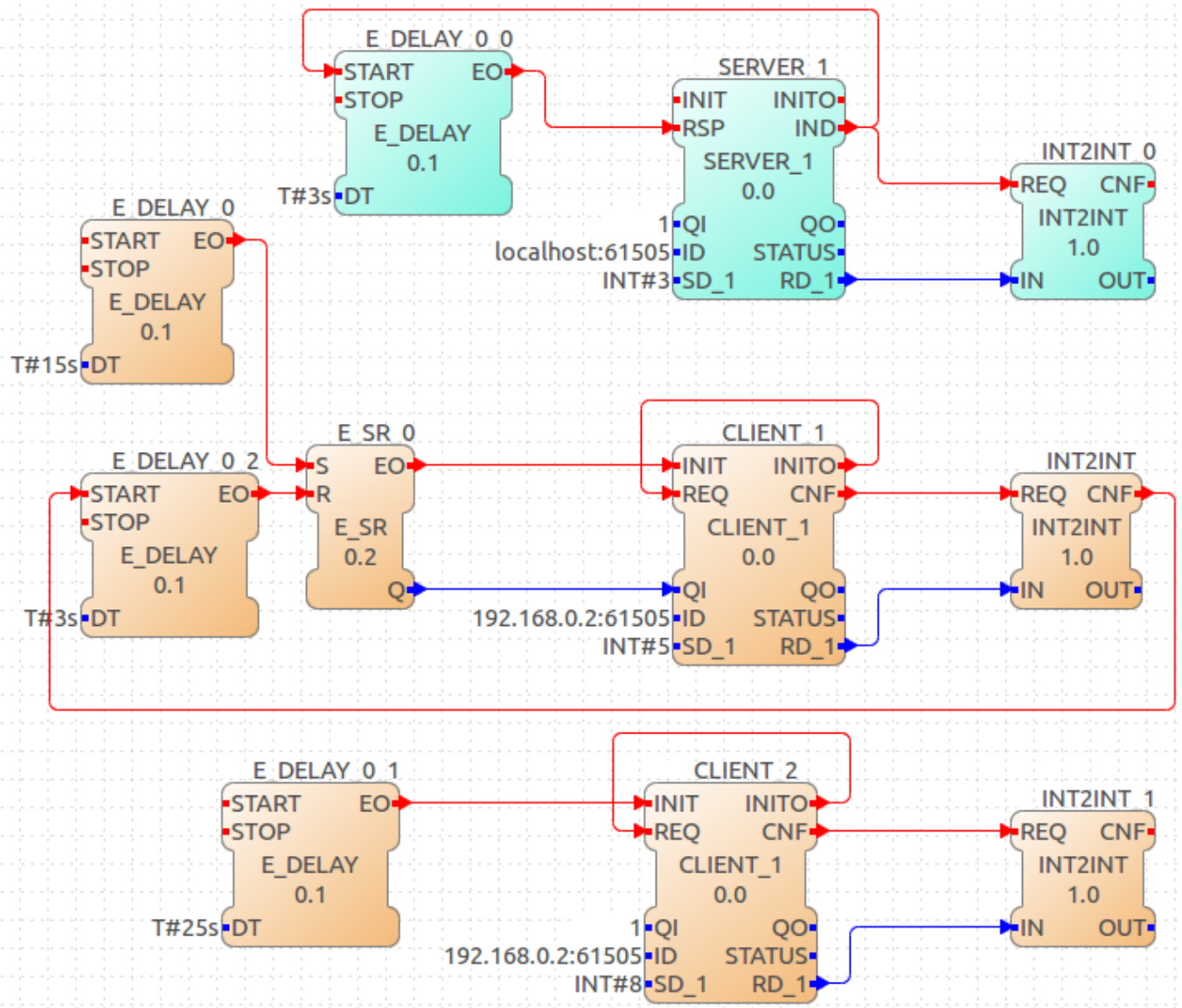
Пример использования FB «публикация/подписка» показан на рисунке ниже и может быть обобщён следующим образом:

1. После задержки в 10 секунд первое устройство (красное) отправляет целое число 2 по протоколу UDP на многоадресный IP-адрес 225.0.0.1 и порт 61500.
2. Второе устройство (зелёное) получает целое число, прослушивая тот же многоадресный IP-адрес.
3. Второе устройство увеличивает полученное целое число на 3.
4. Второе устройство отправляет результат по протоколу UDP на многоадресный IP-адрес 225.0.0.1 и порт 61499.
5. Первое устройство получает результат, прослушивая тот же многоадресный IP-адрес.

Обратите внимание, что:

- каждый FB публикации имеет такое же количество входов, как и количество выходов соответствующего FB подписки.
- каждая пара Публикация/Подписка имеет свой собственный уникальный IP-адрес и порт многоадресной рассылки.
- уникального номера порта в каждой паре Публикация/подписка достаточно для того, чтобы эти две пары не мешали друг другу.
- ввод события RSP в Subscribe FBs не используется, в то время как вывод события CNF в Publish FBs устанавливается напрямую (см. раздел Пары клиент/сервер).
- Как упоминалось выше, FB для публикации и FB для подписки не обязательно должны работать в паре. Любые данные, отправленные с определённого многоадресного IP-адреса любым FB для публикации, будут получены любым FB для подписки, прослушивающим этот многоадресный IP-адрес.

- каждый клиентский FB имеет столько же входов и выходов, сколько входов и выходов соответствующего серверного FB.
- Ответ клиентским FB должен быть отправлен серверному FB, он не отправляется автоматически
- FB SERVER_1 должен быть инициализирован до FB CLIENT_1 и FB CLIENT_2, чтобы они могли подключиться к нему.
- FB CLIENT_1 должен быть закрыт до того, как FB CLIENT_2 сможет подключиться к FB SERVER_1, то есть сервер может обслуживать только одного клиента одновременно.



Куда двигаться дальше?

Вернитесь к разделу «Протоколы»: Раздел «[Коммуникация](#)»

Если вы хотите вернуться на страницу «[Начать здесь](#)», мы оставляем вам ссылку для быстрого перехода

HTTP

В этом разделе показано, как заставить приложения взаимодействовать по протоколу HTTP. Вы должны использовать клиентские и серверные библиотеки в соответствии с вашими потребностями. Блоки «Опубликовать» и «Подписаться» в настоящее время не поддерживаются. В настоящее время поддерживаются только запросы GET, POST или PUT.

Конфигурация

4diac Forte необходимо скомпилировать из исходного кода, чтобы модуль HTTP работал. Следуйте [инструкциям по созданию собственных библиотек](#). На шаге 3, где выбираются функции для компиляции, выберите также FORTE_COM_HTTP. Затем выполните ту же процедуру, скомпилируйте 4diac FORTE, и вуаля, у вас есть поддержка HTTP в 4diac FORTE.

Клиент

Для реализации HTTP-клиента необходим функциональный блок клиента с 3 входами SD и 2 выходами RD, иначе среда выполнения выдаст ошибку. Ожидается, что SD-запросы будут иметь тип STRING или WSTRING, а RD-запросы — тип STRING. В настоящее время поддерживаются только GET, PUT и POST. Входное значение ID блока клиентской функции должно быть указано следующим образом:

```
http[IP:PORT/PATH;POST|PUT|GET;content-type]
```

IP:PORT указывает IP-адрес и порт удалённой конечной точки. PATH указывает путь на сервере, к которому нужно подключиться, и отделяется от порта косой чертой. В качестве типа запроса можно использовать POST, PUT или GET, и он должен быть написан заглавными буквами. Тип запроса является обязательным. Тип содержимого представляет собой HTTP-тип содержимого отправляемого пакета, но является необязательным. Тип содержимого по умолчанию — text/html. Примеры ввода идентификатора:

- `http[34.231.219.193:80/get;GET;text/html]` → отправка запроса GET с типом содержимого text/html
- `http[34.231.219.193:80/put;PUT;application/json]` → отправить запрос PUT с типом содержимого application/json

Первый параметр может использоваться для аутентификации, но его также можно оставить пустым, если соответствующий сервер этого не требует. Второй параметр может использоваться для параметров, которые должны быть разделены символом & в HTML-коде. Этот второй параметр также можно оставить пустым, если параметры не нужны. Третий вход SD используется для тела, которое необходимо отправить. Примеры для трёх входов SD следующие:

- аутентификация: введите здесь токен, предоставленный сервером
- параметры: `org=организация&bucket=датчики`
- тело: `airSensors, идентификатор_датчика=TLM0201, температура=73,97038159354763, влажность=35,23103248356096, CO=0,48445310567793615`

Оба RD используются для ответа сервера. Первый RD используется для кода ответа, а второй RD — для тела ответа сервера.

Сервер

Для реализации HTTP-сервера необходим функциональный блок сервера с 1 входом SD для тела создаваемого им ответа. Вход SD ожидает тип STRING или WSTRING. Вход ID функционального блока сервера должен быть указан следующим образом:

```
http[PATH]
```

Указанный PATH добавляется к серверу при запуске. Функциональный блок сервера запускает событие при обращении к PATH с помощью запроса GET. В ответах сервера используется версия HTTP HTTP/1.1 и тип содержимого text/plain по умолчанию. Если указаны параметры, они должны соответствовать количеству RD в блоке функций и иметь тип (имя=значение&....). Для определения более одного параметра следует использовать символ «&». Имя фактически не используется. Значения сохраняются в RD в порядке их поступления и обрабатываются как строковые значения.

Куда двигаться дальше?

Вернитесь к разделу «Протоколы»: [Индекс сообщений](#)

Если вы хотите вернуться на страницу «[Начать здесь](#)», мы оставляем вам ссылку для быстрого перехода

Поддерживаемые Eclipse 4diac протоколы связи

Эта страница предназначена для ознакомления с протоколами связи, поддерживаемыми 4diac FORTE.

- [OPC UA](#)
- [OPC DA](#)
- [HTTP](#)
- [MQTT](#)
- [Связь с инструментами моделирования](#)
 - [FMI](#)
 - [Связь с помощью Matlab](#)^м
- [TSN](#)
- [FBDK/IP](#): протокол, определенный в профиле соответствия [IEC 61499 для демонстрационных испытаний](#), и протокол по умолчанию в 4diac FORTE.
- [Modbus](#)
- [openPOWERLINK](#)
- [Arrowhead](#)
- [Доступ к базе данных BaseX с помощью XQueries](#)^м

Modbus

Инструкции по установке

Для работы интерфейса Modbus требуется библиотека libmodbus. Перед компиляцией 4diac FORTE выполните следующие действия по настройке среды.

1. Файлы libmodbus должны быть размещены в следующей структуре папок:
 - o [libmodbushome]/include/modbus: include files
 - o [libmodbushome]/lib: libmodbus.lib (windows) or libmodbus.so (posix)
2. При настройке проекта 4diac FORTE с помощью CMake выберите

```
[libmodbushome] as FORTE_COM_MODBUS_LIB_DIR
```

3. forte.exe должен иметь возможность найти динамическую библиотеку libmodbus после сборки.

Поэтому скопируйте libmodbus.dll/so в папку с forte.exe или добавьте libmodbus.dll/so в путь включения библиотеки.

Параметры

Клиент Modbus (TCP)

Использование клиента Modbus (TCP):

```
modbus[ (protocol:) ip:port: (slaveId) :pollFrequency:readAddresses:sendAddresses (:responseTimeout:byteTimeout) ]
```

- протокол: tcp (по умолчанию используется tcp)
- ip: адрес сервера Modbus, например 127.0.0.1
- порт: желательно выше 1024, по умолчанию 502
- slaveId (необязательно): идентификатор ведомого устройства, используемый сервером Modbus (стандартно 0xFF); в принципе, это необязательный параметр, но некоторым устройствам Modbus (например, устройствам измерения мощности Siemens SENTRON PAC) требуется этот параметр для установления соединения. Кроме того, если вы используете шлюз Modbus-RTU для Modbus-TCP, например EX9133C-2-MTCP, то этот идентификатор ведомого устройства указывает, к какому устройству на стороне RTU вы хотите подключиться.
- pollFrequency: частота опроса в миллисекундах
- readAddresses: адреса могут быть указаны в диапазоне от 0 до 65535. Более одного адреса (максимум 100) могут быть указаны с помощью
 - o запятой для разделения адресов 0, 2, 65500
 - o точек для интервала 5..10
 - o комбинации 0, 5..10, 2, 65500
 - o по умолчанию используется регистр хранения, который можно изменить с помощью буквы в начале
 - 'c' для катушки
 - 'd' для дискретного входа
 - 'h' для регистра хранения
 - 'i' для входного регистра
- -> см. подробную таблицу ниже
- sendAddresses: адреса могут быть указаны в диапазоне от 0 до 65535, если данные только считываются. Если данные только считываются, sendAddresses следует оставить пустым. Можно указать более одного адреса (максимум 100), используя
 - o запятую для разделения адресов 0, 2, 65500
 - o точки для интервала 5..10
 - o комбинация 0,5..10,2,65500
 - o функция может быть выбрана, как для readAddresses
 - o responseTimeout (необязательно): время ожидания ответа в миллисекундах (по умолчанию 500 мс)
- byteTimeout (необязательно): время ожидания в миллисекундах между двумя последовательными байтами (по умолчанию 500 мс)
- для повторного использования предыдущего подключения укажите только IP-адрес и порт, а slaveId оставьте пустым

Пример:

```
modbus[tcp:127.0.0.1:502::20000:h0..3:]
```

Клиент Modbus (RTU)

Использование клиента Modbus (RTU):

```
modbus[protocol:port:baudrate:parity:databits:stopbits:flow:(slaveId):pollFrequency:readAddresses:sendAddresses(:responseTimeout:byteTimeout)]
```

- протокол: rtu
- порт: последовательный порт, например /dev/ttyS0 или COM1 скорость передачи данных: скорость передачи данных последовательного порта, например 9600
- чётность: N — без чётности, E — чётная, O — нечётная
- биты данных: количество битов данных, например 8
- стоповые биты: количество стоповых битов, например 1
- поток:
- оставьте пустым для отсутствия потока
- arduino — отключите DTR и дождитесь загрузки Arduino на всякий случай
- delay — подождите 2 секунды после подключения
- longdelay — подождите 3 секунды после подключения
- все остальные параметры такие же, как для TCP
- кроме slaveId, который в некоторых случаях необязателен для Modbus-TCP, но обязателен для Modbus-RTU, и в большинстве случаев заводские настройки устройств по умолчанию — 1
- чтобы повторно использовать предыдущее подключение, укажите только порт и оставьте slaveId пустым

Пример:

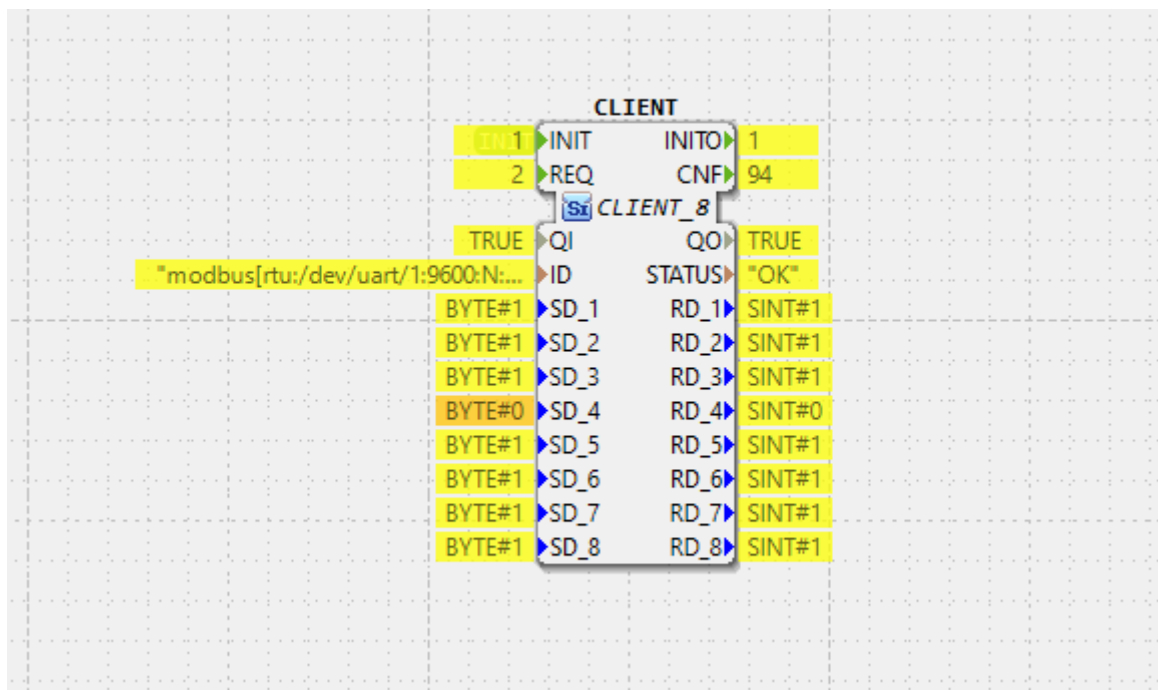
```
modbus[rtu:/dev/ttyUSB0:19200:N:8:1::1:2000:i6142,i6132,i6137:h64025..64028,h63995..63998]
```

Таблица функций:

		read	write
c	coil	modbus read bits 0x01 (read coil status)	modbus write bits 0x0F (force multiple coils)
d	discrete input	modbus read input bits 0x02 (read input status)	
h	holding register	modbus read registers 0x03 (read holding registers)	modbus write registers 0x10 (preset multiple registers)
i	input register	modbus read input registers 0x04 (read input registers)	

Пример: запись выходных данных 8-канальной релейной карты:

```
modbus[rtu:/dev/uart/1:9600:N:8:1::1:2000:c0..7:c0..7]
```



modbus с набором 8Q

Куда двигаться дальше?

Вернуться к списку протоколов: [Индекс коммуникации](#)

Если вы хотите вернуться на страницу «[Начать здесь](#)», мы оставляем вам ссылку для быстрого перехода

MQTT с Eclipse Paho

В этом разделе показано, как заставить приложения взаимодействовать с помощью [MQTT Paho](#). 4diac FORTE использует многоуровневую архитектуру связи. Для MQTT Paho реализован базовый уровень, соединяющий MQTT Paho и уровень FBDK.

Важно! Обратите внимание, что при текущей реализации невозможно подключиться более чем к одному брокеру MQTT Paho на одном устройстве.

Поскольку вам нужен MQTT Paho, вам нужно установить библиотеки на свой компьютер. Поэтому вам нужно скачать код и скомпилировать его. Вам понадобятся те же инструменты, что и для 4diac FORTE (git, make, компиляторы). Процесс основан на обычной компиляции 4diac FORTE, но функция MQTT включена. [Вот](#) краткая ссылка на компиляцию 4diac FORTE. Для установки выполните следующие действия:

1. Скачайте и соберите MQTT Paho (если вы используете Windows, некоторые команды могут отличаться):

```
$ git clone https://github.com/eclipse/paho.mqtt.c.git $ cd paho.mqtt.c
```

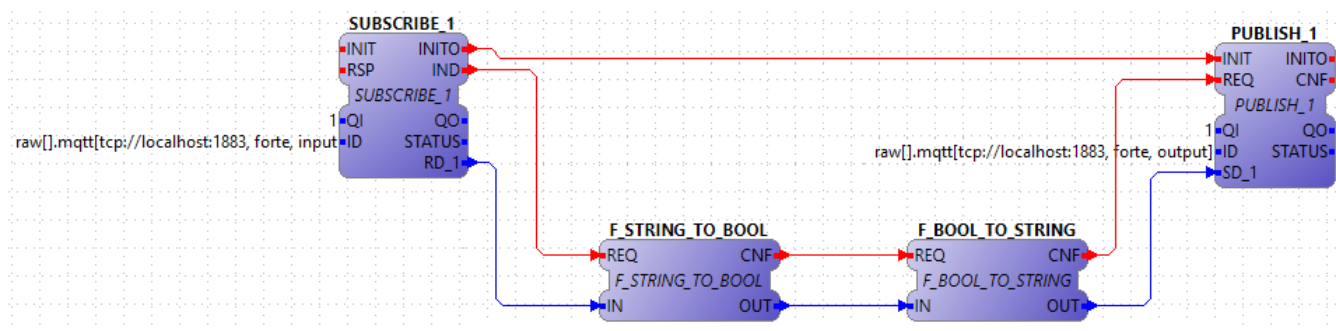
2. Используйте CMake для создания файлов для компиляции так же, как вы использовали бы его для компиляции 4diac FORTE. Параметры CMake для Paho меньше и проще для понимания. Вы экспортируете файлы в [binPaho]. Затем выполните

```
$ cd [binPaho] $ make
```

3. Настройте файлы компиляции 4diac FORTE, запустив CMake, как обычно, и выполните следующие действия:

- i. Проверьте наличие FORTE_COM_PAHOMQTT в CMake
- ii. Задайте для FORTE_COM_PAHOMQTT_INCLUDE_DIR CMake папку, в которой находятся заголовочные файлы Paho.
- iii. Убедитесь, что библиотека, указанная в FORTE_COM_PAHOMQTT_LIB, имеет то же имя, что и скомпилированная вами. Если нет, измените ее.
- iv. Задайте FORTE_COM_PAHOMQTT_LIB_DIR в качестве папки, в которой находится FORTE_COM_PAHOMQTT_LIB.
- v. Настройте CMake так, чтобы ничего не было помечено красным, затем сгенерируйте с помощью CMake.
- vi. Соберите 4diac FORTE
- vii. Если вы используете MQTT Paho на виртуальной машине, используйте NAT и настройте переадресацию для порта 1883, который является портом по умолчанию для брокера MQTT Paho. IP-адреса для переадресации могут быть пустыми.
- viii. Для связи с MQTT Paho используйте блоки функций публикации/подписки 4diac FORTE и настройте ввод данных идентификатора следующим образом:

```
raw[].mqtt[tcp://ip:port, clientID, topic]
```



Пример MQTT

Внимание! При использовании слоя raw[] в идентификаторе RD_1 вашего подписчика и SD_1 ваших издателей должны быть подключены к данным типа STRING. Кроме того, вы можете использовать только PUBLISHER_1 и SUBSCRIBER_1, а не другие функциональные блоки с большим количеством входов или выходов.

Основная причина использования этого дополнительного необработанного параметра заключается в том, что MQTT не определяет формат полезной нагрузки. С помощью дополнительного параметра вы указываете в 4diac IDE так называемый уровень связи, который будет использоваться поверх MQTT. Этот уровень связи

теперь отвечает за получение данных из IEC 61499 и создание соответствующего формата полезной нагрузки или за получение полезной нагрузки и её преобразование для выходных данных FB. Необработанный уровень просто принимает строки и отправляет необработанное содержимое строки.

Куда двигаться дальше?

Вернитесь к индексу протоколов: [Индекс связи](#)

Если вы хотите вернуться на страницу «[Начать здесь](#)», мы оставляем вам быстрый доступ

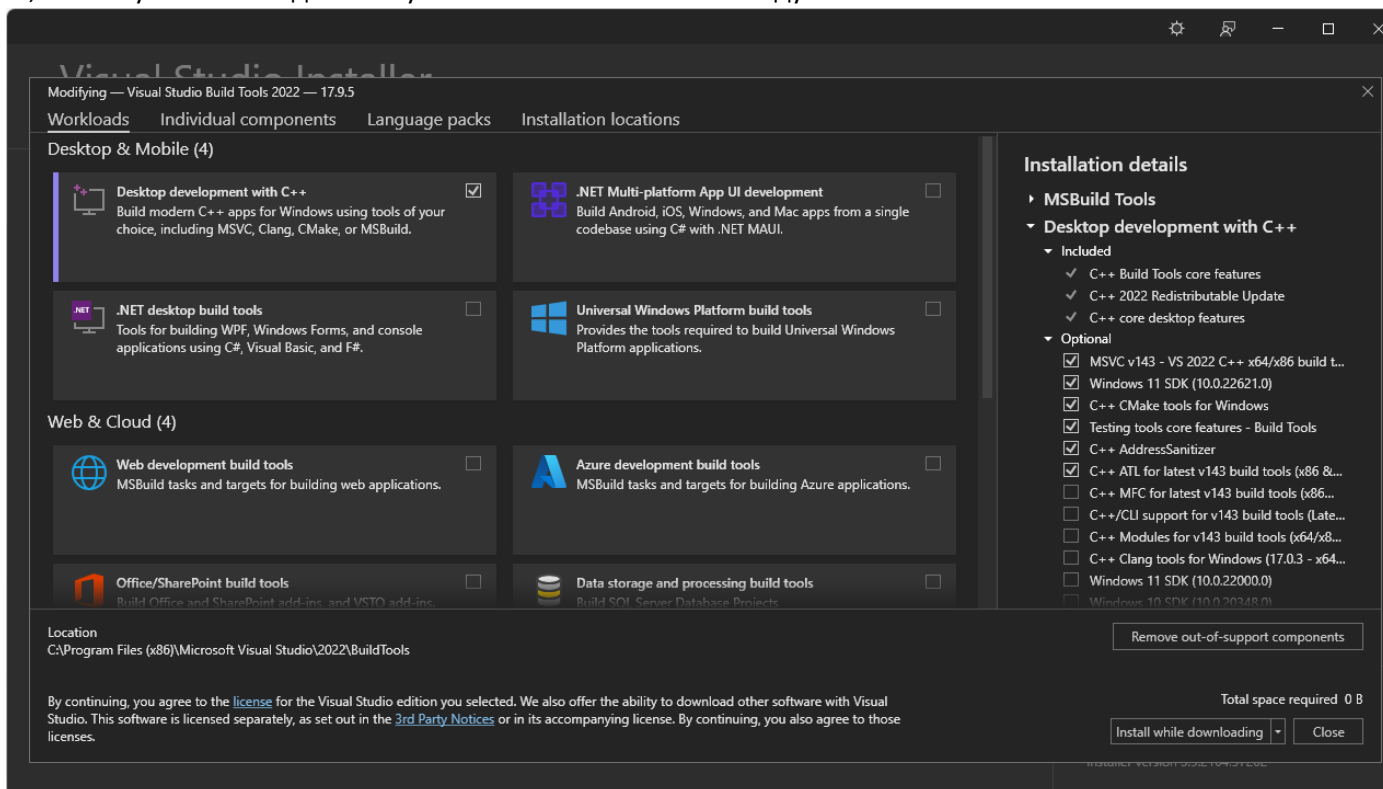
OPC DA

В этом разделе вы узнаете, как создать клиент OPC DA с помощью 4diac IDE и как использовать его с 4diac FORTE. Скачайте следующие пакеты:

- [Библиотека OPC Client версии 0.5.1](#)
- [Boost Lexical Cast](#)
- [Установщик Visual Studio 2022](#)

Подготовьте и скомпилируйте набор инструментов OPC Client

1. Распакуйте библиотеки Boost C++
2. Установите инструменты сборки Visual Studio с компонентами, указанными ниже. Начиная с [Visual Studio 2013](#), atls.lib и atlsd.lib объединены в atls.lib, который вместе с заголовками может быть обнаружен msvc, поэтому нет необходимости указывать в CMakeLists.txt модуль OPC.



3. Клонировать библиотеку OPC Client, перейдите в ветку разработки и откройте папку OPC-Client-X64 в Visual Studio Code. Visual Studio Code с установленным пакетом расширений C/C++ должен распознавать OPC-Client-X64 как проект CMake. Создайте файл settings.json в папке .vscode, как показано ниже.

```
{
  "cmake.configureOnOpen": false,
  "cmake.configureSettings": {
    "CMAKE_INSTALL_PREFIX": "D:/4diac/OPC-Client-X64/",
    "OPCDACLIENT_TESTING": false
  }
}
```

4. Откройте командную палитру (ctrl + shift + p), введите следующую команду:
cmake:select a kit; //choose Visual Studio Build Tools 2022 Release
cmake:select variant; //debug or release
cmake:configure;
cmake:build //compile
cmake:install //install

Скомпилируйте 4diac FORTE с поддержкой OPC DA Client

1. Создайте проект MS Visual Studio Code для 4diac FORTE с помощью CMake, как описано в разделе «[Компиляция и отладка 4diac FORTE с помощью MS Visual Studio Code](#)», и добавьте следующие переменные в settings.json, которые необходимы для поддержки OPC Client:
 - FORTE_COM_OPC : ON,
 - FORTE_COM_OPC_LIB_ROOT: путь к корневой папке библиотеки OPC,
 - FORTE_COM_OPC_BOOST_ROOT: путь к корневой папке Boost,
2. Откройте проект 4diac FORTE в MS Visual Studio Code.

3. [скомпилируйте или отладьте](#) 4diac FORTE

Создайте OPC-клиент

Запустите IDE 4diac и создайте новую систему с новым приложением для одного устройства. Добавьте в приложение функциональный блок CLIENT_1, задайте следующие входные значения и сопоставьте его с оборудованием.

- $QI = 1$
- `ID = opc[localhost:FestoDidactic.EzOPC.2:2000:0.01:VirtualPLC.EB0:VirtualPLC.AB0]`
- `SD_1 = BYTE#210`

при этом входной параметр ID имеет следующий формат:

`opc[host:имя_сервера:частота_обновления:диапазон_неопределенности:входные_элементы_fb:выходные_элементы_fb]`

- `host`: адрес компьютера с OPC-сервером
- `имя_сервера`: имя OPC-сервера, например `Matrikon.OPC.Simulation`
- `частота_обновления`: частота обновления в миллисекундах
- `диапазон_неопределенности`: диапазон неопределенности, используемый для обновления значения (в тех же единицах измерения, что и значение)
- `входные_элементы_fb/выходные_элементы_fb`: добавляемые элементы, элементы должны быть разделены запятыми, например

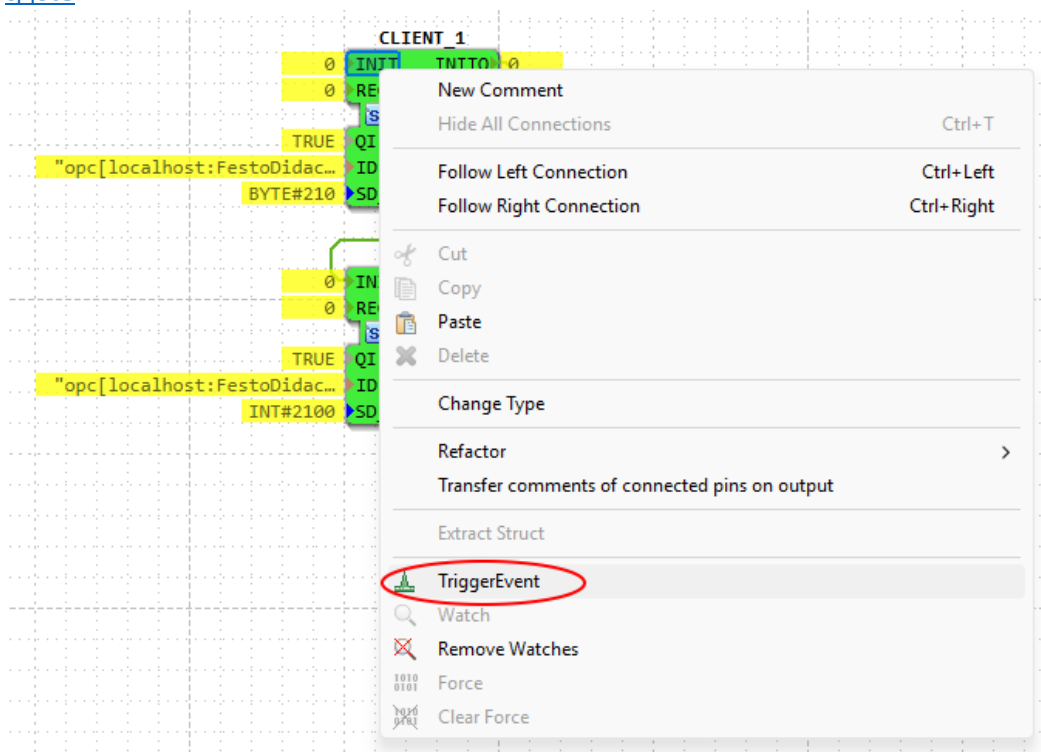
Добавьте в приложение второй функциональный блок CLIENT_1, задайте следующие входные значения и сопоставьте его с оборудованием.

- $QI = 1$
- `ID =`

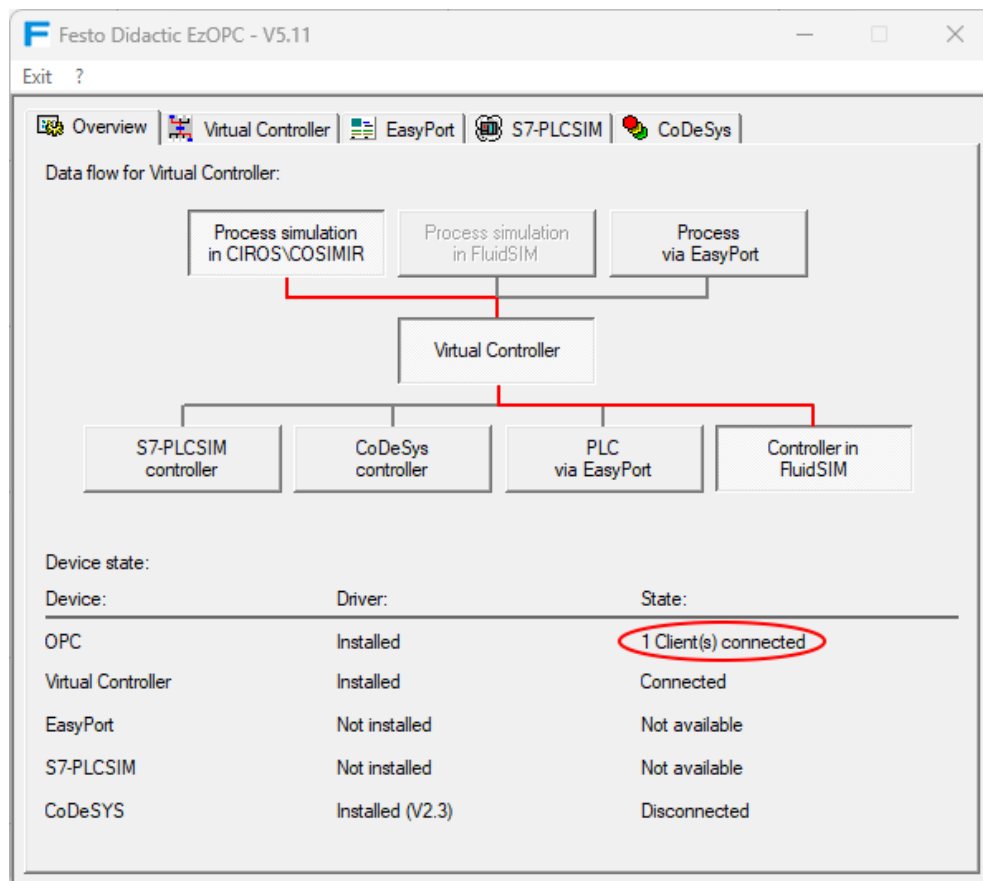
`"opc[localhost:FestoDidactic.EzOPC.2:2000:0.01:VirtualPLC.Аналоговый.ANIN0:VirtualPLC.Аналоговый.ANOU0]"`

- `SD_1 = INT#2100`

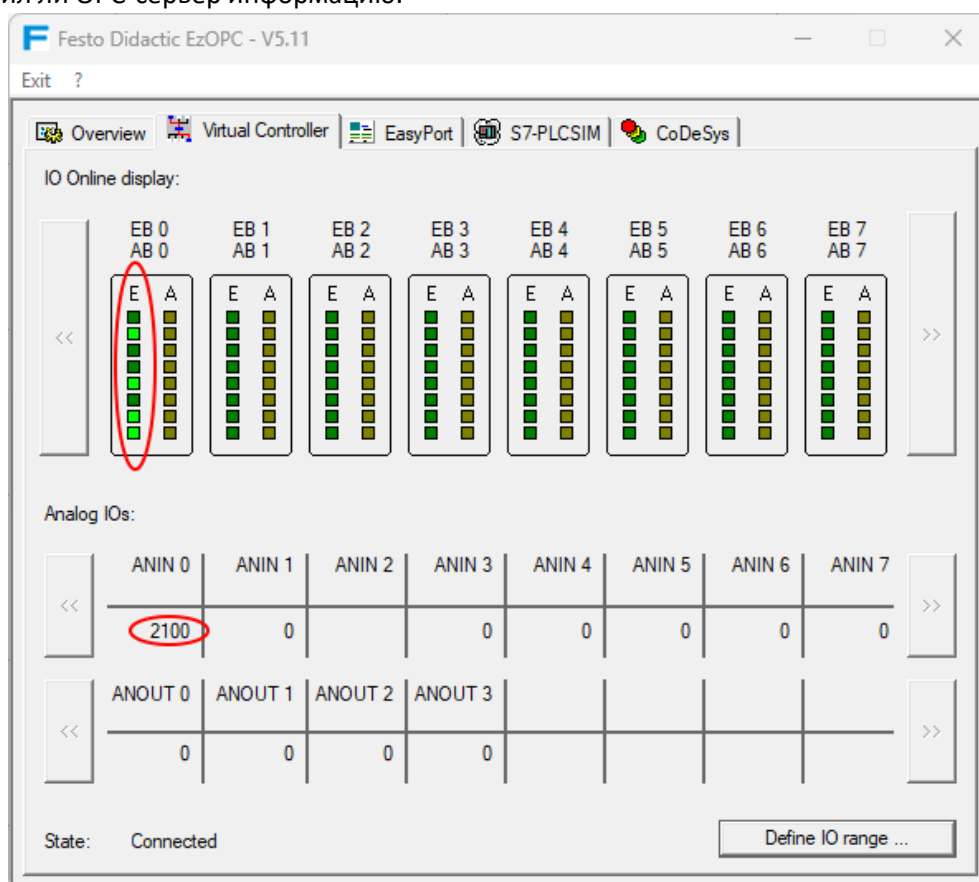
Запустите 4diac FORTE и разверните приложение. Переведите систему в онлайн-режим и начните отслеживать входящие и исходящие данные. Для получения дополнительной информации о [функциях мониторинга](#) перейдите по ссылке. После запуска события INIT OPC-клиент пытается подключиться к OPC-серверу. В этом примере OPC-сервером является FestoDidactic.EzOPC.2. Это инструмент от Festo, который можно скачать [здесь](#)



Когда соединение установлено, появляется окно сервера.



Теперь запустите событие REQ, чтобы отправить данные с клиента на сервер. BYTE#210 должен быть отправлен на порт сервера VirtualPLC.EB0. Перейдите на вкладку «Виртуальный контроллер», чтобы проверить, получил ли OPC-сервер информацию.



Куда двигаться дальше?

Вернитесь к списку протоколов: [Индекс сообщений](#)

Если вы хотите вернуться на страницу «[Начать здесь](#)», мы оставляем вам ссылку для быстрого перехода

OPC UA с руководством по IEC 61499

В этом руководстве показано, как использовать [OPC UA](#) в приложении IEC 61499 с помощью доступных FB. Сначала вам следует ознакомиться с руководством «[Первые шаги в Eclipse 4diac](#)», чтобы привыкнуть к среде разработки 4diac. 4diac FORTE использует стек OPC UA [open62541](#) с открытым исходным кодом, который можно бесплатно использовать в коммерческих проектах.

После версии 1.11.0 4diac FORTE модуль OPC UA изменил свой API, поэтому приложения, разработанные до этой версии, будут несовместимы. Изменения повышают стабильность 4diac FORTE, а также добавляют новые функции. Модуль OPC UA можно использовать с версией 1.0 open62541.

Задачи в этом руководстве:

1. [Создайте 4diac FORTE с помощью open62541](#)
2. [Предоставьте клиентам переменные OPC UA.](#)
3. [Предложите клиентам метод OPC UA для вызова.](#)
4. [Вызовите удалённый метод из приложения IEC 61499.](#)
5. [Создайте подписку OPC UA в приложении IEC 61499.](#)

Создайте 4diac FORTE с помощью open62541.

OPC UA по умолчанию не включён в 4diac FORTE. Чтобы включить его, вам нужно самостоятельно собрать 4diac FORTE с исходным кодом open62541.

Важно! Для компиляции библиотеки OPCUA на вашем компьютере должен быть установлен [Python](#).

Linux

1. Скачайте исходный код 4diac FORTE с <https://github.com/eclipse-4diac/4diac-forte>:

```
$ cd ~/4diac $ git clone https://github.com/open62541/open62541.git --branch=v1.0
open62541
```

2. Скачайте исходный код open62541 с <https://github.com/open62541/open62541>:

```
$ cd ~/4diac $ git clone https://github.com/open62541/open62541.git --branch=v1.0
open62541
```

3. Скомпилируйте open62541. Если вы запускаете код на рабочих устройствах, мы рекомендуем установить тип сборки на Release.

```
$ cd ~/4diac/open62541 && mkdir build && cd $_
$ cmake -DBUILD_SHARED_LIBS=ON -DCMAKE_BUILD_TYPE=Debug -
DUA_ENABLE_AMALGAMATION=ON ..
$ make -j
```

4. Включите 4diac FORTE для open62541. Если вы запускаете код на рабочих устройствах, мы рекомендуем установить тип сборки на Release.

```
$ cd ~/4diac/forte/build
$ cmake -DCMAKE_BUILD_TYPE=Debug \
-DFORTE_ARCHITECTURE=Posix \
-DFORTE_MODULE_CONVERT=ON \
-DFORTE_COM_ETH=ON \
-DFORTE_MODULE_IEC61131=ON \
-DFORTE_COM_OPC_UA=ON \
-DFORTE_COM_OPC_UA_INCLUDE_DIR=$HOME/4diac/open62541/build \
-DFORTE_COM_OPC_UA_LIB_DIR=$HOME/4diac/open62541/build/bin \
-DFORTE_COM_OPC_UA_LIB=libopen62541.so
$ make -j
```

Windows с Visual Studio

1. Используйте инструмент Git в Windows и загрузите исходный код 4diac FORTE с <https://github.com/eclipse-4diac/4diac-forte> и используйте ветку разработки. Ниже приведены команды для Git Bash.

```
$ mkdir "$HOME/4diac" && cd "$HOME/4diac"
$ git clone https://github.com/open62541/open62541.git --branch=v1.0
open62541
$ mkdir build && cd "build"
```

2. Используйте CMake для создания всех файлов сборки. Скачайте и установите: <https://cmake.org/download/>

3. Откройте папку `$HOME/4diac/forte` в CMake и укажите, что выходные двоичные файлы сборки должны находиться в папке `forte/build`.
4. Нажмите «Настроить» и выберите версию Visual Studio. (Убедитесь, что вы выбрали установленную версию VS. В противном случае CMake не найдет компилятор C/C++. Активируйте «Использовать собственные компиляторы по умолчанию».
5. Установите `FORTE_ARCHITECTURE` на Win32, `FORTE_MODULE_CONVERT=ON`, `FORTE_MODULE_IEC61131=ON` и дважды нажмите «Настроить». Теперь все красные записи должны исчезнуть.
6. Скомпилируйте `open62541`. Если вы запускаете код на рабочих устройствах, мы рекомендуем установить тип сборки «Выпуск». Откройте новое окно CMake и выберите папку `$HOME/4diac/open62541` в качестве каталога исходного кода, а папку `$HOME/4diac/open62541/build` — в качестве каталога двоичных файлов.
7. Нажмите «Настроить» и выберите версию Visual Studio. (Убедитесь, что вы выбрали установленную версию VS. В противном случае CMake не найдет компилятор C/C++. Активируйте «Использовать собственные компиляторы по умолчанию».
8. Активируйте `BUILD_SHARED_LIBS` и `UA_ENABLE_AMALGAMATION`
9. Нажмите «Настроить», затем «Сгенерировать», а затем «Открыть проект»
10. Перейдите в «Сборка» → «Собрать решение». Возможно, вам потребуется выполнить команду несколько раз, пока все проекты не будут успешно собраны. Это создаст библиотеку `open62541` в папке `$HOME/4diac/open62541/build/bin/Release` и объединённый заголовочный файл `open62541.h` в папке `$HOME/4diac/open62541/build/`
11. Переключитесь в окно 4diac FORTE CMake, чтобы включить `open62541`:
`FORTE_COM_OPC_UA=ON`
`FORTE_COM_OPC_UA_INCLUDE_DIR=C:\Users\USER\4diac\open62541\build`
`FORTE_COM_OPC_UA_LIB_DIR=C:\Users\USER\4diac\open62541\build\bin\Release`
12. Нажмите «Настроить» и убедитесь, что для `FORTE_COM_OPC_UA_LIB` установлено значение `open62541.dll`
13. Нажмите «Настроить», затем «Сгенерировать» и «Открыть проект»
14. Перейдите в раздел «Сборка» → «Собрать решение». Возможно, вам потребуется выполнить команду несколько раз, пока все проекты не будут успешно собраны. Это создаст файл `$HOME/4diac/forte/build/Debug/forte.exe`
15. В проекте 4diac FORTE VS задайте переменную `PATH`, чтобы она включала путь к `open62541.dll`. Перейдите в свойства Forte, «Свойства конфигурации» | «Отладка» | «Окружающая среда» и добавьте `PATH=C:\Users\USER\4diac\open62541\build\bin\Release;%PATH%` (укажите правильный путь пользователя!)

Общий обзор использования модуля

Параметры для связи FB (ввод идентификационных данных) разделены на две (три для удалённых действий) части, разделённые точкой с запятой ';'

`opc_ua[ДЕЙСТВИЕ; ОКОНЧАНИЕ; ПАРА1; ПАРА2; ...]`

- **ACTION (ДЕЙСТВИЕ):** обязательное поле, может принимать следующие значения (все в верхнем регистре):
 - `READ` ПРОЧИТАТЬ
 - `WRITE` ЗАПИСАТЬ
 - `CREATE_METHOD` СОЗДАТЬ МЕТОД
 - `CALL_METHOD` ВЫЗВАТЬ МЕТОД
 - `SUBSCRIBE` ПОДПИСАТЬСЯ
 - `CREATE_OBJECT` СОЗДАТЬ ОБЪЕКТ
 - `DELETE_OBJECT` УДАЛИТЬ ОБЪЕКТ
 - `CREATE_VARIABLE` СОЗДАТЬ ПЕРЕМЕННУЮ
 - `DELETE_VARIABLE` УДАЛИТЬ ПЕРЕМЕННУЮ
- **ENDPOINT (ОКОНЧАНИЕ):** обязательное и ограниченное поле при использовании для удалённых действий. Он должен заканчиваться символом.
- Пример: `opc.tcp://192.168.0.100:4840`

- PAIR (ПАРА): в формате BROWSENAME, NODE_ID (разделены запятой ','). Если возможно, NODE_ID можно опустить, в этом случае запятая ',' также должна быть опущена. Если BROWSENAME опущено (если возможно), запятая ',' должна присутствовать.

- ИМЯ БРАУЗЕРА: путь к узлу, который всегда должен начинаться с косой черты "/". Двоеточие перед именем позволяет определить пространство имен для имени браузера текущего элемента. Пространство имен browseName по умолчанию равно 1, за исключением первой части, которая по умолчанию равна нулю. Это связано с тем, что в большинстве случаев первая часть, к которой требуется получить доступ, находится в пространстве имен 0. Если вы хотите создать что-либо непосредственно в /Root, вам следует указать свое пространство имен, например /1:directUnderRoot. Это допустимо для локального и удаленного доступа.

Пример: /Objects/myFolder/myNode

Пример: /Objects/2:myFolder/myNode → В объектах будет найдена папка с именем myFolder и пространством имён 2, а внутри — узел с именем myNode и пространством имён 1

- NODE_ID: в формате NAMESPACE_INDEX: IDENTIFIER_TYPE=IDENTIFIER
 - NAMESPACE_INDEX: номер пространства имён. Если не указан, по умолчанию используется 0. Если не указан, также необходимо опустить «:»
 - IDENTIFIER_TYPE: может принимать следующие значения:
 - i: идентификатор узла является числовым
 - s: идентификатор узла является строкой
 - b: идентификатор узла является байтовой строкой
 - g: идентификатор узла является GUID (НЕ РЕАЛИЗОВАНО)
 - IDENTIFIER: идентификатор в соответствии с IDENTIFIER_TYPE
 - 1:i=12345 → пространство имён 1, числовой идентификатор 12345
 - 3:s=hello → пространство имён 3, строковый идентификатор "hello"
 - 2:b=hello2 → пространство имён 2, байтовый идентификатор hello2
 - s=hello5 → пространство имён 0, строковый идентификатор "hello5"
 - Примеры пар:
 - /Objects/myFolder/myNode,1:i=12345 → указаны и путь просмотра, и идентификатор узла
 - /Objects/myFolder/myNode1 → указан путь просмотра, идентификатор узла опущен
 - ,1:i=12345 → путь просмотра опущен, но указан идентификатор узла

Примечание! Чтобы определить, существует ли узел, применяются следующие правила:

- Если указан только путь просмотра, узел существует, если в этом пути просмотра есть узел
- Если указаны и путь просмотра, и идентификатор узла, узел существует, если существующий узел в пути просмотра имеет тот же идентификатор узла, что и указанный.
- Если указан только идентификатор узла, узел с указанным идентификатором узла должен существовать в адресном пространстве.
- При создании узла (только локально) должен быть указан путь просмотра.

Примеры идентификаторов:

opc_ua[READ;/Objects/test1] → чтение локального узла /Objects/test1. Создайте узел, если он не существует, со случайным идентификатором узла

opc_ua[READ;/Objects/test1,i=1] → прочтите локальный узел /Objects/test1. Создайте узел, если он не существует, с числовым идентификатором узла 1 и пространством имён 0. Если узел уже существует и у него другой идентификатор узла, произойдёт сбой.

opc_ua[WRITE;/Objects/test1;/Objects/test1] → Записать узлы /Объекты/test1 и /Объекты/test1 со значениями из SD_1 и SD_2. Если узлы не существуют, они будут созданы со случайными идентификаторами. Если они существуют, они будут записаны при срабатывании.

opc_ua[WRITE;opc.tcp://192.168.0.100:4840#;/Objects/test1;/Objects/test1] → Запись на узлы /Objects/test1 и /Objects/test1, удаленные от значений в SD_1 и SD_2. Если узлы не найдены, 4diac FORTE будет искать их через несколько секунд, пока не найдет. Удаленное создание не поддерживается

Примечание! В зависимости от выполняемого действия путь просмотра и/или идентификатор узла являются обязательными/необязательными. См. таблицу ниже.

Удаленный/локальный	Желаемое действие	Блок функций для использования	Ограничения
Local	READ	SUBSCRIBE	Количество пар должно соответствовать количеству RD. Идентификатор узла необязателен. Если имя браузера не указано, он будет искать узел по идентификатору узла и не создаст его (поскольку не знает, где его создать)
Local	WRITE	PUBLISH	Количество пар должно соответствовать количеству SD. Идентификатор узла необязателен. Если имя узла не указано, он будет искать узел по идентификатору и не создаст его (поскольку не знает, где его создать).
Local	CREATE_METHOD	SERVER	Количество пар должно быть равно 1. Путь просмотра ДОЛЖЕН быть указан. Идентификатор узла необязателен
Local	CALL_METHOD	X	Запрещено вызывать локальные методы
Local	SUBSCRIBE	X	Не разрешается подписываться на локальные переменные
Local	CREATE_OBJECT	PUBLISH_0	Количество пар должно быть равно 2. Первая пара предназначена для типа (необходимо указать имя браузера и/или идентификатор узла. Если указаны оба параметра, они должны совпадать). Вторая пара предназначена для создания экземпляра (необходимо указать путь к браузеру. Идентификатор узла необязателен)
Local	DELETE_OBJECT	PUBLISH_0	Количество пар должно быть равно 1. Путь просмотра ДОЛЖЕН быть указан. Идентификатор узла необязателен
Local	CREATE_VARIABLE	PUBLISH_0	Количество пар должно быть равно 3. Первая пара предназначена для типа переменной (имя просмотра и/или идентификатор узла должны быть указаны. Если указаны оба параметра, они должны совпадать). Вторая пара предназначена для типа значения созданной переменной (имя просмотра и/или идентификатор узла должны быть указаны. Если указаны оба параметра, они должны совпадать). Третий параметр — это

Удаленный/локальный	Желаемое действие	Блок функций для использования	Ограничения
			путь к экземпляру для создания (путь к просмотру ДОЛЖЕН быть указан. Идентификатор узла необязателен).
Local	DELETE_VARIABLE	PUBLISH_0	Количество пар должно быть равно 1. Путь к просмотру ДОЛЖЕН быть указан. Идентификатор узла необязателен
Remote	READ	CLIENT	Количество пар должно соответствовать количеству RD, и не должно быть SD. Необходимо указать имя браузера и/или идентификатор узла. Если указаны оба параметра, они должны совпадать.
Remote	WRITE	CLIENT	Количество пар должно соответствовать количеству SD, RD присутствовать не должно. Необходимо указать имя браузера и/или идентификатор узла. Если указаны оба параметра, они должны совпадать.
Remote	CREATE_METHOD	CLIENT	Не допускается удаленное создание методов
Remote	CALL_METHOD	CLIENT	Количество пар должно быть равно 1. Путь просмотра ДОЛЖЕН быть указан. Идентификатор узла указывается необязательно. Если указаны оба параметра, они должны совпадать
Remote	SUBSCRIBE	SUBSCRIBE	Количество пар должно соответствовать количеству RD. Имя просмотра и/или идентификатор узла должны быть указаны. Если указаны оба параметра, они должны совпадать
Remote	CREATE_OBJECT	X	Не разрешено создавать объекты удаленно
Remote	DELETE_OBJECT	X	Удаленное удаление объектов запрещено
Remote	CREATE_VARIABLE	X	Не разрешено создавать переменные удаленно
Remote	DELETE_VARIABLE	X	Удаленное удаление переменных запрещено

Переменные OPC UA

Триггерное приложение, использующее переменные

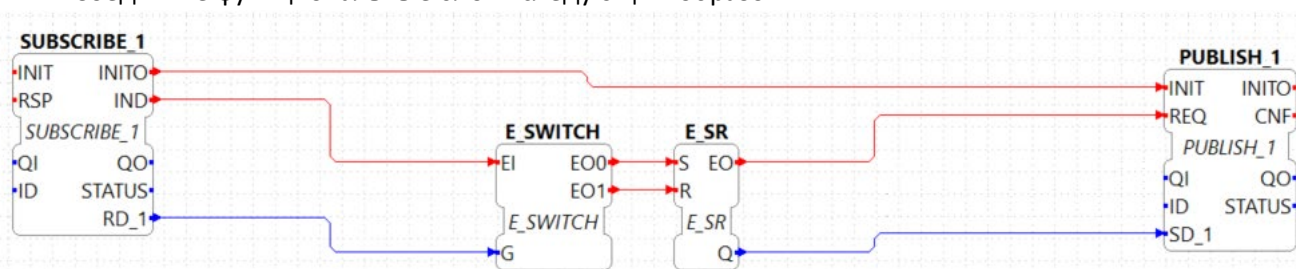
На этом шаге вы создадите простое триггерное приложение, которое использует функциональные блоки ПУБЛИКАЦИИ и ПОДПИСКИ для создания переменных в адресном пространстве сервера OPC UA. Затем клиенты могут считывать данные из этих переменных или записывать новые значения.

Следуйте инструкциям, приведенным в [руководстве по миганию](#), чтобы создать новую систему, приложение и устройство. После создания пустого приложения выполните следующие действия.

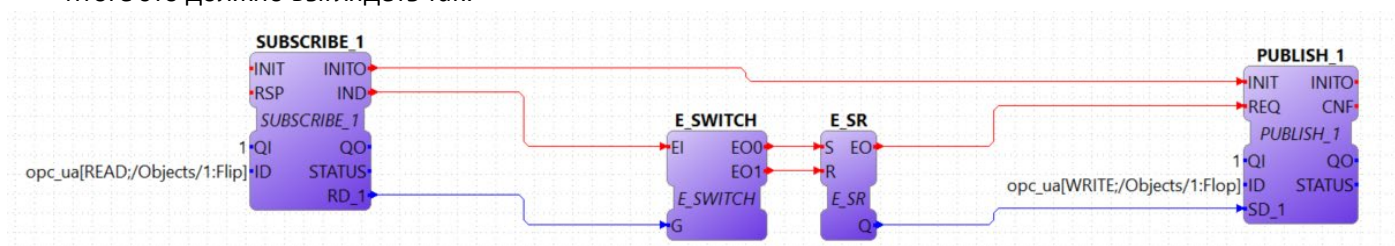
На следующих этапах мы создадим приложение Flip-Flop, в котором логическое значение считывается из переменной OPC UA, а затем его инвертированное значение публикуется в качестве другой переменной. Для этого мы используем блоки функций SUBSCRIBE и PUBLISH. Блок функций SUBSCRIBE используется для подписки на одну или несколько переменных OPC UA, то есть значение переменной будет доступно в приложении. Блок функций PUBLISH используется для публикации значения из приложения, чтобы его могли прочитать клиенты.

При срабатывании события INIT этих функциональных блоков сервер OPC UA инициализируется и запускается по URL-адресу конечной точки по умолчанию: `opc.tcp://localhost:4840`. Обратите внимание, что будет создан только один сервер OPC UA, а модель адреса будет общей для всех функциональных блоков.

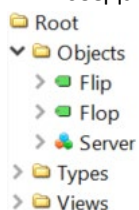
1. Перетащите следующие функциональные блоки из библиотеки типов в редактор приложений:
 - events/E_SWITCH
 - events/E_SR
 - net/SUBSCRIBE_1
 - net/PUBLISH_1
2. Соедините функциональные блоки следующим образом:



3. Сопоставьте функциональные блоки с устройством
4. Чтобы настроить, где в адресном пространстве будут создаваться узлы переменных, используйте поля ID функциональных блоков SUBSCRIBE/PUBLISH. Мы хотим создать переменные в разделе /Objects/, а узлы Flip и Flop должны быть созданы в пространстве имён 1. Поэтому мы задаём идентификаторы `opc_ua[READ;/Objects/1:Flip]` и `opc_ua[WRITE;/Objects/1:Flop]`. Если вы не укажете явно пространство имён (1:), то по умолчанию будет использоваться пространство имён 1. Для включения функционального блока необходимо установить QI на 1.
5. Откройте редактор системы и подключите порты COLD и WARM к порту INIT SUBSCRIBE_1. В итоге это должно выглядеть так:



6. Разверните приложение на 4diac FORTE
7. Откройте UaExpert (вы можете скачать его здесь) и подключитесь к серверу OPC UA, работающему на 4diac FORTE: `opc.tcp://localhost:4840`. Вы должны увидеть две переменные, созданные блоками функций SUBSCRIBE и PUBLISH:



Два параметра VariableNodes - Flip и Flop - создаются с помощью функционального блока SUBSCRIBE и PUBLISH. Тип данных переменных определяется типом данных порта подключенного FB (E_SWITCH.G и E_SR.Q).

8. При желании теперь вы можете отслеживать работу приложения в 4diac FORTE, например, следить за значениями ПОДПИСКИ и ПУБЛИКАЦИИ на FB. О том, как это сделать, смотрите в [руководстве по локальному использованию Eclipse 4diac - Monitor](#).

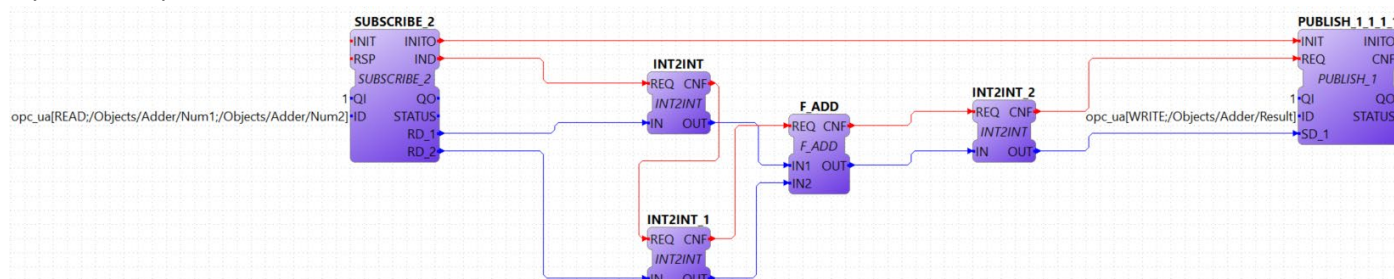
9. В UaExpert перетащите две переменные Flip и Flop в представление «Доступ к данным». Теперь вы можете изменить значение Flip. Это приведёт к срабатыванию события IND в порту SUBSCRIBE, и 4diac FORTE прочтает новое значение переменной, инвертирует его и установит Q в инвертированное значение. Событие REQ в PUBLISH FB должно быть запущено, чтобы установить новое значение из SD_1 в адресной модели OPC UA (переменная Flop). Обратите внимание, что вначале оба значения будут ложными, так как событие ещё не было запущено.

10. В 4diac FORTE проверьте отслеживаемое приложение, чтобы увидеть, что произойдёт, если вы измените переменную.

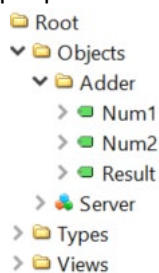
Приложение-сумматор с использованием переменных

Здесь вы можете увидеть ещё один пример использования SUBSCRIBE и PUBLISH для создания приложения IEC 61499, которое складывает два значения, считывая их из переменной OPC UA и предоставляя результат.

Если вы выполнили предыдущие действия, то сможете создать новое приложение, которое выглядит следующим образом:



Функциональный блок F_ADD — это универсальный тип, который может иметь любой поддерживаемый тип данных для портов IN1, IN2 и OUT. С другой стороны, функциональные блоки SUBSCRIBE и PUBLISH должны указывать, какой тип данных должны иметь созданные переменные. Чтобы ввести эту информацию в модель приложения, вы можете использовать функциональный блок INT2INT. Это создаст следующие узлы на сервере OPC UA:



Попробуйте понять, откуда взялись названия Num1, Num2 и Result.

Затем вы можете перетащить переменные в представление доступа к данным UaExpert и изменить значения Num1 и Num2. Значение Result должно быть суммой этих двух значений.

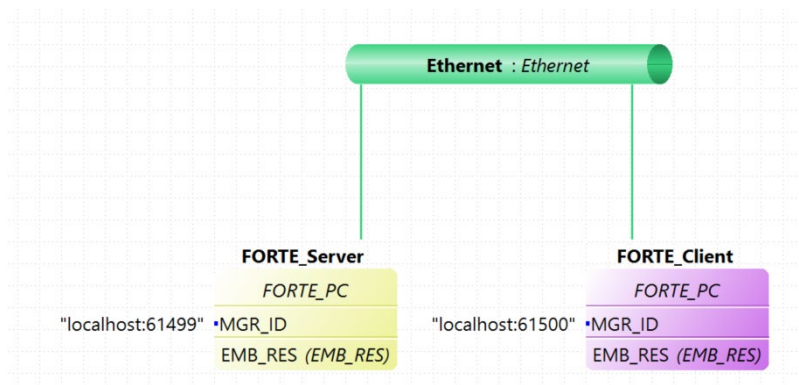
Чтение и запись переменных OPC UA на удалённом сервере

В этом примере показано, как использовать функциональный блок CLIENT для записи и чтения переменной OPC UA на удалённом сервере.

Создайте новую систему, включающую два устройства: одно для удалённого сервера и одно для чтения/записи локальной переменной. Запустив два экземпляра 4diac FORTE на локальном ПК, можно смоделировать сценарий использования, в котором сервер и клиент находятся в отдельных экземплярах 4diac FORTE. Сервер может, например, работать на устройстве, предоставляющем переменные, а клиент — это другое устройство, запрашивающее информацию у предыдущего.

Чтобы запустить два экземпляра 4diac FORTE на одном ПК, вы можете указать порт в командной строке, например, `./forte -c localhost:61500`

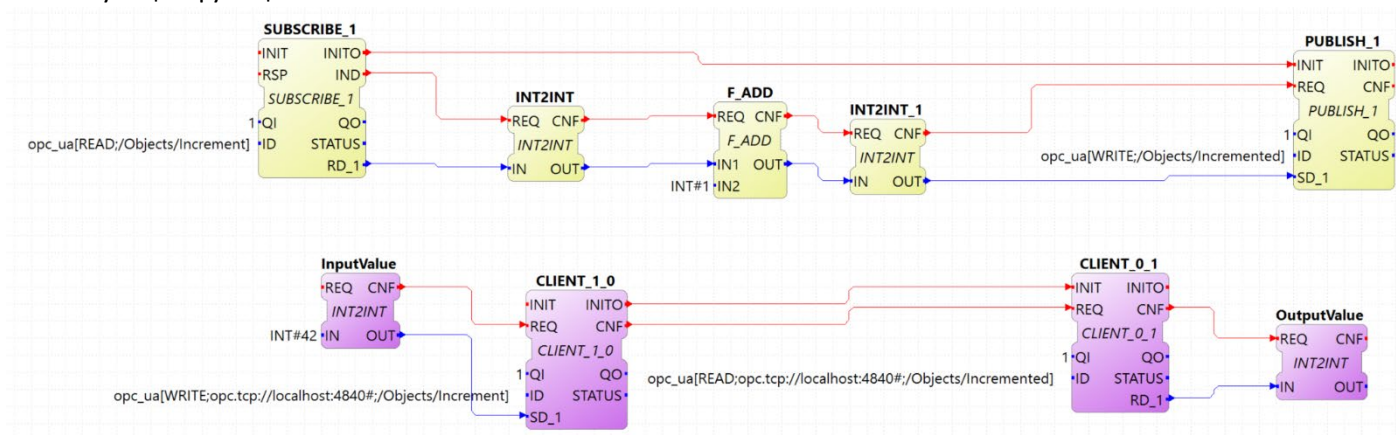
Конфигурация системы должна выглядеть следующим образом:



Затем мы создадим следующее приложение. На стороне сервера мы создаём переменную OPC UA /Objects/Increment (SUBSCRIBE_1). Когда клиент записывает данные в эту переменную, она увеличивается (F_ADD), а затем записывается в другую переменную OPC UA /Objects/Incremented (PUBLISH_1).

На стороне клиента мы создаём FB CLIENT_1_0, который должен записывать число в переменную на удалённом сервере. Чтобы считать увеличенное значение с сервера, мы используем FB CLIENT_0_1, который считывает значение переменной сразу после срабатывания события REQ.

Не забудьте подключить события COLD/WARM в представлении ресурсов к портам событий INIT соответствующих функциональных блоков.



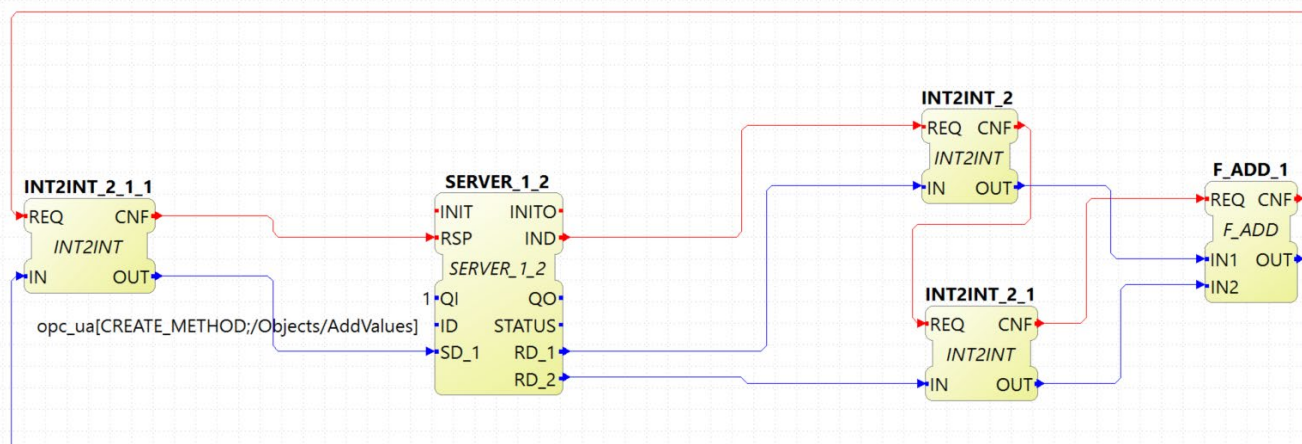
Разверните приложение на двух экземплярах 4diac FORTE. Вы можете получить доступ к переменным сервера с помощью UaExpert. Для клиента вы можете отслеживать состояние системы. Когда вы запускаете событие InputValue.REQ, клиент записывает число 42, а затем считывает приращённое значение, в котором будет присутствовать новое значение. Изменения будут видны и в UaExpert.

Методы OPC UA

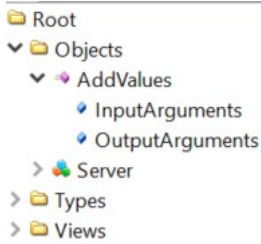
На этом этапе вы увидите, как можно создавать методы OPC UA в приложении IEC 61499. Для этого используется функциональный блок SERVER, который автоматически создаёт соответствующий метод OPC UA в адресной модели.

Следуйте инструкциям в разделе [«Первые шаги» руководства по Eclipse 4diac](#), чтобы создать новую систему, приложение и устройство. После создания пустого приложения выполните следующие действия.

1. Создайте следующее приложение:



2. Разверните приложение в 4diac FORTE
3. Откройте UaExpert, и вы увидите следующую модель адреса:

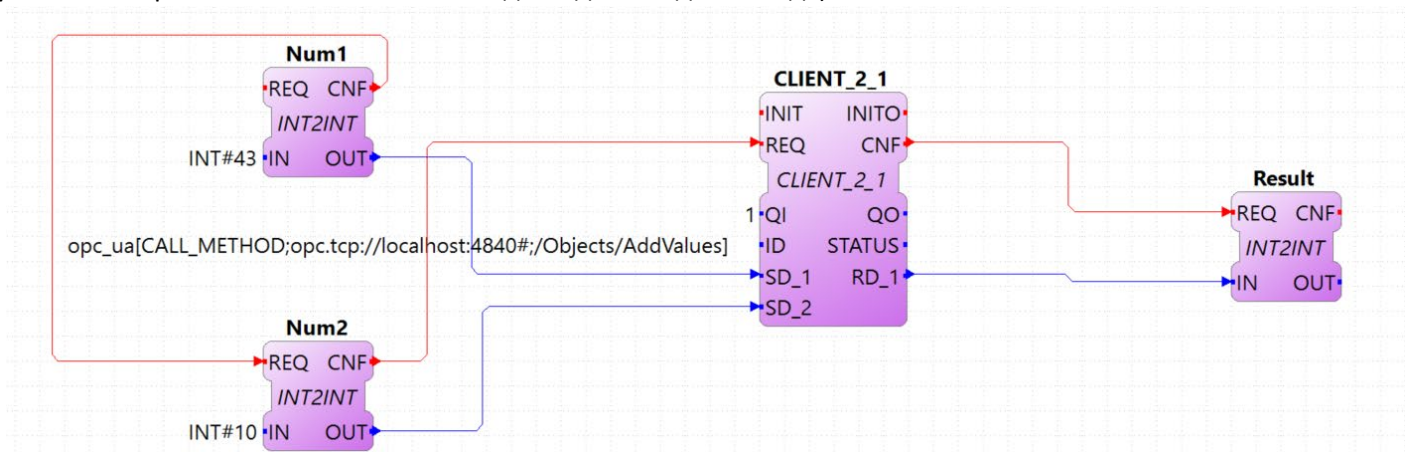


4. При желании вы можете снова выполнить мониторинг приложения в 4diac.
5. В UaExpert щелкните правой кнопкой мыши по узлу метода AddValues и выберите «Вызвать».
6. Вы увидите, что имена входных и выходных аргументов совпадают с именами выходных данных FB сервера. Введите значения для RD_1 и RD_2 и нажмите «Вызвать». 4diac FORTE запустит событие IND FB сервера, в результате чего FB F_ADD вычислит сумму двух чисел. Когда сумма будет готова, на СЕРВЕРЕ сработает событие RSP, и результат будет возвращён вызывающему OPC UA-клиенту, которым в данном случае является UaExpert.

Примечание! После срабатывания события IND результат должен быть готов (т. е. должно сработать событие RSP) в течение 4 секунд. Это время ожидания по умолчанию для вызова метода. В противном случае вызов завершится ошибкой с кодом ошибки тайм-аута.

Вызов метода OPC UA

Чтобы вызвать метод из предыдущего примера, приложение будет выглядеть следующим образом (в этом случае оно отправит 43 и 10 в качестве входных данных для метода):



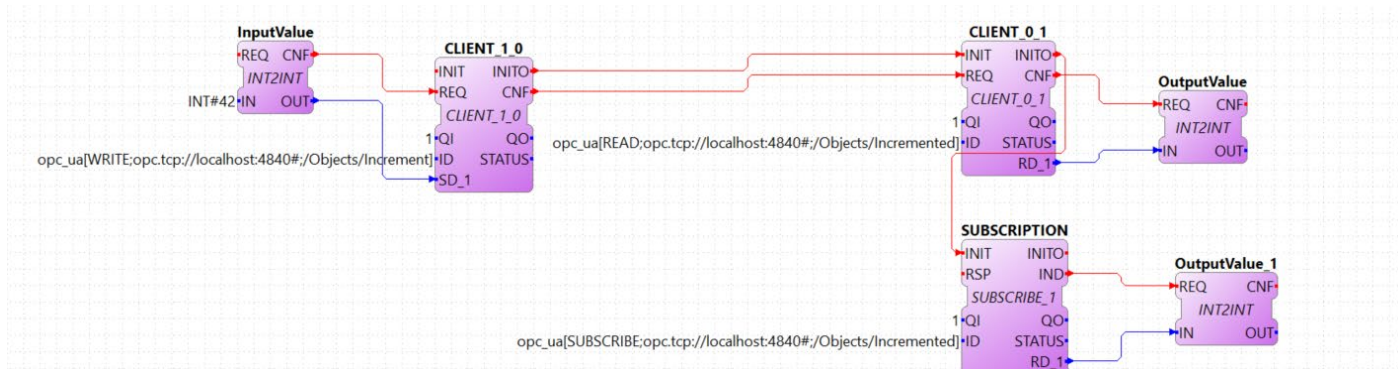
Попробуйте развернуть оба приложения, вызвать метод с клиента и получить результат в Result FB.

Клиентские подписки OPC UA

На этом этапе вы увидите, как можно создать подписку на данные OPC UA в приложении IEC 61499. Она используется для настройки подписки на переменные на сервере OPC UA, чтобы получать изменения значений по мере их появления на сервере. Для этого используется функциональный блок SUBSCRIBE, который позволяет удаленно подписываться на изменения значений переменных.

Мы создадим функциональный блок, который будет подписываться на значение /Objects/Incremented из примера, описанного выше. Поэтому мы расширяем пример переменных OPC UA, добавив функциональный блок подписки для отслеживания изменения значений.

1. Мы добавляем функциональный блок SUBSCRIBE под названием SUBSCRIPTION. Функциональный блок SUBSCRIBE автоматически получает обновленное значение без необходимости вручную запускать событие RSP.
2. Теперь приложение выглядит следующим образом:



3. Обратите внимание, что порт **RD_1** функционального блока **SUBSCRIBE** содержит текущее значение подписанного узла. Кроме того, вам нужно убедиться, что тип данных порта **RD** соответствует типам данных переменной OPC UA, на которую вы хотите подписаться. Для этого используйте соответствующие конвертирующие FB, как показано в этом примере.

4. Разверните приложение в 4diac FORTE

5. Если значение, на которое вы подписались, изменилось на сервере, обновленное значение появится на порту **RD_1**

Подключение к защищенным серверам

Внимание: для использования шифрования необходимо использовать версию 1.0 open62541.

Если вам нужно подключиться к серверам OPC UA, на которых включена какая-либо защита, вам нужно выполнить несколько дополнительных шагов.

1. Сначала вам нужно скомпилировать mbedtls, используя тег mbedtls-2.7.1

```
$ git clone https://github.com/ARMmbed/mbedtls
$ cd mbedtls
$ git checkout mbedtls-2.7.1
$ mkdir build
$ cd build
$ cmake -DENABLE_TESTING=Off ../
$ make
```

2. При компиляции mbedtls вам нужно скомпилировать open62541 с поддержкой безопасности (помните, что в 4diac FORTE с поддержкой безопасности можно использовать только версию 1.0). Для этого вам нужно установить для **UA_ENABLE_ENCRYPTION** значение **true**, что приведёт к появлению нескольких новых переменных в CMake (**MBEDCRYPTO_LIBRARY**, **MBEDTLS_INCLUDE_DIRS**, **MBEDTLS_LIBRARY** и **MBEDX509_LIBRARY**). Задайте для них файлы и папки библиотек в соответствии с их названиями. После этого вы компилируете open62541, как и раньше.

3. Теперь, когда open62541 скомпилирован с шифрованием, вам нужно скомпилировать 4diac FORTE с шифрованием. В CMake установите для переменной **FORTE_COM_OPC_UA_ENCRYPTION** значение **true**. Это также приведёт к появлению новых переменных, как и в open62541:

- **FORTE_COM_OPC_UA_ENCRYPTION_INCLUDE_DIR**: папка с заголовочными файлами в каталоге сборки mbedtls
- **FORTE_COM_OPC_UA_ENCRYPTION_LIB_DIR**: папка в каталоге сборки mbedtls, в которой находятся три библиотеки. Вы можете добавить несколько каталогов, разделенных точкой с запятой

После этого скомпилируйте 4diac FORTE как обычно.

4. Если 4diac FORTE скомпилирован с поддержкой шифрования, вам понадобится файл **CONFIG_FILE** для записи конфигураций безопасности для конечных точек, к которым будет подключаться 4diac FORTE. Файл должен быть записан в формате «ключ=значение» (без пробелов между ними). При попытке подключения к серверу 4diac FORTE будет считывать файл и искать ключ «**endpoint**», пока не найдёт тот, значение которого совпадает со значением подключаемой конечной точки, и будет считывать следующие строки до конца файла или до тех пор, пока не найдёт другой ключ «**endpoint**». Это означает, что для настройки конечной точки сначала нужно указать её, а затем — конфигурации для неё. Допустимые ключи:

- конечная точка: строка для подключения. Пример: **opc.tcp://192.168.1.150:4840**

- имя пользователя: строка для использования в качестве имени пользователя. Пример: john
- пароль: строка для использования в качестве пароля. Пример: doe
- сертификат: файл сертификата. Пример: /home/user/myCertificate/my_certificate.der
- privateKey: файл закрытого ключа. Пример: /home/user/myCertificate/my_key.der
- securityMode: целое число. 1: нет (по умолчанию); 2: подпись; 3: подпись и шифрование
- securityPolicy: строка URI. Например,

<http://opcfoundation.org/UA/SecurityPolicy#Basic256>. По умолчанию используется <http://opcfoundation.org/UA/SecurityPolicy#None>

Пример файла будет выглядеть так (назовем его configSecurity.forte):

```
endpoint=opc.tcp://192.168.1.150:4840
```

```
имя_пользователя=john
```

```
пароль=doe
```

```
сертификат=/home/user/myCertificate/my_certificate.der
```

```
privateKey=/home/user/myCertificate/my_key.der
```

```
securityPolicy=http://opcfoundation.org/UA/SecurityPolicy#Basic256
```

```
securityMode=1
```

Теоретически имя пользователя и пароль можно использовать без подключения библиотек шифрования, но даже в некоторых тестах, где безопасность обеспечивалась только именем пользователя и паролем, а режим безопасности был NONE, требовались сертификат и закрытый ключ.

5. Как только вы получите файл, вам нужно будет указать 4diac FORTE, где его искать. Для этого запустите 4diac FORTE и передайте флаг -oc CONFIG_FILE

```
$ ./forte -oc configSecurity.forte
```

Готово. Теперь, когда ваше приложение попытается подключиться к

opc.tcp://192.168.1.150:4840, оно будет использовать конфигурацию, заданную в файле configSecurity.forte.

Куда перейти отсюда?

Вернитесь к списку протоколов: [Список протоколов](#)

Если вы хотите вернуться на страницу «[Начать здесь](#)», мы оставляем вам быстрый доступ

openPOWERLINK

[openPOWERLINK](#) — это стек промышленного Ethernet с открытым исходным кодом, реализующий протокол [Ethernet POWERLINK](#).

Предварительные условия

Для использования openPOWERLINK с 4diac FORTE необходимо следующее:

- [openPOWERLINK V1.8.0](#)
- [TinyXML](#)

Инструкции

Стек openPOWERLINK V1.8.0 от SourceForge следует компилировать как минимум со следующими параметрами в CMake:

- CFG_POWERLINK_CHM
- CFG_POWERLINK_PROCESS_IMAGE
- CFG_X86_WINDOWS_DLL (доступно только в Windows)

При компиляции в Windows полученные DLL и LIB необходимо скопировать в основную папку openPOWERLINK-V1.8.0.

TinyXML также можно скачать с SourceForge. 4diac FORTE CMake необходимо настроить следующим образом:

- проверьте FORTE_MODULE_POWERLINK
- Задайте FORTE_MODULE_POWERLINK_LIB_DIR в [OPENPOWERLINK-V1.08.0_HOME]
- Задайте FORTE_MODULE_POWERLINK_TINYXML_DIR в каталог, в который вы распаковали

TinyXML

Для связи через openPOWERLINK создайте и настройте блоки функций, расположенные в папке openPOWERLINK. Папка openPOWERLINK является частью библиотеки типов 4diac.

Куда двигаться дальше?

Вернитесь к списку протоколов: [Список протоколов](#)

Если вы хотите вернуться на страницу «[Начало работы](#)», мы оставляем вам быстрый доступ

Взаимодействие с инструментами моделирования

FMI

О FMI

Функциональный макетный интерфейс (FMI) - это стандарт, который создает общий интерфейс для моделирования систем из разных областей. Идея, лежащая в его основе, заключается в том, чтобы обеспечить общий язык между различными инструментами моделирования таким образом, чтобы они могли взаимодействовать друг с другом. Каждая система, реализующая этот интерфейс, экспортируется в так называемый функциональный модуль моделирования (FMU). Стандарт является бесплатным, и более подробную информацию о нем можно найти [здесь](#).

В Eclipse 4diac есть возможность экспортировать каждое устройство системы в виде FMU. Идея заключается в том, чтобы в Eclipse 4diac был контроллер физической системы, который можно экспортировать в виде FMU. С другой стороны, сама физическая система может быть смоделирована с помощью специализированного инструмента, который также позволяет экспортировать систему в виде FMU. Имея на руках оба FMU, можно использовать инструмент совместного моделирования, чтобы соединить входы и выходы обеих систем, совместно смоделировать их и посмотреть, как система будет работать с реализованным контроллером.

Как экспортировать FMU в 4diac IDE?

1. Сначала необходимо скомпилировать специальный 4diac FORTE. Следуйте [инструкциям по созданию собственных FB](#). На шаге 3, где выбираются функции для компиляции, выберите также FORTE_ENABLE_FMU. Переменная FORTE_FMU_INCLUDE должна быть установлена на путь, по которому находятся заголовочные файлы стандарта fmi. Это позволит создать динамическую библиотеку, имя которой будет зависеть от операционной системы, в которой был скомпилирован 4diac FORTE. Эту библиотеку необходимо переименовать в одну из следующих поддерживаемых: win32Forte.dll, win64Forte.dll, linux32Forte.so или linux64Forte.so (для 32-разрядных, 64-разрядных, 32-разрядных и 64-разрядных операционных систем соответственно). FMU может иметь множество различных библиотек, которые позволят имитировать FMU в этих операционных системах.
2. В 4diac IDE перейдите в раздел «Windows» → «Настройки» → «4diac IDE» → «Настройки FMU» и в поле «Расположение двоичных файлов» выберите путь, по которому находится скомпилированный 4diac FORTE (с правильным новым именем). Многие библиотеки из разных ОС могут быть размещены в одной папке, которая будет обнаружена настройками, что позволит включить ОС в FMU. Примените изменения и закройте окно.
3. Перейдите в «Файл» → «Экспорт...» → 4diac → «Создать FMU» → «Далее». В дереве проектов выберите устройства с ресурсами, которые нужно экспортировать в виде FMU. Помните, что будет экспортировано по одному FMU для каждого устройства. Выберите каталог для экспорта FMU, библиотеки, которые нужно включить (можно выбрать только библиотеки, найденные на предыдущем шаге). Последний параметр позволяет пользователю сохранить выбранные библиотеки для следующего раза. Нажмите «Готово», и всё готово.

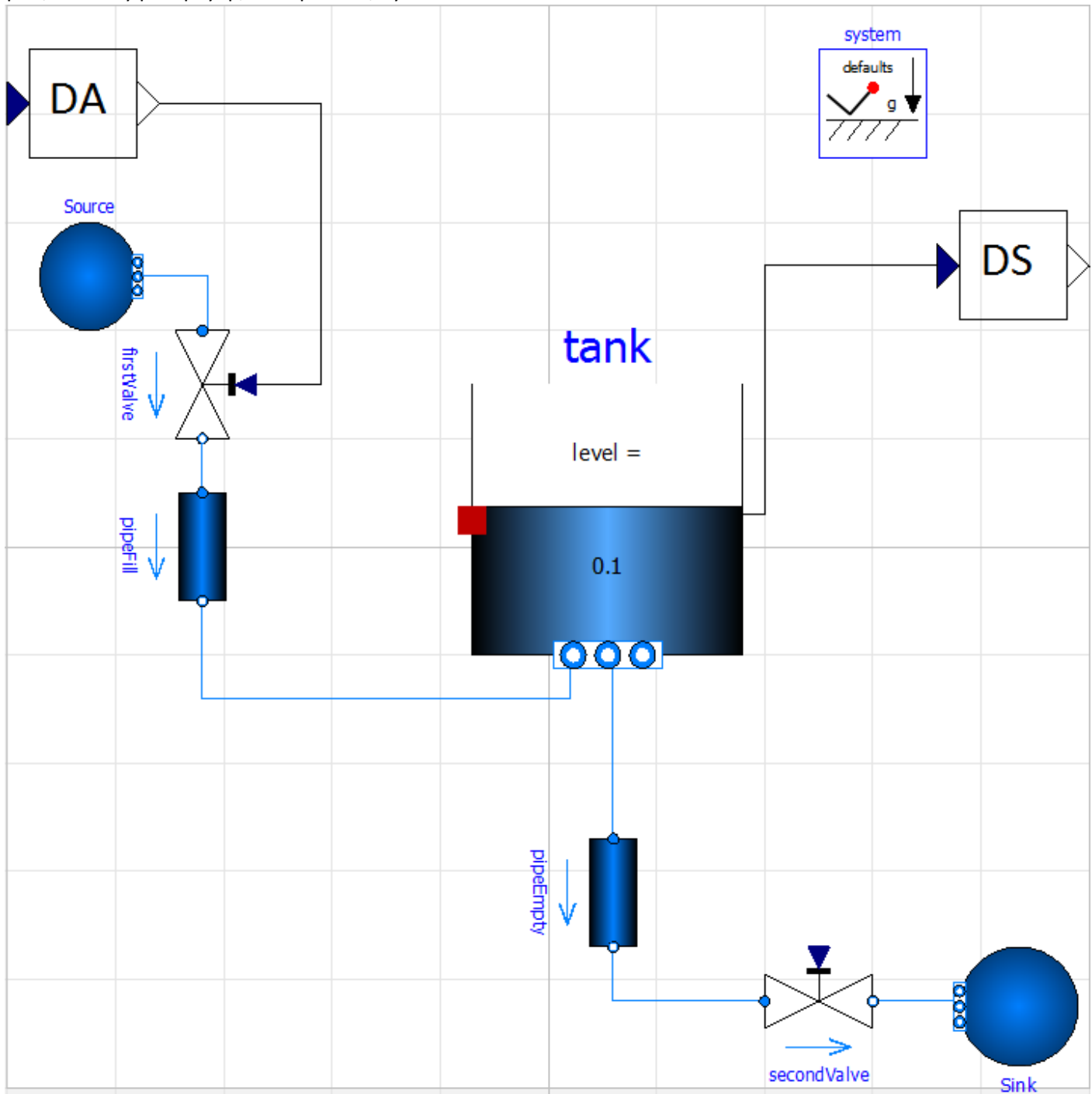
Примечания

- FBS IX и QX обрабатываются как логические входы и выходы FMU.
- FBS IW и QW обрабатываются как целочисленные входы и выходы FMU.
- Литералы на входах данных обрабатываются как параметры FMU.
- Типы BYTE, WORD, DWORD, LWORD, INT, DINT, LINT, SINT, USINT, UINT, UDINT, ULINT, ANY_INT обрабатываются как целочисленные в FMU.
- Типы STRING, WSTRING, ANY_STRING, DATE, DATE_AND_TIME, TIME_OF_DAY, ANY_DATE, TIME в FMU обрабатываются как строки.
- Типы REAL, LREAL, ANY_REAL в FMU обрабатываются как вещественные.
- Тип BOOL в FMU обрабатывается как логический.
- Входы и выходы данных, которые определены для абстрактных типов (не перечисленных выше), не включаются в FMU
- Блоки связи также рассматриваются как входы и выходы FMU, если для входа данных PARAM установлено значение «fmi[]». Каждый SD и RD рассматривается как выход и вход соответственно.

- Все входы и выходы данных, а также внутренние переменные в базовых функциональных блоках (и текущее состояние ECC) и внутренние блоки связи в составном блоке связи экспортируются как внутренние переменные FMU, то есть их можно отслеживать при совместном моделировании. Также доступно количество срабатываний событий. Функциональные блоки интерфейса сервиса имеют только свой интерфейс, поскольку реализация скрыта.
- Функциональные блоки: E_CYCLE, E_F_TRIG, E_R_TRIG, E_TimeOut, E_CTU, E_D_FF, E_DEMUX, E_MERGE, E_PERMIT, E_REND, E_RS, E_SELECT, E_SPLIT, E_SR, E_SWITCH рассматриваются как функциональные блоки интерфейса сервиса.
- Адаптеры пока не поддерживаются.

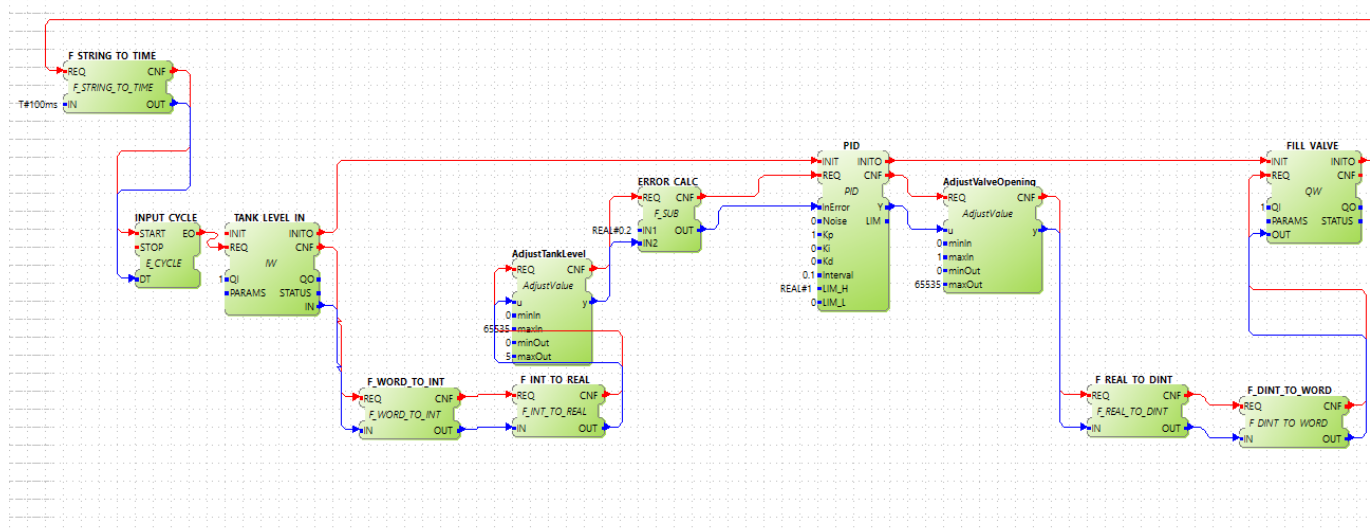
Пример

Простая система резервуаров была смоделирована в OpenModelica, как показано на рисунке ниже. Она имеет «первый клапан», которым нужно управлять и который заполняет резервуар. «Второй клапан» всегда открыт, поэтому резервуар, как правило, пуст.



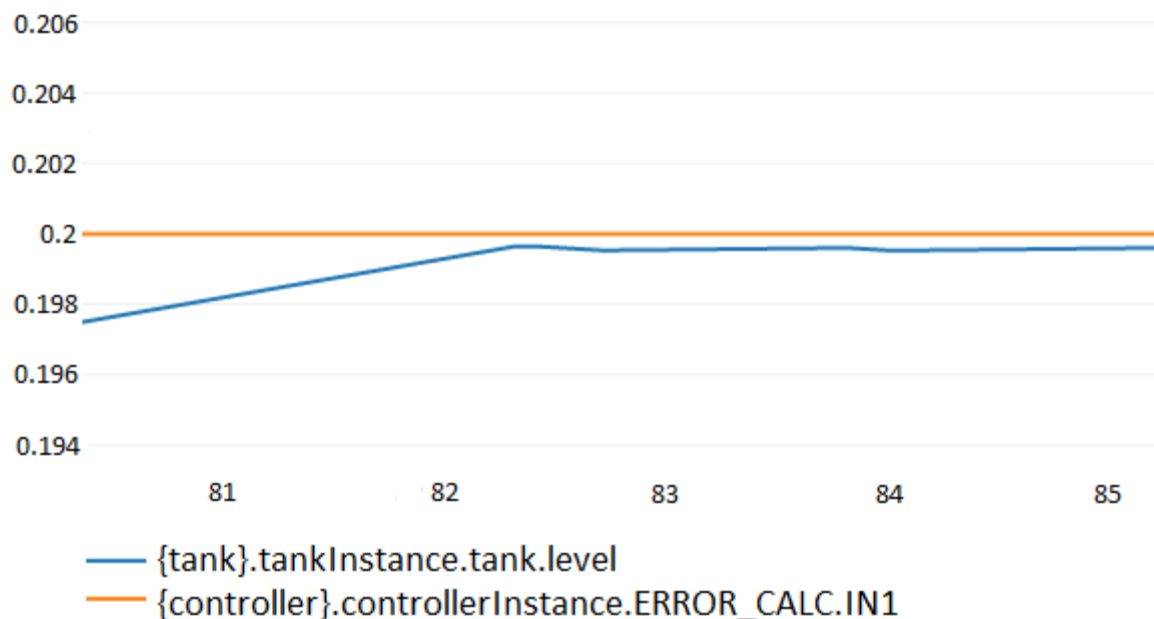
DA и DS обозначают «фиктивный привод» (для управления первым клапаном) и «фиктивный датчик» (который измеряет текущий уровень в резервуаре) соответственно. Они просто преобразуют физические измерения в 16-битное целое число, которое является входом и выходом системы резервуара.

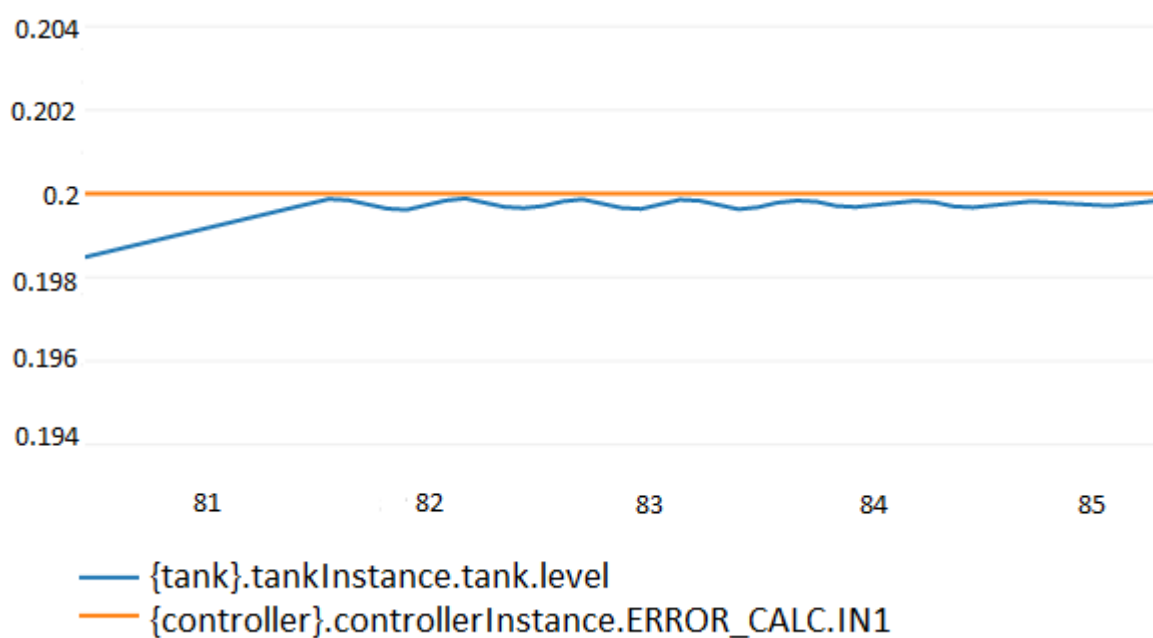
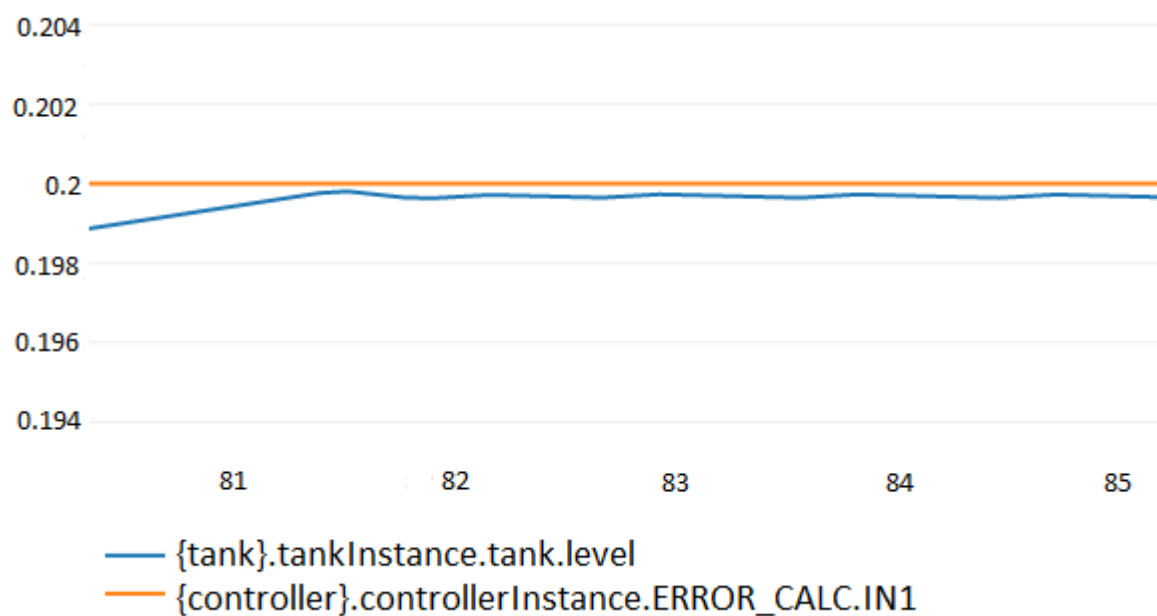
В 4diac IDE был реализован следующий ПИД-регулятор. Основной функциональный блок ПИД-регулятора — это базовый функциональный блок, который поддерживает уровень в резервуаре на заданном уровне (ERROR_CALC.IN2). Резервуар постоянно теряет воду (помните, это всего лишь пример). TANK_LEVEL_IN (IW) и FILL_VALVE (QW) — это FB, которые являются входными и выходными данными экспортируемого FMU.



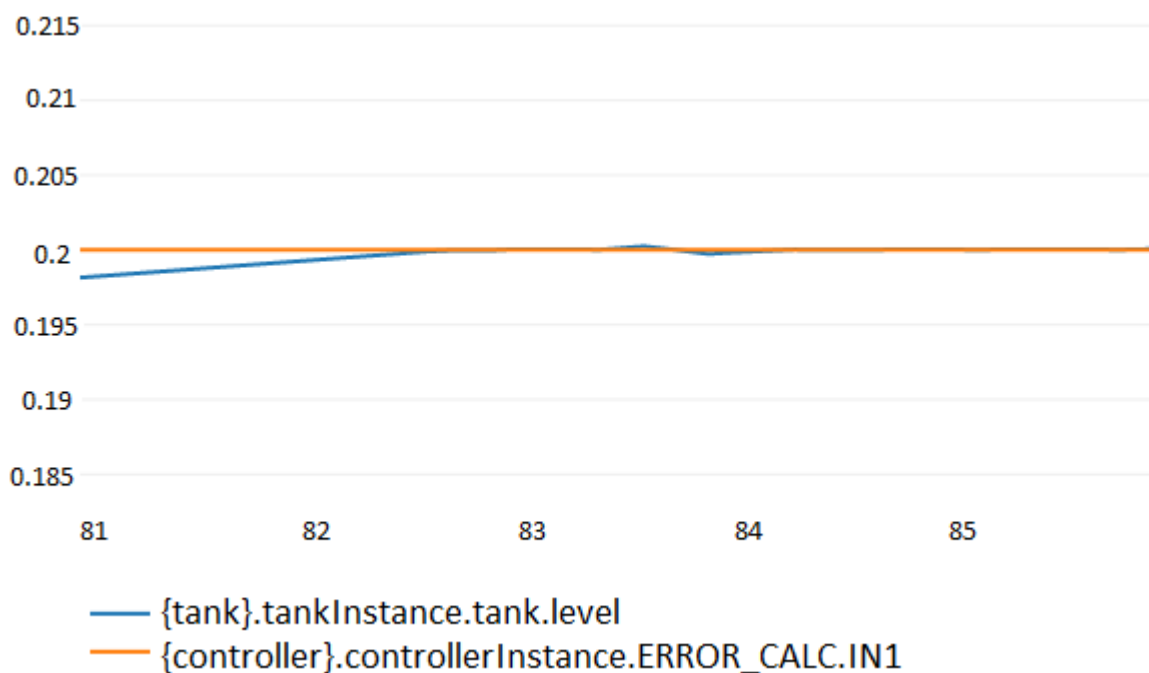
После того как обе системы будут экспортированы в виде FMU, их можно будет совместно моделировать с помощью инструмента [INTO-CPS](#). Входы и выходы обеих моделей подключены правильно (датчик уровня в резервуаре подключен ко входу контроллера, а выход контроллера подключен к приводу клапана резервуара). Поскольку K_p , K_i и K_d ПИД-регулятора являются константами, они рассматриваются как параметры, которые можно задать при каждом совместном моделировании, что упрощает их настройку.

Ниже приведены первые попытки настройки ПИД-регулятора с помощью метода Зиглера-Николса





И окончательный результат настройки показан ниже



Это показывает, как можно легко связать две системы из разных областей. 4-диапазонный FMU также помогает находить ошибки в приложении, чтобы убедиться, что оно работает должным образом.

Matlab

Пользователь Eclipse 4diac Марк Якоби разработал библиотеку для взаимодействия с Matlab, которая позволяет подключать приложения управления IEC 61499 к имитационным моделям в Matlab. Библиотека с открытым исходным кодом реализует протокол клиент-сервер, определенный в профиле соответствия [IEC 61499 для демонстрационных проектов](#). Таким образом, на стороне 4diac вам нужно будет только вставить блок функции взаимодействия с сервером. Подробную документацию и код библиотеки можно найти в проекте [tcpip4diac на GitHub](#).

Куда перейти отсюда?

Вернитесь к индексу протоколов: [Индекс связи](#)

Если вы хотите вернуться на страницу «[Начать здесь](#)», мы оставляем вам быстрый доступ

Как использовать сетевой коммуникационный интерфейс Eclipse 4diac, чувствительный ко времени

Вступление

В этом руководстве показано, как использовать функции 4diac, зависящие от времени работы в сети (TSN). Некоторые стандарты TSN все еще находятся на стадии активной стандартизации (состояние на июль 2018 г.), например, IEEE802.1ASrev и IEEE802.1Qch. Преимуществами TSN являются:

- Конвергентная сеть: сетевой трафик с широким спектром сетевых требований передаётся по одному сетевому кабелю, что повышает эффективность использования пропускной способности и снижает трудозатраты на прокладку кабелей.
- Совместимость: TSN — это расширение стандарта IEEE для уже существующей и широко используемой сети Ethernet.
- Детерминированность: правильно сконфигурированная сеть TSN гарантирует передачу пакетов соответствующим получателям.
- Работа в реальном времени: помимо детерминированности, сети TSN гарантируют передачу сетевых пакетов в течение определённого времени, то есть с максимальной задержкой.

Коммуникационный уровень 4diac реализует соединения по принципу «публикация-подписка» с помощью функциональных блоков интерфейса службы (SIFB). Функциональные блоки публикации дополнены идентификатором VLAN и приоритетом сообщения, представленными во входном параметре данных ID.

Ограничения

Текущая реализация была протестирована на Beaglebone Black на Debian с патчем реального времени (4.1.27-bone-rt-r22) и Ubuntu 16.04LTS.

Текущие ограничения

- Уровень TSN поддерживается в Linux
- Поддерживаются только интерфейсы vlan с eth0 в качестве физического сетевого интерфейса (например, eth0.1, eth0.2, ..., eth0.165, ...)

Как собрать 4diac FORTE с функциями TSN

1. Ознакомьтесь с веткой разработки 4diac FORTE

```
$ git clone git://git.eclipse.org/gitroot/4diac/org.eclipse.4diac.forte.git
$ cd org.eclipse.4diac.forte
$ git checkout develop
```

2. Создайте каталог для двоичных файлов и задайте соответствующие переменные CMake

```
$ mkdir -p bin/posix
$ cd bin/posix/
$ cmake ../.. -DCMAKE_BUILD_TYPE=Debug -DFORTE_ARCHITECTURE=Posix -
DFORTE_COM_TSN=ON
```

3. Соберите 4diac FORTE с помощью:

```
$ make
```

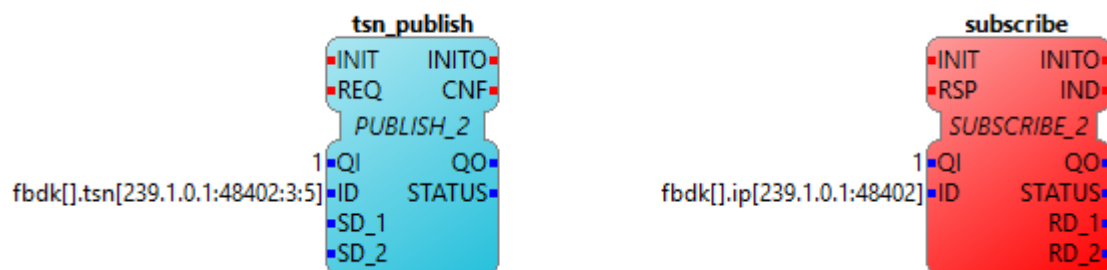
4. Запустите 4diac FORTE с помощью:

```
$ cd src $ ./forte
```

Настройте уровень TSN 4diac FORTE с помощью блока функций публикации

Уровень IP-коммуникации 4diac FORTE с поддержкой TSN сопоставляет поток UDP с конкретным интерфейсом VLAN. Он привязывает поток к виртуальному Ethernet-адаптеру (например, eth0.x). Правильная маркировка с идентификатором VLAN и приоритетом используется сетью с поддержкой TSN для передачи пакетов в соответствии с требованиями реального времени. Различные виртуальные локальные сети связаны с различными стратегиями переадресации и планирования в сети TSN.

Пример публикации FB с поддержкой TSN и стандартного подписчика FB на уровне IP показан на следующем рисунке.



Примечание

Чтобы этот пример работал, необходимо направить многоадресный трафик UDP на устройство подписчика следующим образом:

```
$ route add -net 239.1.0.0 netmask 255.255.0.0 dev eth0.3 $ ./forte
```

и вам нужно настроить интерфейс VLAN и его IP-адрес на отправляющем устройстве, прежде чем привязывать к нему поток, например, с помощью:

```
$ sudo ifconfig eth0.3 up
$ sudo ifconfig eth0.3 "ваш_любимый_IP-адрес"
$ sudo ifconfig eth0.3 netmask "ваша_любимая_маска_сети"
```

Куда двигаться дальше?

Вернитесь к разделу «Протоколы»: Раздел «[Коммуникация](#)»

Если вы хотите вернуться на страницу «[Начало работы](#)», мы оставляем вам быстрый доступ

Доступ к базе данных BaseX с помощью XQueries

В этом разделе показано, как заставить приложения взаимодействовать с базой данных [BaseX](#) с помощью XQueries. Это позволяет компоненту инициализировать себя, обратившись к оболочке управления ресурсами в виде базы данных XML. Компонент управляется специальным функциональным блоком, который генерирует XQueries для инициализации входных данных значениями, полученными или рассчитанными из базы данных BaseX.

Раздел разделён на три части. В первой части объясняется, как создать 4diac FORTE для запуска приложения, отправляющего X-запросы. Во второй части объясняется, как настроить базу данных BaseX. А в третьей части показано, как создать приложение в 4diac IDE, отправляющее X-запросы. Пример файла AutomationML, а также эта реализация были созданы в рамках проекта [BaSys4.0](#), который финансируется [Федеральным министерством образования и научных исследований](#).

Создайте 4diac FORTE для запуска приложения, отправляющего X-запросы

Сначала проверьте [BaseX на GitHub](#), чтобы получить C API от BaseX. В загруженной папке вы найдете C API в разделе `.../basex-api/src/main/c`. Пожалуйста, учтите, что при использовании официально доступного C API от BaseX аутентификация в базе данных BaseX в настоящее время не выполняется в Windows.

```
git clone https://github.com/BaseXdb/basex.git
```

Затем настройте CMake для сборки 4diac FORTE. Если вы не знаете, как собрать 4diac FORTE, сначала [прочтите инструкции по сборке](#). Помимо обычной конфигурации, активируйте Xquery Client в конфигурации 4diac FORTE CMake:

```
FORTE_COM_XqueryClient=ON
```

Нажмите кнопку «Настроить» в CMake, чтобы получить настройки для Xquery Client. Настройте появившиеся параметры. В Linux каталог include для OpenSSL может находиться по адресу `/usr/include/x86_64-linux-gnu/openssl`. Библиотеки OpenSSL `libcrypto.a` и `libssl.a` могут находиться в `/usr/lib/x86_64-linux-gnu/`.

```
FORTE_BASEX_SRC_DIR=путь_к_C-API_BaseX, который_вы_скачали
```

```
OPENSSL_INCLUDE_DIR=путь_к_заголовочным_файлам_OpenSSL
```

```
OPENSSL_LINK_DIR=путь_к_библиотекам_OpenSSL
```

Нажимайте кнопку «Настроить» в графическом интерфейсе CMake, пока ни одна из строк не станет красной. Затем нажмите кнопку «Сгенерировать» в графическом интерфейсе CMake. Затем соберите 4diac FORTE.

Настройте базу данных BaseX

Скачайте BaseX для своей системы [здесь](#). После запуска BaseX загрузите файл AutomationML или XML в BaseX, нажав «База данных/открыть и управлять». В качестве примера файла AutomationML можно использовать [модель системы Pallet](#). Этот пример файла AutomationML был создан в рамках проекта [BaSys4.0](#), который финансируется [Федеральным министерством образования и научных исследований](#). На вкладке «Общие» введите путь к вашему файлу и выберите подходящий формат ввода. Если вы используете файл AutomationML или XML, выберите XML. Затем нажмите «ОК». Ваша база данных будет загружена. Для доступа к созданной базе данных потребуется имя пользователя и пароль. Чтобы создать его, выберите команду в раскрывающемся списке под панелью меню и введите следующие команды в поле рядом с ней.

```
> create user admin > grant write to admin
```

Теперь вы можете протестировать любой XQuery в графическом интерфейсе BaseX. Для получения дополнительных разъяснений или в случае возникновения проблем, пожалуйста, ознакомьтесь с [руководством по BaseX](#).

Чтобы запустить весь сервер BaseX, перейдите в каталог bin вашей установки BaseX. Запустите там сервер BaseX. На выходе должна быть версия BaseX и порт, по которому вы можете подключиться к своей базе данных.

```
> ./basexserver BaseX 10.4 [Server] Server was started (port: 1984).
```

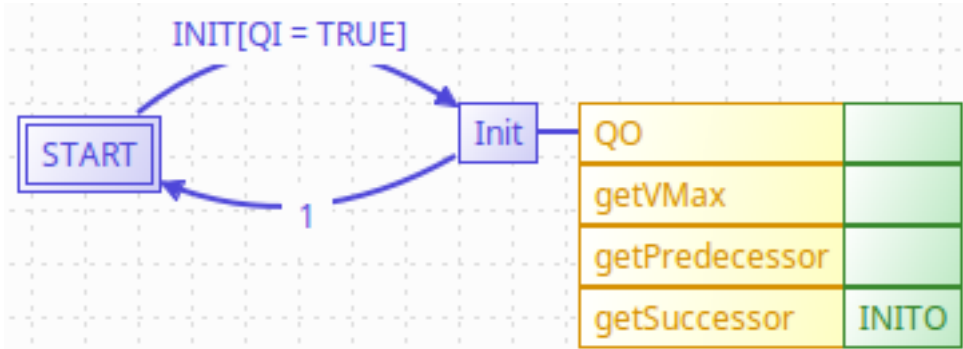
Если вы использовали Java-реализацию BaseX, следующая команда запустит сервер BaseX. Замените номер версии в следующей строке на номер версии загруженного вами JAR-файла.

```
> java -cp BaseX104.jar org.basex.BaseXServer BaseX 10.4 [Server] Server was started (port: 1984).
```

Необходимо создать приложение для отправки X-запросов в BaseX.

Чтобы отправлять XQueries в вашу базу данных, создайте соответствующий базовый функциональный блок, который генерирует нужные XQueries. На следующих изображениях показан возможный интерфейс и схема

управления выполнением для базового функционального блока, который извлекает значения из вашей базы данных BaseX во время инициализации.



Алгоритм `getVMax` содержит X-запрос, который должен вычислять максимальную скорость роликового конвейера на основе значений из базы данных. Поскольку в настоящее время IDE 4diac не поддерживает язык алгоритмов XQuery, в качестве языка алгоритмов необходимо использовать AnyText. Язык AnyText интерпретируется как код C++ во время экспорта вашего блока функций. Ваш X-запрос присваивается желаемому строковому выходу вашего базового блока функций, в данном случае `xVMax`. Если вы также хотите заменить некоторые элементы в XQuery, например, значениями из интерфейса вашего функционального блока, вам нужно разделить строку. Затем значение интерфейса добавляется в качестве параметра метода `append`, поддерживаемого 4diac FORTE, например, `name().getValue()`. Остальная часть XQuery добавляется позже. Это может выглядеть как следующий код алгоритма для `getVMax`.

```
st_xVMax() = CIEC_STRING("xquery ");
st_xVMax().append("let $name := '");
st_xVMax().append(st_name().getValue());
st_xVMax().append("' "
    "let $component := //InstanceHierarchy/*/InternalElement[@Name=$name] "
    "let $compType :=
tokenize($component/RoleRequirements/@RefBaseRoleClassPath/data(), '/') [last()] "
    "let $rollerTable :=
$component/InternalElement/RoleRequirements[contains(@RefBaseRoleClassPath, 'RollConveyo
r')]/.. "
    "let $rollDiameter := $rollerTable/Attribute[@Name='RollDiameter']/Value/data() "
    "let $link := tokenize($component/InternalLink/@RefPartnerSideA/data(), ':') [1] "
    "let $motor :=
//InternalElement[@ID=$link]/RoleRequirements[contains(@RefBaseRoleClassPath, 'ElectricM
otor')]/.. "
    "let $gearFactor := $motor/Attribute[@Name='Gearfactor']/Value/data() "
    "let $rotSpeed := $motor/Attribute[@Name='Rotational Speed']/Value/data() "
    "let $linSpeed := $rotSpeed div (60.0 * $gearFactor) * (2.0 * 3.1416 * 0.5 *
$rollDiameter) "
    "return round-half-to-even($linSpeed, 3)");
```

Код алгоритма XQuery для `getPredecessor` может быть следующим:

```
st_xPredecessor() = CIEC_STRING("xquery ");
st_xPredecessor().append("let $name := '");
st_xPredecessor().append(st_name().getValue());
st_xPredecessor().append("' "
    "let $links := //InstanceHierarchy/InternalElement/InternalLink "
    "let $cp := //InstanceHierarchy/*/InternalElement[@Name=$name] "
    "let $id := $cp/@ID/data() "
    "let $succId := tokenize($links[contains(@RefPartnerSideB, $id)]/@RefPartnerSideA,
':') [1] "
    "let $successor := //InstanceHierarchy/*/InternalElement[@ID=$succId] "
    "return $successor/@Name/data() ");
```

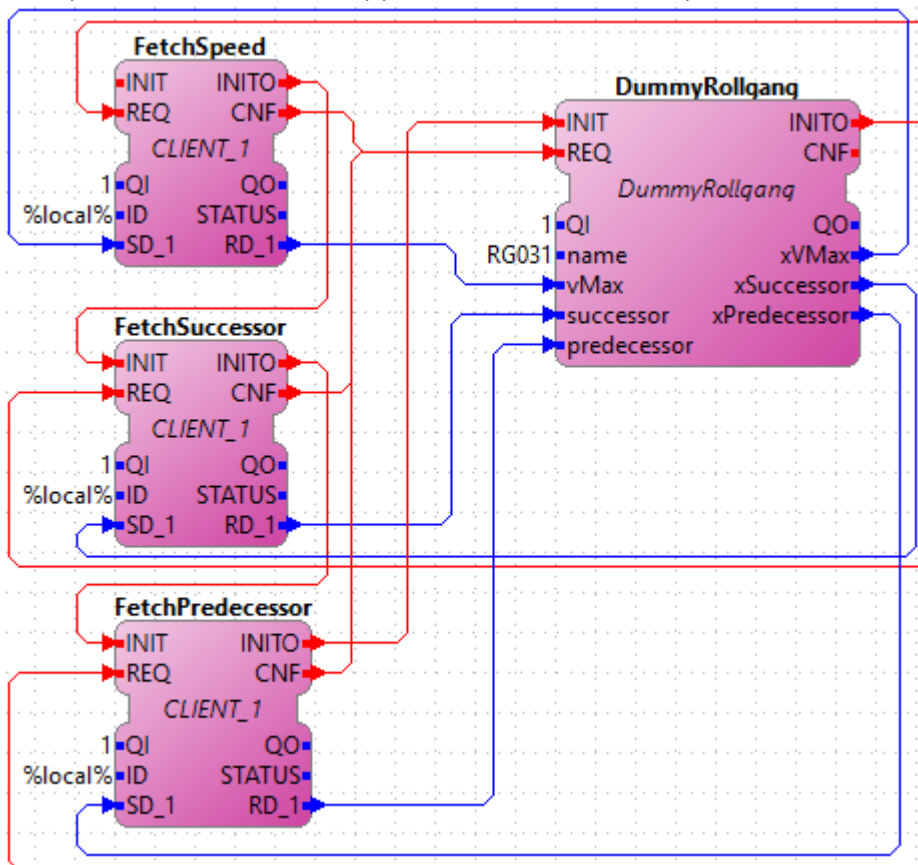
Код алгоритма XQuery для `getSuccessor` может быть следующим:

```
st_xSuccessor() = CIEC_STRING("xquery ");
st_xSuccessor().append("let $name := '");
st_xSuccessor().append(st_name().getValue());
st_xSuccessor().append("' "
    "let $links := //InstanceHierarchy/InternalElement/InternalLink "
    "let $cp := //InstanceHierarchy/*/InternalElement[@Name=$name] "
    "let $id := $cp/@ID/data() "
    "let $succId := tokenize($links[contains(@RefPartnerSideA, $id)]/@RefPartnerSideB,
':') [1] "
    "return $succId/@Name/data() ");
```

```
"let $successor := //InstanceHierarchy/*/InternalElement[@ID=$succId] "
"return $successor/@Name/data() " );
```

Обратите внимание, что ваш XQuery зависит от структуры данных вашей базы данных. В данном случае это структура файла AutomationML для [системы Pallet](#). После того как вы создадите свой тестовый функциональный блок, экспортируйте его и создайте с его помощью 4diac FORTE. Если вы не знаете, как создать собственный функциональный блок, ознакомьтесь с [инструкциями по созданию](#).

Теперь вы можете использовать свой тестовый функциональный блок в приложении. Чтобы отправить X-запросы в базу данных BaseX, добавьте CLIENT_1 для каждого отправляемого запроса. Пожалуйста, обратите внимание, что в настоящее время поддерживается только CLIENT_1, где вы подключаете вход SD_1 к выходу xquery, создающему выходной поток вашего блока тестовой функции. Результат XQuery принимается на выходе RD_1. Возможное тестовое приложение показано на следующем изображении. В этом приложении конвейерная лента инициализируется максимальной скоростью и соседними конвейерными лентами.



Входное значение идентификатора функционального блока CLIENT_1 содержит переменную %local%, которая заменяется соответствующим значением во время развёртывания. Идентификатор настраивается с помощью имени протокола, IP-адреса компьютера, на котором работает ваша база данных BaseX, порта, на котором ваш сервер BaseX ожидает запросы, имени вашей базы данных, имени пользователя и пароля. Для примера базы данных, работающей на вашем локальном компьютере, идентификатор может быть следующим:

```
xquery[127.0.0.1:1984; BaSys_PalletSystem_Model; admin; admin]
```

После того как вы завершите работу с приложением, сопоставьте его с соответствующим устройством и запустите 4diac FORTE, который вы создали ранее. Затем разверните приложение на 4diac FORTE, который вы запустили. Если все прошло успешно, 4diac FORTE должен выдать что-то вроде этого:

```
INFO: T#00ms: FORTE is up and running
INFO: T#00ms: Using default bootfile location: forte.fboot
INFO: T#00ms: Boot file forte.fboot could not be opened. Skipping...
INFO: T#0100722ms: Connection closed by peer
INFO: T#0100737ms: Connected to DB.
INFO: T#0101026ms: DB BaSys_PalletSystem_Model opened..
```

Если вы отслеживаете работу своего приложения, вы должны получать результаты запросов XQuery на соответствующих входах вашего блока тестовой функции.

Вернитесь к разделу «Протоколы»: Раздел [«Связь»](#) Если вы хотите вернуться на страницу [«Начало работы»](#)

Сборка и запуск 4diac IDE

У вас есть 2 варианта сборки и запуска 4diac IDE:

1. [Сборка и запуск 4diac IDE из исходного кода](#)
2. [Сборка двоичного пакета 4diac IDE из исходного кода](#)

1 Сборка и запуск 4diac IDE из исходного кода

Выполните следующие действия для сборки и запуска 4diac IDE из исходного кода:

1. [Установите среду разработки](#)
2. [Ознакомьтесь с репозиторием 4diac IDE](#)
3. [Импортируйте плагины в рабочую область](#)
4. [Установите целевую платформу](#)
5. [Запустите в режиме разработки](#)

1.1 Получите среду разработки

Для 4diac IDE требуется как минимум Java JRE версии 21. Поддерживаются JRE более высоких версий, но может потребоваться дополнительная настройка Eclipse IDE (см. шаг 4 в главе «[Импорт плагинов в рабочую область](#)»).

Получите последнюю версию [Eclipse IDE](#). Используйте версию Eclipse Modeling Tools.

Для корректного разрешения всех зависимостей требуются следующие дополнительные пакеты Eclipse:

- Интеграция с M2E - Maven для Eclipse
- Интеграция с M2E - PDE
- XText Complete SDK

Вы можете установить эти пакеты с помощью Eclipse IDE, выбрав Справка → Установить новое программное обеспечение (репозиторий Eclipse).

1.2 Ознакомьтесь с репозиторием IDE 4diac

В этом разделе показано, как извлечь (клонировать) репозиторий IDE 4diac. Для его загрузки вам понадобится [Git](#). Используйте команду clone из Git для репозитория <https://github.com/eclipse-4diac/4diac-ide.git>. Если вы используете терминал, выполните следующие действия:

```
$ cd ~
```

```
$ git clone https://github.com/eclipse-4diac/4diac-ide.git
```

Код будет скопирован в новую созданную папку ~/4diac-ide.

Для этого процесса вы также можете использовать Git, встроенный в Eclipse. Пожалуйста, ознакомьтесь с документацией Eclipse EGIt о том, как клонировать репозиторий.

После клонирования репозитория убедитесь, что вы переключились на ветку разработки. Ветка по умолчанию, release, может быть устаревшей и, следовательно, несовместимой с этими шагами по настройке.

```
$ cd ~/4diac-ide/
```

```
$ git checkout develop
```

1.3 Импорт плагинов в рабочее пространство

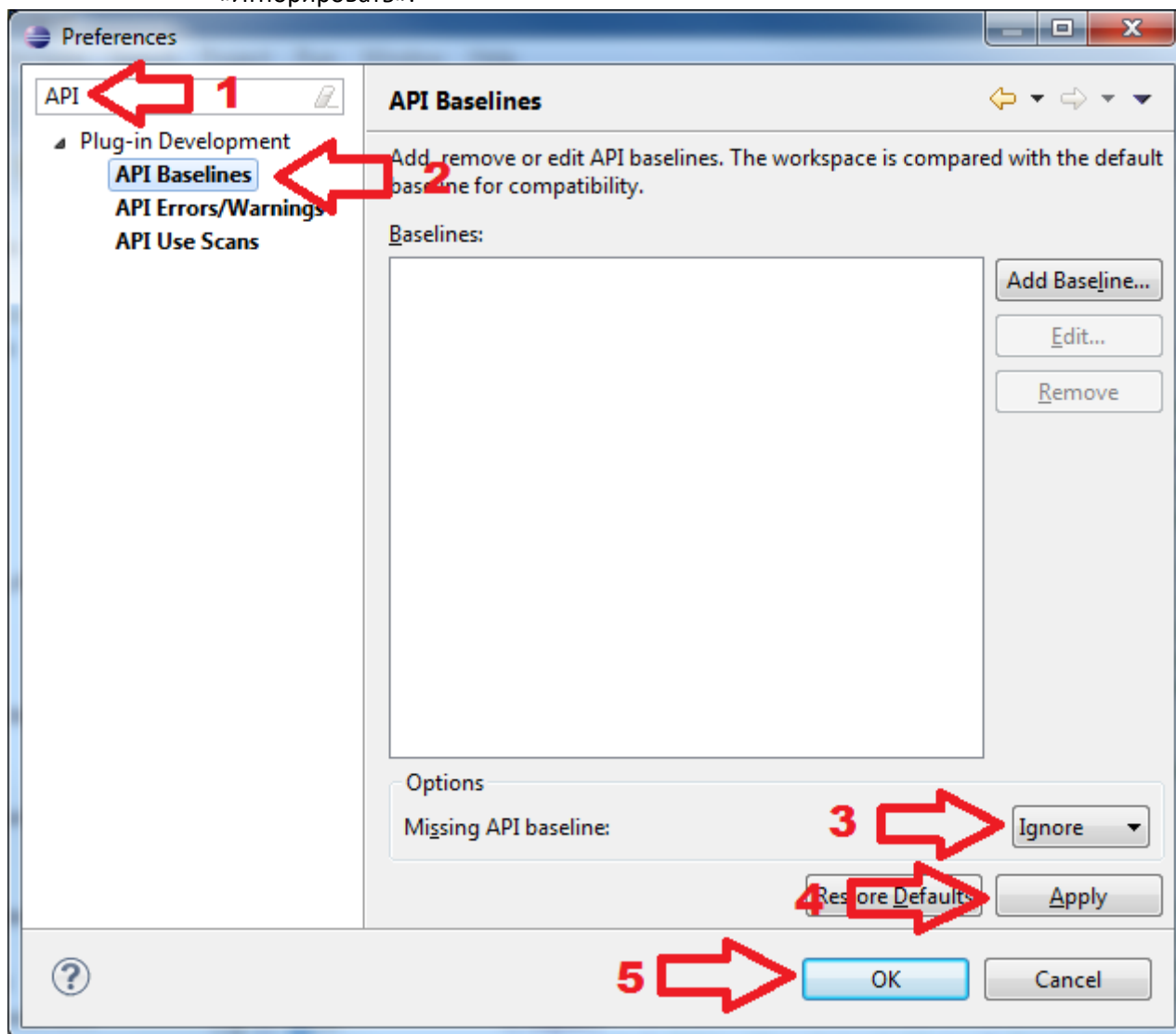
1. переключитесь на режим Git в Eclipse
2. Щёлкните правой кнопкой мыши по репозиторию 4diac IDE в Git и выберите «Импортировать проекты...», чтобы открыть диалоговое окно. Выберите всё в этом диалоговом окне, кроме следующих элементов:

Folder	Import as
☐ 4diac-ide\features	Maven
☐ 4diac-ide\plugins	Maven
☐ 4diac-ide\tests	Maven

4diac IDE не импортирует папки

3. Дождитесь, пока Eclipse завершит сборку проекта (обратите внимание на индикатор выполнения в правом нижнем углу главного окна Eclipse).
4. Если у вас установлена JRE версии > 21, убедитесь, что уровень соответствия компилятора установлен на JDK версии 21, чтобы сборка прошла успешно. Перейдите в «Окно» → «Настройки». Найдите «Компилятор» и в разделе «Соответствие JDK» установите «Уровень соответствия компилятора» на 21. Если этот шаг не был выполнен должным образом, в разделе «Проблемы» будут отображаться ошибки «XText».

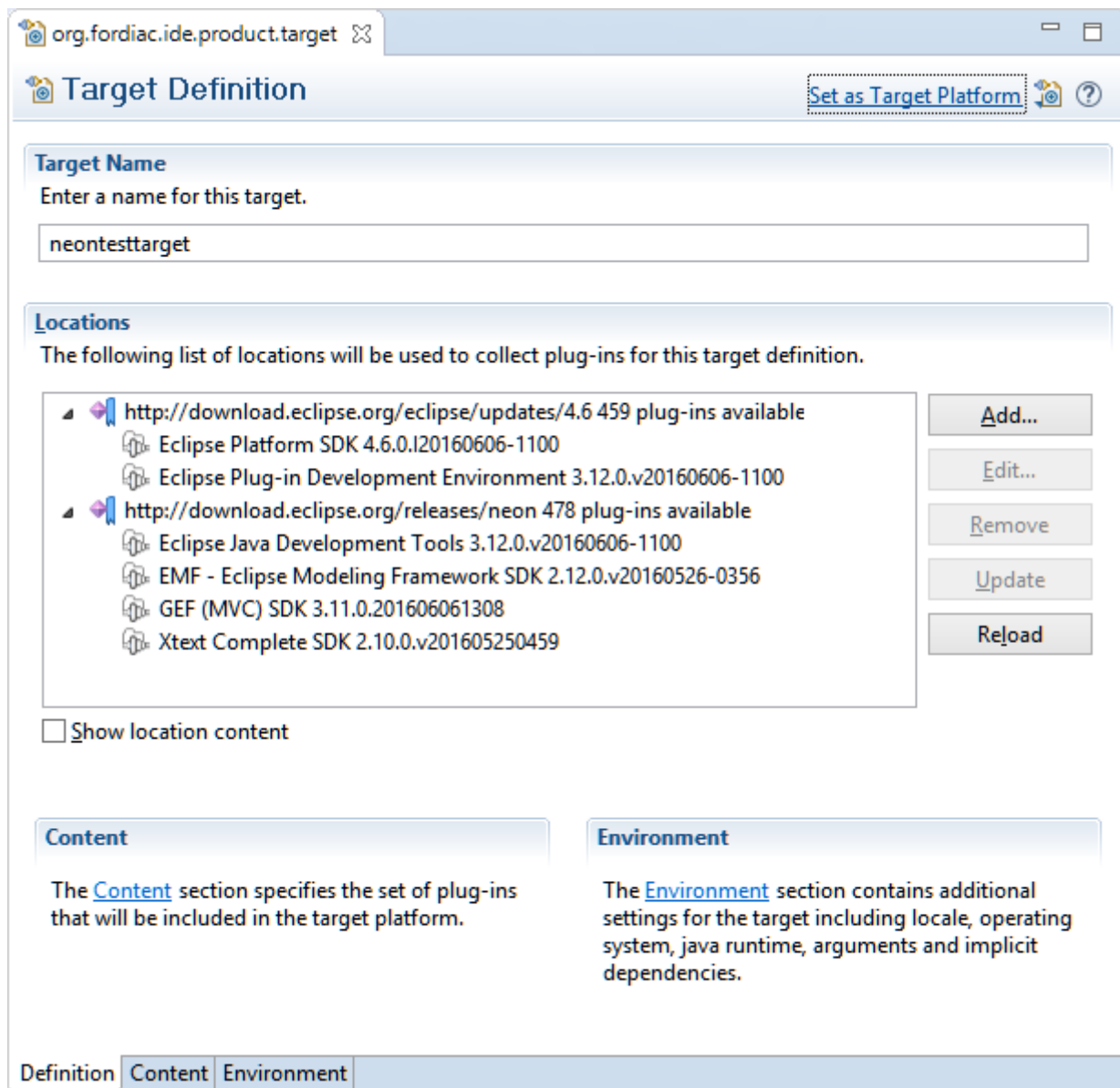
5. Проверьте раздел «Проблемы» на наличие ошибок. Если вы получаете сообщение об ошибке «Базовый уровень API не задан», выполните следующие действия:
- Откройте настройки Eclipse: «Окно» → «Настройки»
 - Введите «API» в поле поиска в левом верхнем углу. Вы должны получить три результата исследования: 1. Базовые настройки API, 2. Ошибки/предупреждения API и 3. Сканирование использования API
 - Теперь у вас есть два варианта:
 - Установите базовую настройку API для своих разработок:
 - Выберите: Базовую настройку API
 - Нажмите «Добавить базовую настройку...»
 - Введите название, например, MyEclipse
 - В качестве местоположения выберите каталог Eclipse
 - Нажмите «Готово»
 - Выберите новую базовую версию в окне «Базовые версии API» и нажмите «OK».
 - Установите для ошибки «Базовая версия API» значение «Игнорировать»:
 - Выберите: «Базовая версия API».
 - Установите для параметра «Отсутствует базовая версия API» значение «Игнорировать», которое можно найти в нижней части окна настроек, изменив значение «Ошибка» на «Игнорировать».



Примечание. Если в разделе «Проблемы» вы видите сообщение «Возможно несоответствие базовой версии API ...», вы можете спокойно игнорировать это сообщение. Это сообщение не мешает успешной сборке IDE.

1.4 Настройка целевой платформы

- Разверните проект `org.eclipse.fordiac.ide.product` и откройте целевую конфигурацию `org.eclipse.fordiac.ide.product.target`:

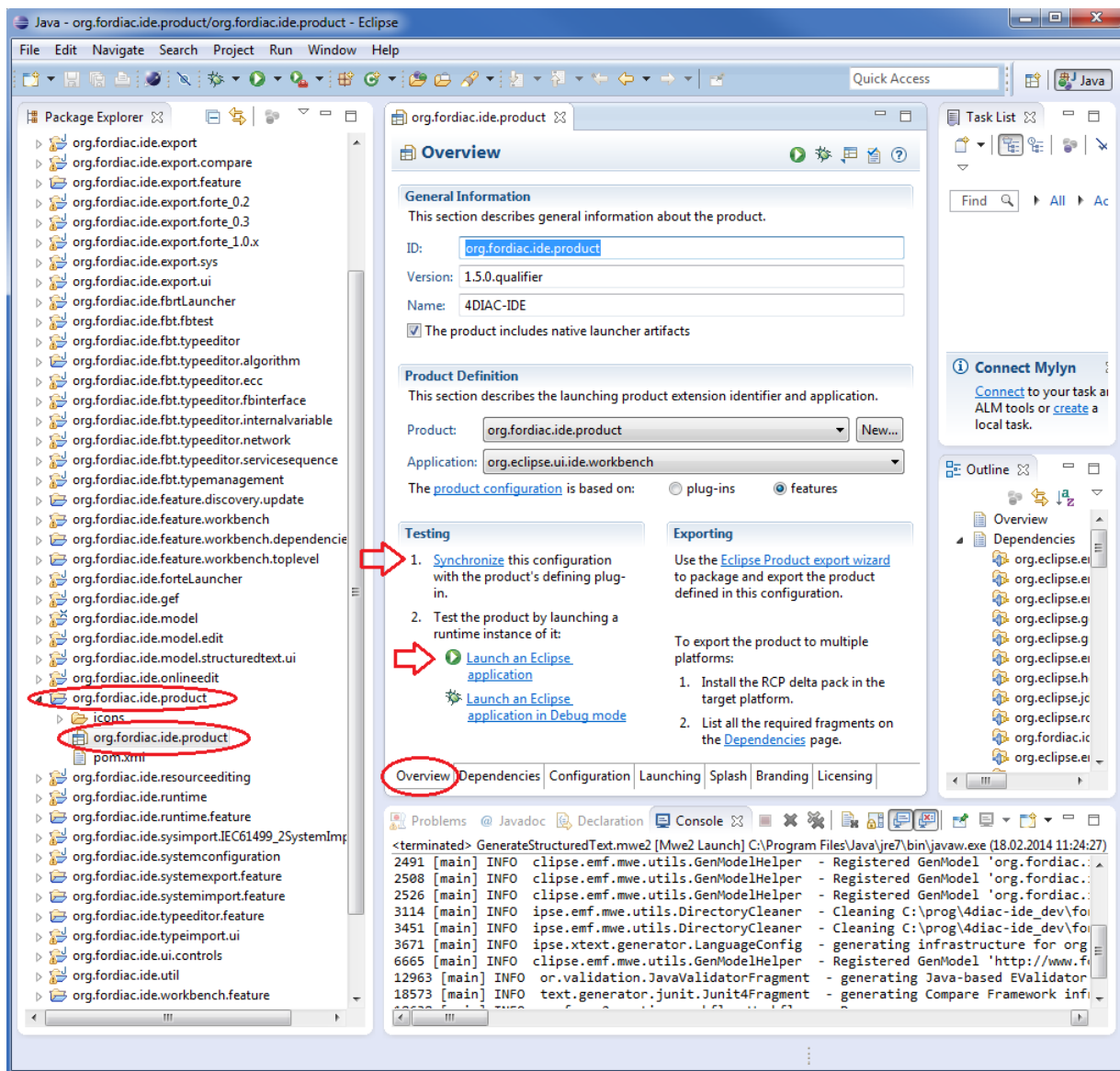


2. Дождитесь, пока целевая платформа будет определена, или нажмите «Перезагрузить» и дождитесь, пока она будет определена

3. Нажмите «Установить как активную целевую платформу» в правом верхнем углу и дождитесь, пока будет создано рабочее пространство.

1.5 Запуск в режиме разработки

1. Чтобы использовать 4diac IDE непосредственно в этом (разрабатываемом) Eclipse, вам нужна библиотека типов и шаблоны. И то, и другое является частью репозитория 4diac IDE и находится в каталоге данных. Чтобы 4diac IDE для разработки могла найти их, их нужно поместить в каталог, где хранится целевая платформа. Для этого скопируйте `template` и `typelibrary` в каталог `.metadata/.plugins/org.eclipse.pde.core/.bundle_pool`.
2. Откройте `org.eclipse.fordiac.ide.product` из подключаемого модуля `org.eclipse.fordiac.ide.product plug-in`.



3. В разделе «Тестирование» на вкладке «Обзор» нажмите «Синхронизировать»

4. Нажмите «Запустить приложение Eclipse» на вкладке «Обзор»

Создание собственной IDE 4diac

Большим недостатком запуска IDE 4diac из исходного кода является то, что вам потребуется как минимум 2 запущенных экземпляра Eclipse. Особенно в системах с небольшим объёмом памяти это может стать проблемой. Если вы по-прежнему хотите быть в курсе изменений, вносимых в репозиторий Git, имеет смысл создать двоичный пакет. Подробнее о том, как это сделать, см. в разделе «[Создание двоичного пакета 4diac IDE из исходного кода](#)».

Кроме того, мы предлагаем ночную сборку 4diac IDE [здесь](#).

[Перейти к началу](#)

2 Создание двоичного пакета 4diac IDE из исходного кода

Для создания двоичного пакета 4diac IDE из исходного кода лучше всего использовать конфигурацию Maven, которая также используется для создания 4diac IDE на нашем сервере сборки. Современная IDE Eclipse поставляется с интеграцией Maven «из коробки». Затем необходимо выполнить следующие действия для создания двоичных пакетов 4diac IDE:

1. поместите исходный код 4diac IDE в рабочую область Eclipse, как описано в разделе «[Создание и запуск 4diac IDE из исходного кода](#)»
2. Перейдите в корневой проект `org.eclipse.fordiac.ide` и щёлкните правой кнопкой мыши по `pom.xml`
3. Выберите «Запуск от имени» → «Установка Maven».
4. После успешной сборки вы найдёте результат в каталоге `plugins/org.eclipse.fordiac.ide.product/target/products`.

В качестве альтернативы вы можете запустить `.mvn .install` в командной строке в корневой папке исходного кода 4diac IDE.

Участие в разработке 4diac IDE или 4diac FORTE

Проекты 4diac IDE и 4diac FORTE хранятся в виде репозитория Git на платформе Eclipse. Это проект с открытым исходным кодом, поэтому вы можете скачать его и даже вносить изменения и улучшать его. Но, конечно, всё не так просто, иначе изменения добавлялись бы без какого-либо контроля и могли бы испортить код. Если вы хотите сообщить об ошибке, см. раздел «[Сообщение об ошибке](#)».

Репозиторий Git

Если вы не знаете, что такое Git, не волнуйтесь, мы все через это проходили. По сути, Git — это распределённая система, которая используется для управления различными версиями чего угодно, но в основном программного кода. Код хранится в так называемом репозитории, который содержит все файлы, связанные с программным обеспечением. После изменения кода вместо того, чтобы сохранять новые файлы (fileV1, fileV2, fileFinalVersion, fileThisIsReallyTheFinalVersion, fileThisIsReallyTheFinalVersion2) в новой папке, вы просто «фиксируете» изменения, и они сохраняются в репозитории. Чтобы просмотреть более старые версии программного обеспечения, перейдите к конкретной фиксации. В определении Git есть слово «распределённый» (уверен, вы его не заметили). Существует не только один репозиторий, но и любой, кто хочет внести свой вклад в репозиторий, копирует всё содержимое репозитория на свой компьютер и работает там, вносит изменения и совершает коммиты. Затем пользователь отправляет изменения в репозиторий, из которого он их скопировал.

Это лишь основной принцип работы Git, но он даёт представление о том, как это работает. Пошаговое руководство, приведённое ниже, поможет вам настроить Git для внесения первых изменений. В интернете можно найти много информации о Git. Вместо того, чтобы начинать с нуля, я бы порекомендовал заглянуть на <https://git-scm.com/>, где вы можете скачать всё необходимое, чтобы начать вносить свой вклад в любой проект Git. В книге-справочнике по Git представлен хороший обзор. Хотя [книга](#) очень длинная, вам понадобятся только главы 1 и 2, чтобы понять, как работает Git, и глава 3, чтобы понять, что такое ветки.

Но я должен предупредить вас, юный падаван, что изучать Git сложно. Это очень мощный инструмент, но для его понимания требуется время. Просто прочтите основы и следуйте пошаговому руководству. Необязательно пытаться понять всё, прежде чем вносить свой вклад. Лучший учитель — это практика, поэтому используйте 4diac FORTE или 4diac IDE в качестве примера для практики. Вы ничего не сломаете. Git надёжен.

Gerrit

Вы знаете, как работает Git, и, возможно, уже использовали его. Но теперь в игре появился новый игрок: Gerrit. Что это такое? Информация в Википедии поможет, но если говорить просто, то это брандмауэр для вкладов. Он используется многими платформами, в том числе Eclipse Foundation, где находятся репозитории 4diac FORTE и 4diac IDE. Когда кто-то, программист с благими намерениями, хочет внести свой вклад в код 4diac FORTE, он клонирует репозиторий Git, вносит изменения, коммитит, а затем отправляет в репозиторий в Eclipse Foundation. Но, конечно, изменения не попадают напрямую в репозиторий. Во-первых, его должны принять люди, отвечающие за 4diac FORTE или 4diac IDE. Поэтому участники отправляют изменения в Gerrit. Gerrit проверяет соответствие законодательным требованиям, а затем уведомляет ответственных за 4diac FORTE о том, что поступило новое изменение. Затем они заходят в Gerrit, видят новое изменение и могут принять или отклонить его. Просто, не так ли?

Но чтобы лучше понять Gerrit, лучше поработать с ним. Мы шаг за шагом будем отправлять код в Gerrit, чтобы внести свой вклад в 4diac FORTE.

Помогите 4diac IDE и 4diac FORTE — СЕЙЧАС!

Как я уже говорил, мы будем шаг за шагом вносить свой вклад в 4diac FORTE. Однако, если вы хотите внести свой вклад, вам нужно будет подписать несколько [юридических документов](#) онлайн. Если вы не хотите подписывать их, вы всё равно можете получить код 4diac FORTE и поиграть с ним.

Создайте учётную запись Eclipse

Прежде всего вам нужно создать учётную запись Eclipse. Все коммиты, созданные учётными записями Eclipse без подписанного ECA, будут автоматически отклонены. Если вы не хотите вносить свой вклад, а просто хотите скачать код и поиграть с ним, переходите к следующему шагу.

Чтобы создать учётную запись Eclipse, просто перейдите на <https://www.eclipse.org/> и нажмите «Создать учётную запись» в правом верхнем углу страницы. Затем просто заполните информацию для создания учётной записи. Адрес электронной почты, указанный при регистрации учётной записи, также следует использовать в Git.

Отправляйте сообщения об ошибках

Если вы заметили проблему, связанную с Eclipse 4diac, вы можете сообщить об ошибке по следующей ссылке: [Bugzilla](#)

Вы можете сообщать о проблемах и вносить предложения для 4diac IDE, 4diac FORTE, веб-сайта и т. д. после входа в свою (новую) учётную запись Eclipse. Следуйте [инструкциям по написанию отчётов об ошибках](#), чтобы другие пользователи могли понять вашу проблему. Сначала выберите компонент, который необходимо улучшить. Затем введите сведения о вашей системе: используемую версию программы, а также вашу систему. Если ошибка не связана с определённой операционной системой, выберите «Все». Краткое описание будет заголовком ошибки и должно быть кратким и содержательным. Дополнительные сведения добавляются в поле описания. Предоставьте всю информацию, необходимую для понимания и, возможно, воспроизведения ошибки. Не стесняйтесь сообщать и о незначительных проблемах.

Подпишите ЕСА

Следующим шагом является подписание ЕСА (Соглашения участников Eclipse). Это обязательно для участия в проектах. Перейдите на страницу ЕСА, чтобы прочитать его и узнать больше. ЕСА защищает, например, авторские права. По сути, когда вы подписываете его, вы заявляете, что имеете полное право на каждое внесённое вами изменение, что помогает защитить интеллектуальную собственность, в том числе вашу собственную. Если ваш вклад является частью вашей работы в какой-либо компании, вам следует сначала обсудить это со своим руководителем и юридическим отделом, чтобы полностью понять, как к этому подойти. Подпись ставится онлайн после входа в учётную запись и выбора «Соглашения участника Eclipse». Вам нужно поставить галочки и заполнить текстовое поле словами «Я СОГЛАСЕН». Это не принесёт ощутимых результатов. Eclipse сохраняет вашу информацию вместе с версией ЕСА, которую вы подписали. Вы можете распечатать ЕСА, но вы не получите сертификат, подтверждающий, что вы его подписали. В вашей учётной записи будет указано, что вы подписали ЕСА, и в Eclipse есть инструмент для проверки того, что учётная запись его подписала.

Используйте EGit для внесения вклада: подготовка

В первом примере используется Eclipse, и вся работа с Git будет выполняться с помощью EGit, который обычно устанавливается автоматически вместе с Eclipse IDE. Если вы считаете, что графический интерфейс предназначен для слабаков, а настоящий программист должен использовать командную строку, вы можете следовать приведённым [ниже](#) инструкциям. Тем не менее, пользователям, не имеющим опыта работы с командной строкой, следует использовать Eclipse.

1. Установите всё необходимое программное обеспечение

Первое, что нужно сделать, — это установить всё программное обеспечение. Для 4diac FORTE скачайте Eclipse для C и установите его. Установка набора инструментов описана [xref:../installation/index.adoc#ownIDE\[здесь\]](#). Если у вас уже есть Eclipse IDE, но она не настроена для работы с C, вам не нужно скачивать ещё одно Eclipse, а только плагин C++. Смотрите руководство [здесь](#). Чтобы установить Eclipse IDE для разработки 4diac IDE, воспользуйтесь этой [инструкцией](#). EGit должен быть установлен автоматически вместе с Eclipse, в противном случае следуйте инструкциям по его установке [здесь](#).

2. Настройте EGit

Первое, что нужно сделать в EGit, — это настроить параметры пользователя. Основные параметры — это имя и адрес электронной почты. Эта информация прикрепляется к вашим коммитам. Чтобы сделать это, просто следуйте простым шагам, описанным в разделе 4.2 по той же [ссылке](#), что и выше. В частности, откройте «Окно» → «Настройки» → «Команда» → «Git» → «Конфигурация в Eclipse». Нажмите «Добавить запись...», чтобы открыть диалоговое окно для пары «ключ-значение». В качестве ключа введите email, а в качестве значения — адрес электронной почты вашей учётной записи Eclipse. Нажмите «Добавить» и добавьте ещё одну запись. Вторая запись состоит из имени ключа и вашего полного имени в качестве значения. Поскольку вы создали учётную запись Eclipse и подписали ЕСА на предыдущем [шаге](#), вы должны использовать тот же адрес электронной почты.

3. Скачайте репозиторий 4diac FORTE или 4diac IDE

В Eclipse перейдите в «Файл» → «Импорт...» → «Git» → «Проекты из Git». Нажмите «Далее», а затем выберите «URI клонирования». Затем вставьте <https://github.com/eclipse-4diac/4diac-forte.git> или <https://git.eclipse.org/r/4diac/org.eclipse.4diac.ide> в поле URI и нажмите «Далее». Eclipse подключится к репозиторию и загрузит все его ветки. Выберите все и нажмите «Далее». Ветки представляют собой различные этапы разработки. Для небольших обновлений/исправлений текущей версии можно использовать ветку с последним номером версии. Выберите папку назначения для репозитория на вашем локальном компьютере и нажмите «Далее». Подождите, пока репозиторий загрузится. Нажимайте «Далее», пока помощник не завершит работу.

4. Настройка Gerrit

Перейдите на страницу [Gerrit](#) в Eclipse, чтобы управлять данными своей учётной записи в Gerrit. Нажмите «Войти» в правом верхнем углу. Имя пользователя — это адрес электронной почты, зарегистрированный в вашей учётной записи Eclipse, созданной на [шаге 1.1](#). Войдите в систему, используя пароль вашей учётной

записи Eclipse. Затем нажмите на своё имя в правом верхнем углу и выберите «Настройки». Там вы можете изменить настройки сообщений, следить за проектами, управлять паролем и т. д. Вы можете отправлять изменения в Gerrit через SSH или https. В этом примере мы будем использовать https. Для SSH необходимо создать открытый и закрытый ключи, которые хранятся в файлах. Eclipse использует эти файлы для подключения к Gerrit. Чтобы узнать больше о создании ключей и настройке Eclipse, перейдите по [ссылке](#), а затем добавьте ключ в настройки открытых ключей SSH в Gerrit.

Перейдите в раздел «Пароль HTTP» на странице «Настройки» Gerrit, скопируйте появившееся имя пользователя и нажмите «Создать пароль». Вы получите случайный длинный пароль, который будете использовать каждый раз, когда захотите отправить что-то в Gerrit. Перейдите в Eclipse, щёлкните правой кнопкой мыши по проекту → «Команда» → «Показать в представлении репозитория». Затем разверните раздел «Удаленные» и щёлкните правой кнопкой мыши по «Источнику» → «Конфигурация Gerrit...». На этом этапе вы можете выбрать ssh, https или другой протокол связи. Выберите https и вставьте свое имя пользователя из Gerrit в поле «Пользователь». Это изменит URI отправки. В поле «Целевая ветка» вы должны указать ветку для отправки. Вы не можете отправить изменения в ветку, которой не существует в репозитории Gerrit. Обратите внимание, что целевая ветка начинается с refs/for/. Нажмите «Готово» и в разделе «Репозитории» увидите, что удалённый сервер Origin изменил URI отправки. Это означает, что когда вы отправляете что-то в 4diac FORTE или 4diac IDE, это отправляется в Gerrit, но когда вы получаете что-то оттуда, вы получаете это из реального репозитория.

5. (Необязательно) Осмотритесь и переключайтесь между ветками

Теперь у вас есть новейший код 4diac FORTE или 4diac IDE. После загрузки кода проекта сначала осмотритесь. Проверьте папки, прочтите документацию и файлы с инструкциями. Постарайтесь разобраться в папках и иерархиях.

Щёлкните правой кнопкой мыши по проекту в проводнике пакетов Eclipse и выберите «Команда». Там вы найдете все возможные команды для EGit. Команда → Показать в истории покажет вам все коммиты в текущей ветке. Обычно основной веткой является master. Если в 4diac FORTE есть другая ветка, и вы хотите работать с ней, вам нужно будет создать локальную ветку, которая будет служить ссылкой на исходную ветку. То есть в репозитории Eclipse есть основная ветка (обозначается как origin/master) и, например, ветка разработки (обозначается как origin/develop). Но у вас локально есть только основная ветка (обозначается как master). Эта локальная ветка master является ссылкой на origin/master, и все изменения, которые вы вносите в неё, будут затем отправлены в origin/master. Но если вы хотите работать с веткой origin/develop, вам нужно создать локальную ветку, которая будет ссылаться на неё. Подробную информацию о ветках можно найти в упомянутой выше [книге](#) (глава 3).

Щёлкните правой кнопкой мыши по проекту → Команда → Переключиться на → Другое.... Там вы увидите локальные и удалённые ветки. Выберите удалённую ветку, с которой вы хотите работать, например origin/develop, и нажмите «Оформить». Вы получите сообщение о том, что вы можете следить за удалённой веткой или создать новую локальную ветку для работы над ней. Нажмите «Оформить как новую локальную ветку», а затем выберите имя для локальной ветки. Обычно имя исходной ветки копируется, поэтому мы назовём её develop (без части «origin/») и нажмём «Готово». Теперь вы можете переключаться между ветками, и изменения в каждой из них будут отражаться в соответствующей исходной ветке.

Проекты следуют описанному [здесь](#) рабочему процессу. По сути, основная ветка используется для выпуска версий и разработки нового контента. Для работы над чем-то новым всегда следует создавать ветку из основной.

Внесите свой вклад с помощью Eclipse и EGit

После того как вы загрузили текущий код, вы можете начать вносить свой вклад.

1. Найдите ошибку, над которой нужно поработать

Чтобы внести свой вклад, вам нужно сначала найти ошибку, над которой нужно поработать. Все ваши изменения должны быть связаны с ошибкой, зарегистрированной в [Bugzilla](#). Вы можете просмотреть код и найти что-то не так или перейти к [списку ошибок](#) и посмотреть те, которые относятся к проекту 4diac FORTE. Если вы обнаружите ошибку или даже нехватку документации, вам следует сообщить об этом. Чтобы узнать, как внести свой вклад, вы можете отредактировать какую-нибудь документацию. Возможно, вы обнаружите опечатку или захотите добавить какие-то детали. Исправление ошибок — это показатель, который демонстрирует, как продвигается разработка программного обеспечения. Но имейте в виду, что при сообщении об ошибке должен быть указан чёткий способ её исправления. Для такой ошибки, как «Нехватка документации», неясно, когда исправление будет завершено. Поэтому укажите, например, какая документация отсутствует. У каждой ошибки в Bugzilla есть идентификатор. Допустим, мы работаем над ошибкой с номером 123456.

2. Внесите изменения и подготовьтесь к коммиту.

Теперь вы действительно меняете код. Откройте файлы, которые хотите изменить, и отредактируйте их. Старайтесь изменять не более 1000 строк за один коммит, чтобы отдельные коммиты были небольшими. Более крупные изменения можно разделить на несколько коммитов.

Откройте представление Git Staging в Eclipse, выбрав «Окно» → «Показать представление» → «Другое...» → «Git» → «Git Staging». В разделе «Незафиксированные изменения» вы увидите все изменённые файлы. Щёлкните по ним правой кнопкой мыши и выберите «Добавить в индекс». Выбранные файлы будут перемещены в раздел «Зафиксированные изменения».

3. Зафиксируйте изменения

Зафиксировать изменения — очень важный шаг. Сначала изменения фиксируются локально. Сообщение о коммите необходимо для отправки в Gerrit. Примеры сообщений приведены ниже. В Gerrit коммит сначала проверяется, чтобы убедиться, что всё в порядке, а затем изменения принимаются. Сообщение о коммите создаётся в представлении Git Staging и состоит из трёх частей: темы, основного текста и нижнего колонтитула. Эти три части разделены пустой строкой.

- Тема должна содержать номер ошибки, над которой вы работаете, в квадратных скобках, а затем следует краткое описание содержимого коммита. Старайтесь, чтобы оно было короче 50 символов, и для лучшей читаемости начинайте сообщение с заглавной буквы. Не ставьте точку в конце темы.
- Основная часть содержит все пояснения к тому, что было сделано. Используйте её, чтобы объяснить, что и почему, но не как.
- Нижний колонтитул должен состоять из двух частей (трёх, если нужен идентификатор изменения):
 - i. [ВСЕГДА] ссылка на ошибку (Bug: https://bugs.eclipse.org/bugs/show_bug.cgi?id=123456)
 - ii. [ТОЛЬКО ДЛЯ ПЕРЕЗАПИСИ КОММИТА] поле Change-Id используется, когда вы добавляете второй коммит к уже отправленному ранее изменению (например: Change-Id: If0b359ad15268ed179194cf5ad4fb5b2d09b3290). См. ПРИМЕЧАНИЕ 2, чтобы узнать больше о том, когда его использовать. Подписывайте с помощью кнопки с кистью, которая находится над сообщением о коммите. Подпись в конце проверяется Gerrit, чтобы убедиться, что коммиттер подписал ЕСА.

Примечание! Пустые строки допустимы (и обязательны) только между темой, текстом и нижним колонтитулом, но не между внутренними частями нижнего колонтитула.

Примечание! Идентификатор изменения не редактируется при первой отправке коммита в Gerrit (CommitOne). Gerrit генерирует идентификатор изменения. Если CommitOne отклонен (из-за ошибочного кода, отсутствия информации и т. д.), вы должны пересмотреть коммит и изменить (перезаписать) CommitOne. Это приведет к созданию CommitTwo. (Вы также можете полностью удалить CommitOne и создать новый коммит под названием CommitTwo.) Помните, что CommitTwo — это совершенно новый коммит. У COMMITWO должен быть идентификатор изменения, сгенерированный Gerrit для CommitOne (Перейдите на веб-страницу Gerrit, найдите изменение CommitOne, и вы увидите его идентификатор изменения). Когда вы нажимаете Committwo, новая запись в Gerrit не создается. Вместо этого COMMITWO появится под CommitOne на той же странице. Теперь COMMITWO ожидает подтверждения.

Пример для CommitOne:

```
+
[123456] Change the initial value of temp variable

The initial value of the temp variable was changed to 1 instead of zero
because of weather conditions
```

Bug: https://bugs.eclipse.org/bugs/show_bug.cgi?id=123456

+ После ввода сообщения нажмите «Отправить».

+ Пример для CommitTwo:+

```
[123456] Change the initial value of temp variable

The initial value of the temp variable was changed to 1 instead of zero
because of weather conditions
```

Bug: https://bugs.eclipse.org/bugs/show_bug.cgi?id=123456

Change-Id: If0b359ad15268ed179194cf5ad4fb5b2d09b3290

+ После ввода сообщения нажмите «Отправить».

4. Отправьте коммит

В разделе «История» вы можете увидеть новый коммит, который вы только что добавили. Чтобы перейти в этот раздел, измените вкладку с «Git Staging» на «История». Пока что коммит хранится только локально, и никто о нём не знает. Пришло время отправить его в репозиторий в Eclipse. Щёлкните правой кнопкой мыши по последнему коммиту → Отправить коммит.... Откроется диалоговое окно. С указанными настройками вы пытаетесь выполнить отправку в ветку разработки Gerrit под названием refs/heads/develop. Но, согласно документации Gerrit, вы должны выполнять отправку в волшебную ветку с названием refs/for/[ветка]. Поэтому в диалоговом окне вместо стандартной ветки разработки выберите refs/for/develop (вместо refs/heads/develop). Нажмите «Далее», и вам будет предложено ввести сгенерированный [выше](#) пароль https Gerrit. Подтверждение отправки уведомляет о создании новой ветки. Не беспокойтесь об этом и нажмите «Готово». Введите пароль ещё раз, и тогда отправка должна пройти успешно. Если что-то пойдёт не так, ошибка появится в журнале.

Вариант 2: Git в командной строке

Используйте Git в командной строке вместо [EGit](#) (этот вариант для безрассудных, рыцарей клавиатуры и особенно для тех, кто боится мышей).

Следующие инструкции применимы к системам Linux на базе Debian, таким как LinuxMint или Ubuntu. Предполагается, что вы уже создали учётную запись Eclipse и подписали ECA, как описано [выше](#). Также применяются те же [правила](#) для создания отчёта об ошибке и работы с кодом 4diac FORTE или 4diac IDE. Конечно, вы сами можете решить, какой редактор использовать для программирования. Существует множество вариантов: VI, Emacs и многие другие. Для компиляции и связывания кода в системе Linux обычно лучше всего подходит набор компиляторов GNU. Но подробности об этом здесь рассматриваться не будут.

1. Проверьте и установите инструменты командной строки Git

Если вы не уверены, что инструменты командной строки Git уже установлены на вашем компьютере с Linux, вы можете ввести следующую команду в дистрибутивах на основе Debian (например, LinuxMint, Ubuntu):

```
john DOE@linuxmint ~/$ dpkg --get-selections | grep git
```

Если инструменты Git уже установлены, вы должны получить ответ, подобный следующему:

```
git      install
git-core install
git-gui  install
git-man  install
git-review install
gitk     install
```

Если вы не видите приведенный выше вывод, вам нужно установить инструменты Git с помощью следующей команды:

```
john DOE@linuxmint ~/$ sudo apt-get update && sudo apt-get install git
```

2. Клонировать репозиторий 4diac FORTE или 4diac IDE в выбранный вами каталог

Сначала создайте каталог в своём домашнем каталоге и перейдите в него. В моём примере это /home/john DOE/develop/repos, но вы можете выбрать другой.

```
john DOE@linuxmint ~/$ mkdir -p develop/repos
john DOE@linuxmint ~/$ cd develop/repos
```

Теперь пришло время клонировать исходники. В следующем примере используется URL-адрес репозитория 4diac FORTE. Для 4diac IDE используйте вместо этого <https://git.eclipse.org/r/4diac/org.eclipse.4diac.ide>.

```
john DOE@linuxmint ~/develop/repos$ git clone
https://git.eclipse.org/r/4diac/org.eclipse.4diac.forte
Cloning into 'org.eclipse.4diac.forte'...
remote: Counting objects: 1, done
remote: Finding sources: 100% (1/1)
Receiving objects: 100% (2283/2283), 1.27 MiB | 458.00 KiB/s, done.
remote: Total 2283 (delta 0), reused 2283 (delta 0)
Resolving deltas: 100% (1445/1445), done.
Checking connectivity... done.
```

После клонирования вы можете просмотреть ветки репозитория, но сначала вам нужно перейти в новый каталог исходных кодов 4diac FORTE, созданный автоматически.

```
john DOE@linuxmint ~/develop/repos$ cd org.eclipse.4diac.forte
john DOE@linuxmint ~/develop/repos/org.eclipse.4diac.forte $ git branch -a
* master
remotes/origin/1.8.x
remotes/origin/HEAD → origin/master
remotes/origin/OPC-UA
remotes/origin/develop
remotes/origin/master
```

«*» указывает на текущую активную ветку. Теперь переключитесь на ветку «develop», потому что именно в неё должны попадать коммиты.

```
johndoe@linuxmint ~/develop/repos/org.eclipse.4diac.forte $ git checkout develop
Branch develop set up to track remote branch develop from origin.
Switched to a new branch 'develop'
```

3. Настройте установку Git для работы с проверкой кода в Gerrit

Вы уже должны были создать учётную запись Eclipse и войти в Gerrit, следуя описанному выше способу. Предположим, что ваш адрес электронной почты — john.doe@example.com, а логин для Gerrit — jdoexy5. Сначала мы настроим это в конфигурации git.

```
johndoe@linuxmint ~/develop/repos/org.eclipse.4diac.forte $ git config --global
user.email "john.doe@example.com"
johndoe@linuxmint ~/develop/repos/org.eclipse.4diac.forte $ git config --global
user.name "jdoexy5"
```

Обратите внимание, что настройки git config --global обычно хранятся в файле конфигурации для конкретного пользователя. Этот файл называется .gitconfig и хранится в вашем домашнем каталоге, а не в репозитории 4diac FORTE. Теперь мы создадим открытый ключ SSH, который вам нужно будет загрузить в свою учётную запись Gerrit позже.

```
johndoe@linuxmint ~/develop/repos/org.eclipse.4diac.forte $ cd ~/.ssh
johndoe@linuxmint ~/.ssh $ ssh-keygen -t rsa -C "john.doe@example.com"
Generating public/private rsa key pair.
Enter file in which to save the key (/home/johndoe/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/johndoe/.ssh/id_rsa.
Your public key has been saved in /home/johndoe/.ssh/id_rsa.pub.
The key fingerprint is:
4d:c7:4f:8f:71:07:89:cb:c9:dc:e5:ad:54:77:9a:64 john.doe@example.com
```

Вы можете просто принять расположение файла ключа по умолчанию, нажав клавишу Return. Фонд Eclipse настоятельно рекомендует использовать парольную фразу для дополнительной безопасности. Теперь скопируйте только что созданный открытый SSH-ключ в свою учётную запись Gerrit на сайте eclipse.org. Просмотрите содержимое файла открытого ключа с помощью следующей команды:

```
johndoe@linuxmint ~/.ssh $ cat id_rsa.pub
```

Скопируйте всё, что отображается в вашем буфере обмена, от начала (включая ssh-rsa) до конца (включая john.doe@example.com). Теперь войдите в свою учётную запись Gerrit на eclipse.org, нажмите на маленькую стрелку рядом с вашим именем пользователя, отображаемым в правом верхнем углу, и выберите «Настройки». В меню слева выберите «Открытые ключи SSH» и нажмите «Добавить ключ...». Теперь вставьте всё из буфера обмена в текстовое поле и нажмите «Добавить». Теперь ваш открытый ключ должен появиться в списке. Давайте проверим, правильно ли Gerrit принимает ваш ключ. Давайте проведём небольшой тест входа по ssh.

```
johndoe@linuxmint ~/.ssh $ ssh -p 29418 jdoexy5@git.eclipse.org
The authenticity of host '[git.eclipse.org]:29418 ([198.41.30.196]:29418)' can't be
established.
RSA key fingerprint is 1a:b6:dc:be:0e:1f:ab:01:70:aa:43:82:4d:54:51:37.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '[git.eclipse.org]:29418,[198.41.30.196]:29418' (RSA) to the
list of known hosts.
```

```
**** Welcome to Gerrit Code Review ****
```

```
Hi John, you have successfully connected over SSH.
```

```
Unfortunately, interactive shells are disabled.
To clone a hosted Git repository, use:
```

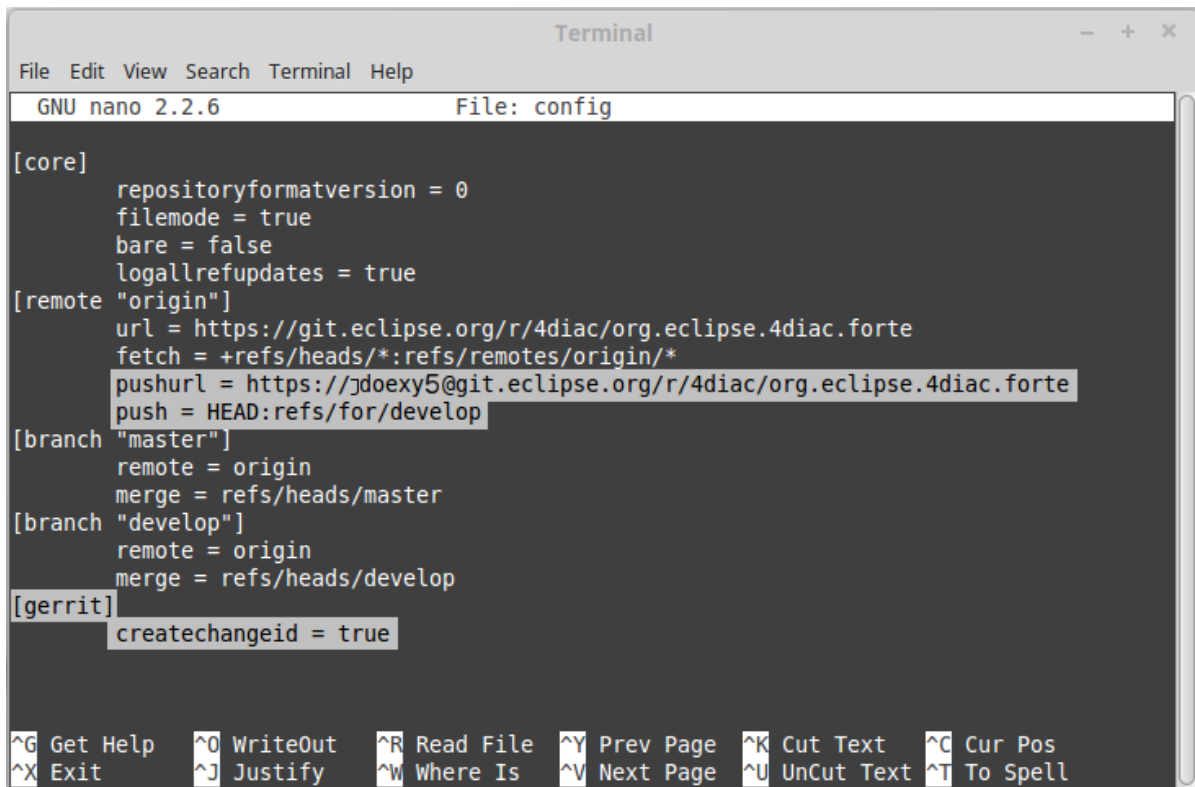
```
git clone ssh://jdoexy5@git.eclipse.org:29418/REPOSITORY_NAME.git
```

```
Connection to git.eclipse.org closed.
```

Вам нужно будет настроить URL-адрес Gerrit Push в конфигурации Git. Перейдите в скрытый каталог Git в репозитории 4diac FORTE и отредактируйте файл с именем "config"

```
johndoe@linuxmint ~/.ssh $ cd ..
johndoe@linuxmint ~/ $ cd develop/repos/org.eclipse.4diac.forte/.git
johndoe@linuxmint ~/develop/repos/org.eclipse.4diac.forte/.git $ nano config
```

На следующем скриншоте показаны части, которые нужно добавить или изменить. После этого сохраните и выйдите.



```
File Edit View Search Terminal Help
GNU nano 2.2.6 File: config

[core]
  repositoryformatversion = 0
  filemode = true
  bare = false
  logallrefupdates = true
[remote "origin"]
  url = https://git.eclipse.org/r/4diac/org.eclipse.4diac.forte
  fetch = +refs/heads/*:refs/remotes/origin/*
  pushurl = https://jdoexy5@git.eclipse.org/r/4diac/org.eclipse.4diac.forte
  push = HEAD:refs/for/develop
[branch "master"]
  remote = origin
  merge = refs/heads/master
[branch "develop"]
  remote = origin
  merge = refs/heads/develop
[gerrit]
  createchangeid = true

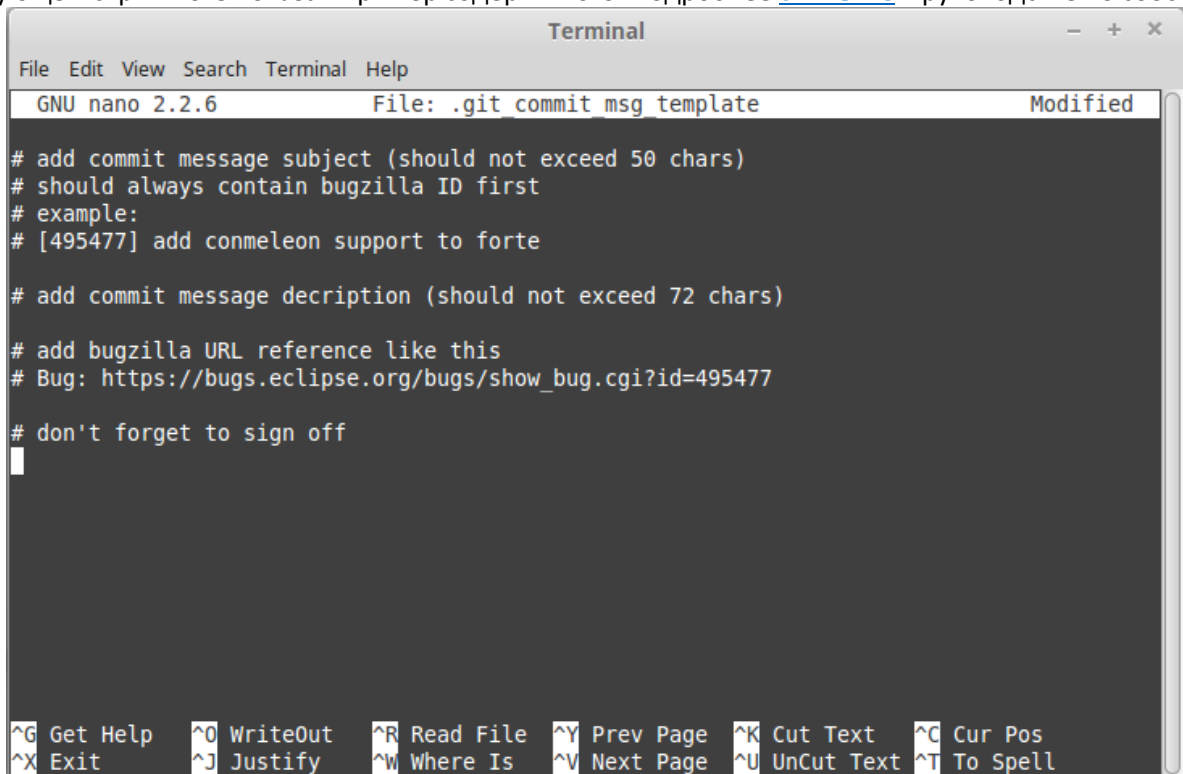
^G Get Help  ^O WriteOut  ^R Read File  ^Y Prev Page  ^K Cut Text  ^C Cur Pos
^X Exit      ^J Justify   ^W Where Is  ^V Next Page  ^U UnCut Text ^T To Spell
```

4. Создайте свой собственный шаблон сообщения о коммите (необязательно)

Вы можете создать файл шаблона сообщения о коммите, который будет использоваться при каждом новом коммите. Вы можете добавлять полезные комментарии, чтобы не забыть важные части сообщения, или даже добавлять текст, который должен быть частью каждого сообщения о коммите. Это зависит только от вас. Перейдите в свой домашний каталог и создайте новый файл с именем `.git_commit_msg_template` в вашем любимом текстовом редакторе. Для удобства я использовал nano.

```
johndoe@linuxmint ~/develop/repos/org.eclipse.4diac.forte/.git $ cd /home/johndoe
johndoe@linuxmint ~/ $ nano .git_commit_msg_template
```

На следующем скриншоте показан пример содержимого. Подробнее [см. выше](#) в руководстве по сообщениям



```
File Edit View Search Terminal Help
GNU nano 2.2.6 File: .git commit msg template Modified

# add commit message subject (should not exceed 50 chars)
# should always contain bugzilla ID first
# example:
# [495477] add conneleon support to forte

# add commit message decription (should not exceed 72 chars)

# add bugzilla URL reference like this
# Bug: https://bugs.eclipse.org/bugs/show_bug.cgi?id=495477

# don't forget to sign off

```

5. Выполните первую фиксацию в командной строке

Новая фиксация всегда должна быть связана с ошибкой в Bugzilla, как уже упоминалось [выше](#). Ошибка также может добавить новые функции в 4diac FORTE. Одна фиксация не должна содержать более 1000 строк кода (да, вы правы, об этом уже упоминалось выше, но лучше повторить). Чтобы проверить, было ли что-то

изменено в ваших локальных источниках 4diac FORTE и нужно ли выполнить фиксацию, используйте команду `git status`. Сначала вам нужно переключиться на локальный репозиторий 4diac FORTE Git.

```
johndoe@linuxmint ~/develop/repos/org.eclipse.4diac.forte $ git status
On branch develop
Your branch is up-to-date with 'origin/develop'.
```

```
Untracked files:
(use "git add file..." to include in what will be committed)
```

```
src/modules/conmeleon_c1/
```

```
nothing added to commit but untracked files present (use "git add" to track)
```

В приведенном выше примере я добавил только пустой каталог, который теперь распознается Git как неотслеживаемый. Если вы хотите добавить новые файлы, просто скопируйте их в локальный репозиторий 4diac FORTE или отредактируйте существующие файлы. Git распознает изменения, и вы увидите файлы с помощью команды «`git status`». Чтобы что-то зафиксировать, сначала нужно добавить соответствующие файлы. Таким образом, файлы будут перемещены в так называемую промежуточную область. Так что пошевелите мозгами и добавьте их правильно.

```
johndoe@linuxmint ~/develop/repos/org.eclipse.4diac.forte $ git add
src/modules/conmeleon_c1/util
johndoe@linuxmint ~/develop/repos/org.eclipse.4diac.forte $ git status
On branch develop
Your branch is up-to-date with 'origin/develop'.
```

```
Changes to be committed:
(use "git reset HEAD file..." to unstage)
```

```
new file: src/modules/conmeleon_c1/util/fileres.cpp
new file: src/modules/conmeleon_c1/util/fileres.h
new file: src/modules/conmeleon_c1/util/uncopyable.h
```

```
Untracked files:
(use "git add file..." to include in what will be committed)
```

```
src/modules/conmeleon_c1/CMakeLists.txt
src/modules/conmeleon_c1/gpio/
src/modules/conmeleon_c1/processinterface.cpp
src/modules/conmeleon_c1/processinterface.h
src/modules/conmeleon_c1/spi/
```

Теперь мы делаем сам коммит. Не забудьте опцию `-s` для автоматической подписи, которая требуется для проекта `org.eclipse.4diac.forte`, и следуйте [инструкциям по созданию сообщения](#), если вы ещё не сделали это в качестве шаблона. После команды `git commit` откроется редактор по умолчанию с шаблоном сообщения о коммите, и вам нужно будет ввести информацию о сообщении (идентификатор Bugzilla, что было изменено и почему, а также URL-адрес записи в Bugzilla).

```
johndoe@linuxmint ~/develop/repos/org.eclipse.4diac.forte $ git commit -s
[develop 3a8de79] [495477] Add conmeleon support to forte
3 files changed, 272 insertions(+)
create mode 100644 src/modules/conmeleon_c1/util/fileres.cpp
create mode 100644 src/modules/conmeleon_c1/util/fileres.h
create mode 100644 src/modules/conmeleon_c1/util/uncopyable.h
```

Пока всё идёт хорошо. Это было не так уж сложно, не так ли? Следующий шаг — отправить код на проверку в Gerrit, и тогда ваш новый код будет тщательно изучен недремлющими глазами рецензента кода проекта.

6. Отправьте код в Gerrit

Это несложно, если вы следовали описанным выше шагам. Вам осталось выполнить всего одну команду, чтобы закончить.

```
johndoe@linuxmint ~/develop/repos/org.eclipse.4diac.forte $ git push origin
HEAD:refs/for/develop
Password for 'https://jdoexy5@git.eclipse.org':
Counting objects: 38, done.
Delta compression using up to 2 threads.
Compressing objects: 100% (8/8), done.
Writing objects: 100% (9/9), 3.84 KiB | 0 bytes/s, done.
Total 9 (delta 3), reused 0 (delta 0)
remote: Resolving deltas: 100% (3/3)
remote: Processing changes: new: 1, refs: 1, done
```

```
remote: -----
remote: Reviewing commit: 3a8de79f
remote: Authored by: jdoexy5 (john.doe@example.com)
remote:
remote: The author is not a committer on the project.
remote: The author has a current Contributor License Agreement (CLA) on file.
remote: The author has "signed-off" on the contribution.
remote:
remote: This commit passes Eclipse validation.
remote:
remote: New Changes:
remote: https://git.eclipse.org/r/74832 [495477] Add conmeleon support to forte
remote:
To https://jdoexy5@git.eclipse.org/r/4diac/org.eclipse.4diac.forte
* [new branch] HEAD → refs/for/develop
```

Вы справились, молодец! Теперь очередь рецензента, и вы увидите его или её комментарии на странице проверки кода в Gerrit.

Что нужно учитывать при внесении изменений:

- При создании новых файлов для проекта в начале добавьте условия авторского права. После года внесения изменений добавьте название. Кроме того, добавьте свое имя под заголовком «Участники» вместе с кратким описанием вашего вклада. Пример кода:

```
{year} {owner}[ and others]
```

```
This program and the accompanying materials are made available under the
terms of the Eclipse Public License 2.0 which is available at
http://www.eclipse.org/legal/epl-2.0.
```

```
SPDX-License-Identifier: EPL-2.0
```

Contributors:

```
{name} - initial API and implementation
```

- Всегда вносите свой вклад в исправление ошибок.
- Укажите номер ошибки в скобках в теме сообщения о коммите, а ссылку на ошибку — в нижнем колонтитуле.
- Не забудьте подписать коммит.
- Используйте [веб-сайт](#) Gerrit, чтобы посмотреть отправленные вами коммиты и их статус.
- Не бойтесь вносить свой код.
- В случае сомнений воспользуйтесь форумами.
- Дополнительную информацию можно найти [здесь](#).

Внешнее событие SIFB

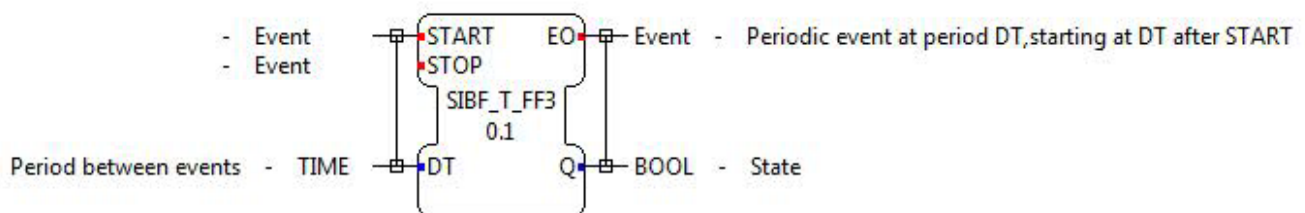
Внешние события — это события в 4diac FORTE, которые скрыты и не отображаются в 4diac IDE. Они не отображаются в виде красных линий событий. Обычно они используются для связи между аппаратным обеспечением и функциональными блоками. Например, E_CYCLE — это функциональный блок, использующий эту функцию. Он регистрируется в классе CTimerHandler для периодического получения внешних событий.

Получение внешних событий

Это расширенный пример, показывающий, как создать собственный функциональный блок интерфейса Т-триггера с таймером. Он основан на блоке E_CYCLE и работает точно так же. В этом руководстве показано, как создать его с нуля. Имейте в виду, что это всего лишь пример, который не соответствует стандарту IEC 61499.

Интерфейс

Т-триггера в качестве SIFB со встроенным таймером. [Экспортируйте интерфейс в 4diac FORTE](#) (файлы .h и .cpp) и [включите их в свою сборку](#).



Отредактируйте заголовочный файл

На этот раз мы хотим создать SIFB с таймером, поэтому мы не наследуем от CFunctionBlock. Вместо этого мы наследуем от CEventSourceFB. CEventSourceFB может реагировать на внешние события. Поэтому вам следует изменить первую строку объявления вашего класса следующим образом:

```
class FORTE_SIFB_T_FF3 : public CEventSourceFB{
```

В этом случае мы хотим получать внешние события от включенного обработчика таймера. Включите 2 файла: timerha.h и resource.h и измените funcbloc.h на esfb.h. Ваши параметры включения должны выглядеть так:

```
#include "timerha.h" //added
#include "resource.h" //added
#include "esfb.h" //instead of #include "funcbloc.h"
#include "forte_time.h"
#include "forte_bool.h"
```

Затем нам нужен новый конструктор, а также два новых частных члена bool mActive; которые будут указывать на то, что блок функций с таймером в данный момент активен.

```
EVENT_SOURCE_FUNCTION_BLOCK_CTOR(FORTE_SIFB_T_FF3){
    setEventChainExecutor(pa_poSrcRes->getResourceEventExecution());
    mActive = false;
};
```

Удалите конструктор по умолчанию FUNCTION_BLOCK_CTOR и замените его на приведенный выше. Этот конструктор регистрирует данный функциональный блок в качестве внешнего прослушателя событий, который запускает этот функциональный блок при возникновении события. Затем настраивается запись в списке таймеров.

Отредактируйте исходный файл

Затем вам нужно отредактировать метод executeEvent. Обычные события запуска и остановки зарегистрируют этот блок в CTimerHandler. Добавьте новое событие cgExternalEventID, которое определено в CEventSourceFB, и настройте его выходы. Оно будет срабатывать, если ваш функциональный блок получит внешнее событие.

```
void FORTE_SIFB_T_FF3::executeEvent(TEventID paEIID, CEventChainExecutionThread *const
paECET){
    switch(paEIID){
        case cg_nExternalEventID:
            Q() = !Q();
            sendOutputEvent(scm_nEventEOID, paECET);
            break;
        case scm_nEventSTOPID:
            if(m_bActive){
                CTimerHandler::sm_poFORTETimer->unregisterTimedFB(this);
```

```

        m_bActive = false;
    }
    break;
case scm_nEventSTARTID:
    if(!m_bActive){
        CTimerHandler::sm_poFORTETimer->registerPeriodicTimedFB(this, DT());
        m_bActive = true;
    }
    break;
default:
    break;
}
}

```

Скомпилируйте 4diac FORTE и запустите его. Протестируйте свой новый функциональный блок с помощью тестера. Если что-то пошло не так, сравните свой код со следующими файлами.

Отправка внешних событий

Чтобы отправлять внешние события, ваш класс должен наследоваться от CExternalEventHandler. В конструкторе вы должны зарегистрировать этот класс:

```
m_nExtEvHandID = sm_poDeviceExecution->registerExternalEventHandler(this);
```

Затем вы можете отправлять внешние события с помощью:

```

if(sm_poDeviceExecution->extEvHandlerIsAllowed(m_nExtEvHandID)) {
    sm_poDeviceExecution->startNewEventChain(pointerToTargetFB);
}

```

Обеспечение качества кода 4diac FORTE

Сокращение количества ошибок и поддержание высокого качества кода — это постоянная борьба. Для 4diac FORTE мы применяем несколько средств и инструментов, которые мы обобщаем здесь, поскольку считаем, что они могут быть полезны пользователям, планирующим расширять 4diac FORTE.

Статический анализ кода

Статические анализаторы кода, как следует из названия, выполняют автономный анализ вашего кода и пытаются выявить потенциальные проблемы. Для 4diac FORTE мы в настоящее время используем следующие два статических анализатора кода с открытым исходным кодом:

- Cpp Check: это инструмент, который вы можете напрямую использовать для проверки своего кода. Однако для Eclipse также существует плагин Eclipse, который вы можете установить с Eclipse market. Что делает его использование намного более удобным. Проверка Cpp помогает, например, определить, используются ли правильные пары new/delete, а также возможные утечки памяти.
- Clang's scan-build: scan-build — это инструмент, поставляемый с компилятором C++ clang. Он использует компилятор clang для выполнения статического анализа во время компиляции.

Чтобы использовать его для проверки 4diac FORTE, сначала нужно изменить C на clang's ccc-analyzer, а компилятор C на clang's 'c-analyzer' во время настройки CMake для 4diac FORTE. Поэтому рекомендуется использовать отдельный каталог для сборки.

Кроме того, вместо запуска

```
make all
```

что вам нужно для вызова команды.d

```
scan-build -o ./static make
```

для сборки 4diac FORTE. Преимущество scan-build в том, что он создаёт красивые HTML-файлы с пояснениями о найденных проблемах.

Динамический анализ

Инструменты динамического анализа помогают находить проблемы во время выполнения. Типичные проблемы связаны с памятью, например, утечки памяти, выход за пределы массива, переполнение/недостаточное заполнение стека или использование памяти после её освобождения.

- Valgrind: Valgrind, безусловно, является наиболее известным инструментом динамического анализа. Он проверяет в основном проблемы с памятью. В настоящее время он доступен только для Linux и только для платформ Intel. Главный недостаток Valgrind заключается в том, что он значительно замедляет выполнение тестируемого приложения.
- Sanitizers: Sanitizers — это средства проверки во время выполнения, которые компилятор добавляет в ваше приложение. Их можно активировать с помощью переключателей компилятора, и их большое преимущество в том, что в настоящее время они работают намного быстрее, чем Valgrind. Однако они доступны только для Clang и gcc. Для последнего — только частично. В настоящее время мы работаем со следующими средствами проверки:
- Address: проверяет наличие проблем с памятью и кучей. Можно активировать, добавив `-fsanitize=address` в параметр CMake CMAKE_CXX_FLAGS.
- Undefined Behavior: проверяет наличие неопределённого поведения в вашем коде (например, переполнения, деления на ноль). Можно активировать, добавив `-fsanitize=undefined` в параметр CMake CMAKE_CXX_FLAGS.
- Thread: тестируется

Модульные тесты

Мы использовали фреймворк модульных тестов, предоставляемый библиотекой Boost, под названием Boost Test. Текущий набор доступных модульных тестов можно найти в каталоге test. Новые модульные тесты всегда приветствуются.

Установите Boost Test

Для выполнения модульных тестов требуется как минимум [Boost 1.59](#). Скачайте и распакуйте его.

Linux

Проверьте

```
wget -O boost_1_63_0.tar.gz
http://sourceforge.net/projects/boost/files/boost/1.63.0/boost_1_63_0.tar.gz/download
tar -xzf boost_1_63_0.tar.gz cd boost_1_63_0/
./bootstrap.sh gcc
sudo ./b2 install --prefix=installation/path toolset=gcc
```

Windows

Подробные инструкции по установке в Windows можно найти в [этом](#) или [этом](#) блоге. В противном случае следуйте этому краткому руководству для cygwin:

1. откройте командную строку cygwin и перейдите в распакованную папку
2. Запустите bootstrap.bat и укажите набор инструментов. Поддерживаемые наборы инструментов: borland, como, gcc, gcc-посыгwin, intel-win32, metrowerks, mingw, msvc, vc7, vc8, vc9, vc10, vc11. Используйте следующую команду для cygwin
3. определите каталог установки и укажите набор инструментов. Наборы инструментов здесь немного отличаются от тех, которые мы использовали раньше. Следующая команда создает две папки: include и lib. Впоследствии в обеих папках должны быть файлы, что занимает много времени.

```
./b2 install --prefix=c:/installation/path toolset=gcc
```

Настройте CMake для 4diac FORTE

Настройте следующую конфигурацию в CMake:

```
FORTE_TESTS=ON
FORTE_TESTS_INC_DIRS=path to generated include files
FORTE_TESTS_LINK_DIRS=path to generated libraries
```

В Windows необходимо указать каталоги заголовков и ссылок. В Linux эти каталоги не нужно указывать, если boost был установлен с помощью `sudo apt install`, но при загрузке архива и выполнении сценария оболочки, как описано выше, их также необходимо указать. Цель сборки «all» генерирует исполняемый файл `forte_test` в каталоге тестов вашего каталога сборки, который выполняет все тесты. Если вам нужны результаты анализа покрытия кода, также проверьте параметр `FORTE_TEST_CODE_COVERAGE_ANALYSIS` и создайте в Eclipse цель сборки под названием `TestCoverage`. Цель сборки `TestCoverage` генерирует папку с html-файлами, содержащими результаты анализа покрытия. Если возникнет ошибка, раскомментируйте

```
line 133: #ADD_DEFINITIONS (-fsanitize=address)
line 166: remove -fsanitize=address
```

Тесты для функциональных блоков

На основе Boost Test мы разработали вспомогательный класс, который позволяет писать модульные тесты для функциональных блоков на C++. Примеры использования можно найти в тестовых сценариях в каталоге `test/stdfblib`. Напишите положительные и отрицательные тестовые сценарии для своих модульных тестов.

Архитектура связи

Многоуровневый сетевой интерфейс

Сетевой интерфейс спроектирован таким образом, чтобы быть максимально гибким. Это означает, что интеграция новых протоколов должна быть простой в реализации, а повторное использование реализованных протоколов должно быть максимально возможным. В основе сетевого интерфейса лежит реализация шаблона проектирования уровней. Каждый уровень выполняет роль одного из уровней OSI. Интерфейс между уровнями определяется для всех уровней. Таким образом, в принципе можно реализовать больше или меньше уровней из модели OSI.

Обзор сетевого интерфейса показан на рисунке 1. На рисунке показаны различные типы классов и их базовое взаимодействие. Класс функционального блока (CCommFB) — это единственный класс, который взаимодействует с приложениями IEC 61499 и должен иметь возможность отправлять и получать события IEC 61499. Класс функционального блока и классы уровней взаимодействуют с помощью следующих функций, подробное описание которых приведено в этом разделе:

- `openConnection`: вызывается функциональным блоком для открытия соединения, должен быть реализован на каждом уровне и в сетевом стеке.
- `closeConnection`: вызывается функциональным блоком для закрытия соединения, должен быть реализован на каждом уровне и в сетевом стеке.
- `sendData`: вызывается функциональным блоком для отправки сообщения по сети, должен быть реализован на каждом уровне и в сетевом стеке.
- `recvData`: вызывается при получении новых сообщений из сети, должен быть реализован на каждом уровне.
- `processInterrupt`: вызывается из сетевого стека, когда слою необходимо обработать данные.

Настройка функциональных блоков

Существует множество вариантов реализации идентификатора. Очень удобный способ указания протокола используется в URL-адресах веб-страниц: веб-браузер использует первую часть URL-адреса для определения протокола, например `http://` или `ftp://`. Чтобы можно было указывать больше протоколов, необходим более гибкий способ их указания. Следующая спецификация позволяет указать все уровни модели OSI.

- `protocol`: строка для идентификации протокола
- `parameter`: строка, определяемая поставщиком
- `ID`: ``{protocol[parameter].}protocol[parameter]`

В следующем примере эта спецификация используется для описания соединения с кодировкой ASN.1 и TCP-соединением в качестве транспортного уровня:

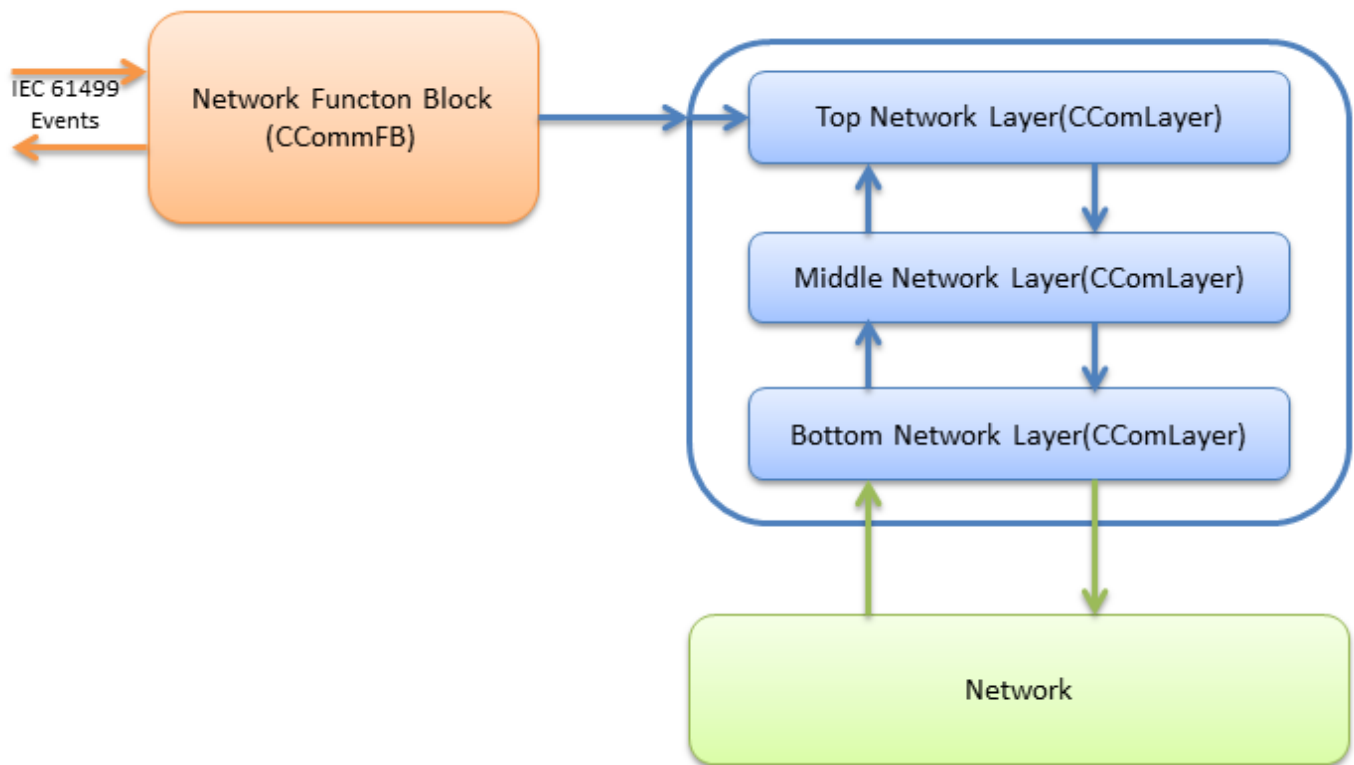
```
fbdk[] .ip[192.168.20.1:61499]
```

Имена слоёв указаны в файле `comlayersmanager.cpp`, который будет сгенерирован в соответствии с настройками в CMake.

Сетевой уровень

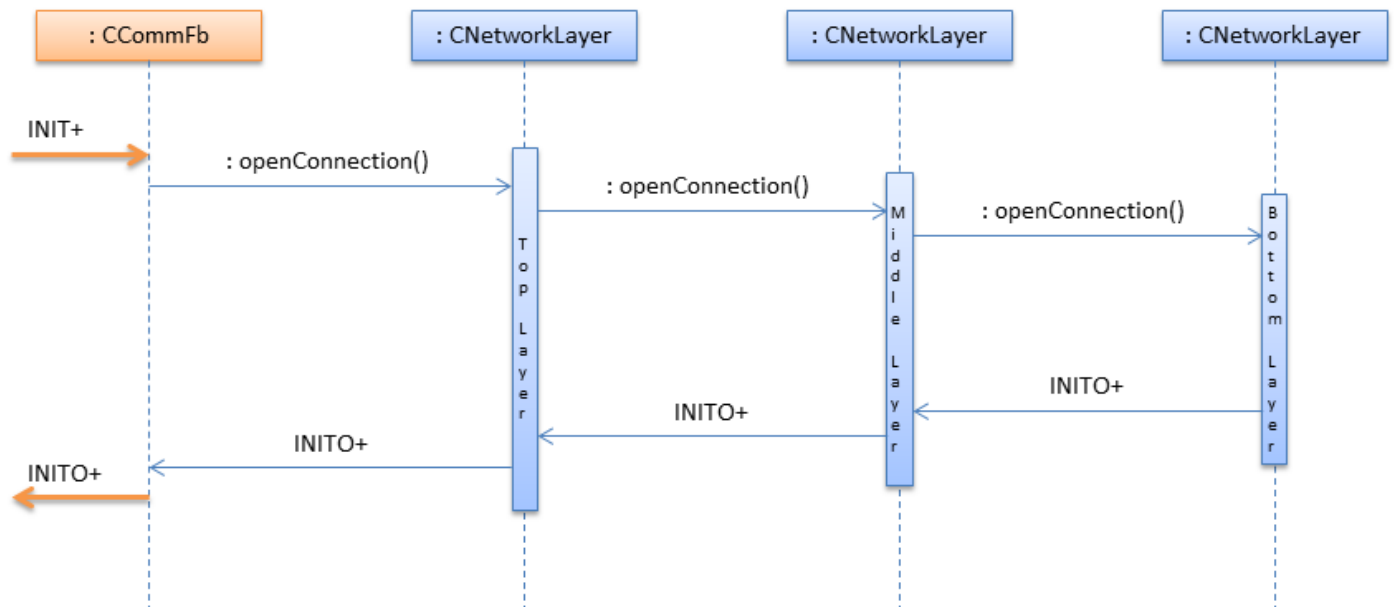
Один сетевой уровень соответствует одному уровню OSI. Поскольку нет информации о том, какой уровень будет реализован, для всех уровней был выбран общий интерфейс. Общий интерфейс определяет только базовые функции: открытие соединения, закрытие соединения, отправка данных, получение данных и общие прерывания. Функции подробно описаны в следующих разделах. Уровень также может выполнять три функции: верхний уровень, средний уровень и нижний уровень.

- **Верхний уровень:** Верхний уровень может напрямую обращаться к портам данных функционального блока, а функция отправки используется только для указания того, что все порты данных готовы к отправке.
- **Средний уровень:** Для среднего уровня должны быть реализованы все функции, определенные в универсальном интерфейсе. В большинстве случаев средний уровень добавляет к сообщению некоторую дополнительную информацию, такую как кодировка, сжатие, информация о сеансе и т.д.
- **Нижний уровень:** нижний уровень может напрямую обращаться к сети. Нижний уровень также должен иметь возможность асинхронно получать данные из сети. В большинстве случаев нижний уровень не реализует функцию `recvData`, так как она обрабатывается непосредственно сетевым интерфейсом.



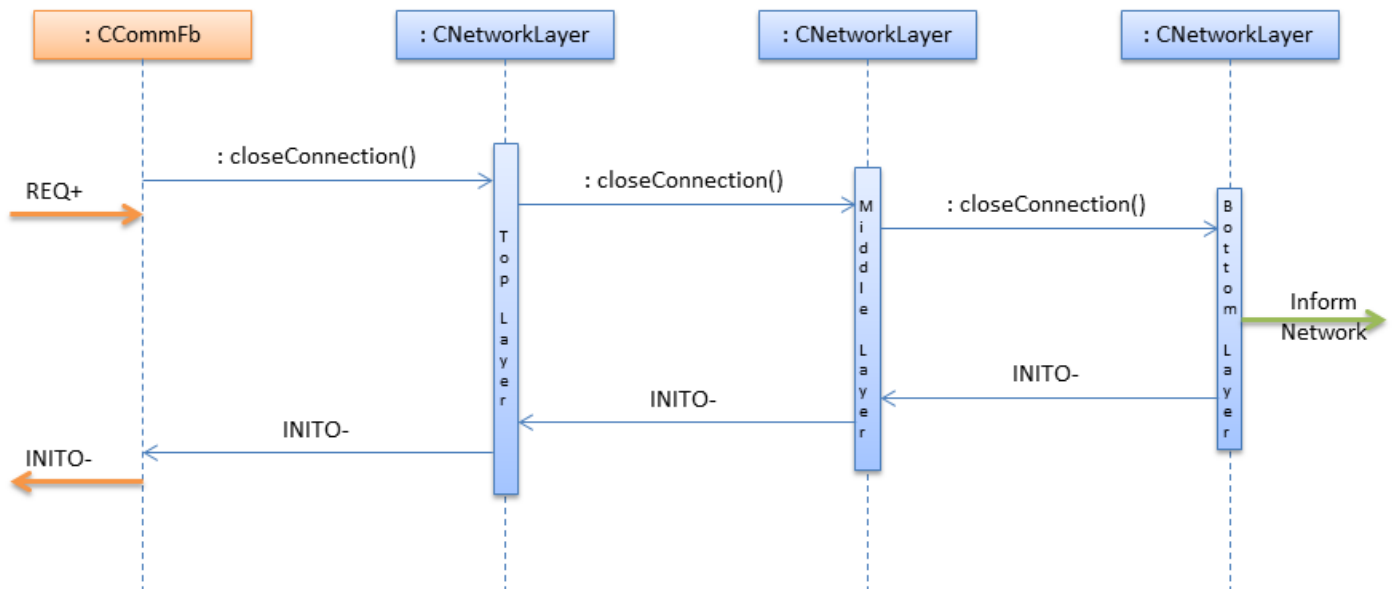
Открыть соединение

Функция слоя `openConnection` используется для инициализации текущего слоя. Слой получает два типа информации: первый — параметр для инициализации, второй — для создания нижнего слоя. Слой ничего не знает о слое, находящемся под ним, поэтому фабричный класс должен создать нижний слой. Нижний слой создаётся с помощью второго параметра функции `openConnection`, если он равен 0, то ниже не будет создано ни одного слоя. Функция `openConnection` вызывается, когда функциональный блок получает событие `INIT+`. Функция также может завершиться ошибкой, например, если сетевое устройство не готово.



Закреть соединение

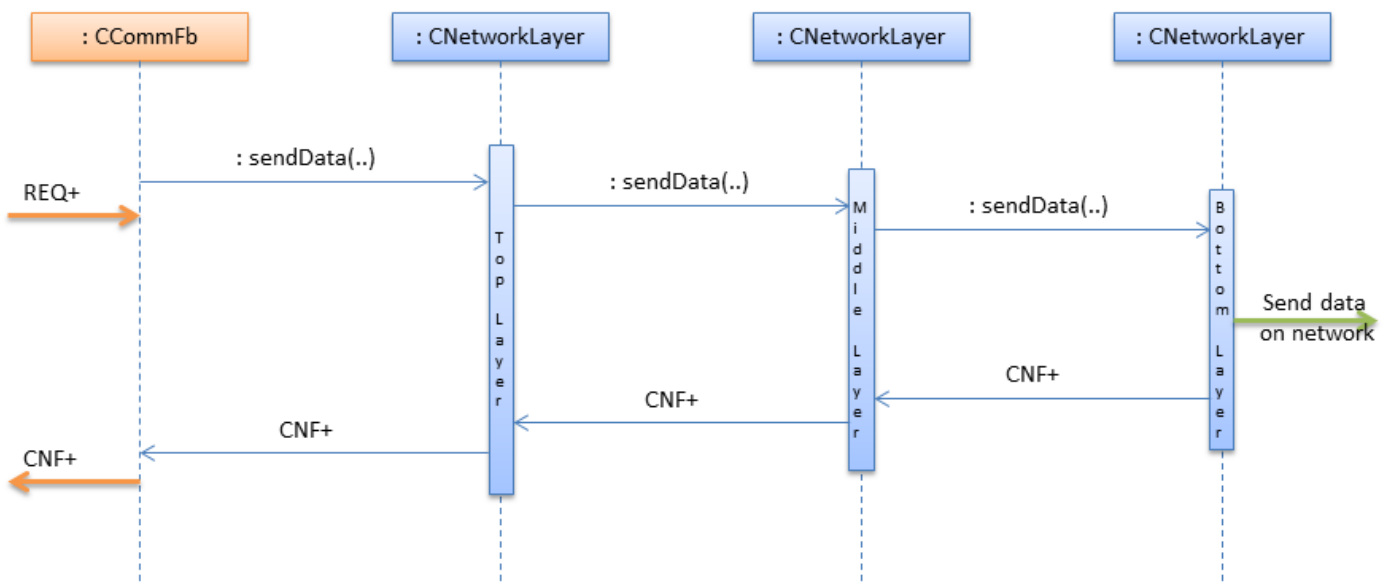
Функция `closeConnection` зависит от функции `openConnection`. Функция закрывает все нижележащие слои, а затем закрывает собственное соединение.



Отправка данных

Эта функция может быть реализована тремя способами: как верхний, средний или нижний уровень. Основная функция этой функции — получать данные с верхнего уровня, упаковывать их и отправлять на нижний уровень. Средний уровень — самый гибкий, так как в него можно добавлять дополнительные средние уровни практически в любой стек.

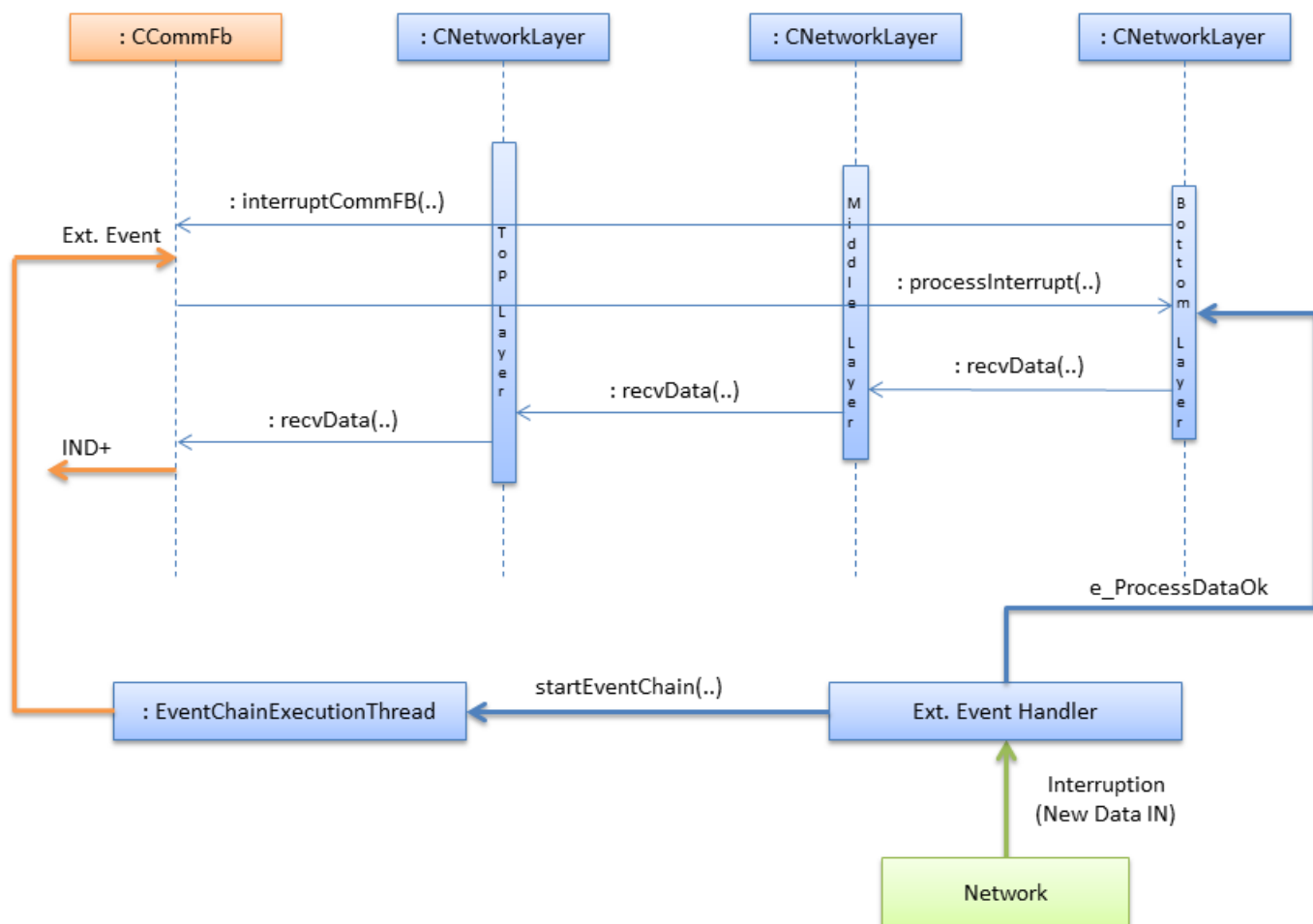
- Реализация верхнего уровня отличается только тем, кто и как вызывает функцию. Верхний уровень вызывается из функционального блока и должен получать информацию непосредственно из функционального блока.
- Нижний уровень должен записывать данные в сеть. Поэтому функция `sendData` не будет вызывать другую функцию `sendData`, как на верхнем или среднем уровне. В большинстве случаев данные отправляются операционной системой с помощью вызова функции API операционной системы.
- Сообщение передаётся с одного уровня на другой, пока не будет достигнут последний уровень. Последний уровень (нижний уровень) отправляет сообщение в сеть и возвращает событие подтверждения в функциональный блок.



Получение данных

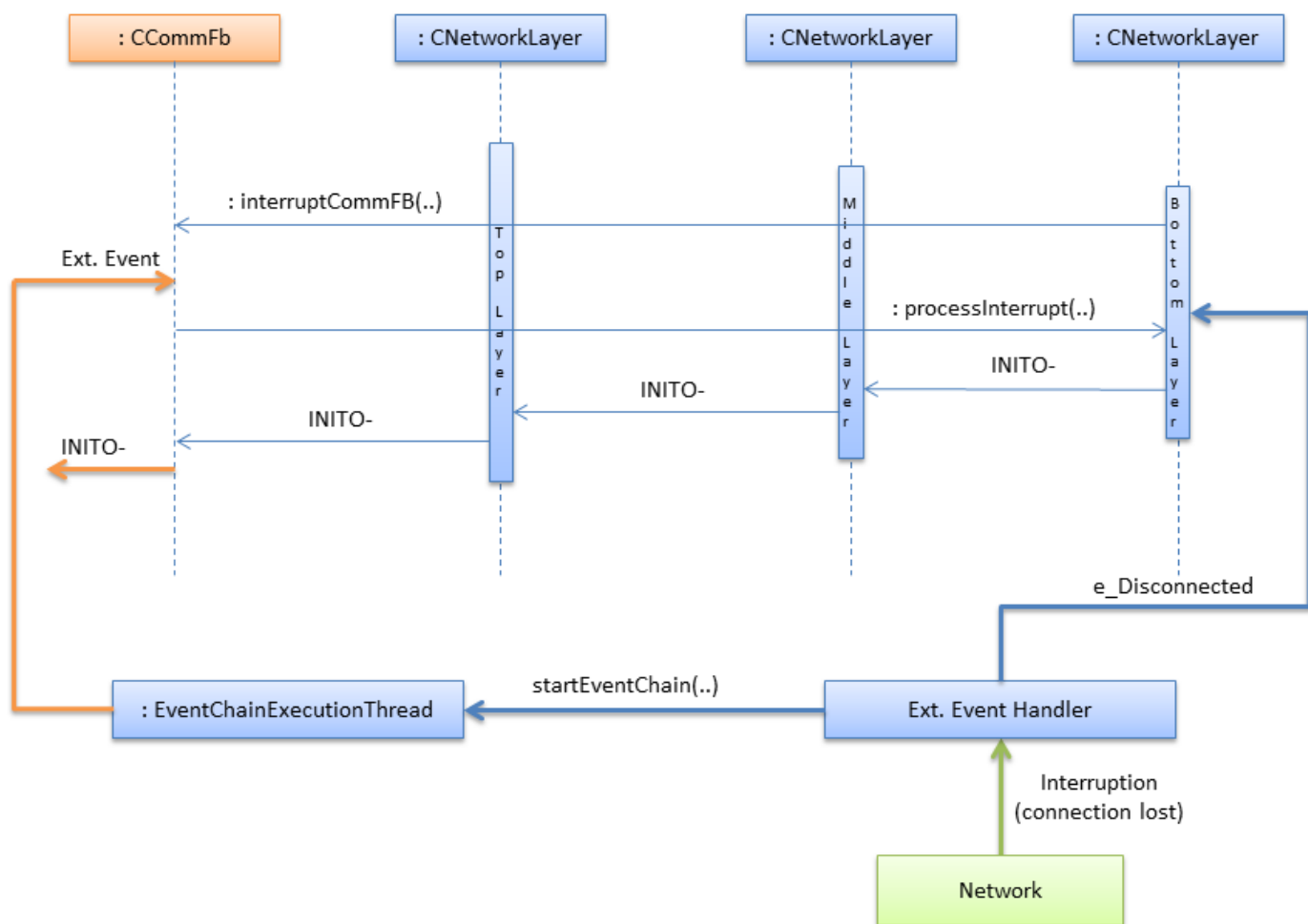
Функция `recvData` разделена на те же части, что и функция `sendData`. Верхний и средний уровни аналогичны функции `sendData`, только нижний уровень требует дополнительного обсуждения. Проблема нижнего уровня заключается в том, что система должна генерировать прерывание. Сообщение из сети может быть обнаружено с помощью прерывания, опроса или системного вызова, который ожидает получения новых данных. Поток выполнения функционального блока нельзя использовать, потому что этот поток используется для выполнения функциональных блоков. Если бы использовался поток функциональных блоков, то

функциональные блоки не выполнялись бы во время ожидания события из сети. Чтобы решить эту проблему, 4diac FORTE реализует обработчик внешних событий, который отвечает за обработку внешних прерываний и запуск потоков выполнения цепочки событий. На рисунке 6 показан приём данных на основе прерывания. Сначала сеть вызывает прерывание в обработчике внешних событий. Это событие передаётся на нижний уровень и запускает цепочку событий выполнения. Затем нижний уровень вызывает поток функционального блока, вызывая функцию `interruptCommFB(..)`. Как только функциональный блок получает внешнее событие, он вызывает функцию `processInterrupt` сетевого уровня. Прерывание процесса обычно выполняется на нижнем или среднем уровне в зависимости от того, как обрабатываются данные. После этого функция `recvData(..)` будет вызываться от нижнего к верхнему уровню до тех пор, пока функциональный блок не получит данные и не отправит сигнал `CNF+/IND+`



Универсальное прерывание

Получение данных — это частный случай универсального прерывания. Универсальное прерывание также можно использовать для получения информации об ошибках в сети, например о потере соединения. Как и в примере с получением данных, обработчик внешнего события информирует нижний сетевой уровень, который вызывает прерывание функционального блока (или в некоторых случаях промежуточного уровня) и запускает цепочку событий. Затем, после получения сигнала внешнего события, функциональный блок отправляет `processInterrupt()` на сетевой уровень, который возвращает событие `INIT-`, указывающее на потерю соединения.



Функциональные блоки

Реализация функциональных блоков основана на профиле соответствия требованиям. Она проста, единственная реализация — это правильная обработка событий ввода/вывода и создание экземпляров функциональных блоков с переменным количеством входных/выходных данных. Обработка событий описана в этом разделе. Это часть, которая напрямую взаимодействует с сетевым интерфейсом.

Несмотря на то, что имена событий в функциональных блоках различаются, сетевой интерфейс поддерживает только две формы событий: события инициализации и события запросов. События можно разделить на две категории: входные/выходные события и успешные/неуспешные события.

Event name	I/O names	Direction	Description
Init Positive	INIT+	Input	Открыть новое соединение
Init Negative	INIT-	Input	Закрыть соединение или завершить соединение
Request Positive	REQ+/RSP+	Input	Отправить данные
Request Negative	REQ-/RSP-	Input	Игнорировать значение
Init Positive	INITO+	Output	Установлено новое соединение
Init Negative	INITO-	Output	Соединение закрыто
Confirm Positive	CNF+/IND+	Output	Доступны новые данные
Confirm Negative	CNF-/IND-	Output	Полученные данные недействительны или ошибка соединения без закрытия соединения

События в сетевом интерфейсе

Направление события от функционального блока к сетевому интерфейсу - это простой вызов функции. Положительное событие Init приводит к вызову openConnection, отрицательное событие Init приводит к вызову CloseConnection. Положительный вызов запроса используется либо для разрешения новых событий

получения, если используется на сервере или подписчике, либо для отправки вызова отправки, если используется на клиенте или издатель. Отрицательное событие запроса не обрабатывается, поскольку оно указывает на недопустимые данные на входном порту функционального блока.

События сетевого интерфейса

Существует два типа событий, которые запускает сетевой интерфейс: в ответ на входное событие или асинхронно, когда сетевой интерфейс запускается сам.

В большинстве случаев входное событие приводит к соответствующему выходному событию, например, входное событие Init Positive запускает выходное событие Init Positive сразу после установления соединения. Для упрощения создания событий все функции, описанные в разделе «Сетевой уровень», могут возвращать событие с соответствующим сообщением о состоянии. Поддерживаемые в настоящее время события перечислены в таблице. Возвращаемые события имеют приоритет, поскольку возможно, что один уровень возвращает положительное значение, а нижний уровень возвращает отрицательное значение: в этом случае возвращается наихудший вариант. Также возможно не возвращать ни одного события. Это используется, если сетевой интерфейс не вызывает никаких событий.

События, возвращаемые вызовом функции уровня, упорядочены по возрастанию приоритета:

Event	Symbol	Description
No Event	e_Nothing	Событие не запущено
Init Positive	e_InitOk	Соединение установлено
Process Data Positive	e_ProcessDataOk	Обработка данных прошла успешно
Init Invalid ID	e_InitInvalidId	Некорректный идентификатор при открытии соединения
Init Terminated	e_InitTerminated	Соединение закрыто.
Invalid Object	e_ProcessDataInvalidObject	Сбой обработки данных из-за неверного объекта
Data Type Error	e_ProcessDataDataTypeError	Сбой обработки данных из-за ошибки типа данных
Data Inhibited	e_ProcessDataInhibited	Попытка отправить данные без разрешения
Socket Error	e_ProcessDataNoSocket	Сбой обработки данных из-за отсутствия сокета
Data Send Failed	e_ProcessDataSendFailed	Данные были отправлены некорректно
Data Receive Failed	e_ProcessDataRecvFailed	Данные были получены некорректно

Второй способ создания выходного события — это событие, асинхронно запускаемое сетевым интерфейсом. Этот вопрос рассматривается в шаблоне проектирования прерываний, описанном в разделе «[Получение данных](#)». Асинхронное событие может быть запущено в двух случаях: при получении данных или при возникновении ошибки в сети. Выходное событие запускается по завершении функции stackProcess.

Сетевой стек

Сетевой стек — это контейнер уровней, который используется функциональным блоком для отправки сообщений в сеть. По сути, это интерфейс функционального блока, который отправляет данные на сетевой интерфейс. Для сетевых уровней стек является интерфейсом для доступа к функциональному блоку и его ресурсам. Сетевой стек отвечает за получение потока функционального блока и корректное выполнение процедур прерывания.

На данный момент сетевой стек используется для выполнения описанных функций. Сетевой стек также используется для расширения функций, если слоям необходимо управлять ресурсами. Возможной функцией является блок управления памятью, в котором все слои запрашивают память из единого пула. Это обеспечивает более точный контроль использования памяти каждым сетевым стеком.

Фабрика слоёв

Невозможно создать экземпляр универсального стека без настройки инженером, поскольку это означало бы потерю гибкости. Поэтому стек должен создаваться во время выполнения. Это означает, что неизвестно, какие слои будут созданы. Общий шаблон проектирования для решения такой задачи — это фабрика.

Фабрика не только создаёт новый слой, но и инициализирует его. Вышестоящий слой получает информацию от нового слоя, расположенного под ним, и новый слой инициализируется строкой параметров и соответствующим сетевым стеком. Эта структура слоёв представляет собой древовидную структуру. У каждого слоя есть только один вышестоящий слой, но под ним может быть несколько слоёв.

Классы и методы:

CComLayersManager:

Создает новые соединения по запросу FB и реализует шаблон проектирования «фабрика».

Методы:

- `static CComLayer* createCommunicationLayer(char paLayerIdentifier, CComLayer paUpperLayer, CCommFB * paComFB)`: Создает сетевой уровень. Параметр `paLayerIdentifier` используется для создания всех нижних уровней, `paUpperLayer` используется для отправки данных на вышестоящий уровень, а `paComFB` является ответственным функциональным блоком.

CComLayer:

Этот класс реализует один уровень сетевого стека. В большинстве случаев этот класс упаковывает или распаковывает данные в более крупный фрейм. Последний уровень обеспечивает доступ к сети. На последнем уровне функция `recvData` не используется.

Методы:

- `EComResponse openConnection(char *paConnectionParams, char *paRemainingConnectionID)`: Эта функция настраивает данный уровень и при необходимости создает необходимые нижние уровни. Параметр `paConnectionParams` используется для настройки реализуемого слоя, а `paRemainingConnectionID` идентифицирует данные конфигурации слоёв, расположенных ниже. Если 0 или `paRemainingConnectionID` равно '0x00', то ниже этого слоя нет других слоёв. Возвращает статус процесса открытия (`e_InitOk`).
- `virtual void closeConnection() = 0`: Виртуальная функция. Реализация этой функции должна выполнять действия, необходимые для закрытия слоя, а затем вызывать `closeConnection()` нижнего слоя.
- `virtual EComResponse sendData(void *paData, unsigned int paSize) = 0`: Виртуальная функция. Принимает заданные данные и выполняет необходимый процесс отправки данных. Параметр `paData` — это указатель на отправляемые данные, а `paSize` — размер данных. Возвращает статус процесса отправки.
- `virtual EComResponse recvData(const void *paData, unsigned int paSize) = 0`: Виртуальная функция. Принимает заданные данные и выполняет необходимый процесс для получения данных. Параметр `paData` — это указатель на полученные данные, а `paSize` — размер полученных данных. Возвращает статус процесса получения.
- `virtual EComResponse processInterrupt()`: Виртуальная функция. Используется для завершения получения данных в контексте, отличном от прерывания связи.

CCommFB:

Этот класс представляет собой функциональный блок, который использует сетевой стек для отправки и получения данных. Функциональный блок должен иметь возможность запускать новую цепочку событий, иначе будет невозможно переключаться между потоком прерывания и потоком выполнения ресурса. Здесь мы приводим лишь несколько методов, для получения дополнительной информации обратитесь к исходному коду.

Методы:

- `void interruptCommFB(CComLayer *paComLayer)`: Запускает новую цепочку событий. Эта функция используется для переключения потока из вызова `interruptCommFB()` в поток, выполняющий ресурс, в котором создан функциональный блок.

- `getFBTypeId(void)`: Этот метод используется командой Query для получения правильного названия типа экземпляра (например, «CLIENT_3_2»).
- `EComServiceType getComServiceType()`: Возвращает значение, указывающее, является ли FB издателем, подписчиком, сервером или клиентом.

Разработка — мониторинг

DTD будет добавлен после завершения всех работ по внедрению и стабилизации интерфейса для всех команд. В настоящее время поддерживаются следующие команды мониторинга:

- **AddWatch:** где {0} — это идентификатор, {1} будет заменен на FB-Instance-Name.Portname, а {2} — на '*'

```
<Request ID="\{0}" Action="CREATE">
  <Watch Source="\{1}" Destination="\{2}" />
</Request>
```

- **Удалить наблюдение:** где {0} — это идентификатор, {1} будет заменен на `FB-Имя-экземпляра.Имя-порта, а {2} — на '*'

```
<Request ID="\{0}" Action="DELETE">
  <Watch Source="\{1}" Destination="\{2}" />
</Request>
```

- **Прочитать наблюдения:** запрашивает текущие значения наблюдаемых элементов

```
<Request ID="\{0}" Action="READ">
  <Watches/>
</Request>
```

- **Ответное сообщение Read_Watches:**

```
<Resource name="ResName">
  <FB name="fbName">
    <Port name="portname">
      <--if variable port -->
      <Data value="value" forced="true | false">
      </Data>
      <-- if event port -->
      <Data value="value" time="time">
      </Data>
    </Port>
  </FB>
</Resource>
```

- **ForceValue:** где {0} — идентификатор, {1} — значение, {2} будет заменено на FB-Instance-Name.Portname, а {3} — на значение true или false, чтобы удалить принудительное значение

```
<Request ID="\{0}" Action="WRITE">
  <Connection Source="\{1}" Destination="\{2}" force="\{3}" />
</Request>
```

- **TriggerEvent:** где {0} - идентификатор, {1} будет '\$e', а {2} - имя экземпляра FB.Имя_события

```
<Request ID="\{0}" Action="WRITE">
  <Connection Source="\{1}" Destination="\{2}" />
</Request>
```

Обновление целевой платформы, используемой 4diac IDE

Для перехода на новые версии платформы Eclipse необходимо выполнить следующие действия:

- Обновить org.eclipse.fordiac.ide.product.target в плагине org.eclipse.fordiac.ide.product

- Перейдите по ссылке <https://github.com/eclipse-4diac/4diac-ide/blob/develop/plugins/org.eclipse.fordiac.ide.product/org.eclipse.fordiac.ide.product.target>
- Откройте его в текстовом редакторе
- Просмотрите все записи, проверьте наличие новых URL-адресов и новых версий
- В конце откройте `org.eclipse.fordiac.ide.product.target` в редакторе целей, чтобы убедиться, что все обновления работают. При необходимости устраните проблемы
- Обновите `pom.xml` в корневом каталоге 4diac IDE. Необходимо выполнить как минимум следующие действия:
 - Проверьте `tycho.version`
 - Обновите `tycho-repo.url` до новой целевой версии платформы
 - Проверьте диапазон версий `org.eclipse.equinox.executable`. Обратите внимание, что это нужно сделать в двух местах в файле. Для проверки версии перейдите на страницу [обзора последней версии](#). Там проверьте исполняемые файлы Eclipse Platform Launcher.
- Регенерируйте весь код модели и редактирования модели из подключаемого модуля моделей.
- Запустите продукт из Eclipse IDE с помощью редактора продукта, то есть нажмите кнопку синхронизации, а затем «Запустить приложение Eclipse»
- Запустите Maven для установки `pom.xml` в корневом каталоге исходного кода 4diac IDE локально
- Проверьте, можно ли выполнить сгенерированный продукт. Сгенерированные продукты находятся в:


```
`/org.eclipse.4diac.ide/plugins/org.eclipse.fordiac.ide.product/target/products/org.eclipse.fordiac.ide.produ
ct
```
- Отправьте в Gerrit, чтобы CI-сборщик также мог проверить, работает ли он

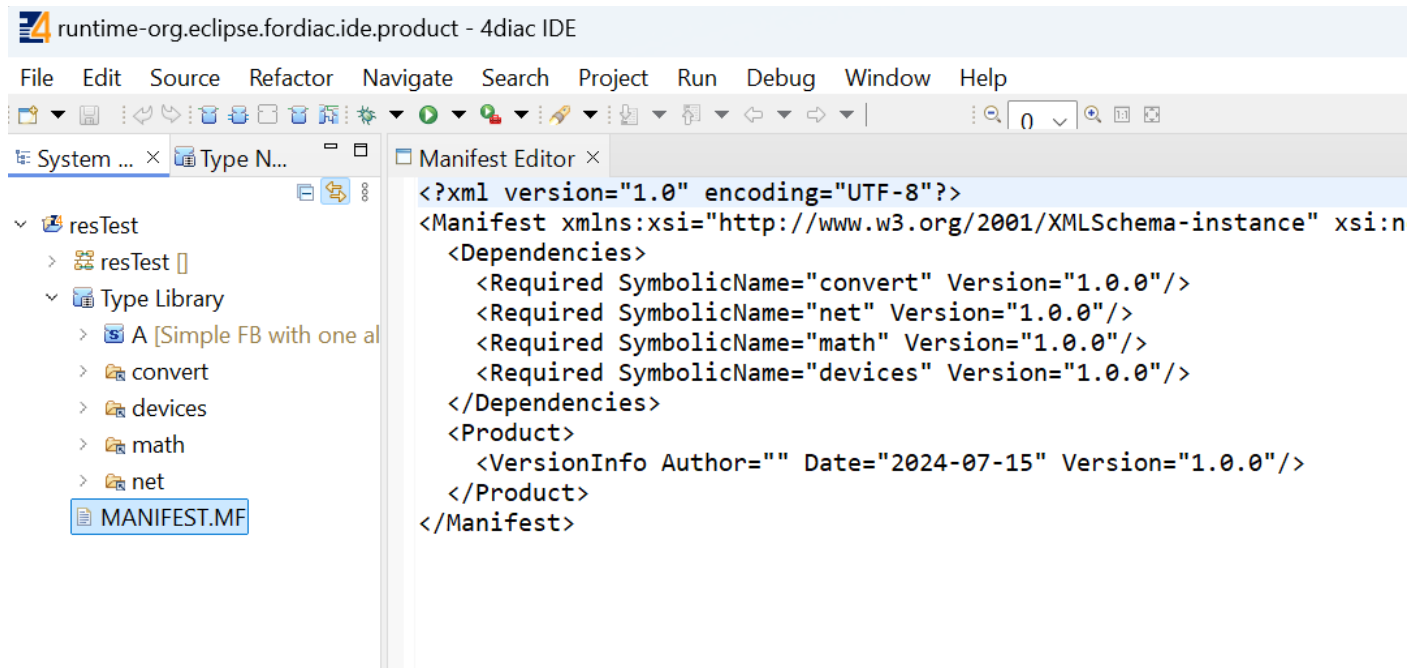
Разработка 4diac IDE или 4diac FORTE

На этой странице представлен обзор тем, которые вы можете найти в документации по разработке 4diac IDE или 4diac FORTE.

- 4diac IDE

- [Сборка и запуск 4diac IDE из исходного кода](#)
 - [Сборка двоичного пакета 4diac IDE из исходного кода](#)
 - [Тесты SWTBot для 4diac IDE](#)
- 4diac FORTE
 - [Мониторинг 4diac FORTE](#)
 - [Архитектура связи](#)
 - [Service FB как внешние события](#)
 - [Обеспечение качества кода 4diac FORTE](#)
- Содействие
 - [Содействие 4diac IDE или 4diac FORTE](#)

Управление библиотеками



Работа с библиотеками

Библиотеки добавляются в проект с помощью доступных мастеров импорта, которые при необходимости обновляют манифест проекта. Библиотеки ссылаются на себя с помощью символических ссылок, чтобы избежать ненужного копирования.

Место хранения

Существует три отдельных места хранения:

- Стандартные библиотеки поставляются в комплекте с дистрибутивом 4diac
- Архивы хранятся в папке .download рабочей области
- Библиотеки хранятся в папке .lib рабочей области

Запуск проекта

При открытии проекта фоновая задача проверяет манифест проекта на наличие зависимостей и выполняет следующие действия:

1. Проверяет, связана ли уже подходящая версия библиотеки
2. Если это не так, ищет и связывает подходящую версию в .lib
3. Если подходящая версия не найдена, пытается загрузить её из настроенных источников загрузки

Структура библиотеки

- Папка с именем <Символическое имя>-<Версия>, содержащая:
 - Файл MANIFEST.MF, определяющий библиотеку
 - Папка с именем typelib, содержащая содержимое библиотеки (функциональные блоки, структуры, типы и т. д.)

Ожидается, что архивы библиотек (в формате zip) будут содержать только эту папку.

Файл манифеста

XML-документ, содержащий всю информацию о библиотеках, используемых как библиотеками, так и проектами. Следующие части являются общими для обоих:

- Область действия (атрибут) — библиотека или проект
- Зависимости (необязательно) — ссылки на другие необходимые библиотеки
- Обязательные (от 0 до неограниченного количества) — ссылки
- SymbolicName (атрибут) — символическое имя зависимости библиотеки
- [Version](#) (атрибут) — диапазон версий зависимости библиотеки
- Product (обязательно) — базовая информация о проекте/библиотеке
- VersionInfo (обязательно)
- Author (атрибут) — имя автора
- Date (атрибут) — дата создания/изменения (дата по стандарту ISO 8601)

- Version (атрибут) — версия проекта/библиотеки

Библиотечный манифест

Имеет следующие дополнения:

- Продукт
 - Имя (атрибут) — отображаемое имя
 - Символическое имя (атрибут) — символическое имя
 - Комментарий (атрибут) — комментарий/описание

Пример манифеста библиотеки

```
<?xml version="1.0" encoding="UTF-8"?>
<Manifest xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation="platform:/resource/org.eclipse.fordiac.ide.library.model
/model/library.xsd" Scope="Library">
  <Dependencies>
    <Required SymbolicName="firstLib" Version="1.1.0"/>
    <Required SymbolicName="secondLib" Version="[1.0.0-2.0.0)"/>
  </Dependencies>
  <Product Name="Example Library" SymbolicName="exampleLib" Comment="example for
demonstration">
    <VersionInfo Author="Eclipse 4diac" Date="2024-07-24" Version="1.0.0"/>
  </Product>
</Manifest>
```

Манифест проекта

Имеет следующие дополнения:

- Экспорт (необязательно) — содержит библиотеки, которые будут экспортированы из этого проекта
 - Библиотека (от 0 до неограниченного количества) — определение экспорта библиотеки
 - Имя, Символическое имя, Комментарий — эквивалентно манифесту библиотеки
 - Включает (обязательно) — фильтры для включения типов в экспортируемую библиотеку
 - Элемент библиотеки (от 0 до неограниченного количества) — шаблон фильтра для включения
- Исключает (необязательно) — фильтры для исключения типов
 - Элемент библиотеки (от 0 до неограниченного количества) — шаблон фильтра для исключения

Пример манифеста проекта

```
<?xml version="1.0" encoding="UTF-8"?>
<Manifest xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation="platform:/resource/org.eclipse.fordiac.ide.library.model/model/library.xsd"
Scope="Project">
  <Dependencies>
    <Required SymbolicName="exampleLib" Version="[1.0.0-1.2.5]"/>
    <Required SymbolicName="math" Version="1.0.0"/>
  </Dependencies>
  <Product>
    <VersionInfo Author="" Date="2024-07-24" Version="1.3.0"/>
  </Product>
  <Exports>
    <Library Name="Export Library 1" SymbolicName="exportLib1" Comment="example for export">
      <Includes>
        <LibraryElement>base::**</LibraryElement>
        <LibraryElement>extended::**</LibraryElement>
      </Includes>
      <Excludes>
        <LibraryElement>*::test::**</LibraryElement>
      </Excludes>
    </Library>
  </Exports>
```

</Manifest>

Версия (диапазон)

Используется формат версии major.minor.micro, где все три части являются положительными целыми числами. Стандартное значение для всех них равно нулю, что приводит к тому, что 1 эквивалентно 1.0 и 1.0.0. Диапазон версий (используемый в зависимостях) форматируется как <слева><версия>-<версия><справа>, где <слева> — [(включительно) или ((исключительно), а <справа> —] (включительно) или) (исключительно). Особый случай <версия> будет интерпретироваться как [<версия>-<версия>] (диапазон, содержащий только эту версию).

Элемент библиотеки

Шаблон фильтра для включения/исключения типов в экспортируемую библиотеку, состоящий из следующих элементов:

Идентификаторы — в соответствии со стандартом IEC61131-3

:: — разделитель пакетов

* — соответствует любому идентификатору

** — соответствует любому идентификатору на одном или нескольких уровнях

Примеры:

A::B::C — соответствует определённому типу

test::** — соответствует чему угодно непосредственно внутри test (но не test:a:b в качестве примера)

test::** — соответствует всему внутри test и его нижним уровням

***::basetype — соответствует любому вхождению basetype, где бы оно ни находилось (но не basetype без какого-либо пакета)

*::test::** — соответствует всему, что имеет test в качестве второго уровня иерархии

SWTBot тестирует документацию

- [Улучшение качества пользовательского Интерфейса Eclipse 4diac IDE за счет автоматизированного Тестирования](#)
 - [Что такое Eclipse SWTBot](#)
 - [Требование](#)
 - [Настройки для запуска тестов SWTBot](#)
- [Набор тестов и структура тестов](#)
- [Выполнение тестов SWTBot](#)
- [Создание нового тестового класса или метода тестирования](#)
 - [Настройки, которые необходимо выполнить в программе @beforeAll](#)
- [Результаты и оценка тестов](#)

Улучшение качества пользовательского интерфейса Eclipse 4diac IDE за счет автоматизированного тестирования

Часто в проектах с открытым исходным кодом участвует всего несколько человек. Они стараются привнести все новые идеи и улучшения. Однако это оставляет мало времени на тестирование, а значит, страдает качество программного обеспечения. Начиная с Eclipse 4diac, тестирование также по-прежнему выполняется вручную, что, с одной стороны, неэффективно, а с другой стороны, часто не выполняется полностью из-за нехватки времени. По этой причине автоматизированный набор тестов для пользовательского интерфейса IDE 4diac был бы хорошим улучшением.

Что такое Eclipse SWTBot

с сайта <https://wiki.eclipse.org/SWTBot>:

Eclipse SWTBot™ — это инструмент для UI/функционального тестирования на основе Java для тестирования приложений на базе SWT и Eclipse. SWTBot предоставляет простые в использовании API. API также скрывают сложности, связанные с SWT и Eclipse. Это делает его подходящим для UI/функционального тестирования для всех, а не только для разработчиков Eclipse. SWTBot также предоставляет собственный набор утверждений, полезных для SWT. Вы также можете использовать собственную систему утверждений с SWTBot. SWTBot может записывать и воспроизводить тесты и интегрируется с Eclipse и технологиями сборки (Tycho, Ant), что упрощает использование в непрерывной интеграции.

Требования

Все файлы, связанные с SWTBot, можно найти в исходном коде IDE 4diac в папке `org.eclipse.fordiac.ide.test.ui`. Для выполнения расширенных тестов были расширены предоставленные классы SWTBot. Например, стало ясно, что с помощью стандартных классов Eclipse SWTBot невозможно создавать соединения в 4diac IDE. По умолчанию перетаскивание работает путём выбора 1-го контакта, а затем перехода ко 2-му контакту и его выбора. К сожалению, в 4diac IDE это не работает, необходимо перемещать мышь между выборами. Поэтому были созданы 4 класса расширения, которые можно найти в папке с исходным кодом 4diac IDE `org.eclipse.fordiac.ide.test.ui.swtbot`. Это классы `SWT4diacGefBot`, `SWTBot4diacGefViewer`, `SWTBot4diacGefEditor` и `SWTBot4diacFigureCanvas`.

Настройки для запуска тестов SWTBot

Все настройки, необходимые для запуска теста SWTBot, уже выполнены, и пользователю не нужно ничего настраивать дополнительно.

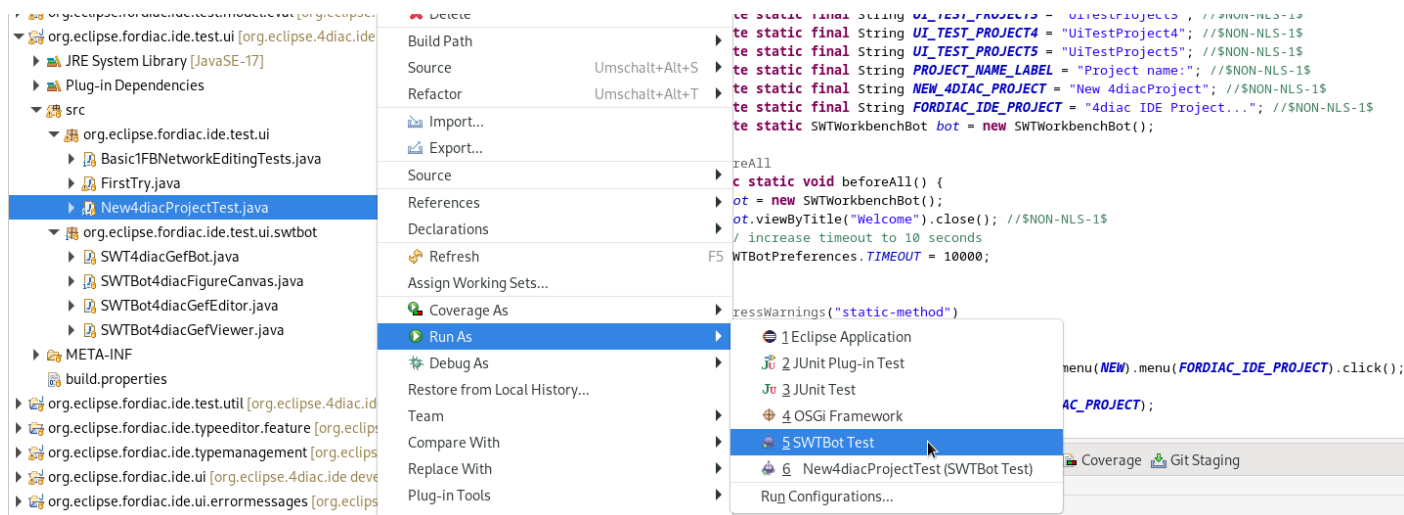
Набор тестов и структура тестов

Для управления тестами были созданы пакеты, чтобы обеспечить лучший обзор. В каждом пакете есть один или несколько тестовых классов. Все тестовые классы наследуются от `Abstract4diacUITest`, за исключением `New4dacProjectTest`, который является независимым, поскольку для него требуется другой метод `@beforeAll`. Идеи, которые еще не реализованы, были собраны в соответствующем тестовом классе с пустым методом и `Javadoc`. Будущие тесты помечаются как `@Disabled` ("до реализации"). Структуру тестов SWTBot также можно найти в разделе «[Структура тестов SWTBot](#)».

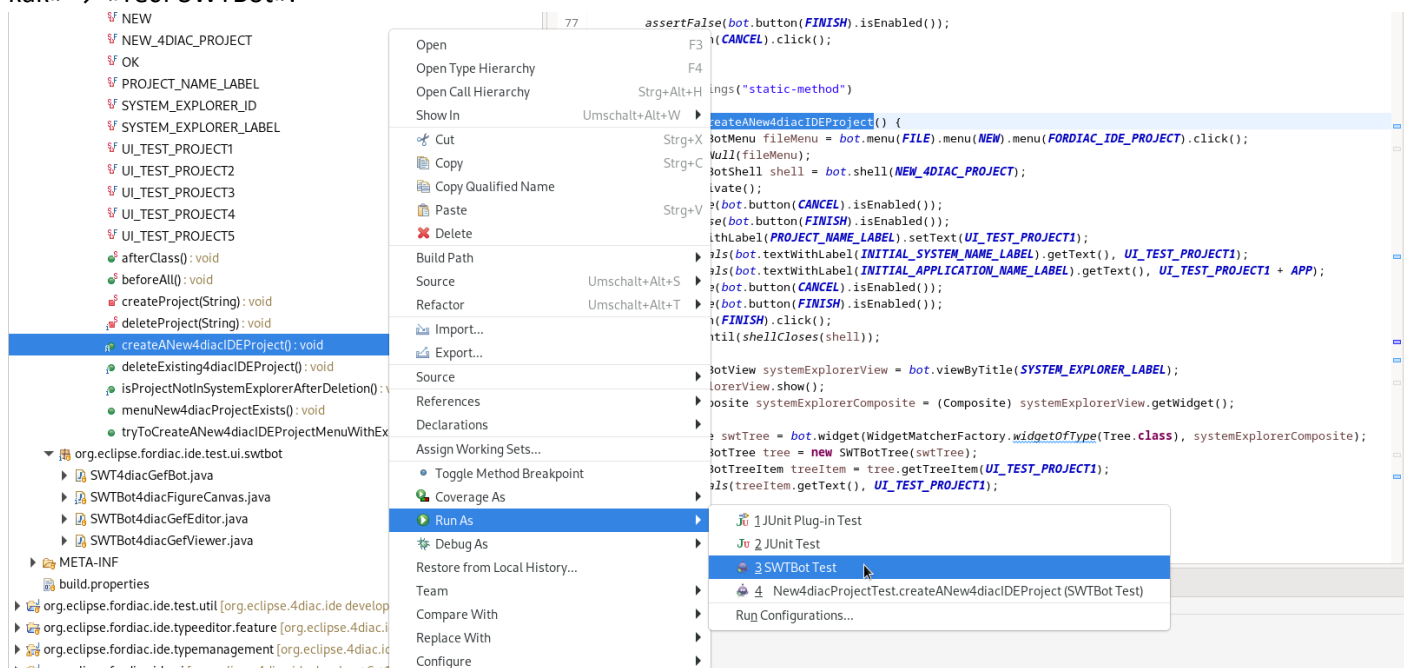
Выполните тесты SWTBot

Важно: было замечено, что во время автоматического выполнения тестов нельзя вручную нажимать на кнопку мыши, так как это мешает работе бота и приводит к сбою тестов.

Чтобы запустить все тесты класса в папке `org.eclipse.fordiac.ide.test.ui`, **right click on the class** → **Run As** → **SWTBot Test**.

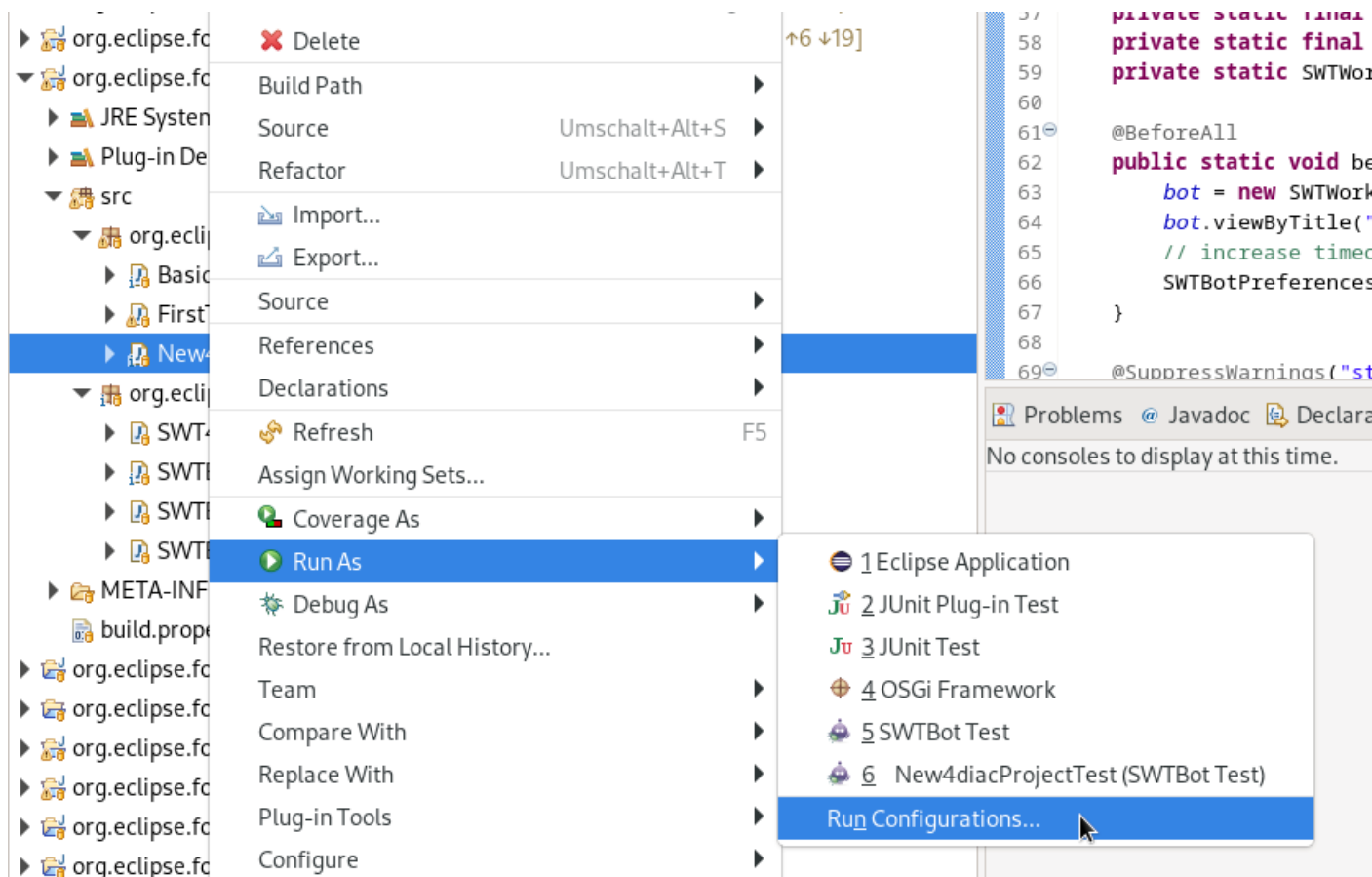


Чтобы выполнить один метод тестирования, выберите метод и щелкните правой кнопкой мыши → «Запустить как» → «Тест SWTBot».

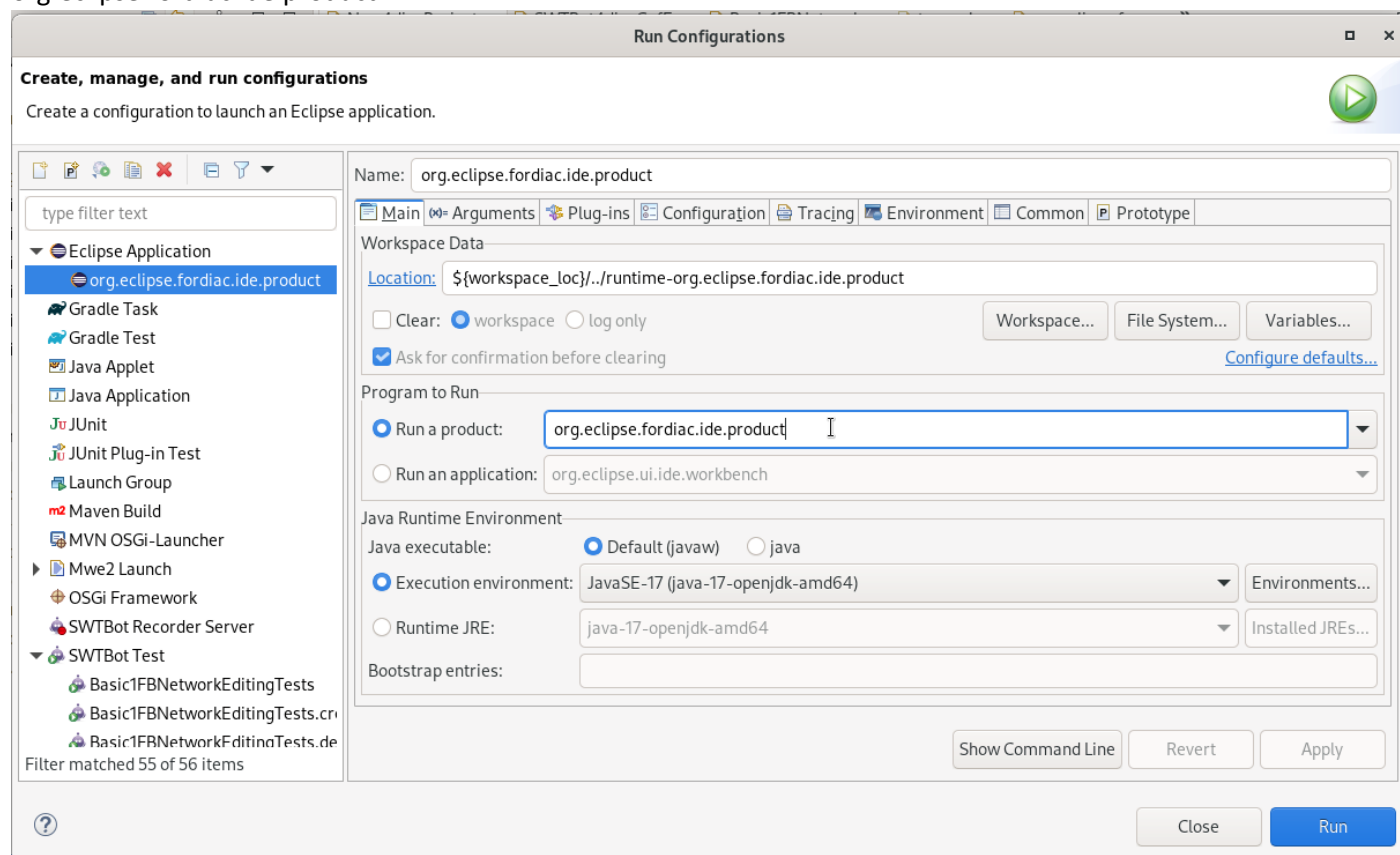


Создайте новый класс тестирования или метод тестирования

После добавления нового класса необходимо изменить конфигурации запуска с Eclipse Platform на Eclipse 4diac, чтобы запустить тесты. Самый быстрый способ сделать это — щелкнуть правой кнопкой мыши по классу в проводнике системы → «Запустить как» → «Конфигурации запуска»...



В появившемся окне нажмите на вкладку « **Main tab** » и выберите справа от « **Run a project** » опцию `org.eclipse.fordiac.ide.product`.



Чтобы иметь возможность запускать тесты по отдельности, необходимо выполнить те же настройки для отдельных тестовых методов. Однако было замечено, что иногда первые 1-2 запуска тестов не проходят, несмотря на эти настройки, потому что они сбрасываются до настроек платформы Eclipse. В этом случае повторите описанные выше действия.

Аннотации для тестов такие же, как в Junit5. С введением в JUnit 5 можно ознакомиться [здесь](#).

Настройки, которые необходимо выполнить в @BeforeAll

Тестовые классы Basic1FBNetworkEditingTests и будущие тестовые классы структурированы таким образом, что сначала запускается 4diac IDE. Метод beforeAll() с аннотацией @BeforeAll создает бота и закрывает окно приветствия, которое появляется при запуске 4diac IDE. Это закрытие важно, иначе тесты не смогут запуститься.

Здесь также задаются такие параметры, как время ожидания и скорость воспроизведения.

```
@BeforeAll
public static void beforeAll() {
    bot = new SWT4diacGefBot();
    bot.viewByTitle("Welcome").close(); //$NON-NLS-1$
    // increase timeout to 10 seconds
    SWTBotPreferences.TIMEOUT = 10000;
    createProject();
}
```

Результаты и оценка тестов

При запуске тестов откроется новая вкладка JUnit. На ней отображается информация о том, какой тест прошел успешно, а какой — нет. В приведённом ниже примере показано, что все тесты, кроме одного, прошли успешно. Это означает, что ожидаемый результат был достигнут. Однако один тест не прошёл. Этот тест интересен тем, что он, вероятно, обнаружил ошибку в программе. Теперь необходимо подробнее изучить, что пошло не так.

При создании теста необходимо позаботиться о том, чтобы запрашивалось нужное событие. Это единственный способ исключить неверные результаты из-за некорректных тестов. В нашем случае тест пройден успешно, мы обнаружили ошибку: FB на самом деле не отображается в дереве иерархии. Расследование показало, что об ошибке уже сообщили, поэтому в данном случае никаких дополнительных действий предпринимать не нужно.

The screenshot displays the Eclipse IDE interface. The top menu bar includes File, Edit, Source, Refactor, Navigate, Search, Project, Run, Window, and Help. Below the menu is a toolbar with various icons. The main workspace is divided into three panes. The left pane shows the 'Package Explorer' with a tree view of test results for 'Basic1FBNetworkEditingTests'. The tree lists several tests with their execution times: deleteExistingFB() (4,671 s), moveFB() (2,723 s), isAddedFbInProjectAppNode() (4,254 s), editDTofECycle() (3,261 s), createValidConnection() (3,000 s), directEditorDefaultValueTest() (3,157 s), directEditorNewValueTest() (3,071 s), and dragAndDrop1FB() (2,632 s). The right pane shows the 'JUnit' view with a progress bar indicating 'Runs: 8/8', 'Errors: 1', and 'Failures: 0'. The bottom pane shows the 'Source' view with the code for 'dragAndDrop1FB()' in the 'SWTBot Tests Documentation.html' file. The code is annotated with @Test and includes comments for selecting the FB E_CYCLE and waiting for the tree item to have the node.

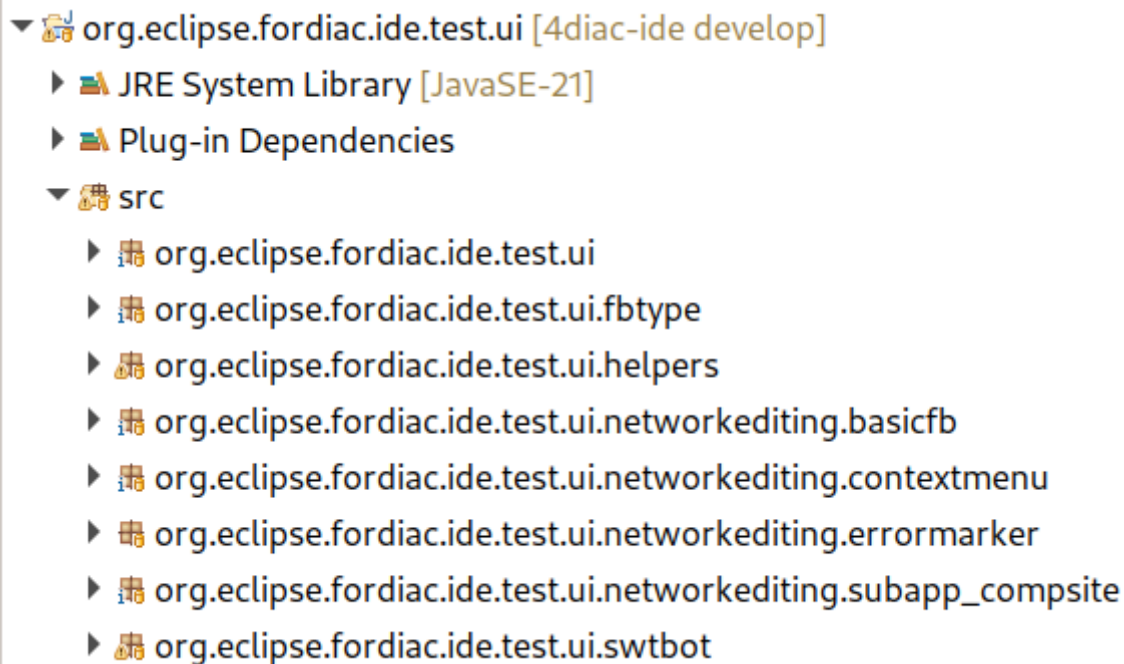
```
4diacIDE_SWTBot - org.eclipse.fordiac.ide.test.ui/src/org/eclipse/fo
File Edit Source Refactor Navigate Search Project Run Window Help
Package Expl Type Hierarc JUnit x SWTBot Tests Documentation.html Basic1FBNetworkEditingTests.java x
Finished after 31,137 seconds
Runs: 8/8 Errors: 1 Failures: 0
Basic1FBNetworkEditingTests [Runner: JUnit 5] (26,772)
deleteExistingFB() (4,671 s)
moveFB() (2,723 s)
isAddedFbInProjectAppNode() (4,254 s)
editDTofECycle() (3,261 s)
createValidConnection() (3,000 s)
directEditorDefaultValueTest() (3,157 s)
directEditorNewValueTest() (3,071 s)
dragAndDrop1FB() (2,632 s)
@Test
public void dragAndDrop1FB() {
    // select FB E_CYCLE
    final SWTBotView systemExplorerView = bot.viewById(SYSTEME
    systemExplorerView.show();
    final Composite systemExplorerComposite = (Composite) sys
    final Tree swtTree = bot.widget(WidgetMatcherFactory.wid
    final SWTBotTree tree = new SWTBotTree(swtTree);
    final SWTBotTreeItem treeProjectItem = tree.getTreeItem(/
    treeProjectItem.select();
    treeProjectItem.expand();
    final SWTBotTreeItem typeLibraryNode = treeProjectItem.ge
    typeLibraryNode.select();
    typeLibraryNode.expand();
    final SWTBotTreeItem eventsNode = typeLibraryNode.getNode
    eventsNode.select();
    eventsNode.expand();
    bot.waitForTreeItemHasNode(eventsNode, E_CYCLE_TREE_I
    final SWTBotTreeItem eCycleNode = eventsNode.getNode(E_C
    eCycleNode.select();
```

Набор тестов пользовательского интерфейса SWTBot и структура тестов

- [Где расположены тесты пользовательского интерфейса SWTBot в исходном коде IDE 4diac?](#)
- [Иерархия SWTBot с расширениями для IDE 4diac](#)
- [Структура тестов пользовательского интерфейса SWTBot](#)
- [Набор тестов пользовательского интерфейса SWTBot](#)
- [Документация по тестированию пользовательского интерфейса SWTBot разработчика 4diac](#)

Где в исходном коде IDE 4diac расположены тесты пользовательского интерфейса SWTBot?

Тесты пользовательского интерфейса с Eclipse SWTBot можно найти в пакете /org.eclipse.fordiac.ide.test.ui. Он разделён на следующие пакеты:

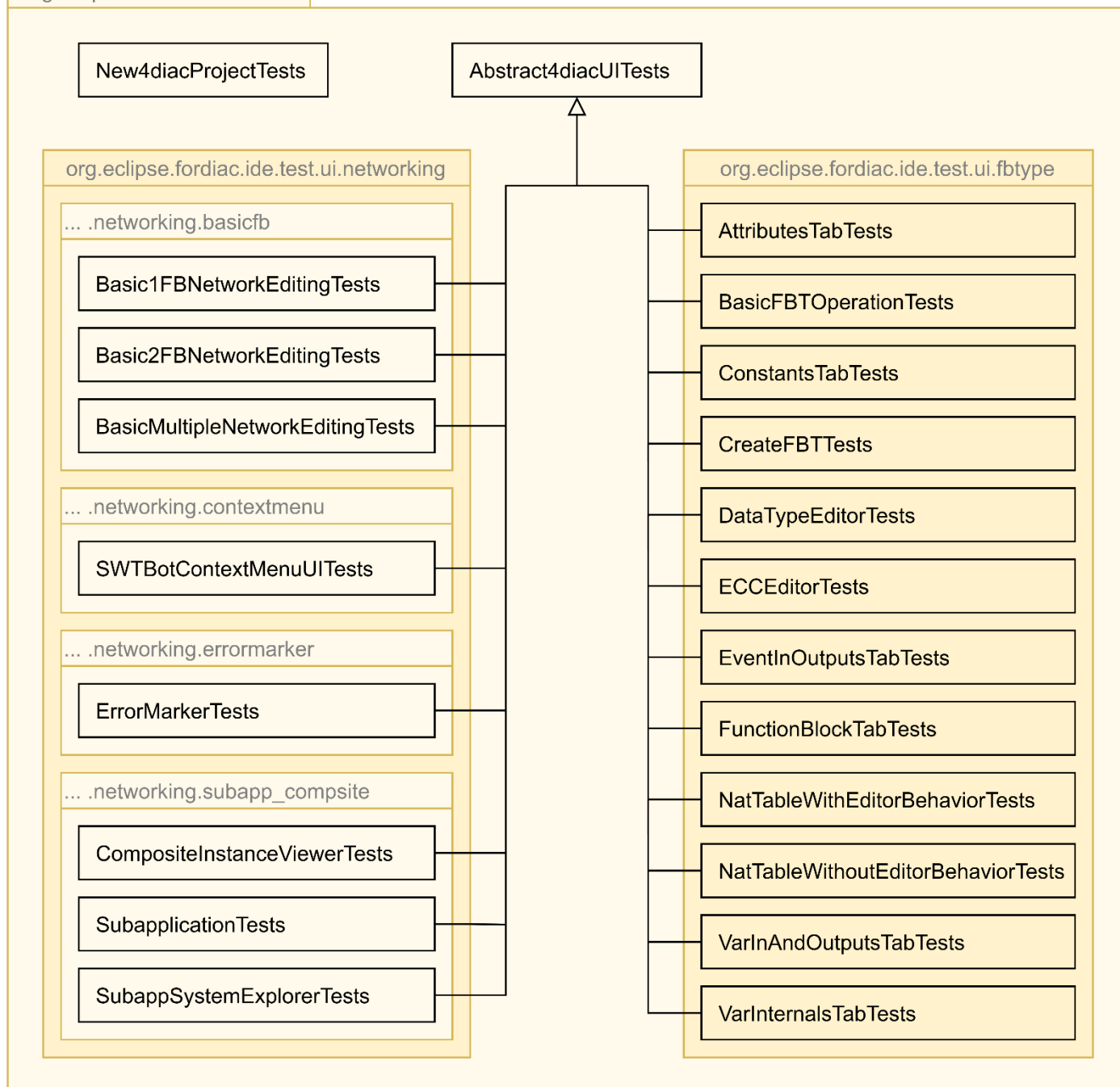


Иерархия SWTBot с расширениями для 4diac IDE

В пакете org.eclipse.fordiac.ide.test.ui.swtbot содержатся расширения для IDE 4diac, так как 4diac обладает некоторыми характеристиками, которые требуют этих расширений для тестовой среды. Например, SWTBot будет переходить из одной точки в другую при перетаскивании FB. Однако для выполнения этого действия 4diac также требуется небольшое движение мыши. Переопределённый метод mouseDrag класса SWTBot4diacFigureCanvas добавляет необходимое движение мыши, а также вносит необходимые изменения в вызывающие классы.

Четыре класса:

- SWT4diacGefBot расширяет SWTGefBot
- SWTBot4diacGefEditor расширяет SWTBotGefEditor
- SWT4diacGefViewer расширяет SWTBotGefViewer и
- SWTBot4diacFigureCanvas расширяет SWTBotGefFigureCanvas.



[Документация по тестам SWTBot UI для разработчиков 4diac IDE](#)

Документация по разработке была создана для того, чтобы все разработчики 4diac IDE могли добавлять новые тесты.

Её можно найти в установленной 4diac IDE в разделе

Help → *Help Contents* → *4diac User Guide* → *Development Documentation* → *4DIAC-IDE* → *SWTBot testing* или

[Документация по тестам SWT Bot](#)

ПРИМЕРЫ

Запустите контроллер примера с светофором на BeagleBoardBlack (BBB)

Хорошей практикой для знакомства с 4diac IDE и концепцией распределённой автоматизации является применение примера с светофором, но вместо того, чтобы запускать его полностью на компьютере или другой платформе, мы поместим контроллер примера с светофором на BBB, а лампочки и кнопки включения/выключения будут работать на обычном компьютере с использованием среды выполнения FBRT.

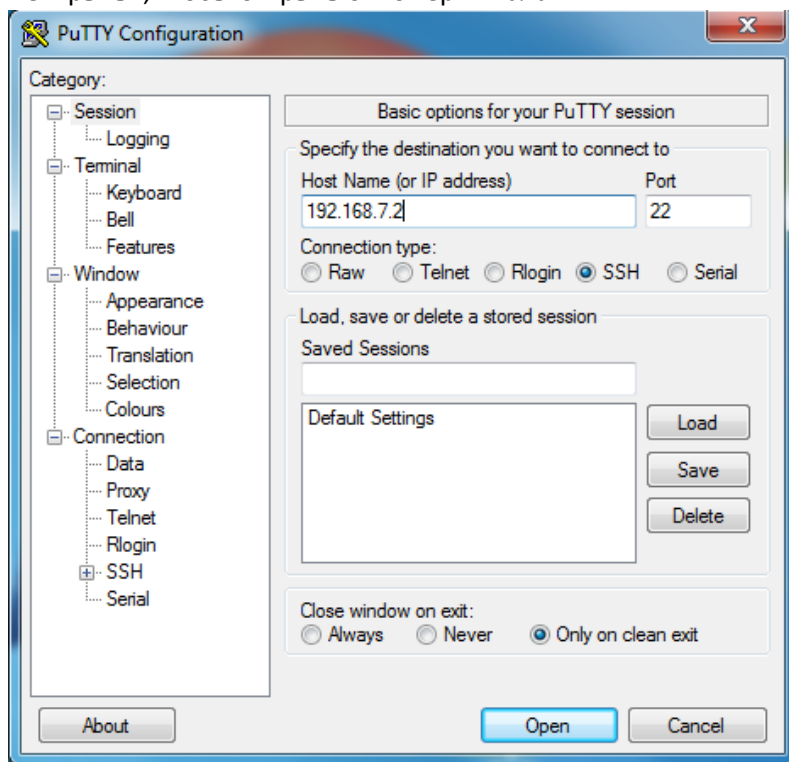
Подготовка BBB

BBB имеет функцию, которая позволяет эмулировать Ethernet-порт через USB-интерфейс. Для этого требуются специальные драйверы на компьютере, подключённом к BBB. Для получения точной информации о том, как подготовить BBB, ознакомьтесь с официальной страницей [BeagleBone](#).

После подключения BBB обычно получает IP-адрес 192.168.7.2, поэтому пинг с компьютера должен быть успешным.

Доступ к BBB через SSH

После подключения BBB к компьютеру через USB следующим шагом будет подключение к нему через SSH. Для этого потребуется SSH-клиент. На компьютерах с Linux он уже установлен, и подключение можно выполнить через терминал, но на компьютерах с Windows проще всего использовать Putty. Чтобы подключиться к BBB с помощью [Putty](#), просто введите IP-адрес BBB (192.168.7.2), порт 22 и нажмите «Открыть», чтобы открыть окно терминала.



Имя пользователя root без пароля должно работать на BBB, к которому ранее не подключались.



Настройка сети в BBB

Если вы используете USB-порт в качестве интерфейса Ethernet в BBB, необходимо внести некоторые изменения, чтобы связь с компьютером работала. Команда `ip route` покажет текущие маршруты, сохранённые в BBB.

```
root@at-tl001:~# ip route
default via 10.200.0.1 dev eth0
10.200.0.0/16 dev eth0 proto kernel scope link src 10.200.4.120
192.168.7.0/30 dev usb0 proto kernel scope link src 192.168.7.2
root@at-tl001:~#
```

Сначала мы удалим маршрут по умолчанию, который проходит через интерфейс `eth0`. Используйте команду `ip route del default via [ip] dev eth0`, где `[ip]` — это IP-адрес, указанный в записи маршрутизации по умолчанию для Ethernet (в данном случае `10.200.0.1`). Далее мы добавим запись по умолчанию, которая будет направлять трафик на USB-порт с помощью команды `ip route add default via [ip] dev usb0`, где `[ip]` — это IP-адрес компьютера на другой стороне соединения с BBB, обычно `192.168.7.1`. Некоторые пользователи сообщают, что интерфейс BBB не поддерживает многоадресные пакеты, что не позволяет приложению обмениваться данными. Чтобы решить эту проблему, проверьте, поддерживает ли интерфейс, который вы используете в BBB, многоадресную рассылку. Для этого введите команду `ifconfig`, и вы увидите слово `MULTICAST` (все заглавные буквы) в описании интерфейса, как на изображении ниже. Если нет, выполните команду: `ifconfig usb0 multicast`

```

root@at-tl001:~/fortel.8/bin/posix/src# ifconfig
eth0      Link encap:Ethernet  HWaddr 78:a5:04:bc:bf:9d
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:40

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:24 errors:0 dropped:0 overruns:0 frame:0
          TX packets:24 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:2410 (2.3 KiB)  TX bytes:2410 (2.3 KiB)

usb0      Link encap:Ethernet  HWaddr 5a:26:00:54:5f:02
          inet addr:192.168.7.2  Bcast:192.168.7.3  Mask:255.255.255.255
          inet6 addr: fe80::5826:ff:fe54:5f02/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:139084 errors:0 dropped:3 overruns:0 frame:0
          TX packets:253817 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:6730777 (6.4 MiB)  TX bytes:59501649 (56.7 MiB)

          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:24 errors:0 dropped:0 overruns:0 frame:0
          TX packets:24 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:2410 (2.3 KiB)  TX bytes:2410 (2.3 KiB)

```

Запуск 4diac FORTE на BBB

Чтобы запустить 4diac FORTE на BBB, вы можете либо использовать кросс-компилятор на своем компьютере, а затем перенести исполняемый файл 4diac FORTE на BBB, либо вы можете перенести все исходные файлы на BBB и скомпилировать непосредственно в нем. Это руководство посвящено компиляции непосредственно в BBB. В любом случае, ещё один шаг, который необходимо сделать, — это сообщить CMake, что вы хотите включить IO в 4diac FORTE. Для этого сначала найдите файл `setup_posix.sh` в корневой папке 4diac FORTE, найдите команду `cmake` почти в самом низу и добавьте текст `-DFORTE_MODULE_SysFs=ON` (без кавычек) перед `-DFORTE_ARCHITECTURE=Posix` (оставив пробелы между ними).

Перенос исходного кода на BBB

Для переноса файлов с компьютера с Windows на BBB можно воспользоваться программой [WinSCP](#), которая позволяет подключаться к BBB и перемещаться по папкам в графическом интерфейсе. Подключитесь к BBB, используя IP-адрес так же, как и в SSH. После подключения вы увидите два дерева папок. Слева — ваша локальная машина, а справа — BBB. В WinSCP перейдите по обеим веткам к нужным папкам и выберите файлы для передачи, а затем нажмите «Загрузить», чтобы передать файлы с компьютера → BBB, и «Скачать», чтобы передать файлы с BBB → на компьютер.

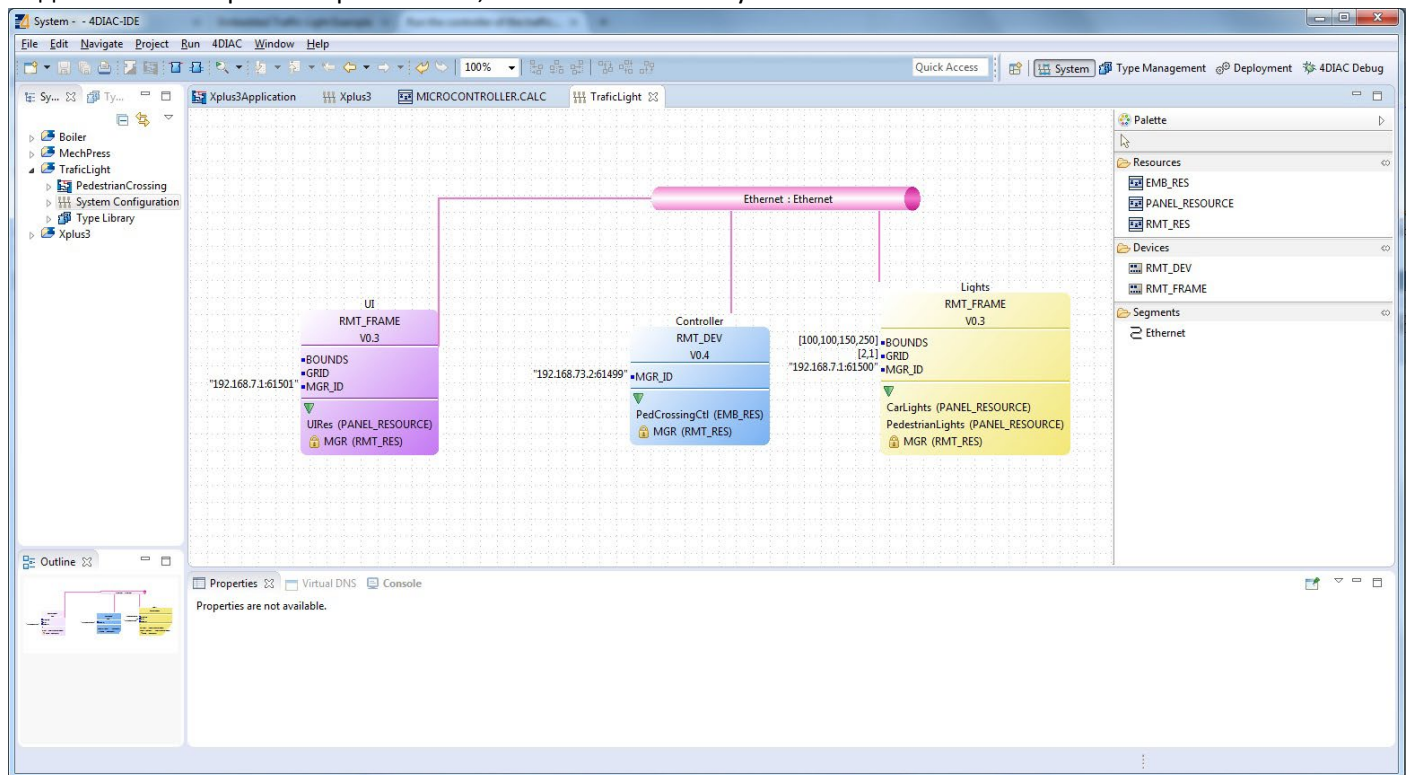
Если вы не можете или не хотите использовать WinSCP, вы можете сделать это через терминал. Для этого вам понадобится [PSCP](#), у которого нет графического интерфейса, но он работает в командной строке (терминале). После загрузки вы можете добавить имя папки, в которой находится файл `pscp.exe`, в переменную `PATH`. Если нет, вы можете перейти в терминале к папке, в которой находится `pscp.exe` найдите файл и выполните там следующую команду: `pscp -r "[локальный путь к 4diac FORTE]" [имя пользователя BBB]@[ip]:[удаленная папка для 4diac FORTE]`, где [локальный путь к 4diac FORTE] - это папка на компьютере с Windows, где находится исходный код 4diac FORTE, [имя пользователя BBB] - это имя пользователя для входа в BBB, [ip] - это IP-адрес BBB, а [удаленная папка 4diac FORTE] - это папка, в которую копируются файлы в BBB. Опция `-r` указывает на рекурсивное копирование всех папок. Примером команды может быть `pscp.exe -r "c:\Users\JohnDoe\Documents\forte_repository" root@192.168.7.2:/root/bbb_forte_repository`

Сборка 4diac FORTE в BBB

Чтобы собрать 4diac FORTE на BBB, вам нужно перейти в папку, куда вы скопировали все файлы, и запустить: `./setup_posix.sh`, который запустит CMake и подготовит все файлы. Затем перейдите в папку `bin/posix` в папке 4diac FORTE и выполните там команду: `make`. Это займёт некоторое время, потому что BBB работает не так быстро, как обычный компьютер. Поэтому кросс-компилятор может оказаться хорошей идеей. После завершения компиляции вы найдёте исполняемый файл 4diac FORTE в папке `bin/posix/src` в папке 4diac FORTE. Если у вас возникли проблемы со сборкой 4diac FORTE, вы можете выполнить те же действия, что и для [Raspberry Pi](#). Если в BBB отсутствуют какие-либо программы, дополнительную информацию можно найти в разделе [подготовки](#) Raspberry Pi.

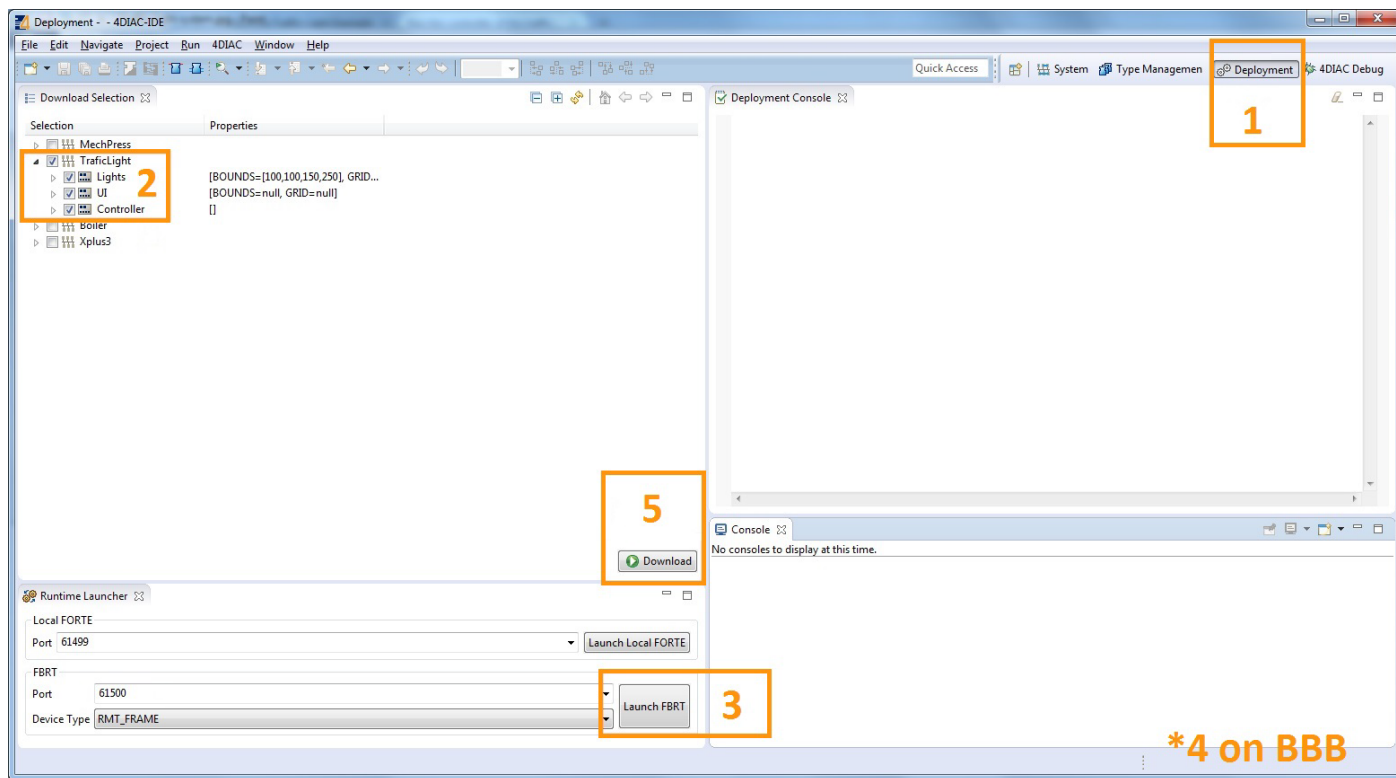
Настройте систему в 4diac IDE

Как только 4diac FORTE сможет работать на BBB, мы воспользуемся 4diac IDE, чтобы загрузить на него часть приложения, а остальная часть приложения будет работать на компьютере. Для этого мы открываем приложение Traffic Light и настраиваем систему так, чтобы два устройства-кадра имели локальный IP-адрес, а устройство-контроллер имело IP-адрес BBB. 4diac FORTE по умолчанию работает на порту 61499, поэтому убедитесь, что порт устройства-контроллера настроен правильно. Порты двух фреймовых устройств должны отличаться друг от друга. Подключите устройства с помощью Ethernet-соединения в 4diac IDE, если они не подключены. Сохраните приложение, и вы готовы его запустить.

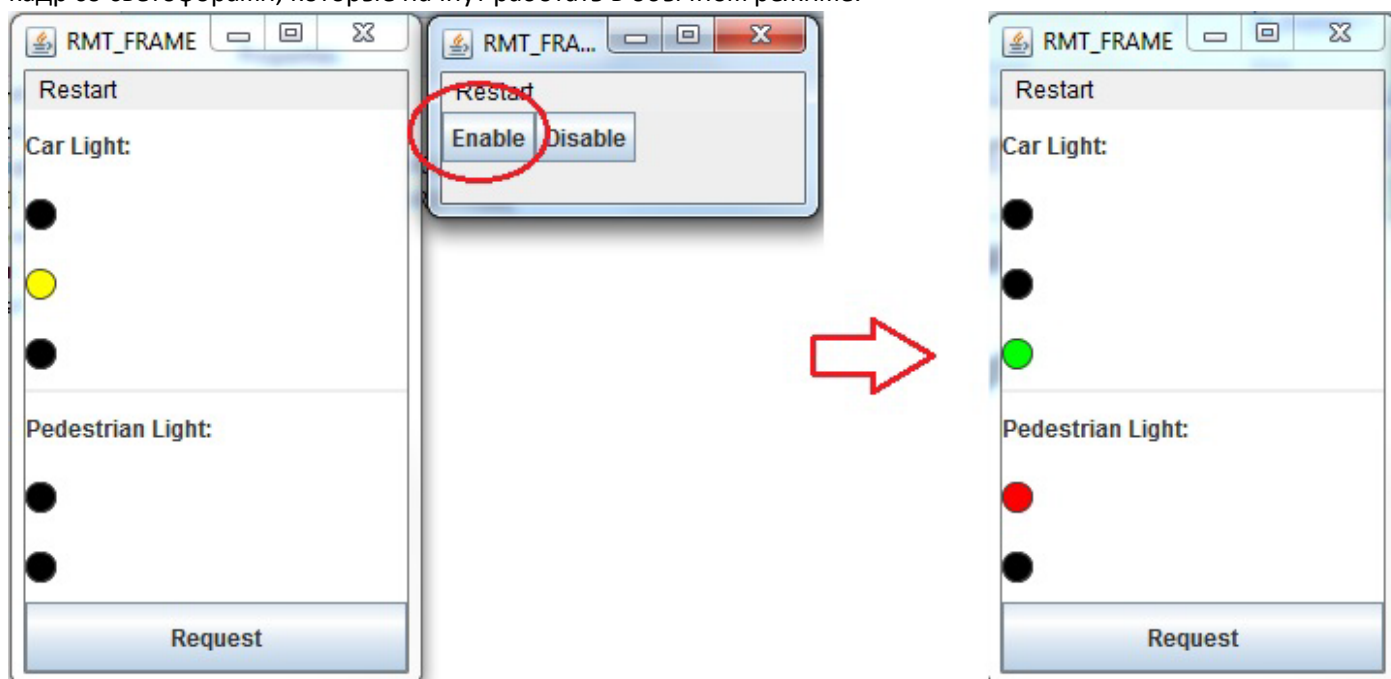


Запуск примера

После правильной настройки приложения перейдите в раздел «Развёртывание» (1) в 4diac IDE и выберите «Светофор» (2) в левом дереве. На вкладке «Запуска среды выполнения» (3) под ним дважды запустите среду выполнения FBRT, настроив каждый раз нужный порт, как и раньше, и откроются два окна Java. Запустите 4diac FORTE на BBB (4). Нажмите «Загрузить» (5), и каждая часть приложения должна быть загружена на BBB и в два экземпляра FBRT. Убедитесь, что на вкладке «Консоль развёртывания» не возникло ошибок.



Один кадр будет содержать кнопки «Включить» и «Отключить», а другой — светофоры для автомобилей и пешеходов, а также кнопку запроса. Жёлтый свет автомобилей начнёт мигать, и при нажатии кнопки «Включить» информация будет отправлена в BBB, который обработает её, а затем отправит информацию в кадр со светофорами, которые начнут работать в обычном режиме.



Этот пример должен прояснить суть распределённой автоматизации и то, насколько просто её можно реализовать с помощью 4diac IDE.

Устранение неполадок

Во многих случаях связь между ПК и BBB прерывается, но в основном из-за проблем с конфигурацией сети. Поскольку для связи между устройствами используются многоадресные пакеты, могут потребоваться некоторые настройки, особенно на компьютерах с Windows. Первым делом нужно добавить многоадресные адреса, используемые приложением, в таблицу маршрутизации. Для этого вам понадобятся права администратора. Запустите командное окно (CTRL+R → cmd → Enter) и введите route PRINT, чтобы увидеть все маршруты, используемые вашим ПК. Убедитесь, что многоадресный адрес, используемый приложением, отображается там и использует ваш IP-адрес BBB в качестве шлюза. В примере с светофором используется адрес 225.0.0.x. На рисунке ниже видно, что этот адрес отсутствует.

```

C:\Windows\system32>route PRINT
=====
Schnittstellenliste
19...78 a5 04 bc bf 9f .....Linux USB Ethernet/RNDIS Gadget
15...5e 51 4f 74 99 bf .....Microsoft Virtual WiFi Miniport Adapter #2
14...5e 51 4f 74 99 c0 .....Microsoft Virtual WiFi Miniport Adapter
13...5c 51 4f 74 99 bf .....Intel(R) Dual Band Wireless-AC 7260
12...02 80 37 ec 02 00 .....N5321 gw Mobile Broadband Driver
11...28 d2 44 48 0d 20 .....Intel(R) Ethernet Connection I217-LM
1.....Software Loopback Interface 1
18...00 00 00 00 00 00 00 e0 Microsoft-ISA-TAP-Adapter
20...00 00 00 00 00 00 00 e0 Microsoft-ISA-TAP-Adapter #2
21...00 00 00 00 00 00 00 e0 Microsoft-ISA-TAP-Adapter #3
16...00 00 00 00 00 00 00 e0 Microsoft-ISA-TAP-Adapter #4
17...00 00 00 00 00 00 00 e0 Microsoft-ISA-TAP-Adapter #5
22...00 00 00 00 00 00 00 e0 Microsoft-ISA-TAP-Adapter #6
=====

IPv4-Routentabelle
=====
Aktive Routen:
   Netzwerkziel   Netzwerkmaske   Gateway   Schnittstelle   Metrik
   0.0.0.0         0.0.0.0         192.168.16.1 192.168.17.188    10
   127.0.0.0         255.0.0.0       Auf Verbindung 127.0.0.1      306
   127.0.0.1         255.255.255.255 Auf Verbindung 127.0.0.1      306
   127.255.255.255   255.255.255.255 Auf Verbindung 127.0.0.1      306
   192.168.7.0       255.255.255.252 Auf Verbindung 192.168.7.1     266
   192.168.7.1       255.255.255.255 Auf Verbindung 192.168.7.1     266
   192.168.7.3       255.255.255.255 Auf Verbindung 192.168.7.1     266
   192.168.16.0      255.255.254.0   Auf Verbindung 192.168.17.188  266
   192.168.17.188    255.255.255.255 Auf Verbindung 192.168.17.188  266
   192.168.17.255    255.255.255.255 Auf Verbindung 192.168.17.188  266
   224.0.0.0         240.0.0.0       Auf Verbindung 127.0.0.1      306
   ??? 224.0.0.0         240.0.0.0       Auf Verbindung 192.168.17.188  266
   224.0.0.0         240.0.0.0       Auf Verbindung 192.168.7.1     266
   255.255.255.255   255.255.255.255 Auf Verbindung 127.0.0.1      306
   255.255.255.255   255.255.255.255 Auf Verbindung 192.168.17.188  266
   255.255.255.255   255.255.255.255 Auf Verbindung 192.168.7.1     266
=====

Ständige Routen:
Keine

IPv6-Routentabelle
=====
Aktive Routen:
  If Metrik Netzwerkziel   Gateway
  1 306 ::1/128           Auf Verbindung
  11 266 fe80::/64         Auf Verbindung
  19 266 fe80::/64         Auf Verbindung
  11 266 fe80::53a:1c38:7b2b:bfc/128
  Auf Verbindung
  19 266 fe80::b8dc:d4f7:6360:c637/128
  Auf Verbindung
  1 306 ff00::/8          Auf Verbindung
  11 266 ff00::/8          Auf Verbindung
  19 266 ff00::/8          Auf Verbindung
=====

```

Итак, адрес должен быть добавлен и направлен на IP-адрес BBB (192.168.7.2). Используйте команду route ADD 225.0.0.0 MASK 255.255.0.0 192.168.7.2 METRIC 255, чтобы он появился в таблице при повторном выполнении команды route PRINT. После этого пакеты, отправленные на ваш BBB, должны приходить.

```

C:\Windows\system32>route ADD 225.0.0.0 MASK 255.255.0.0 192.168.7.2 METRIC 255
OK!

C:\Windows\system32>route PRINT
=====
Schnittstellenliste
19...78 a5 04 bc bf 9f .....Linux USB Ethernet/RNDIS Gadget
15...5e 51 4f 74 99 bf .....Microsoft Virtual WiFi Miniport Adapter #2
14...5e 51 4f 74 99 c0 .....Microsoft Virtual WiFi Miniport Adapter
13...5c 51 4f 74 99 bf .....Intel(R) Dual Band Wireless-AC 7260
12...02 80 37 ec 02 00 .....N5321 gw Mobile Broadband Driver
11...28 d2 44 48 0d 20 .....Intel(R) Ethernet Connection I217-LM
1.....Software Loopback Interface 1
18...00 00 00 00 00 00 00 e0 Microsoft-ISATAP-Adapter
20...00 00 00 00 00 00 00 e0 Microsoft-ISATAP-Adapter #2
21...00 00 00 00 00 00 00 e0 Microsoft-ISATAP-Adapter #3
16...00 00 00 00 00 00 00 e0 Microsoft-ISATAP-Adapter #4
17...00 00 00 00 00 00 00 e0 Microsoft-ISATAP-Adapter #5
22...00 00 00 00 00 00 00 e0 Microsoft-ISATAP-Adapter #6
=====

IPv4-Routentabelle
=====
Aktive Routen:
   Netzwerkziel   Netzwerkmaske   Gateway   Schnittstelle   Metrik
   0.0.0.0         0.0.0.0         192.168.16.1 192.168.17.188   10
   127.0.0.0         255.0.0.0       Auf Verbindung 127.0.0.1       306
   127.0.0.1       255.255.255.255 Auf Verbindung 127.0.0.1       306
   127.255.255.255 255.255.255.255 Auf Verbindung 127.0.0.1       306
   192.168.7.0     255.255.255.252 Auf Verbindung 192.168.7.1     266
   192.168.7.1     255.255.255.255 Auf Verbindung 192.168.7.1     266
   192.168.7.3     255.255.255.255 Auf Verbindung 192.168.7.1     266
   192.168.16.0    255.255.254.0   Auf Verbindung 192.168.17.188   266
   192.168.17.188  255.255.255.255 Auf Verbindung 192.168.17.188   266
   192.168.17.255  255.255.255.255 Auf Verbindung 192.168.17.188   266
   224.0.0.0       240.0.0.0       Auf Verbindung 127.0.0.1       306
   224.0.0.0       240.0.0.0       Auf Verbindung 192.168.17.188   266
   224.0.0.0       240.0.0.0       Auf Verbindung 192.168.7.1     266
   225.0.0.0       255.255.0.0     192.168.7.2   192.168.7.1     265
   255.255.255.255 255.255.255.255 Auf Verbindung 127.0.0.1       306
   255.255.255.255 255.255.255.255 Auf Verbindung 192.168.17.188   266
   255.255.255.255 255.255.255.255 Auf Verbindung 192.168.7.1     266
=====

Ständige Routen:
Keine

IPv6-Routentabelle
=====
Aktive Routen:
If Metrik Netzwerkziel Gateway
1 306 ::1/128 Auf Verbindung
11 266 fe80::/64 Auf Verbindung
19 266 fe80::/64 Auf Verbindung
11 266 fe80::53a:1c38:7b2b:bfc/128 Auf Verbindung
19 266 fe80::b8dc:d4f7:6360:c637/128 Auf Verbindung
1 306 ff00::/8 Auf Verbindung
11 266 ff00::/8 Auf Verbindung
19 266 ff00::/8 Auf Verbindung
=====

```

Другая обнаруженная проблема заключается в том, что, даже если BBB получает пакеты и отправляет ответ, приложение их не получает. Даже если пакет отображается в [Wireshark](#), приложение его не получает. Решение состоит в том, чтобы полностью отключить ---→ БРАНЬ ←---. После этого всё должно работать нормально.

Примеры 4diac

Эта страница предназначена для ознакомления с примерами 4diac, которые помогают понять и разработать приложения. Если вы хотите внести свой вклад, вы можете сделать это, отправив пример в репозиторий [Eclipse 4diac Examples на Github](#).

Общие примеры

Для следующих примеров вам не понадобится специальное оборудование:

- [X+3](#): в этом примере используются 2 устройства, на одном из которых работает 4diac FORTE, а на другом — FBRT. У него очень простая логика, которая позволяет пользователю взаимодействовать с ним, вводя число, которое отправляется в 4diac FORTE, где к введённому числу добавляется 3, и возвращается результат, который отображается пользователю.

Примеры, требующие специального оборудования

Для следующих примеров вам потребуется специальное оборудование, чтобы их опробовать:

- [ПИД-регулирование на двигателе Lego Mindstorms](#): в этом примере используется Lego Mindstorms EV3. Он создаёт блок базовых функций, который реализует ПИД-регулирование положения двигателя.
- [Пример управления трафиком, работающий на Beagle Bone Black](#): это управление трафиком, поставляемое с Eclipse 4diac, но вместо локального запуска управление работает на BBB, а HMI работает локально. Этот пример важен, потому что в нём можно найти множество проблем с сетью, которые были решены.

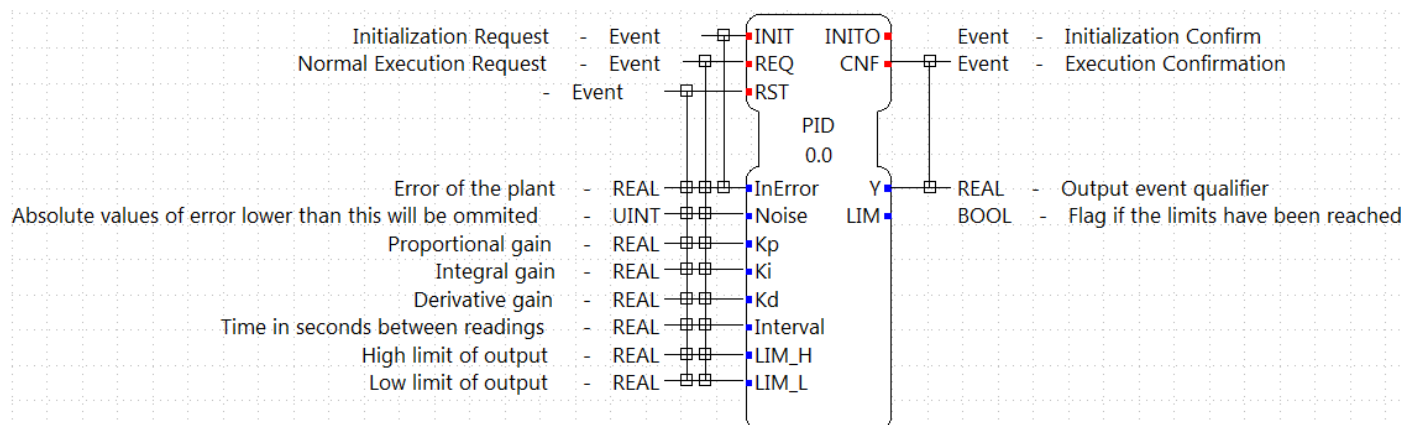
Реализуйте ПИД-регулятор для управления положением двигателя EV3

В этом руководстве представлена реализация ПИД-регулятора для двигателя EV3. Положение двигателя будет управляться с помощью ШИМ. Реализация включает все этапы: от создания функционального блока ПИД в 4diac IDE до настройки ПИД-регулятора и вывода значений на график.

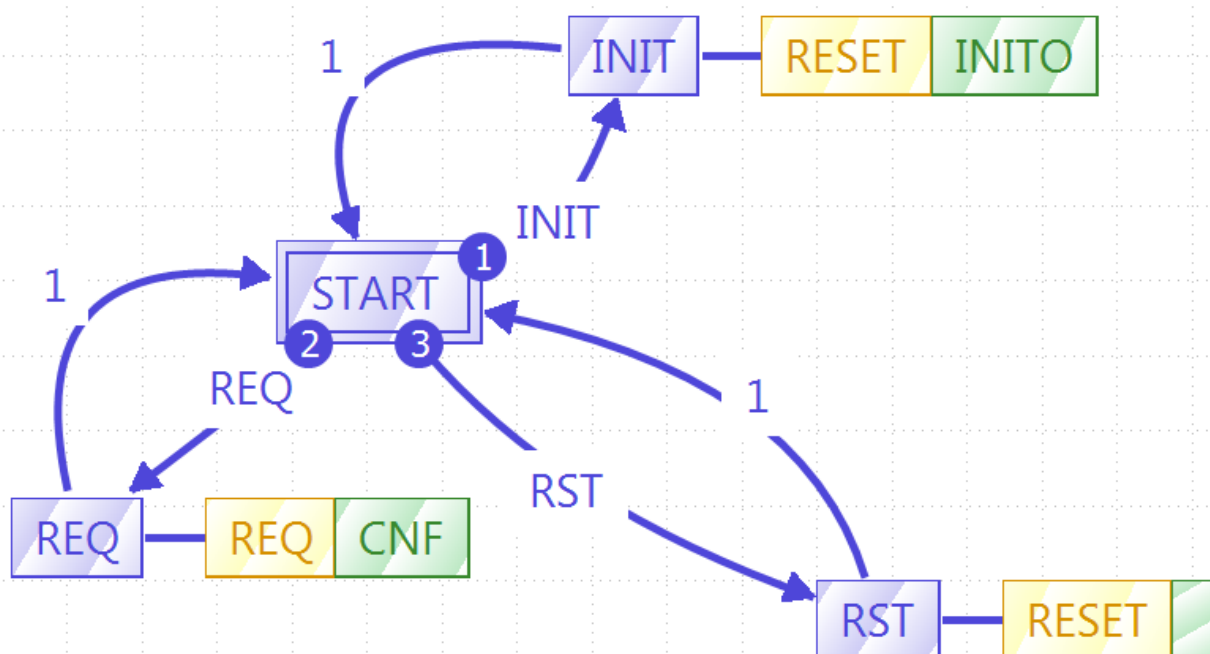
Для выполнения этих шагов необходимы базовые знания об использовании Eclipse 4diac.

Создание функционального блока ПИД

Первое, что нужно сделать, — это создать функциональный блок PID. Перейдите в «Файл» → «Создать» → «Новый тип». Во всплывающем меню выберите родительскую папку для хранения нового FB, в качестве названия типа выберите PID, а в качестве типа — «Базовый». Нажмите «Готово», и откроется окно для редактирования нового FB. Сначала необходимо настроить интерфейс. Воспользуйтесь приведённым ниже изображением, чтобы завершить настройку:



Интерфейс использует K_p , K_i и K_d в качестве настраиваемых констант вместо значений T_i и T_d . Это позволяет легко обнулить интегральную часть. Интервал — это время между запросами к FB, и ожидается, что он будет постоянным. После настройки интерфейса следующим шагом будет определение ECC и алгоритма. График ECC очень простой и должен выглядеть так, как показано на изображении ниже. Возможно, вам придётся создать новые алгоритмы с указанными именами, прежде чем заполнять график.



Когда график ECC готов, можно приступить к написанию кода, который будет управлять поведением FB. На вкладке «Свойства» под графиком ECC на одной из боковых вкладок под названием «Все внутренние переменные» добавьте две переменные типа REAL, которые будут называться LAST_ERROR и INTEGRATION_ACCUM. Эти переменные являются частью FB и сохраняют свои значения между событиями. Далее мы отредактируем алгоритмы, указанные на графике ECC. Простой алгоритм RESET просто инициализирует внутренние переменные. Код для сброса выглядит следующим образом:

```
LAST_ERROR := 0.0;
INTEGRATION_ACCUM := 0.0;
```

Алгоритм REQ выглядит следующим образом, но, поскольку он более сложный, мы объясним его по частям.

```
VAR
    proportionalPart: REAL;
    integralPart: REAL;
    derivativePart: REAL;
    Ydesired: REAL;
    errorFiltered: REAL;
END_VAR;

IF ABS(InError) <= Noise THEN
    errorFiltered := 0;
```

```

ELSE
    errorFiltered := InError;
END_IF;

(* calculate proportional part *)
proportionalPart := Kp * errorFiltered;

(* run integrator *)
integralPart := errorFiltered * Interval * Ki + INTEGRATION_ACCUM;

(* run derivative*)
derivativePart := (errorFiltered - LAST_ERROR) * Kd / Interval;

LAST_ERROR := errorFiltered;

(* calculate output Y *)
Ydesired := proportionalPart + integralPart + derivativePart;

(* check output for limits *)
IF Ydesired >= LIM_H THEN
    Y := LIM_H;
    IF Ki <> 0.0 THEN
        IF (errorFiltered < 0 AND Kp > 0) OR (errorFiltered > 0 AND Kp < 0) THEN
            INTEGRATION_ACCUM := integralPart;
        END_IF;
    ELSE
        INTEGRATION_ACCUM := 0.0;
    END_IF;
    LIM := TRUE;
ELSIF Ydesired <= LIM_L THEN
    Y := LIM_L;
    IF Ki <> 0.0 THEN
        IF (errorFiltered > 0 AND Kp > 0) OR (errorFiltered < 0 AND Kp < 0) THEN
            INTEGRATION_ACCUM := integralPart;
        END_IF;
    ELSE
        INTEGRATION_ACCUM := 0.0;
    END_IF;
    LIM := TRUE;
ELSE
    Y := Ydesired;
    INTEGRATION_ACCUM := integralPart;
    LIM := FALSE;
END_IF;

```

Часть между VAR и END_VAR — это объявление локальных переменных, которые будут использоваться внутри этого алгоритма. Значения не сохраняются между событиями.

Первый оператор IF

```

IF ABS(InError) <= Noise THEN
    errorFiltered := 0;
ELSE
    errorFiltered := InError;
END_IF;

```

отключает ПИД-регулятор, когда значение ошибки меньше шума, указанного на входе. Только когда ошибка превышает порог шума, выполняется вычисление ПИД-регулятора.

Следующая часть — это ядро вычисления ПИД-регулятора. Все значения сохраняются в локальных переменных перед их отражением в фактических выходных и внутренних переменных.

```

(* calculate proportional part *)
proportionalPart := Kp * errorFiltered;

(* run integrator *)
integralPart := errorFiltered * Interval * Ki + INTEGRATION_ACCUM;

(* run derivative*)
derivativePart := (errorFiltered - LAST_ERROR) * Kd / Interval;
LAST_ERROR := errorFiltered;

```

```

(* calculate output Y *)
Ydesired := proportionalPart + integralPart + derivativePart;
Последняя часть очень важна, потому что она ограничивает выходной сигнал и устраняет интегральную
задержку. Поскольку ШИМ-сигнал двигателя может иметь значения от -100 до 100, если выходной сигнал
выходит за пределы этого диапазона, ПИД-регулятор должен ограничивать это значение. Более подробную
информацию об интегральной задержке можно найти в интернете, но в целом, когда ошибка слишком
велика, интегральная часть работает не так, как должна, и приводит к неожиданному поведению
контроллера.
(* check output for limits *)
IF Ydesired >= LIM_H THEN
    Y := LIM_H;
    IF Ki <> 0.0 THEN
        IF (errorFiltered < 0 AND Kp > 0) OR (errorFiltered > 0 AND Kp < 0) THEN
            INTEGRATION_ACCUM := integralPart;
        END_IF;
    ELSE
        INTEGRATION_ACCUM := 0.0;
    END_IF;
    LIM := TRUE;
ELSIF Ydesired <= LIM_L THEN
    Y := LIM_L;
    IF Ki <> 0.0 THEN
        IF (errorFiltered > 0 AND Kp > 0) OR (errorFiltered < 0 AND Kp < 0) THEN
            INTEGRATION_ACCUM := integralPart;
        END_IF;
    ELSE
        INTEGRATION_ACCUM := 0.0;
    END_IF;
    LIM := TRUE;
ELSE
    Y := Ydesired;
    INTEGRATION_ACCUM := integralPart;
    LIM := FALSE;
END_IF;

```

IF и ELSEIF являются ограничениями для значения, и в ELSE обрабатываются допустимые значения.

Ограничение вывода является простым, но решение для завершения может привести к путанице. Что нужно сделать, так это активировать интегрирующую часть только тогда, когда желаемый результат находится в пределах допустимых значений (в ELSE) или когда он выходит за пределы допустимых значений, и ошибка заставляет вернуться обратно в пределы допустимых значений.

Информацию о защите от сбоев можно найти, например, [здесь](#).

После обработки алгоритмов FB готов к экспорту и компиляции с помощью 4diac FORTE.

Экспорт и компиляция с помощью 4diac FORTE для EV3

Как только FB будет готов, перейдите в «Файл» → «Экспорт...» и выберите 4diac IDE → 4diac IDE Тип экспорта. Затем просмотрите дерево папок слева и выберите PID FB, созданный справа. Выберите место для экспорта и нажмите «Готово». Чтобы подготовить исходный код для компиляции 4diac FORTE с помощью созданного FB, необходимо добавить сгенерированные файлы .cpp и .h в конфигурацию CMake. Самый простой способ сделать это — поместить оба файла в папку src/modules/lms_ev3 и добавить в эту папку файл CMakeLists.txt со следующей строкой в конце:

```
forte_add_sourcefile_hcpp(PID)
```

Далее нужно скомпилировать 4diac FORTE для EV3. Самый простой способ сделать это — скопировать весь код на EV3, получить инструменты для компиляции внутри EV3 и скомпилировать его. Этот процесс действительно медленный по сравнению с тем, если бы он выполнялся на обычном компьютере, но для однократной компиляции он намного быстрее, чем настройка всей среды кросс-компиляции, описанной здесь. Более подробная информация представлена в разделе выше.

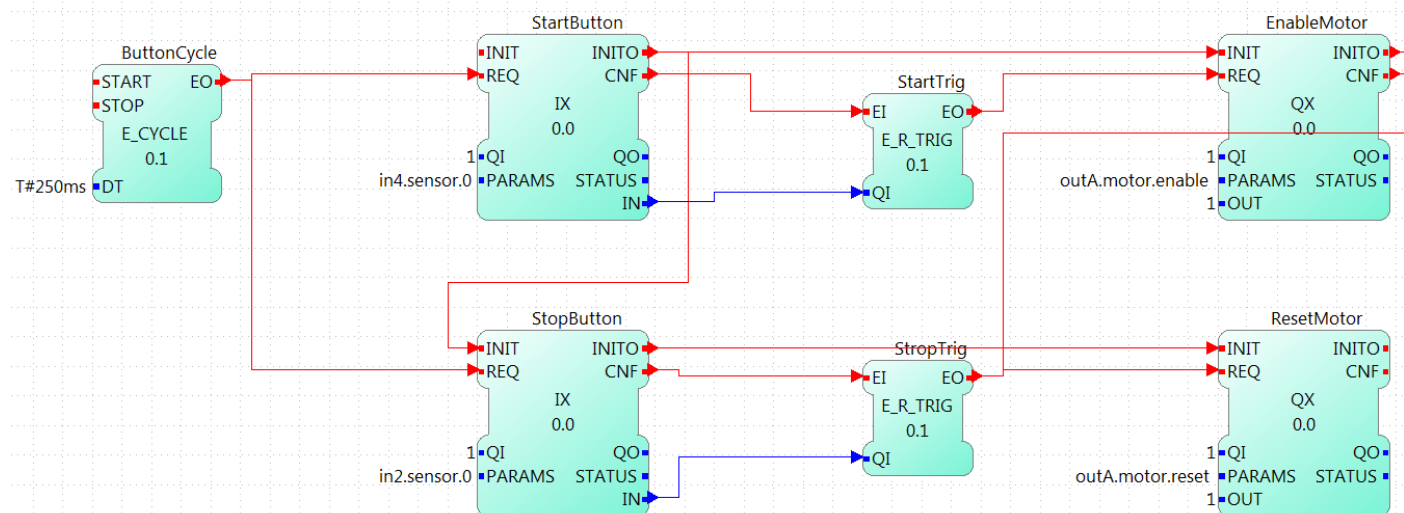
Создание приложения PID в 4diac IDE

После того как 4diac FORTE с PID будет запущен на EV3, пришло время создать приложение, которое будет управлять положением двигателя и сохранять значения. Приложение будет показано по частям для лучшего понимания.

ВКЛЮЧЕНИЕ/ВЫКЛЮЧЕНИЕ контроллера

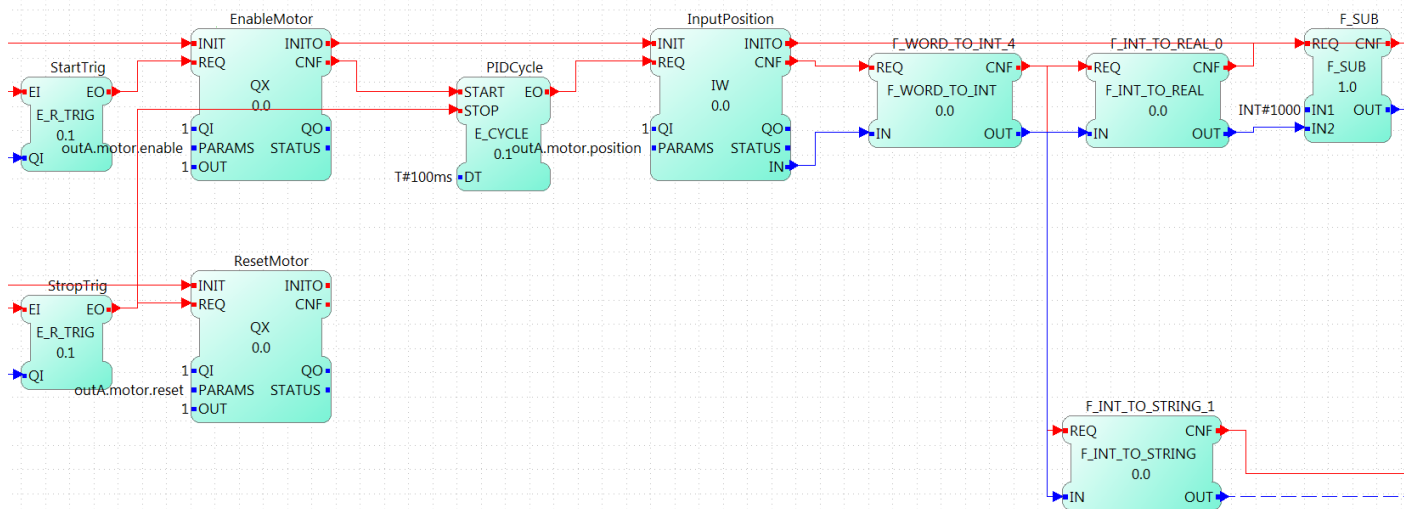
Первая часть приложения может быть выполнена различными способами, что позволяет лучше контролировать тесты. В этом примере он состоит из двух кнопок, подключенных к in2 и in4, одна из которых предназначена для запуска двигателя и определения PID, а другая - для сброса всех параметров (что также останавливает двигатель). У него есть цикл считывания данных с обеих кнопок, ожидающий срабатывания триггера на них, который отправляет событие на включение или сброс двигателя.

Инициализация кнопки «Пуск» выполняется с помощью кнопки «Холодный запуск» при настройке системы.



Чтение данных системы

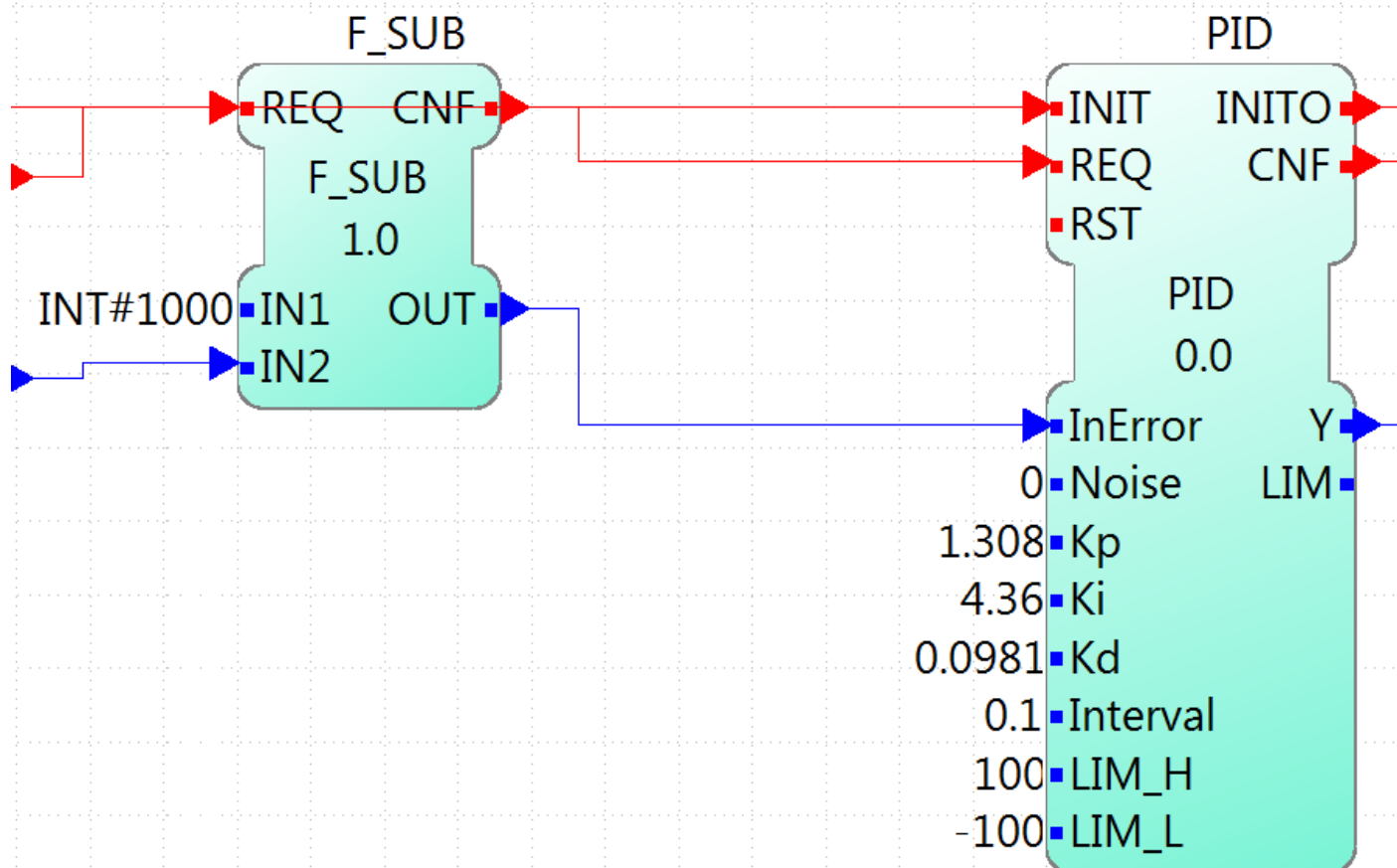
Эта часть начинается с блока цикла, который каждые 100 мс генерирует событие, запускающее вычисление PID. Цикл начинается, когда двигатель включается, и заканчивается, когда двигатель выключается. Цикл запускает считывание данных системы, в данном случае положения двигателя. Эта позиция считывается как WORD (на самом деле это должно быть значение типа double, но пока его можно ограничить до WORD), а затем преобразуется в INT, допускающий отрицательные значения (блок WORD_TO_INT был обновлён, чтобы допускать отрицательные значения), а затем в REAL для вычисления разницы с заданной пользователем точкой (INT#1000 в примере). Блок F_SUB ожидает значения типа REAL, а во входных данных IN1 задаётся точка. Обратите внимание, что на выходе блока WORD_TO_INT есть ещё одно преобразование в строку. Это преобразование отправляется на ПК для хранения значений координат и их построения. В последней части будет показано, как это делается.



Входное значение ПИД-регулятора

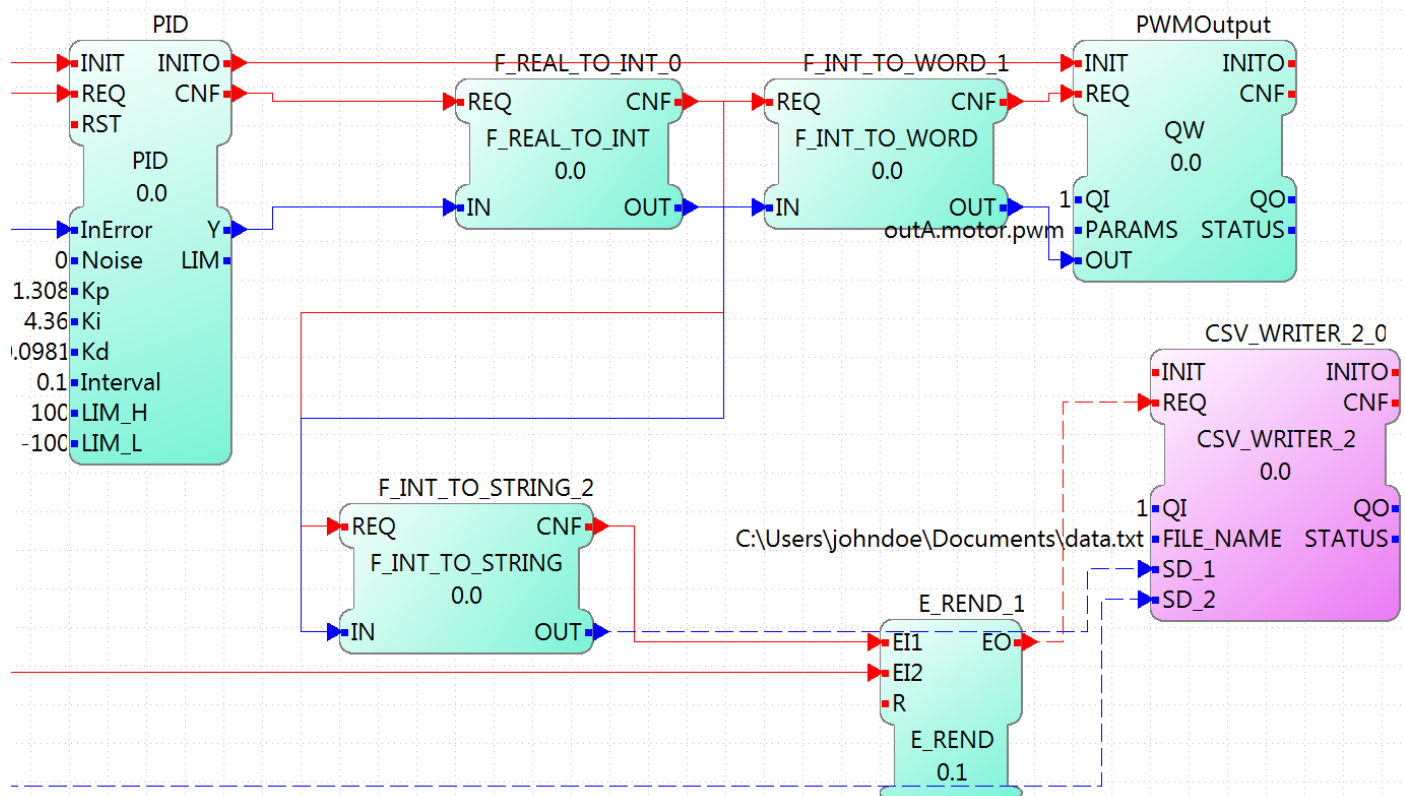
Эта часть вычисляет ошибку как разницу между заданным значением и положением, считываемым с двигателя, и отправляет её в ПИД-регулятор. Остальные входные значения задаются пользователем. Коэффициенты (Kp, Ki и Kd) настраиваются в соответствии с методом. В этом примере использовался метод Цайглера-Николса, который будет рассмотрен позже. Интервал составляет 0,1, что соответствует 100 мс цикла

ПИД-регулятора. Пределы соответствуют предельным значениям выходного сигнала, в данном случае ШИМ-сигнала двигателя.



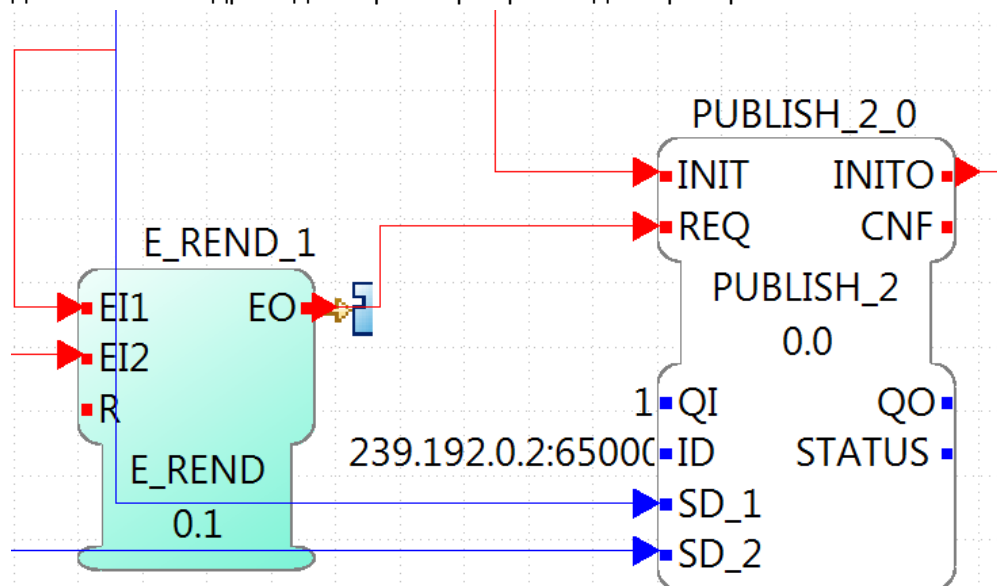
Вывод данных ПИД-регулятора

Последняя часть соответствует выводу данных ПИД-регулятора и отправке информации в журнал на ПК. Данные ПИД-регулятора преобразуются в INT, а затем в WORD, чтобы затем записать их в ШИМ-сигнал двигателя. Кроме того, значение INT преобразуется в STRING, аналогично положению во входной части, и после выполнения обоих преобразований (положения и вывода) они записываются в файл с помощью функции **CSV_WRITER_2** из модуля **UTILS**. Этот FB позволяет записывать 2 значения (**SD_1** и **SD_2**) в формате CSV. Блок другого цвета, потому что он находится на другом устройстве, а не на EV3.

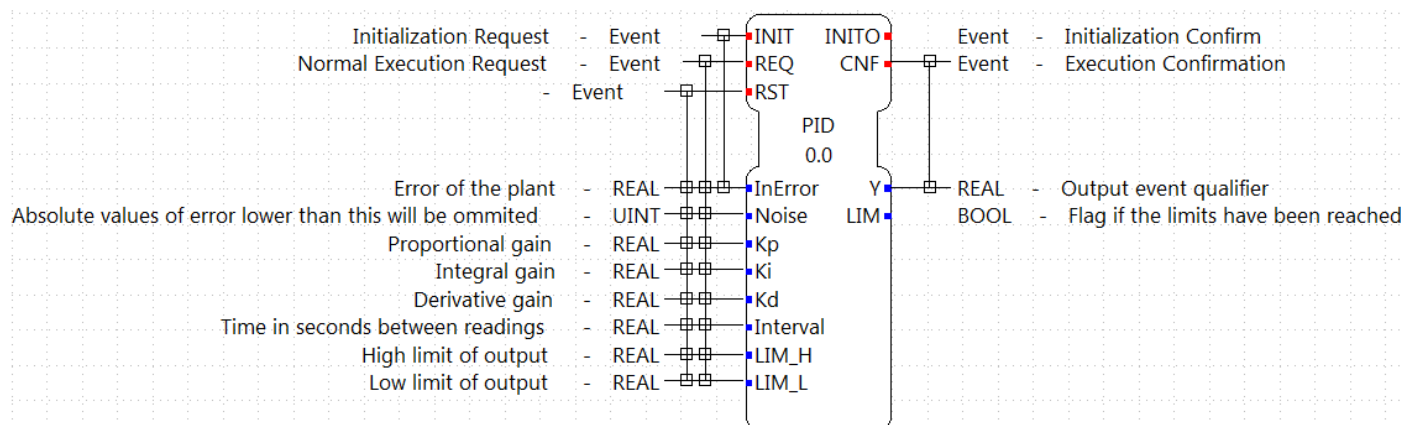


Настройка системы

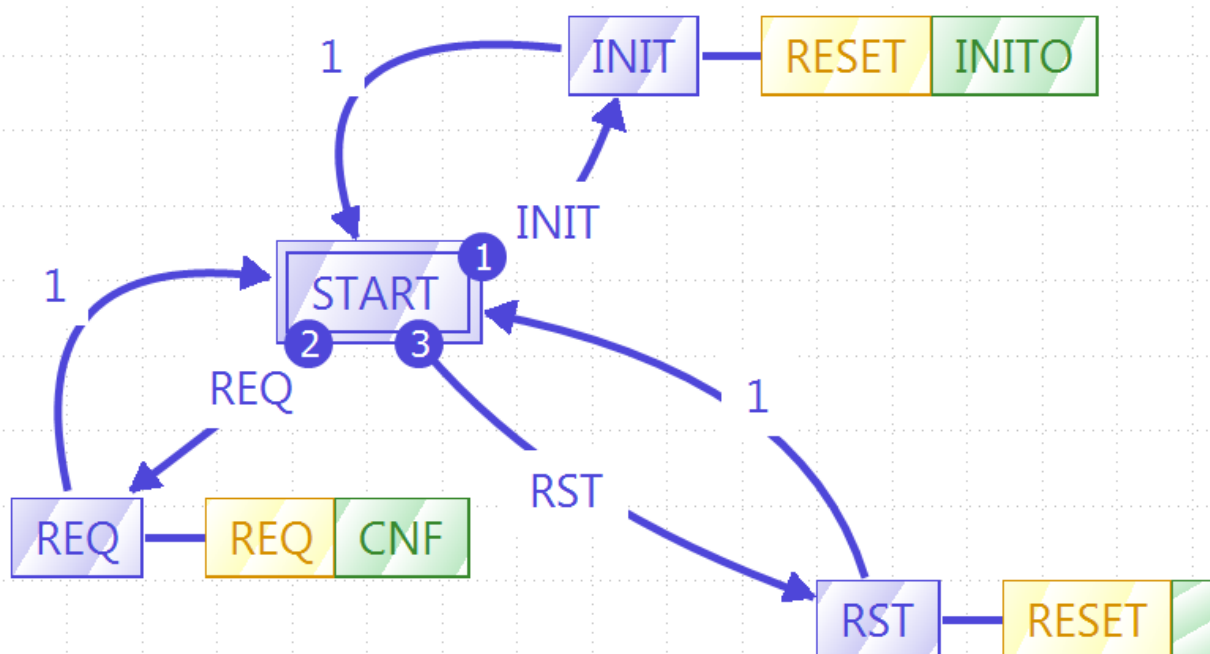
Теперь, когда приложение готово, в системе необходимо выполнить некоторые другие настройки. Во-первых, нужно создать оба устройства и подключить их с помощью сегмента Ethernet. В ресурсе EV3 сначала подключите COLD START к инициализации кнопки Start, как упоминалось ранее. Другая настройка в ресурсе EV3 — публикация преобразований положения и выхода PID. С помощью PUBLISH_2_0 с входными данными и идентификатором многоадресной рассылки UDP (в данном случае 239.192.0.2:65000), а также REQ, запускаемого FB E_RENDER из приложения. Таким образом, EV3 будет отправлять положение и выходные данные PID на адрес идентификатора при каждом преобразовании.



Конфигурация ресурса ПК, на котором будут храниться данные, состоит из FB SUBSCRIBE_2, который будет получать информацию, опубликованную EV3. Идентификатор совпадает с идентификатором издателя, а выходные данные преобразуются с помощью STRING2STRING, поскольку выходные данные имеют тип ANY. После преобразования данные записываются в файл. Имейте в виду, что FB CSV_WRITER не сразу очищает выходные данные, поэтому приложение 4diac FORTE должно завершиться, чтобы прочитать информацию в файле.



Интерфейс использует Kp, Ki и Kd в качестве настраиваемых констант вместо значений Ti и Td. Это позволяет легко обнулить интегральную часть. Интервал — это время между запросами к FB, и ожидается, что он будет постоянным. После настройки интерфейса следующим шагом будет определение ECC и алгоритма. График ECC очень простой и должен выглядеть так, как показано на изображении ниже. Возможно, вам придётся создать новые алгоритмы с указанными именами, прежде чем заполнять график.



Когда график ECC готов, можно приступать к написанию кода, который будет управлять поведением FB. На вкладке «Свойства» под графиком ECC на одной из боковых вкладок под названием «Все внутренние переменные» добавьте две переменные типа REAL, которые будут называться LAST_ERROR и INTEGRATION_ACCUM. Эти переменные являются частью FB и сохраняют свои значения между событиями.

Далее мы отредактируем алгоритмы, указанные на графике ECC. Простой алгоритм RESET просто инициализирует внутренние переменные. Код для сброса выглядит следующим образом:

```
LAST_ERROR := 0.0;
INTEGRATION_ACCUM := 0.0;
```

Алгоритм REQ выглядит следующим образом, но, поскольку он более сложный, мы объясним его по частям.

```
VAR
    proportionalPart: REAL;
    integralPart: REAL;
    derivativePart: REAL;
    Ydesired: REAL;
    errorFiltered: REAL;
END_VAR;

IF ABS(InError) <= Noise THEN
    errorFiltered := 0;
ELSE
    errorFiltered := InError;
```

```

END_IF;

(* calculate proportional part *)
proportionalPart := Kp * errorFiltered;

(* run integrator *)
integralPart := errorFiltered * Interval * Ki + INTEGRATION_ACCUM;

(* run derivative*)
derivativePart := (errorFiltered - LAST_ERROR) * Kd / Interval;

LAST_ERROR := errorFiltered;

(* calculate output Y *)
Ydesired := proportionalPart + integralPart + derivativePart;

(* check output for limits *)
IF Ydesired >= LIM_H THEN
    Y := LIM_H;
    IF Ki <> 0.0 THEN
        IF (errorFiltered < 0 AND Kp > 0) OR (errorFiltered > 0 AND Kp < 0) THEN
            INTEGRATION_ACCUM := integralPart;
        END_IF;
    ELSE
        INTEGRATION_ACCUM := 0.0;
    END_IF;
    LIM := TRUE;
ELSIF Ydesired <= LIM_L THEN
    Y := LIM_L;
    IF Ki <> 0.0 THEN
        IF (errorFiltered > 0 AND Kp > 0) OR (errorFiltered < 0 AND Kp < 0) THEN
            INTEGRATION_ACCUM := integralPart;
        END_IF;
    ELSE
        INTEGRATION_ACCUM := 0.0;
    END_IF;
    LIM := TRUE;
ELSE
    Y := Ydesired;
    INTEGRATION_ACCUM := integralPart;
    LIM := FALSE;
END_IF;

```

Часть между VAR и END_VAR — это объявление локальных переменных, которые будут использоваться внутри этого алгоритма. Значения не сохраняются между событиями. Первый оператор IF

```

IF ABS(InError) <= Noise THEN
    errorFiltered := 0;
ELSE
    errorFiltered := InError;
END_IF;

```

отключает ПИД-регулятор, когда значение ошибки меньше шума, указанного на входе. Только когда ошибка превышает порог шума, выполняется вычисление ПИД-регулятора.

Следующая часть — это ядро вычисления ПИД-регулятора. Все значения сохраняются в локальных переменных перед их отражением в фактических выходных и внутренних переменных.

```

(* calculate proportional part *)
proportionalPart := Kp * errorFiltered;

(* run integrator *)
integralPart := errorFiltered * Interval * Ki + INTEGRATION_ACCUM;

(* run derivative*)
derivativePart := (errorFiltered - LAST_ERROR) * Kd / Interval;
LAST_ERROR := errorFiltered;

(* calculate output Y *)
Ydesired := proportionalPart + integralPart + derivativePart;

```

Последняя часть очень важна, потому что она ограничивает выходной сигнал и устраняет интегральную задержку. Поскольку ШИМ-сигнал двигателя может иметь значения от -100 до 100, если выходной сигнал выходит за пределы этого диапазона, ПИД-регулятор должен ограничивать это значение. Более подробную информацию об интегральной задержке можно найти в интернете, но в целом, когда ошибка слишком велика, интегральная часть работает не так, как должна, и приводит к неожиданному поведению контроллера.

```
(* check output for limits *)
IF Ydesired >= LIM_H THEN
    Y := LIM_H;
    IF Ki <> 0.0 THEN
        IF (errorFiltered < 0 AND Kp > 0) OR (errorFiltered > 0 AND Kp < 0) THEN
            INTEGRATION_ACCUM := integralPart;
        END_IF;
    ELSE
        INTEGRATION_ACCUM := 0.0;
    END_IF;
    LIM := TRUE;
ELSIF Ydesired <= LIM_L THEN
    Y := LIM_L;
    IF Ki <> 0.0 THEN
        IF (errorFiltered > 0 AND Kp > 0) OR (errorFiltered < 0 AND Kp < 0) THEN
            INTEGRATION_ACCUM := integralPart;
        END_IF;
    ELSE
        INTEGRATION_ACCUM := 0.0;
    END_IF;
    LIM := TRUE;
ELSE
    Y := Ydesired;
    INTEGRATION_ACCUM := integralPart;
    LIM := FALSE;
END_IF;
```

IF и ELSEIF являются ограничениями для значения, и в ELSE обрабатываются допустимые значения.

Ограничение вывода является простым, но решение для завершения может привести к путанице. Что нужно сделать, так это активировать интегрирующую часть только тогда, когда желаемый результат находится в пределах допустимых значений (в ELSE) или когда он выходит за пределы допустимых значений, и ошибка заставляет вернуться обратно в пределы допустимых значений.

Информацию о защите от сбоев можно найти, например, [здесь](#).

После выполнения алгоритмов FB готов к экспорту и компиляции с помощью 4diac FORTE.

Экспорт и компиляция с помощью 4diac FORTE для EV3

Когда FB будет готов, перейдите в «Файл» → «Экспорт...» и выберите 4diac IDE → 4diac IDE Тип экспорта. Затем просмотрите дерево папок слева и выберите PID FB, созданный справа. Выберите место для экспорта и нажмите «Готово».

Чтобы подготовить исходный код для компиляции 4diac FORTE с созданным FB, необходимо добавить сгенерированные файлы .cpp и .h в конфигурацию CMake. Самый простой способ сделать это — поместить оба файла в папку src/modules/lms_ev3 и добавить в эту папку файл CMakeLists.txt со следующей строкой в конце:\nforte_add_sourcefile_hcpp(PID)

Следующее, что нужно сделать, — это скомпилировать 4diac FORTE для EV3. Самый простой способ сделать это — скопировать весь код на EV3, получить инструменты для компиляции внутри EV3 и скомпилировать его. Этот процесс очень медленный по сравнению с компиляцией на обычном компьютере, но для однократной компиляции он намного быстрее, чем настройка всей среды кросс-компиляции, описанной [здесь](#). Более подробная информация представлена в разделе [выше](#).

Создание приложения PID в 4diac IDE

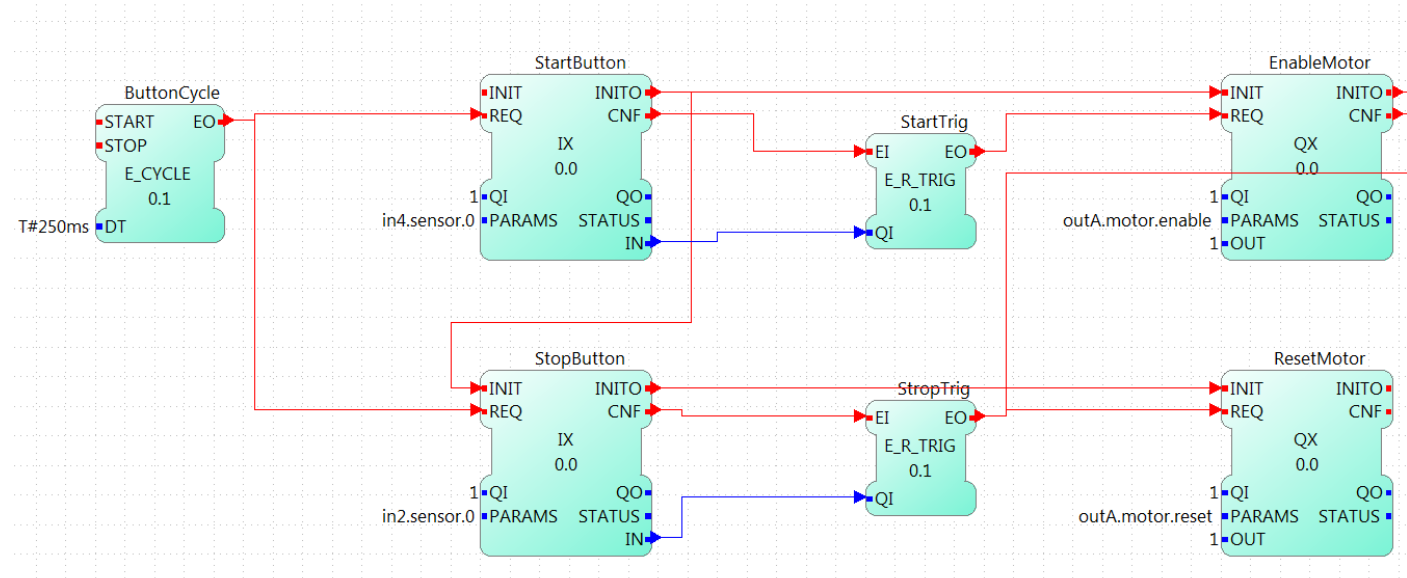
После того как 4diac FORTE с PID-регулятором запустится в EV3, можно будет создать приложение, которое будет управлять положением двигателя и сохранять значения. Приложение будет показано по частям для лучшего понимания.

Включение/выключение контроллера

Первая часть приложения может быть реализована по-разному, что позволяет лучше контролировать тесты. В этом примере он состоит из двух кнопок, подключенных к in2 и in4, одна из которых предназначена для

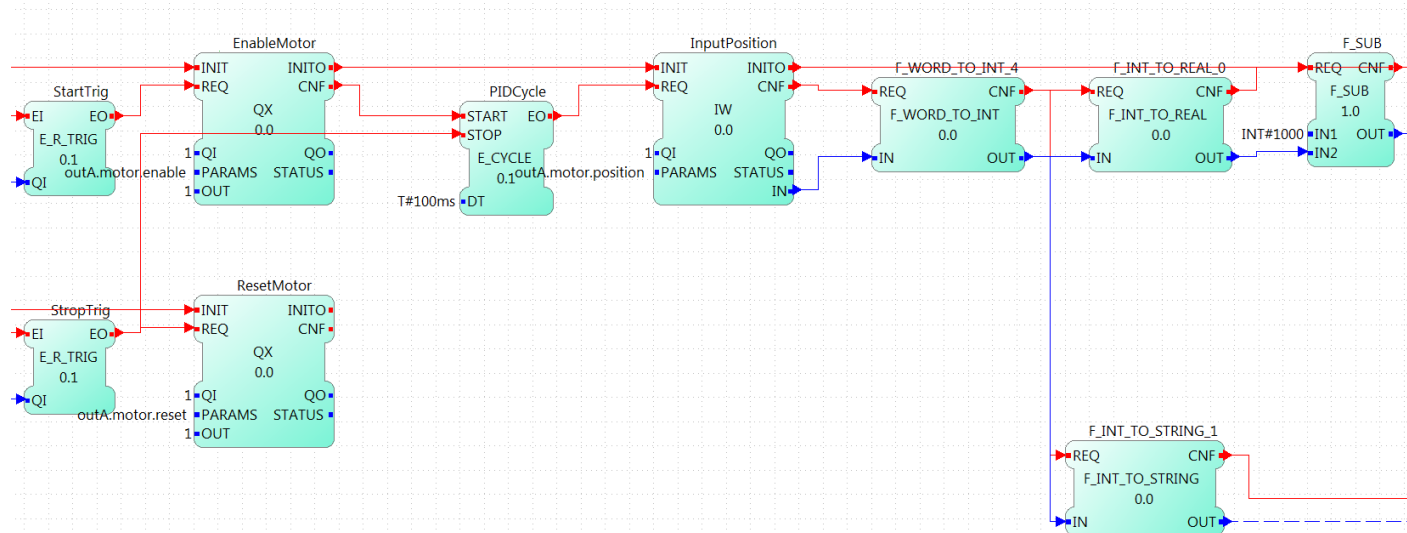
запуска двигателя и определения PID, а другая - для сброса всех параметров (что также останавливает двигатель). В нем есть цикл считывания данных с обеих кнопок в ожидании срабатывания на них, которое отправляет событие на включение или сброс двигателя.

Инициализация кнопки запуска FB выполняется с кнопки ХОЛОДНОГО ЗАПУСКА FB при настройке системы.



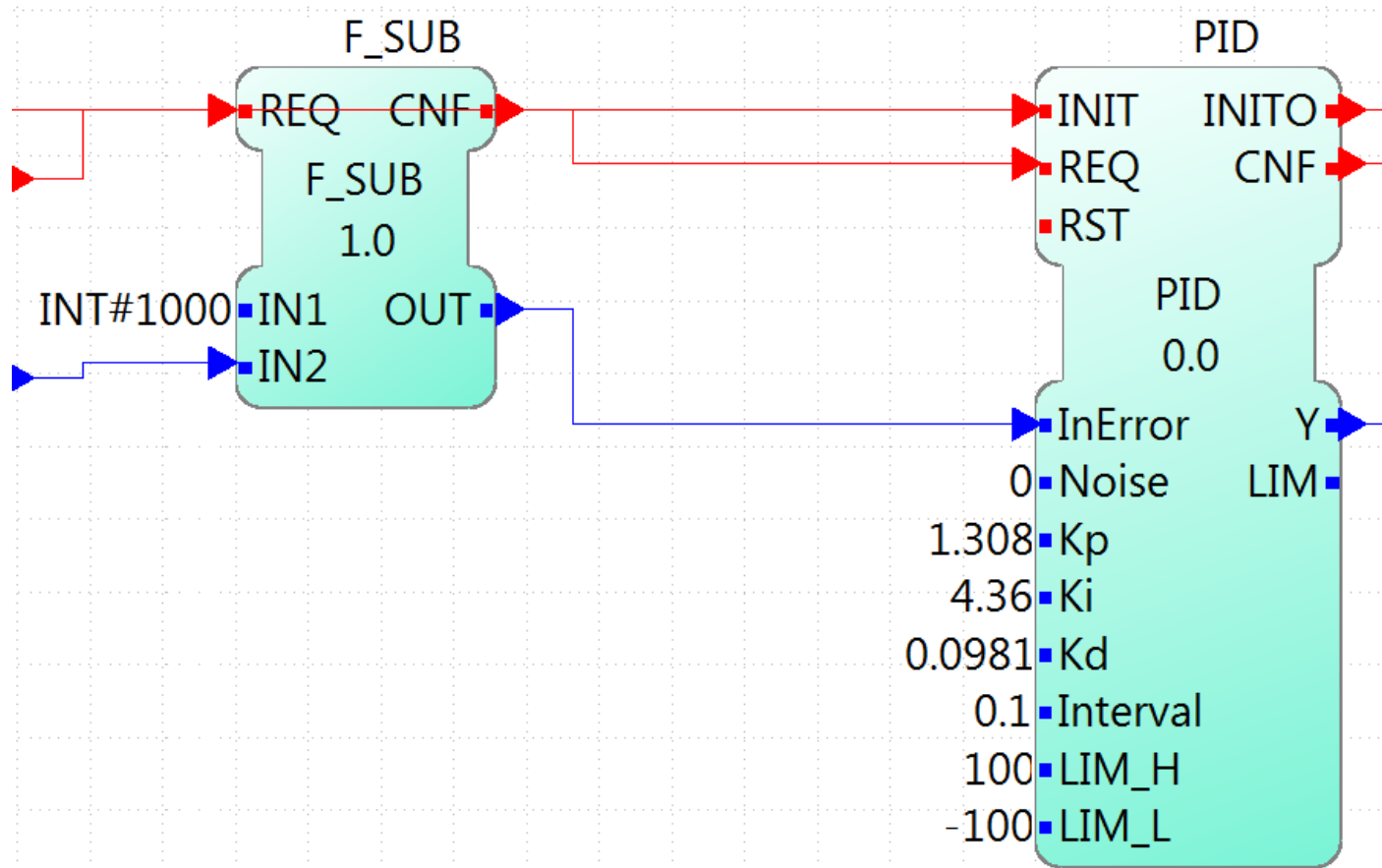
Считывание данных из системы

Эта часть начинается с блока цикла, который каждые 100 мс генерирует событие, запускающее вычисление PID. Цикл начинается, когда двигатель включается, и заканчивается, когда двигатель выключается. Цикл запускает считывание данных системы, в данном случае положения двигателя. Эта позиция считывается как WORD (на самом деле это должно быть значение типа double, но пока его можно ограничить до WORD), а затем преобразуется в INT, допускающий отрицательные значения (блок **WORD_TO_INT** был обновлён, чтобы допускать отрицательные значения), а затем в REAL для вычисления разницы с заданной пользователем точкой (INT#1000 в примере). Блок **F_SUB** ожидает значения типа REAL, а во входных данных **IN1** задаётся точка. Обратите внимание, что на выходе блока **WORD_TO_INT** есть ещё одно преобразование в строку. Это преобразование отправляется на ПК для хранения значений положения и их построения. В последней части будет показано, как это делается.



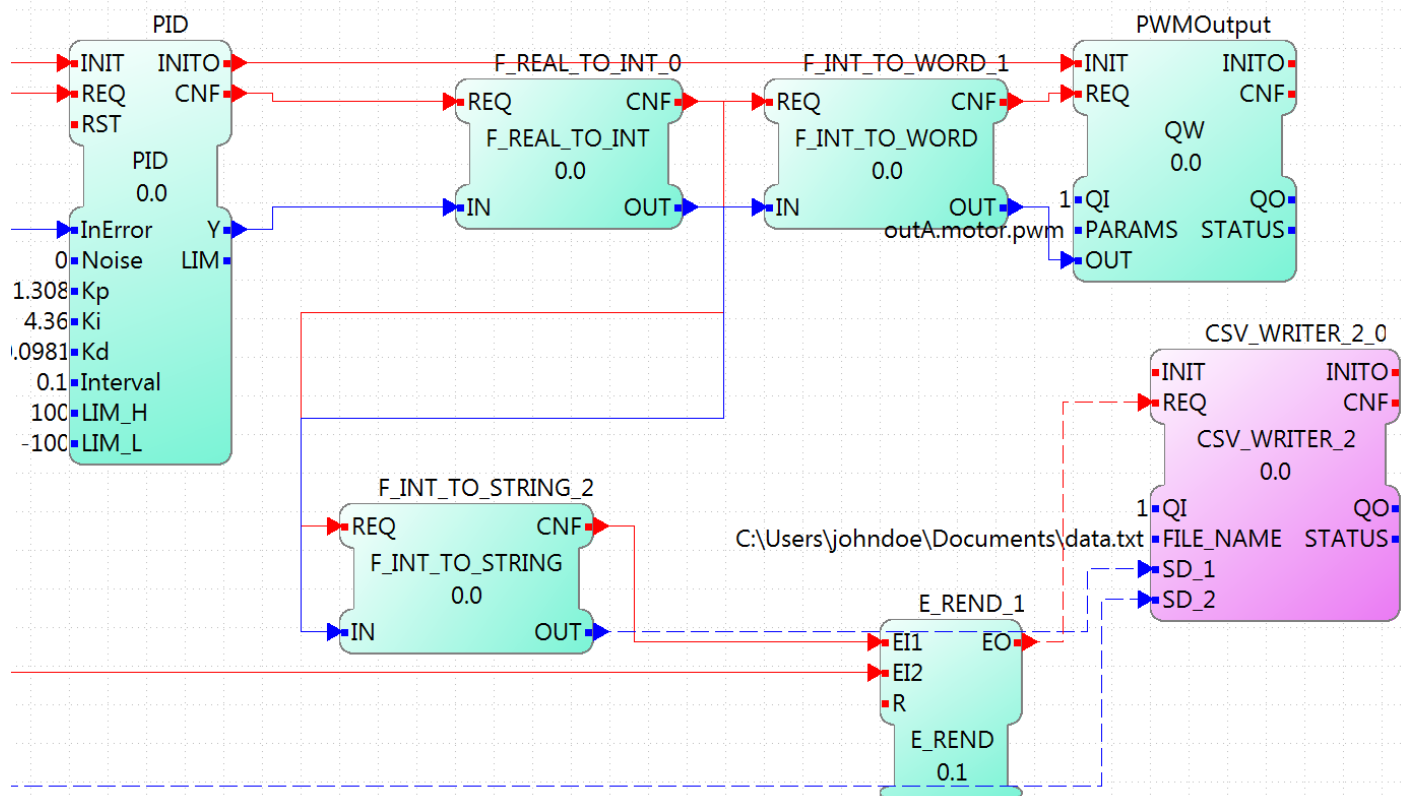
Входные данные ПИД-регулятора

Эта часть принимает ошибку как разницу между заданным значением и положением, считанным с двигателя, и отправляет её в ПИД-регулятор. Остальные входы задаются пользователем. Коэффициенты (K_p , K_i и K_d) настраиваются в соответствии с методом. В этом примере использовался метод Цайглера-Николса, который будет рассмотрен позже. Интервал равен 0,1, что соответствует 100 мс цикла PID. Пределы соответствуют предельным значениям выхода, в данном случае ШИМ-сигнала двигателя.



Выходные данные PID-регулятора

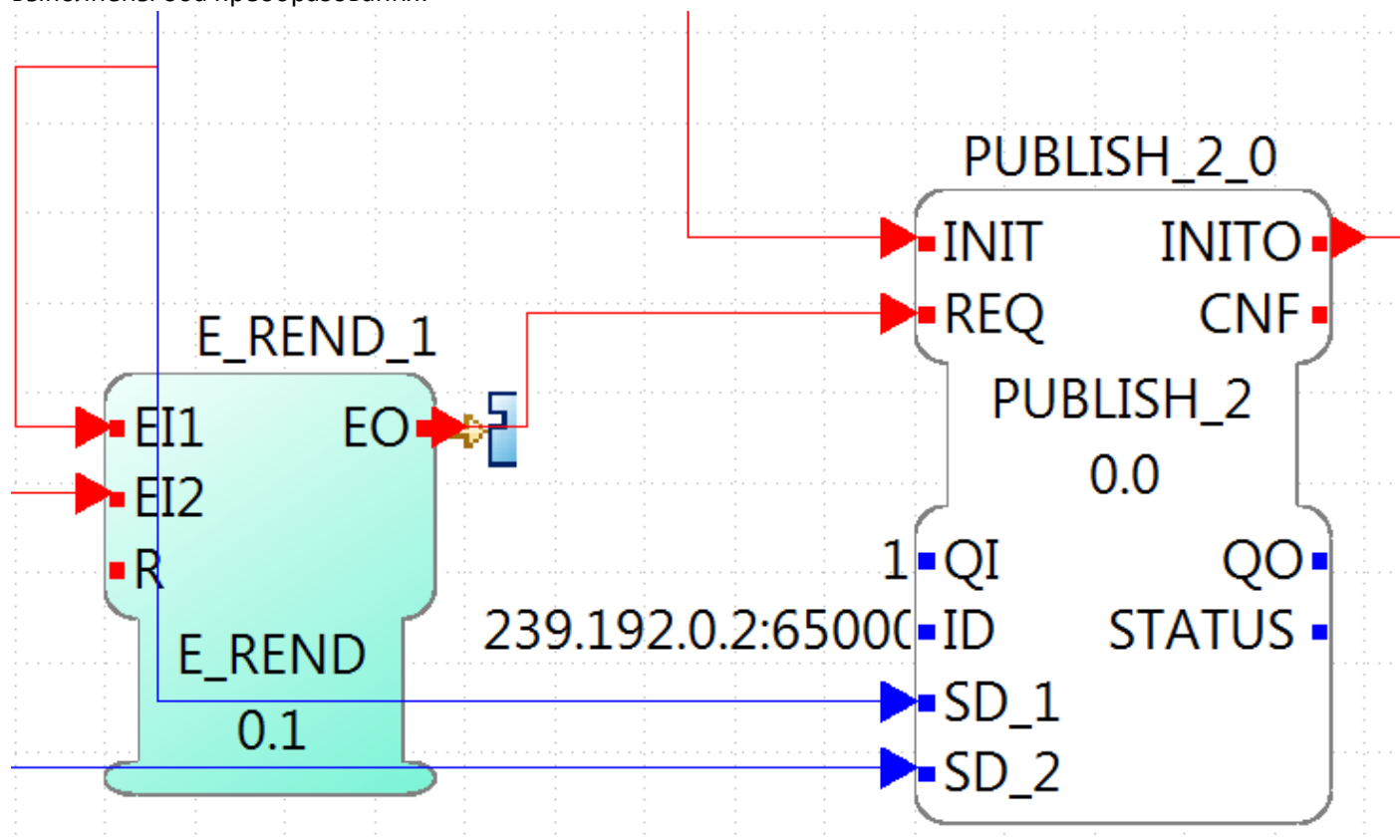
Последняя часть соответствует выходу ПИД-регулятора и отправке информации в журнал на ПК. Выход ПИД-регулятора преобразуется в INT, а затем в WORD, чтобы затем записать выход в ШИМ-сигнал двигателя. Кроме того, значение INT преобразуется в STRING, аналогично положению во входной части, и после выполнения обоих преобразований (положения и выхода) они записываются в файл с помощью функции CSV_WRITER_2 из модуля UTILS. Этот FB позволяет записывать 2 значения (SD_1 и SD_2) в формате CSV. Блок выделен другим цветом, потому что находится на другом устройстве, а не на EV3.



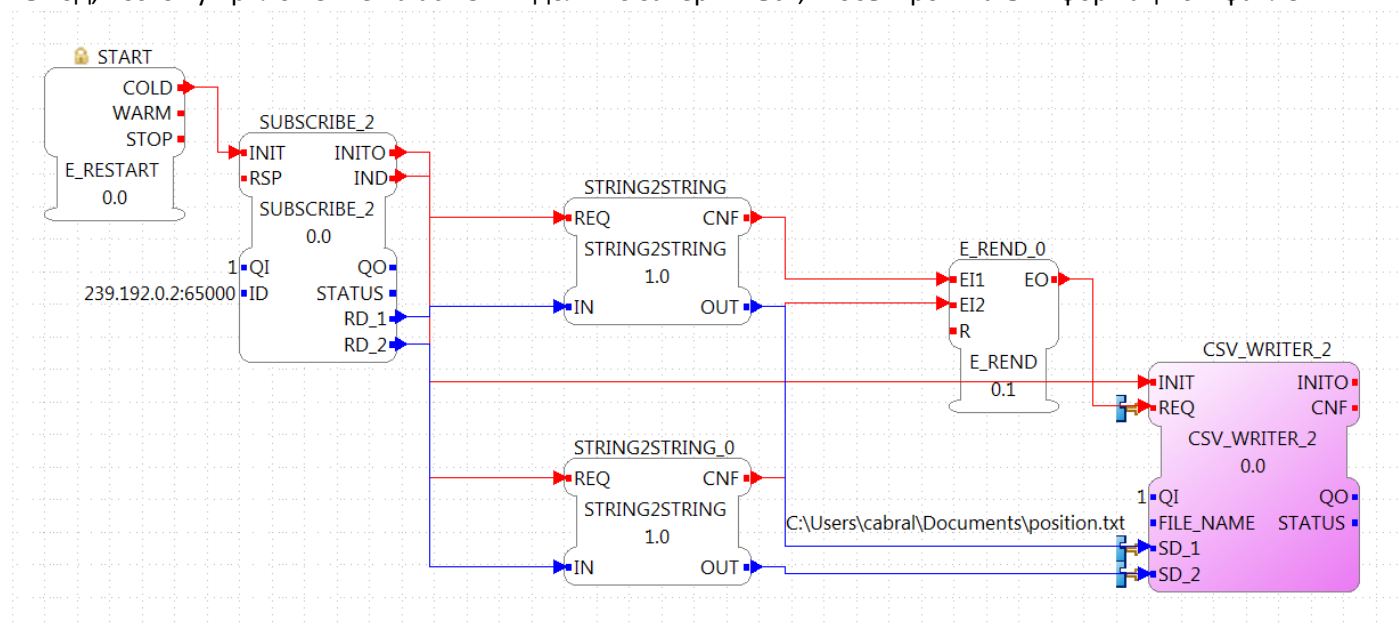
Настройка системы

Теперь, когда приложение готово, в системе необходимо выполнить некоторые другие настройки. Сначала нужно создать оба устройства и подключить их с помощью сегмента Ethernet. В ресурсе EV3 сначала подключите COLD START к инициализации кнопки Start, как упоминалось ранее.

Другой конфигурацией в ресурсе EV3 является публикация преобразований положения и выходных данных PID. Используя **PUBLISH_2_0** как с входными данными, так и с UDP-идентификатором многоадресной рассылки (в данном случае 239.192.0.2:65000), а также запрос запроса, иницируемый **E_REND** FB из приложения. При этом EV3 будет отправлять положение и выходные данные PID на адрес ID каждый раз, когда будут выполнены оба преобразования.



Конфигурация ресурса ПК, на котором будут храниться данные, состоит из FB SUBSCRIBE_2, который будет получать информацию, опубликованную EV3. Идентификатор совпадает с идентификатором издателя, а выходные данные преобразуются с помощью STRING2STRING, поскольку выходные данные имеют тип ANY. После преобразования данные записываются в файл. Имейте в виду, что CSV_WRITER FB не сразу очищает вывод, поэтому приложение 4diac FORTE должно завершиться, чтобы прочитать информацию в файле.

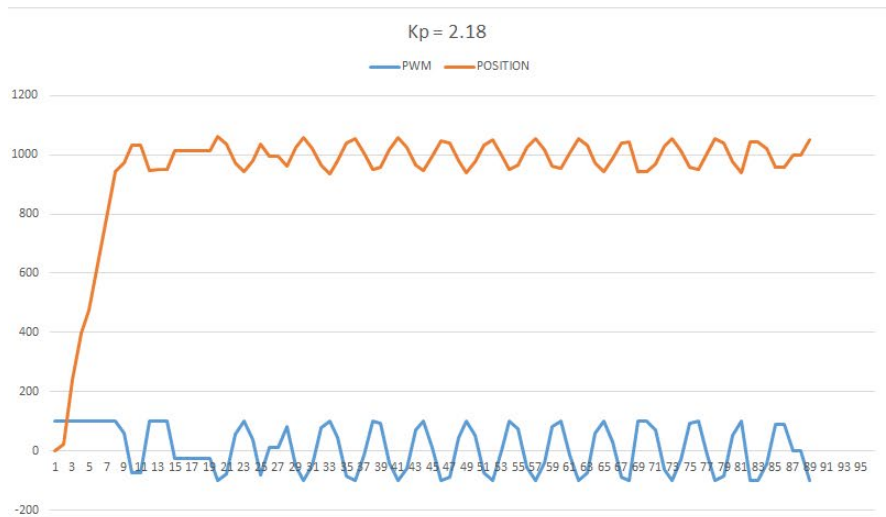
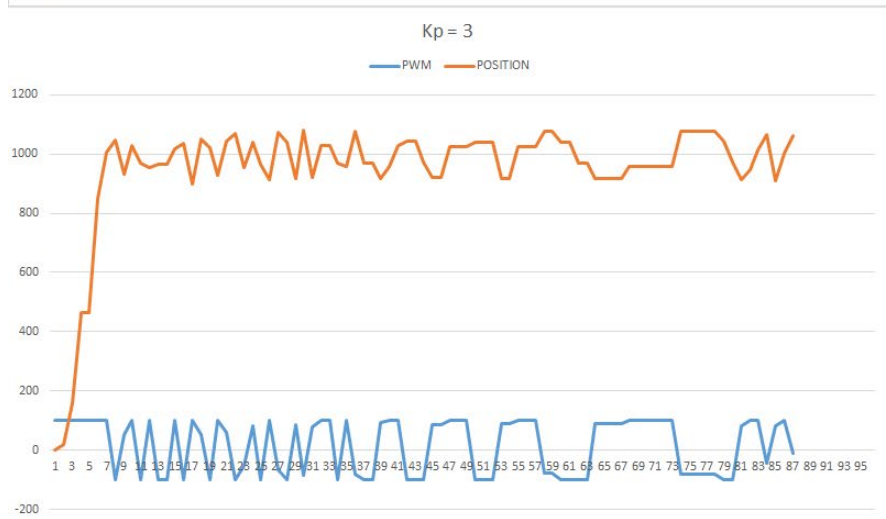
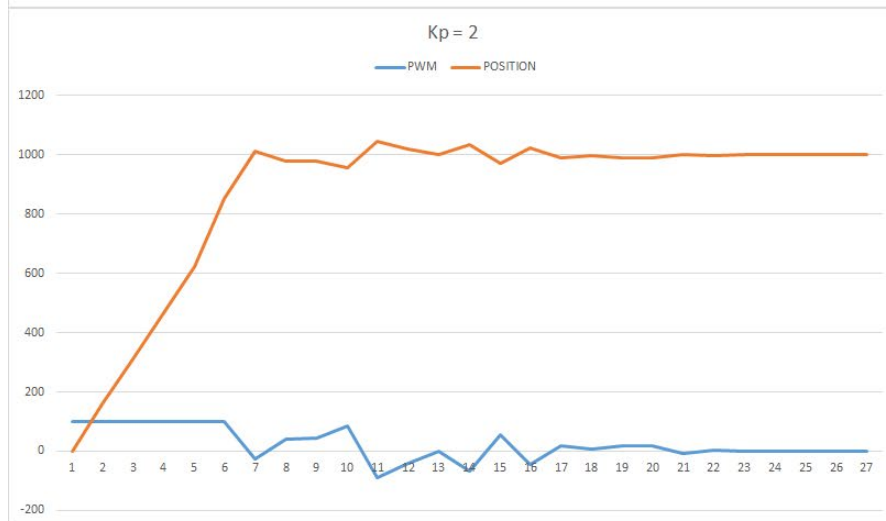
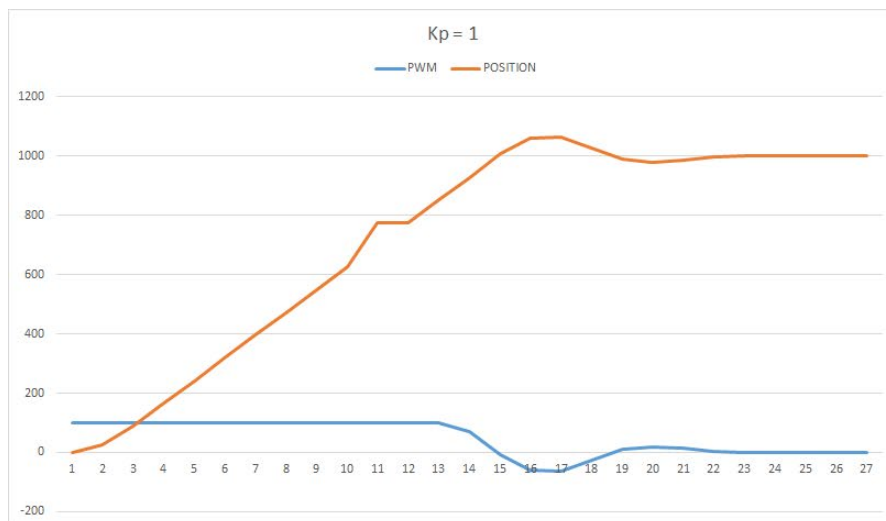


Примечание! На компьютерах с Windows используемый многоадресный адрес не всегда определяется, и пакеты с EV3 не поступают в приложение 4diac FORTE на ПК. Это часто происходит, если ПК подключен к EV3 через специальное соединение. Чтобы решить эту проблему, попробуйте добавить маршрут адреса со шлюзом на свой компьютер, как показано в разделе «Устранение неполадок» в нижней части примера «[Управление трафиком с помощью BVB](#)».

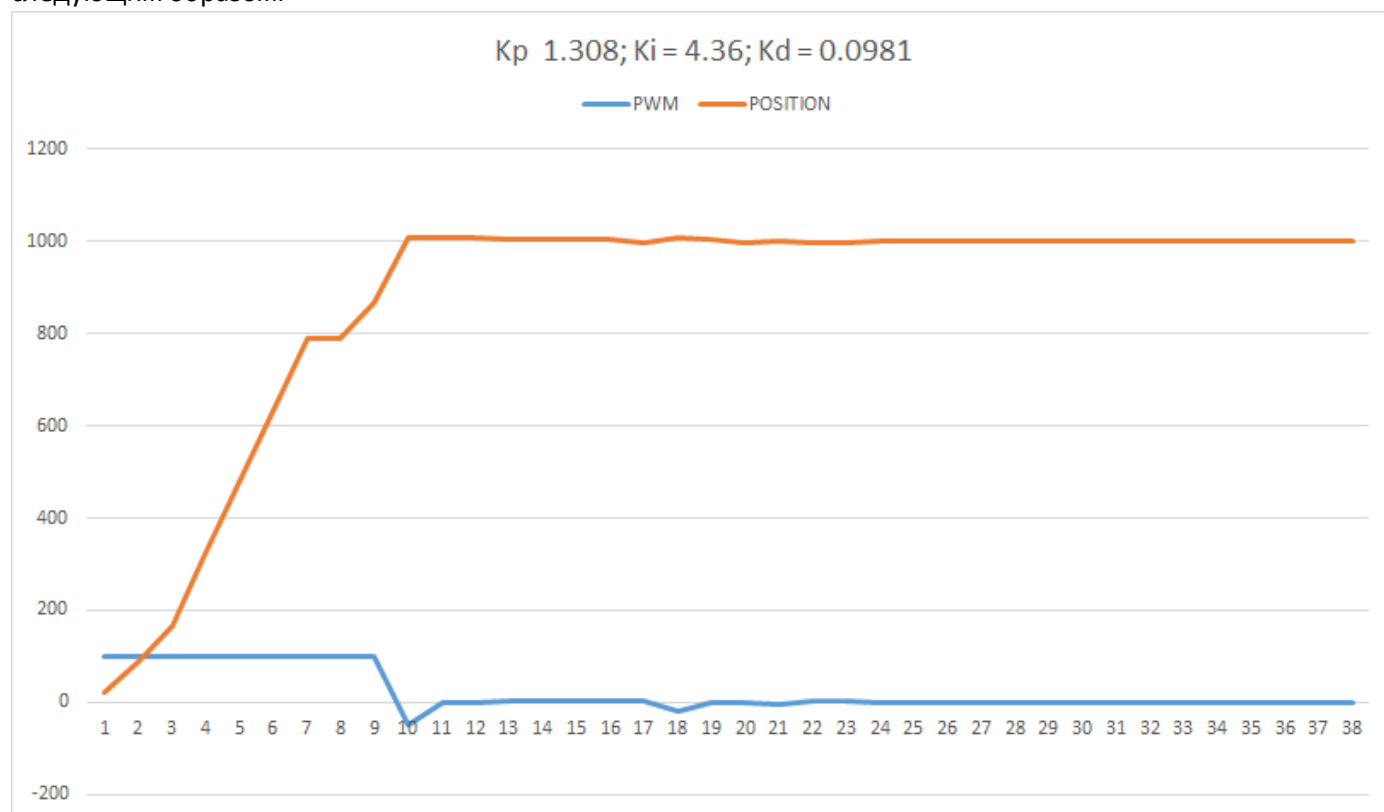
Настройка ПИД-регулятора

После того как приложение будет готово и заработает, для настройки ПИД-регулятора можно использовать [метод Циглера-Николса](#). Основная идея заключается в том, чтобы отключить интегральную и производную части и увеличивать K_p до тех пор, пока на выходе не появятся стабильные и постоянные колебания. Значение K_p , при котором на выходе появляются такие колебания, называется K_u , а период колебаний при таком K_u называется T_u . При этих двух значениях K_p , K_i и K_d выбираются в соответствии с таблицей, приведённой выше. Помните, что $K_d = K_p/T_i$, а $K_i = K_p \cdot T_d$.

Задающая точка — это положение 1000. Первое значение K_p равно 1 и даёт результат без колебаний. При $K_p = 2$ процесс занимает больше времени, но всё равно без стабильных колебаний. При $K_p = 3$ результаты показывают стабильные колебания. Таким образом, критическое значение K_u должно находиться в диапазоне от 2 до 3. После бинарного поиска в диапазоне от 2 до 3 было обнаружено, что $K_u = 2,18$, и его график также показан ниже. Из данных видно, что колебания имеют период 600 мс (0,6 с).



Значения K_p , K_i и K_d для ПИД-регулятора рассчитываются по таблице с учётом значений K_u и T_u . Значения $K_p = 1,308$, $K_i = 4,36$ и $K_d = 0,0981$. С этими значениями ПИД-регулятор снова тестируется, и результат выглядит следующим образом:



Х+3 Учебник

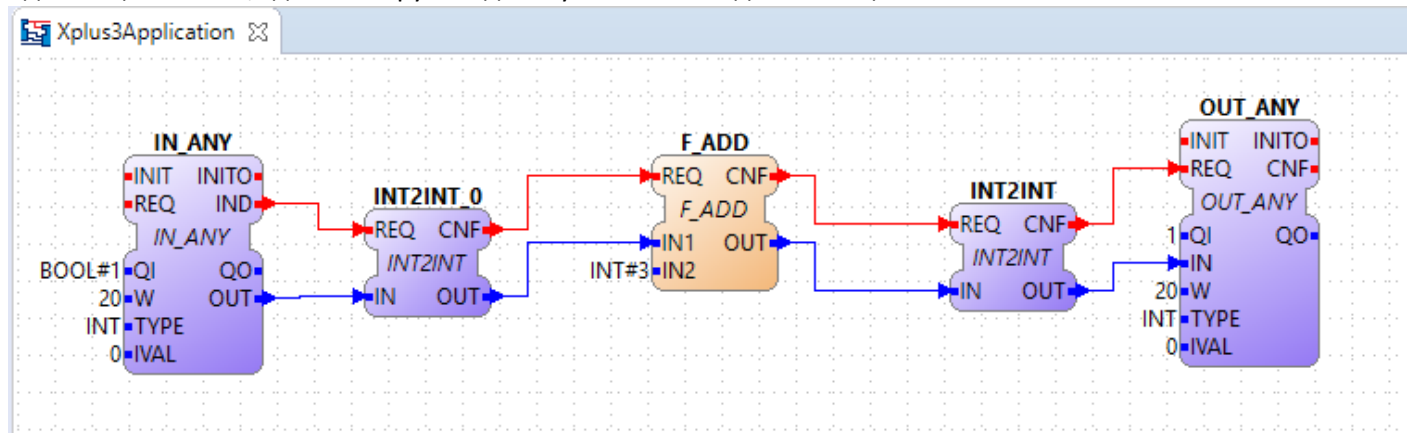
Изучите существующую систему

В этом примере вы познакомитесь с тем, как работать с Eclipse 4diac, изучив и развернув систему Xplus3 и приложение Xplus3Application. Система состоит из двух устройств. После изучения сопоставления частей приложения с ресурсами мы настроим необходимые параметры и протестируем их.

- Задача примера Х+3: вычислить и отобразить Х+3.
- Вы можете получить систему Xplus3 из репозитория [примеров Eclipse 4diac на Github](#).
- Рассматривается простое приложение IEC 61499 с двумя устройствами.
- Связь между различными устройствами интегрирована в ресурс.

Изучите существующее приложение

Приложение IEC 61499 разрабатывается независимо от устройств и ресурсов. Блоки приложения сопоставляются с ресурсом устройства, на котором они будут работать после создания приложения. Нажмите на стрелку рядом с системой Xplus3 и дважды щёлкните, чтобы открыть приложение под названием Xplus3Application. ПРИМЕЧАНИЕ: в зависимости от вашей операционной системы может быть достаточно одного щелчка там, где в этом руководстве упоминаются двойные щелчки.



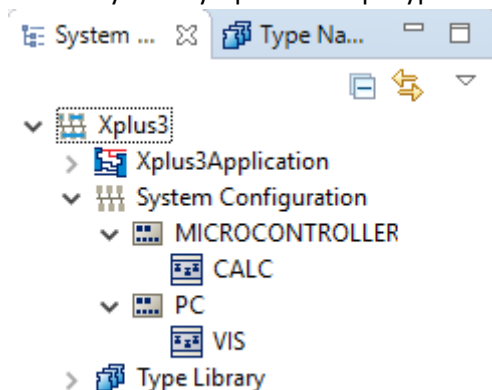
Подключены 3 группы функциональных блоков. Особый цвет функционального блока показывает, что этот блок связан с устройством. Два разных цвета функциональных блоков указывают на то, что они связаны с двумя разными устройствами, поэтому приложение распределено. Пунктирные линии — это просто информация. Мы увидим, как данные передаются между устройствами в разделе «Ресурсы».

Входные и выходные данные приложения находятся на одном устройстве, а вычисление $(X+3)$ — на втором устройстве. При наведении курсора мыши часто появляется дополнительная информация, например, о типе переменной данных. При изучении приложения можно понять, что используются три типа функциональных блоков:

- IN_ANY и OUT_ANY — это входные и выходные данные, которые визуализируют входное значение и результат вычисления. Входной параметр QI — это входной квалификатор, который позволяет функциональному блоку работать, если он истинен. Он используется для отключения функциональных блоков, когда в предшествующем функциональном блоке возникает ошибка. В нашем приложении для OUT_BOOL.QI установлено значение 1.
- F_ADD складывает два любых входных значения. Одно значение поступает с входного соединения, а другое является жестко заданной константой.

Изучите существующую конфигурацию системы

В системе Xplus3 есть конфигурация системы. Нажмите один раз на «Конфигурацию системы» (слева) и продолжайте раскрывать «Конфигурацию системы», нажимая на стрелки слева. Здесь показаны используемые устройства и ресурсы.



Дважды нажмите на «Конфигурацию системы», чтобы открыть модель «Конфигурации системы», которая показывает систему, устройства, их ресурсы и коммуникационную сеть, к которой подключены устройства.

INT2INT используется несколько раз. Вы можете спросить себя: зачем преобразовывать INT в INT — разве это не одно и то же!?

1. Наведите курсор мыши на вывод данных IN_ANY.OUT в VIS. Обратите внимание на его тип: это ANY.
2. Наведите курсор мыши на вывод данных PUBLISH_1_0.SD_1. Обратите внимание на его тип: это ANY.

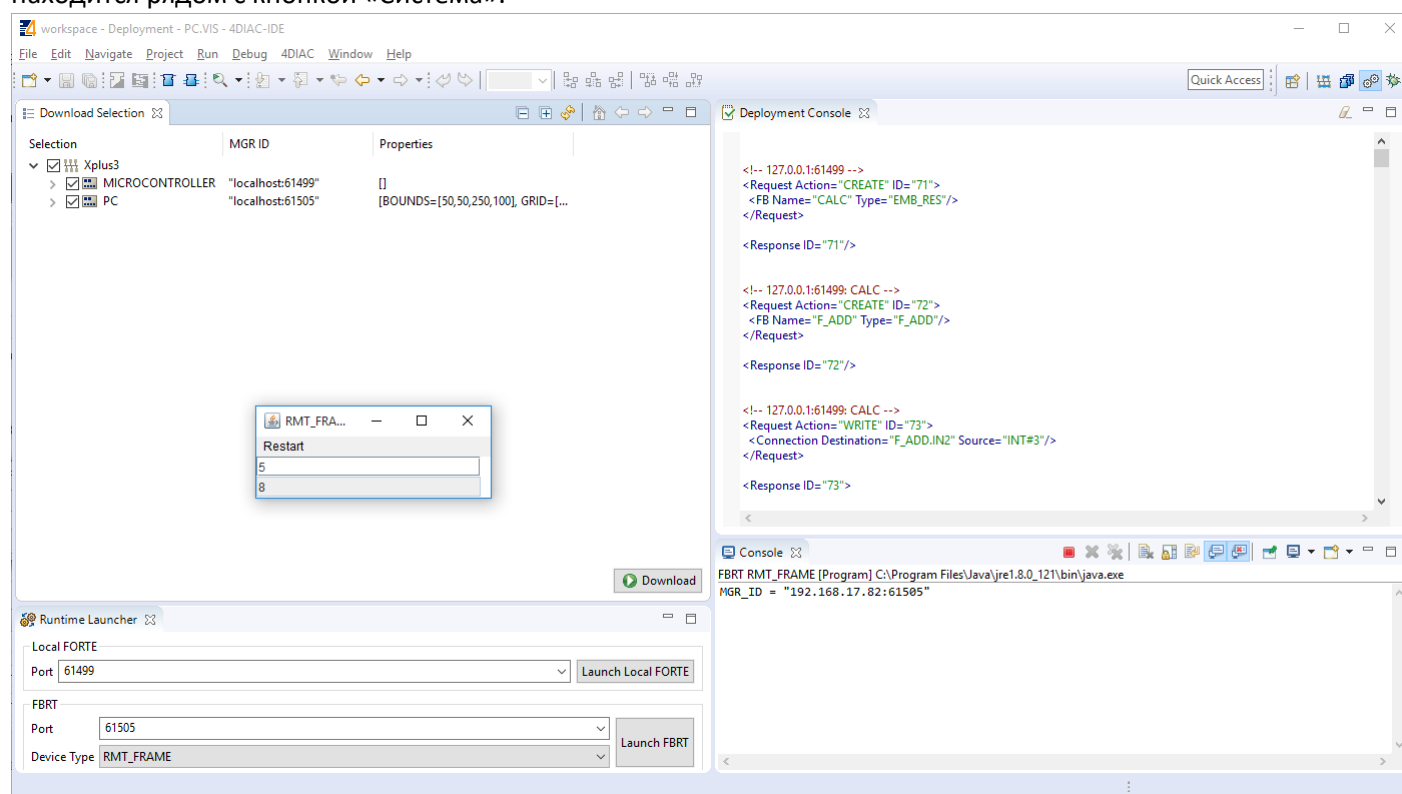
Итак, если тип вывода — ANY, а тип ввода — ANY, как можно узнать фактический тип данных? Вот почему нам нужен функциональный блок преобразования, например INT2INT, с любым типом ввода и фактическим типом данных на выходе. Функциональные блоки, использующие тип ANY для ввода или вывода, должны указывать функциональным блокам тип ввода, а следующим за ними функциональным блокам — тип вывода. Для обычных функциональных блоков тип ввода должен соответствовать его типу. Посмотрите на приложение, где все три части — ввод, вывод и вычисление — отображаются вместе. Обратите внимание, что функциональные блоки «Публикация» и «Подписка», используемые для обмена данными между ресурсами устройства, находятся непосредственно в ресурсе. Это делает приложение менее перегруженным и более понятным.

Разверните свое приложение

Переключитесь на перспективу развертывания и задайте настройки среды выполнения. Следующие настройки находятся в разделе Window → Preferences → 4diac IDE. Дополнительные сведения о настройке этих расположений см. в разделе [Свойства 4diac IDE](#). Для запуска теста необходимо указать местоположение исполняемых файлов стандарта IEC 61499 для используемых устройств.

- В программе MICROCONTROLLER.CALC используется 4diac FORTE. Поэтому местоположение forte.exe должно быть задано в настройках 4diac FORTE. Как правило, это находится где-то ниже /runtime/FORTE/. Относительные пути работают не во всех настройках, поэтому рекомендуется указывать здесь абсолютный путь.
- PC.VIS использует FBRT. Поэтому расположение файла fbrt.jar должно быть задано в настройках FBRT. Как правило, этот путь — ./runtime/FBRT/fbrt.jar.

Теперь вы можете выполнить тестирование системы IEC 61499 на двух устройствах, если настройки заданы правильно. Нажмите кнопку «Открыть перспективу» в верхней части строки заголовка и добавьте «Развертывание» в окно перспективы. Кнопка «Открыть перспективу» в перспективе «Развертывание» находится рядом с кнопкой «Система».



Чтобы загрузить и протестировать приложение IEC 61499:

1. проверьте систему Xplus3 и обратите внимание, что устройства отображаются в разделе «Система», но не в разделе «Приложение», связанном с устройствами.

2. Нажмите кнопку «Запустить FBRT». Обратите внимание, что номер устройства и порта соответствуют свойствам устройства, использующего FBRT (т. е. ПК), в конфигурации системы. Вывод консоли FBRT отображается в правом нижнем углу, и открывается пустое окно Java.
3. Нажмите кнопку «Запустить FORTE». Обратите внимание, что номер хоста и порта соответствуют свойствам устройства, использующего 4diac FORTE (т. е. MICROCONTROLLER), в конфигурации системы. Вывод консоли снова отображается в правом нижнем углу. Вывод на консоль 4diac FORTE зависит от уровня ведения журнала, который был настроен с помощью CMake. Конфигурация по умолчанию - LOGDEBUG. В ней отображается отладочная информация.
4. При первом запуске FBRT или 4diac FORTE может появиться окно настройки системного брандмауэра. Вам следует разрешить доступ к сети, чтобы два устройства могли взаимодействовать друг с другом через Ethernet.
5. Нажмите "Загрузить" и обратите внимание на вывод консоли развертывания справа.
6. Нажмите на окно приложения Java (RMT-FRAME). При необходимости измените его размер, чтобы увидеть содержимое. Введите целое число в верхнее поле и нажмите клавишу Enter. Результат отобразится в нижнем поле: Это должно быть значение, которое вы ввели, плюс три.

Инсталляции

Запустите 4diac FORTE после загрузки в системах на базе Linux

Обычно программируемые логические контроллеры запускают свою среду выполнения сразу после загрузки, чтобы пользователь мог либо вручную загрузить пользовательское приложение управления без входа в программируемый логический контроллер, либо загрузить файл загрузки, содержащий пользовательское приложение, при запуске среды выполнения. Для этого в 4diac FORTE можно добавить службу Linux, которая обеспечивает эту функцию.

Создайте службу Linux

Существуют различные способы создания службы Linux, которая запускает 4diac FORTE после загрузки:

- создание службы в виде скрипта /etc/init.d
- создание службы systemd

Поскольку первый способ является более старым механизмом, предоставляющим меньше возможностей, служба systemd является предпочтительным решением, которое будет описано в этом разделе.

Создание службы systemd

Создайте файл службы /etc/systemd/system/forte.service. Для создания этого файла вам потребуются права суперпользователя. Он может содержать следующий контент:

```
[Unit]
Description=start 4diac runtime
After=network.target dbus.service

[Install]
WantedBy=multi-user.target

[Service]
Environment="DAEMON_PATH=/home/pi/forte/" "DAEMON_NAME=forte"
"DAEMON_BOOT_FILE=BootFile.fboot"
ExecStartPre=/bin/bash -c '[ -x "${DAEMON_PATH}/${DAEMON_NAME}" ] && [ -x "${DAEMON_PATH}/${DAEMON_BOOT_FILE}" ] && rm "${DAEMON_PATH}/${DAEMON_NAME}.log'
ExecStart=/bin/bash -c 'echo "Start 4diac FORTE..." &&
${DAEMON_PATH}/${DAEMON_NAME} -f ${DAEMON_PATH}/${DAEMON_BOOT_FILE} >>
${DAEMON_PATH}/${DAEMON_NAME}.log'
```

```
TimeoutStartSec=10s
```

В этом примере службы systemd предполагается, что 4diac FORTE находится в папке /home/pi/forte/, называется forte и что в той же папке есть файл загрузки MesseDemo_RevolutionPi.fboot. В каждой системе папка может называться по-разному в зависимости от доступных пользователей. С помощью записи ExecStartPre служба systemd проверяет наличие файлов и удаляет все существующие файлы журналов. С помощью записи ExecStart служба systemd запускает 4diac FORTE с загрузочным файлом и записывает сообщения в файл журнала.

Тестирование службы systemd

Тестируйте службу systemd с помощью следующей команды

```
sudo systemctl start forte.service
```

Проверьте состояние службы с помощью следующей команды

```
sudo systemctl status forte.service
```

Службу можно остановить с помощью следующей команды

```
sudo systemctl stop forte.service
```

Включите службу systemd

Если служба работает правильно, включите запуск службы systemd после загрузки с помощью следующей команды

```
sudo systemctl enable forte.service
```

При перезагрузке устройства служба также должна запуститься, а при выполнении команды

```
top
```

Создание образов Docker 4diac FORTE

Создание образа

В папке ./docker/ репозитория 4diac FORTE есть два файла Docker. Первый — это образ Docker, в котором работает 4diac FORTE. Второй — это образ Docker, который используется для создания 4diac FORTE для многих архитектур и устройств. В настоящее время он создается для:

- linux32
- raspberry pi (и BeagleBone Black). Не подходит для Raspberry Pi Zero
- embrick
- (Скоро появится) RevPi

- (Скоро появится) WAGO

В обоих образах Docker 4diac FORTE построен со следующими модулями:

- UTILS
- CONVERT
- IEC61131
- Arrowhead
- HTTP (порт 8080)
- LUA
- MQTT
- OPC UA (с защитой)

и устройства с соответствующим модулем ввода-вывода.

В образе Docker 4diac FORTE двоичный файл находится в `/usr/forte_output/linux32/fort`.

В образе Docker Builder двоичные файлы находятся в `/usr/forte_output`

Чтобы создать образы Docker, вам нужно перейти в корневую папку репозитория 4diac FORTE и выполнить (не забудьте поставить точку в конце):

```
docker build -t forte -f docker/Dockerfile_forte .
```

и

```
docker build -t forte_builder -f docker/Dockerfile_builder .
```

Запустите контейнеры

Чтобы запустить экземпляр 4diac FORTE, просто выполните:

```
docker run -d -p 61499:61499 -p 4840:4840 forte:latest
```

Опция `-d` запускает контейнер в фоновом режиме. Опция `-p` сопоставляет внешний порт с внутренним.

Чтобы запустить 4diac FORTE с другим портом прослушивания, выполните:

```
docker run -d -p 61500:61500 -p 4840:4840 forte:latest  
/usr/forte_output/linux32/forte -c localhost:61500
```

Самый распространенный способ запуска образа `forte_builder` выглядит следующим образом:

```
docker run -it forte_builder:latest
```

После этого вы окажетесь внутри контейнера и сможете просматривать его.

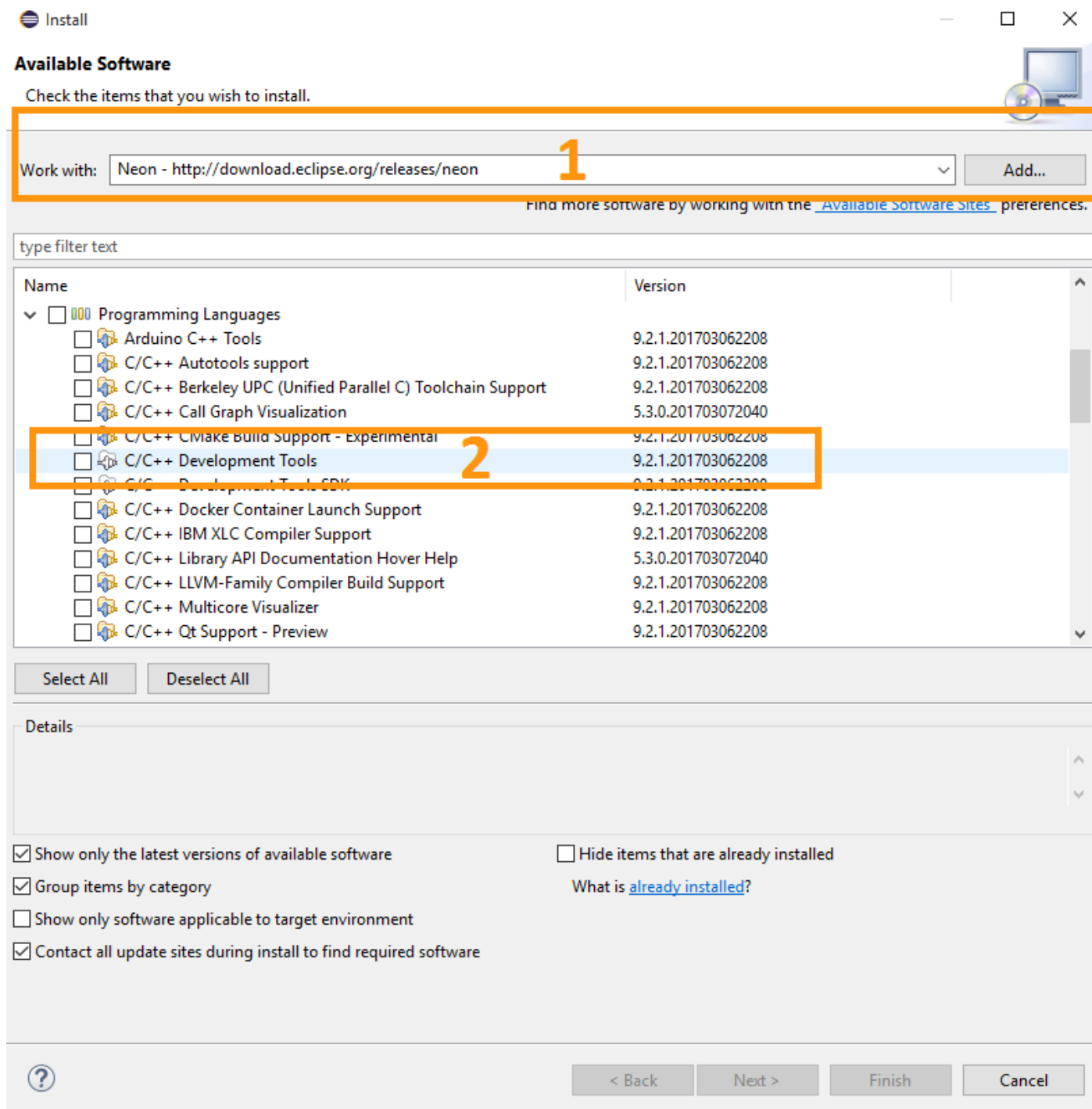
Чтобы скопировать двоичные файлы 4diac FORTE в локальную папку, выполните команду:

```
docker create --name temp_container forte_builder:latest && docker cp  
temp_container:/usr/forte_output/ ./ && docker rm temp_container
```

Компиляция и отладка 4diac FORTE в Eclipse

Получите поддержку C/C++ для Eclipse

1. Если вы еще не используете Eclipse, например, для сборки 4diac IDE из исходного кода, скачайте <http://www.eclipse.org/downloads/> → Установите новое программное обеспечение, выбрав свою версию Eclipse (1) и отметив «Языки программирования» → «Инструменты разработки C/C++» (2).



2. Перезапустите eclipse.exe
3. Выберите каталог рабочей области. Это будет каталог, в котором будут находиться ваши проекты на C/C++. Рекомендуется поместить папку рабочей области на один уровень выше папки с исходным кодом. Пожалуйста, избегайте пробелов в её названии. В противном случае у вас могут возникнуть проблемы при компиляции 4diac FORTE.

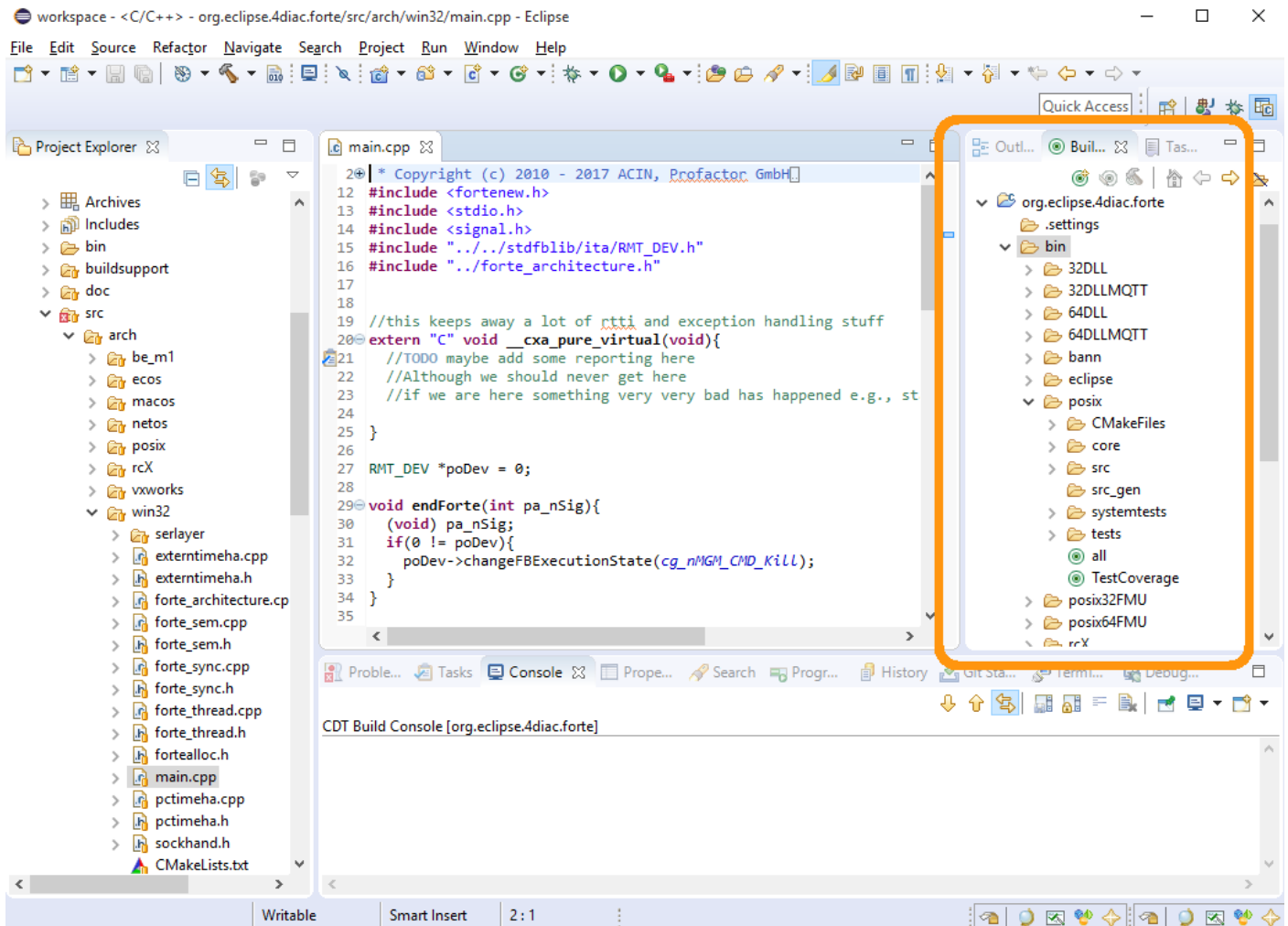
Компилируйте 4diac FORTE в Eclipse

Процесс сборки описан в [руководстве по установке](#). Выполните описанные там шаги, но с учётом следующих изменений: При создании проекта с помощью CMake необходимо выбрать правильный тип проекта. Вам нужно указать CMake на создание проекта Eclipse. Это также зависит от установленного вами компилятора. Для MinGW64 это Eclipse CDT 4 — Makefile для Unix.

Затем скомпилируйте проект следующим образом:

1. Импортируйте 4diac FORTE по файлу → Импорт... → Общие → Существующий проект в рабочую область. Если ваша рабочая область находится на одну папку выше папки с исходным кодом, вам нужно только нажать «Обзор...» и затем «ОК», чтобы импортировать 4diac FORTE в Eclipse. В противном случае выберите каталог, в который вы распаковали 4diac FORTE, и установите флажок «Копировать в рабочую область» (в противном случае исходный код останется в исходном месте).
2. Чтобы открыть «Просмотр целей сборки», выберите «Окно» → «Показать вид» → «Цели сборки», если он ещё не открыт. Дважды щелкните по make-файлу: bin/[ВАША_СИСТЕМА]/all, чтобы скомпилировать 4diac FORTE в

исполняемое приложение. Если вы не видите целей (все или очистка), вы можете создать их в том же окне. Убедитесь, что вы создаете их в нужной папке.

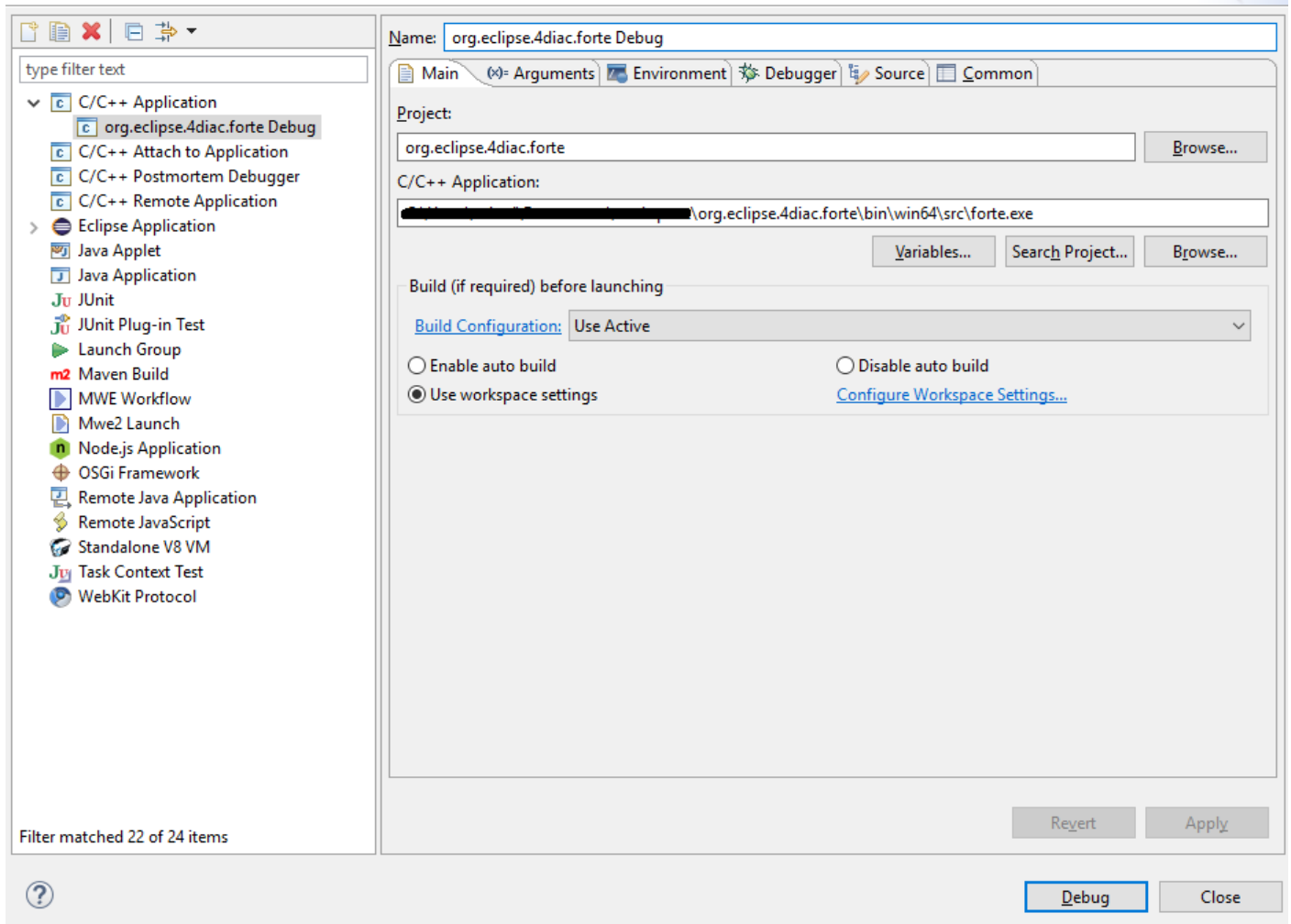


Отладка 4diac FORTE в Eclipse

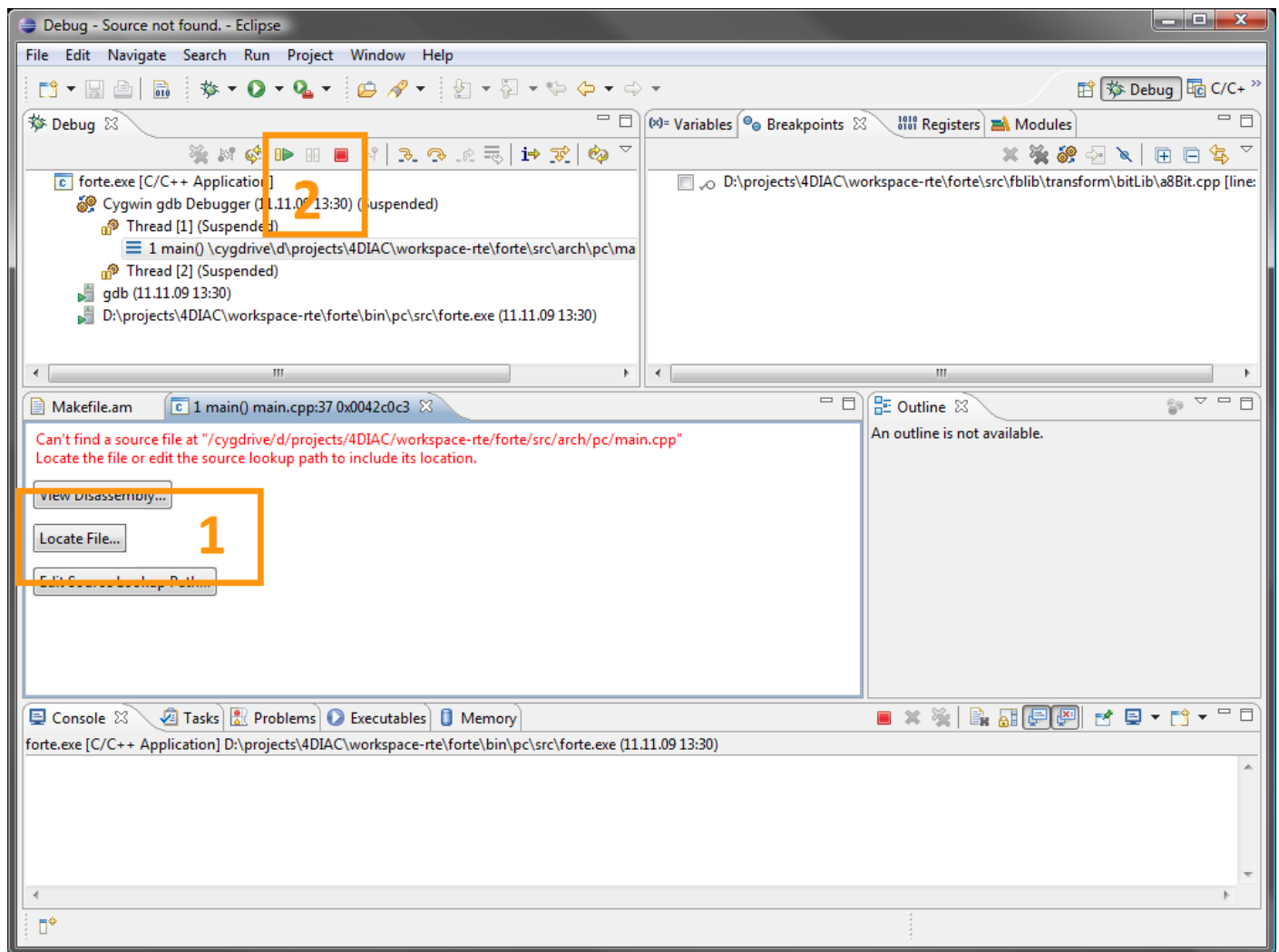
Для отладки собственных функциональных блоков выполните следующие действия:

1. Установите точку останова в файле исходного кода нужного функционального блока.
2. Перейдите в раздел «Выполнить» → «Конфигурации отладки»...
3. Щелкните правой кнопкой мыши по приложению C/C → «Создать»#. На вкладке [.tab4diac]#Main# вам нужно заполнить поле [.specificText]#C/C Application: сгенерированным двоичным файлом (обычно это FORTE_FOLDER_ROOT/bin/[БАША_СИСТЕМА]/src/forte[.exe]). На вкладке «Отладчик» выберите нужный отладчик. В Windows вы обычно используете MinGW64 (вам нужно установить его, чтобы он появился в списке).

Create, manage, and run configurations



4. Нажмите «Отладка».
5. Подтвердите переход в режим отладки.
6. Если вы не видите файл, в котором отладчик остановился в начале, нажмите кнопку «Найти файл» (1) и перейдите в рабочую область Eclipse, содержащую извлечённый 4diac FORTE, введите main в поле поиска проводника Windows и выберите main.cpp. Теперь 4diac FORTE остановлен в функции main().



7. Представление отладки C++ в Eclipse
8. Нажмите кнопку «Продолжить» (2), чтобы запустить 4diac FORTE и наблюдать за поведением вашего приложения.
9. Для остановки 4diac FORTE, запущенного в Eclipse, всегда нажимайте кнопку «Завершить». Это позволит избежать запуска 4diac FORTE в фоновом режиме (не всегда отображается в диспетчере задач).

Сборка 4diac FORTE для freeRTOS + LwIP

Введение

Это руководство по сборке 4diac FORTE для freeRTOS и LwIP. [freeRTOS](#) — это бесплатная операционная система реального времени, которая не привязана к конкретному оборудованию, но может работать на многих разных устройствах. Операционная система очень маленькая и простая, изначально она не поддерживала TCP/IP, поэтому обычно её использовали вместе с [LwIP](#) — небольшим стеком TCP/IP. В freeRTOS теперь также есть дополнительная поддержка TCP/IP (см. [здесь](#)), но на данный момент 4diac FORTE компилируется с использованием LwIP.

Это руководство поможет вам запустить 4diac FORTE на некоторых устройствах, на которых уже могут работать freeRTOS и LwIP. Поскольку вы можете использовать любое оборудование, здесь не предусмотрена поддержка ввода-вывода, и это

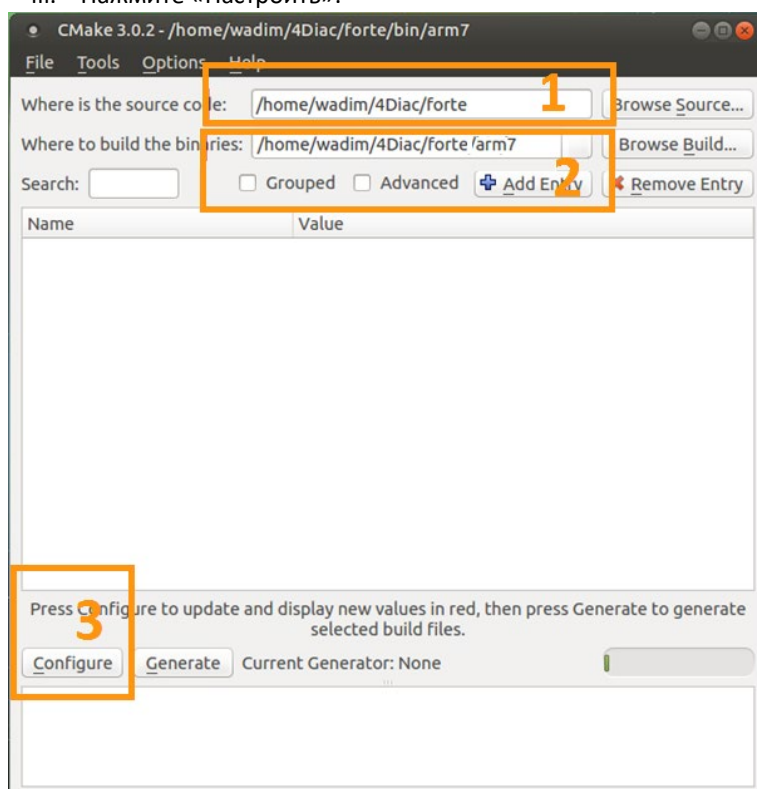
руководство поможет вам скомпилировать 4diac FORTE как статическую библиотеку, которую вы позже сможете добавить в какой-нибудь проект, уже работающий на вашем оборудовании. Перед началом работы с этим руководством предполагается следующее:

- На вашем компьютере установлены IDE и компилятор, которые позволяют компилировать код для вашего оборудования (далее — кросс-IDE и кросс-компилятор).
- У вас есть пример использования freeRTOS и LWIP, работающий на вашем оборудовании, то есть код freeRTOS и LWIP присутствует на вашем компьютере, и
- у вас есть базовое понимание задач freeRTOS

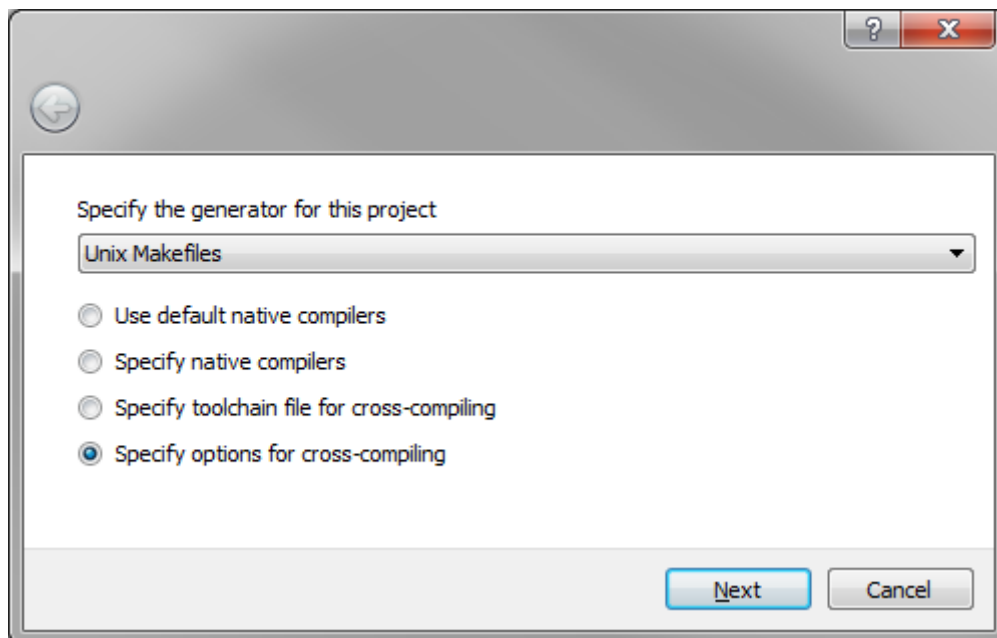
Если вы проверили все пункты, продолжайте. Если у вас нет примера использования freeRTOS и LWIP, это руководство вам не поможет.

Установите CMake и выполните компиляцию

1. Откройте CMake-GUI и выполните действия, как показано на изображении
 - i. Задайте путь к исходному коду 4diac FORTE, куда вы клонировали репозиторий Git.
 - ii. Задайте путь к двоичным файлам, где вы хотите создать библиотеку. Обычно используется bin/freertos.
 - iii. Нажмите «Настроить».

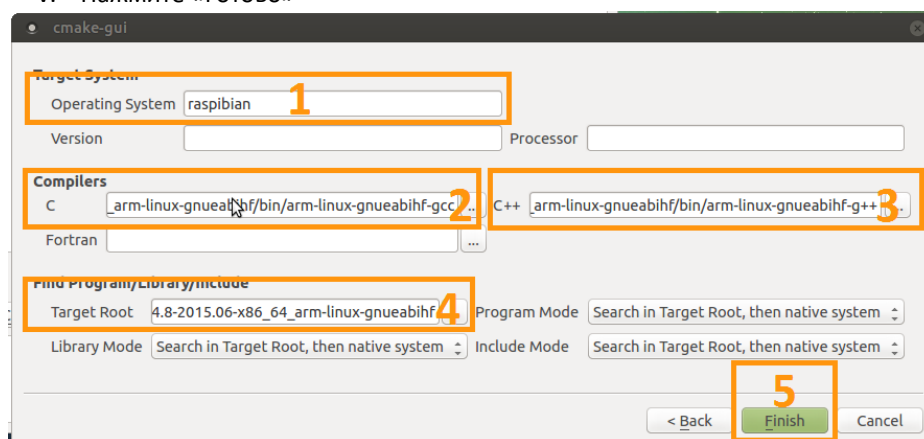


2. Выберите нужный вариант
 - i. Выберите инструмент, который вы обычно используете для компиляции своих программ. В этом примере используется список файлов Makefile для UNIX.
 - ii. Выберите «Указать инструменты для кросс-компиляции».(вы также можете сделать это, выбрав «Указать файл набора инструментов для кросс-компиляции», но для этого сначала нужно создать файл, что вы сможете сделать позже, когда у вас будет больше опыта работы с CMake).
 - iii. Нажмите «Далее».



3. Настройка для кросс-компиляции

- i. Напишите название ОС (обычно это freeRTOS, это не повлияет на компиляцию).
- ii. Выберите путь к кросс-компилятору C для вашего оборудования. Это можно найти в свойствах вашего проекта в кроссплатформенной IDE.
- iii. Выберите путь к кроссплатформенному компилятору C++ для вашего оборудования. Это можно найти в свойствах вашего проекта в кроссплатформенной IDE.
- iv. Поле целевого корня можно оставить пустым.
- v. Нажмите «Готово»



4. Настройте компиляцию

В CMake должен отображаться список со всеми переменными 4diac FORTE, выделенными красным. .. Задайте для переменной FORTE_ARCHITECTURE значение FreeRTOSLwIP и нажмите «Настроить». Проверьте информацию, которая отображается в CMake о конфигурации LwIP. .. Теперь должна появиться переменная FORTE_FreeRTOSLwIP_INCLUDES, которая является самой важной. Задайте для неё несколько путей, по которым находятся заголовки freeRTOS и LwIP, каждый из которых разделён точкой с запятой. Например:

```
{MAIN_DIRECTORY}/FreeRTOS/portable;{MAIN_DIRECTORY}/include;{MAIN_DIRECTORY}/lwip/src/include;{MAIN_DIRECTORY}/lwip/port,
```

где {MAIN_DIRECTORY} — это путь к папке, в которой находятся ваш код FreeRTOS и LwIP. Если при последующей компиляции возникнет ошибка, сообщающая об отсутствии некоторых «включений», эту переменную следует обновить, указав папки, в которых находятся отсутствующие файлы. .. Возможно, вам также потребуется установить переменные CMAKE_C_FLAGS и CMAKE_CXX_FLAGS с флагами, необходимыми для вашего оборудования. Вы можете проверить, какие флаги вам нужны, в свойствах вашего примера проекта в кроссплатформенной IDE. .. Установите для FORTE_BUILD_EXECUTABLE значение false, а для FORTE_BUILD_STATIC_LIBRARY — значение true. .. Нажмите «Настроить», и переменные, которые нужно изменить, снова отобразятся красным цветом, а остальные — белым. Проверьте эти переменные и нажимайте «Настроить», пока ни одна переменная не будет выделена красным. Здесь вы можете добавить модули, которые вы хотите использовать в 4diac FORTE, но с точки зрения freeRTOS вам больше ничего не нужно.

5. Сгенерируйте файлы
 - o Нажмите «Сгенерировать»
6. Соберите 4diac FORTE

- Перейдите в недавно созданную папку и выполните команду make (или соберите его так, как вы обычно это делаете)
\$ cd bin/freertos \$ make
- Если ошибок не возникло, в папке bin/freertos/src вы найдёте файл libforte-static.a. Если возникла ошибка, проверьте ещё раз переменные в CMake, особенно FORTE_FreeRTOSLWIP_INCLUDES, CMAKE_C_FLAGS и CMAKE_CXX_FLAGS.

Добавьте библиотеку 4diac FORTE в свой проект

1. Добавьте недавно скомпилированную библиотеку (libforte-static.a) в свой проект. Этот шаг зависит от того, какая у вас кроссплатформенная IDE.
2. Скопируйте файл src/arch/freertos/forte_init.h в свой проект.
3. В файле, в котором находится ваша «основная» функция, вам нужно включить forte_init.h и добавить задачу, которая запустит задачу «forte_thread». Рабочий пример задачи, выполняемой 4diac FORTE, выглядит следующим образом (PRINTF может вызвать проблемы, поэтому замените его на свою функцию ведения журнала или удалите):

```
static void forte_thread(void *arg) {
    forteGlobalInitialize();
    TForteInstance forteInstance = 0;
    int resultForte = forteStartInstanceGeneric(0, 0, &forteInstance);
    if(FORTE_OK == resultForte) {
        forteJoinInstance(forteInstance);
    } else {
        PRINTF("Error %d: Couldn't start forte\n", resultForte);
    }
    forteGlobalDeinitialize();
    vTaskDelete(NULL);
}
```

Если все в порядке, 4diac FORTE запустится, и вы сможете подключиться к нему по порту 61499. Если у вас возникли проблемы, прочтите следующий раздел

Устранение неполадок

Возможно, 4diac FORTE не запустится с первой попытки. Вам следует позаботиться о следующем:

- Убедитесь, что в вашей системе достаточно оперативной памяти для стека и кучи. Чтобы узнать, не хватает ли вашей системе памяти, в файле FreeRTOSConfig.h убедитесь, что у вас есть:

```
#define configCHECK_FOR_STACK_OVERFLOW 1 #define
configUSE_MALLOC_FAILED_HOOK 1
```

И добавьте следующие функции в свой основной файл:

```
void vApplicationMallocFailedHook() {
    for(;;) {
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}
```

```
void vApplicationStackOverflowHook(TaskHandle_t xTask, char *pcTaskName ) {
    for(;;) {
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}
```

и установите точку останова в обоих vTaskDelay. Если программа достигнет одной из этих точек останова, у вас проблема с памятью.

- Вам следует настроить оборудование и запустить задачу LwIP до 4diac FORTE.
- Конфигурация создания задачи, которая сработала для некоторых пользователей, выглядит следующим образом:
 1. Создайте задачу под названием «stack_init»
 2. Запустите планировщик
 3. "stack_init" запустит и инициализирует стек LwIP (который внутренне создает задачу LwIP).
 4. После этого "stack_init" создает задачу "forte_thread", как и раньше.
 5. "stack_init" завершается вызовом vTaskDelete(NULL);
- Для проектов на C++ (например, 4diac FORTE) необходим вызов __libc_init_array(); В некоторых случаях это не делается по умолчанию в сгенерированном коде примера. Попробуйте добавить этот вызов в начало функции main() перед любым другим вызовом.

Установка

4diac IDE

Чтобы установить 4diac IDE, просто скачайте последнюю версию для вашей операционной системы в разделе [загрузок](#). Распакуйте ее в любую нужную папку и запустите 4diac IDE. Она уже содержит библиотеку функциональных блоков. Если вы хотите использовать только доступные функциональные блоки, вы готовы к работе.

Создание собственной 4diac IDE из исходного кода

Преимущество запуска 4diac IDE из исходного кода в том, что вы можете легко следить за изменениями, вносимыми в репозиторий Git. Если вы хотите запустить 4diac IDE из исходного кода, следуйте [инструкциям по установке](#). Этот вариант предназначен в основном для разработчиков 4diac IDE или для тех, кто хочет расширить возможности 4diac IDE.

4diac FORTE

4diac FORTE можно использовать двумя способами: для проведения первых экспериментов с Eclipse 4diac можно использовать готовую версию 4diac FORTE, которую можно [скачать](#) для Linux и Windows. Её можно использовать для приложений, если вам не нужны собственные функциональные блоки. Пошаговое руководство поможет вам создать простое приложение с помощью 4diac IDE и готовой версии 4diac FORTE. Если вы хотите сразу начать использовать 4diac IDE, вы можете пропустить остальную часть страницы и перейти непосредственно к [пошаговому руководству](#) или на страницу «[Начать здесь](#)».

Если вы хотите разрабатывать собственные функциональные блоки или запускать 4diac FORTE на устройствах управления, вам нужно скачать и настроить 4diac FORTE для конкретной используемой вами платформы, как показано в следующих разделах.

Создание 4diac FORTE из исходного кода

Чтобы использовать собственные функциональные блоки, вам нужно будет собрать 4diac FORTE из исходного кода. В этом разделе описаны подробности для систем на базе Windows и Unix, поскольку большинство пользователей впервые тестируют свои приложения на этих платформах. Ссылки на другие платформы приведены [здесь](#).

Вам потребуется выполнить несколько шагов:

1. Установить компилятор для систем на базе [Windows](#) или [Unix](#)
2. Установить [cmake](#)
3. Создать [дерево папок](#)
4. Скачайте исходный код 4diac FORTE в виде zip-файла [здесь](#)

5. Распакуйте исходный код в нужную вам папку с исходным кодом 4diac FORTE
6. [Соберите проект с помощью CMake](#)
7. [Скомпилируйте](#) проект
8. [Добавьте внешние модули](#): свои собственные FB
9. (необязательно) Установите среду разработки для C++, например [Eclipse](#) или [MS Visual Studio](#)

Компилятор C++ для Windows

Прежде всего вам нужно установить компилятор C++. Скачать и установить его можно здесь:

- [Настройка MinGW-w64 \(рекомендуется\)](#)
- [Настройка Visual Studio](#)

Приведенные ниже инструкции по установке основаны на MinGW-w64, поэтому они рекомендуются неопытным пользователям. Если вы планируете участвовать в разработке 4diac FORTE в качестве разработчика, вам понадобится интегрированная среда разработки, например Eclipse или MS Visual Studio. Последняя уже поставляется с компилятором, поэтому вам не нужно устанавливать отдельный компилятор. Однако пользователям Eclipse рекомендуется установить MinGW-w64.

[Вернуться](#)

Компилятор C++ для систем на базе Unix

В Unix-подобных системах (Linux, Raspberry Pi и т. д.) необходимо установить пакеты binutils, gcc, gdb и make.

[Вернуться](#)

Установите CMake

Программное обеспечение с открытым исходным кодом CMake позволяет создавать проект C++ или файл Makefile из исходного кода. Мы будем использовать его для создания проекта для компиляции. Он генерирует конкретные файлы из общего исходного кода для вашей целевой платформы и компилятора. Кроме того, он используется для настройки вашего 4diac FORTE.

Для Windows вы можете скачать CMake [здесь](#). Следуйте инструкциям мастера установки и не забудьте выбрать добавление CMake в переменную PATH на соответствующем шаге. Чтобы избежать каких-либо проблем, вам следует выбрать путь установки, в котором не будет пробелов.

Для Linux просто установите пакет CMake, входящий в ваш дистрибутив.

[назад](#)

Создайте дерево папок

Вам нужно создать папку, в которой вы будете сохранять все файлы 4diac FORTE. Опять же, путь к файлу не должен содержать пробелов. Мы рекомендуем использовать основную папку C:/FORTE_dev (в этом руководстве мы называем её FORTE_FOLDER_ROOT). В этой папке используйте подпапки для кода (например, org.eclipse.forte), собранной версии (например, build) и ваших собственных модулей (например, ext_modules).

Исходный код 4diac FORTE доступен для скачивания в виде zip-файла [здесь](#). Распакуйте исходный код в нужную вам папку с исходным кодом 4diac FORTE, например C:/FORTE_dev/org.eclipse.forte).

[назад](#)

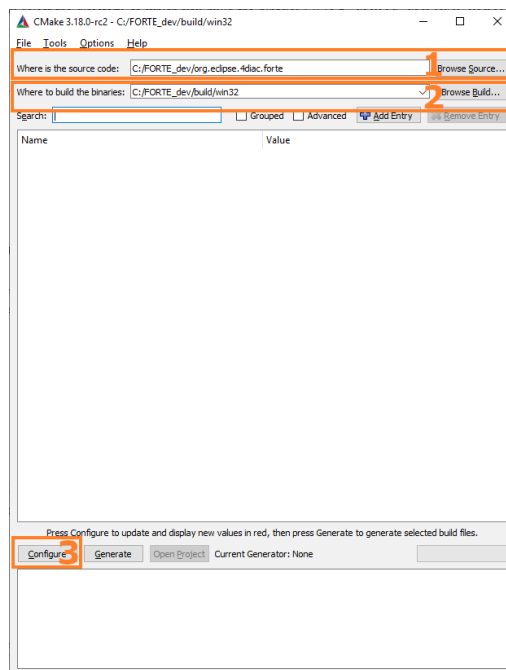
Создайте Makefile для 4diac FORTE с помощью CMake

После того как вы получите код 4diac FORTE на свой компьютер, вам нужно будет создать Makefile для компиляции. Зачем? Поскольку 4diac FORTE предназначен для многих платформ, многие функции могут быть включены, отключены или настроены. Для упрощения этих настроек используется [CMake](#). Новичкам мы рекомендуем использовать графический интерфейс, поставляемый с CMake.

1. Запустите CMake-GUI.
2. Сначала вам нужно выбрать исходный каталог FORTE_FOLDER_ROOT/org.eclipse.forte (1), в который вы извлекли исходный код.

Далее вы выбираете каталог, в который будут создаваться двоичные файлы (2) (например, FORTE_FOLDER_ROOT/bin/posix или FORTE_FOLDER_ROOT/bin/win32, ...). Это выходной каталог (в руководстве он называется OUTPUT_FOLDER), в который CMake помещает данные конфигурации и любые файлы сборки.

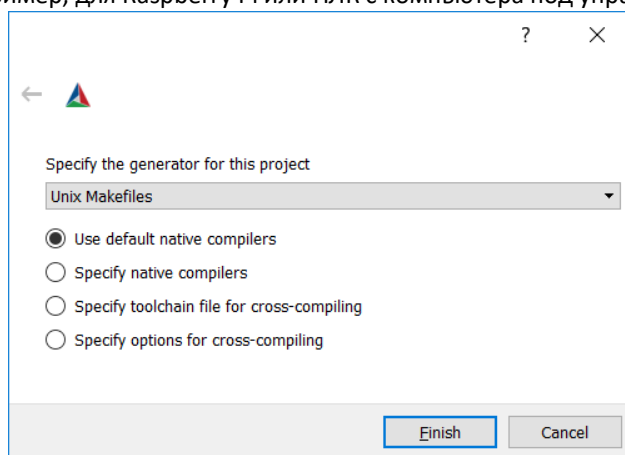
Эти файлы необходимы для компиляции на следующем шаге. Нажмите кнопку «Настроить» (3) внизу. Если вас просят создать папку, которой не существует, подтвердите действие.



3. Во всплывающем окне выберите тип проекта, который зависит от целевой системы. CMake необходимо знать, нужно ли экспортировать файлы в виде проекта для Visual Studio, Eclipse или другой программы, а также какой компилятор вы будете использовать для компиляции исполняемого файла.

Если вы установили MinGW-w64 для Windows, выберите здесь «Makefile для Unix», установите флажок «Использовать собственные компиляторы по умолчанию» и нажмите «Готово».

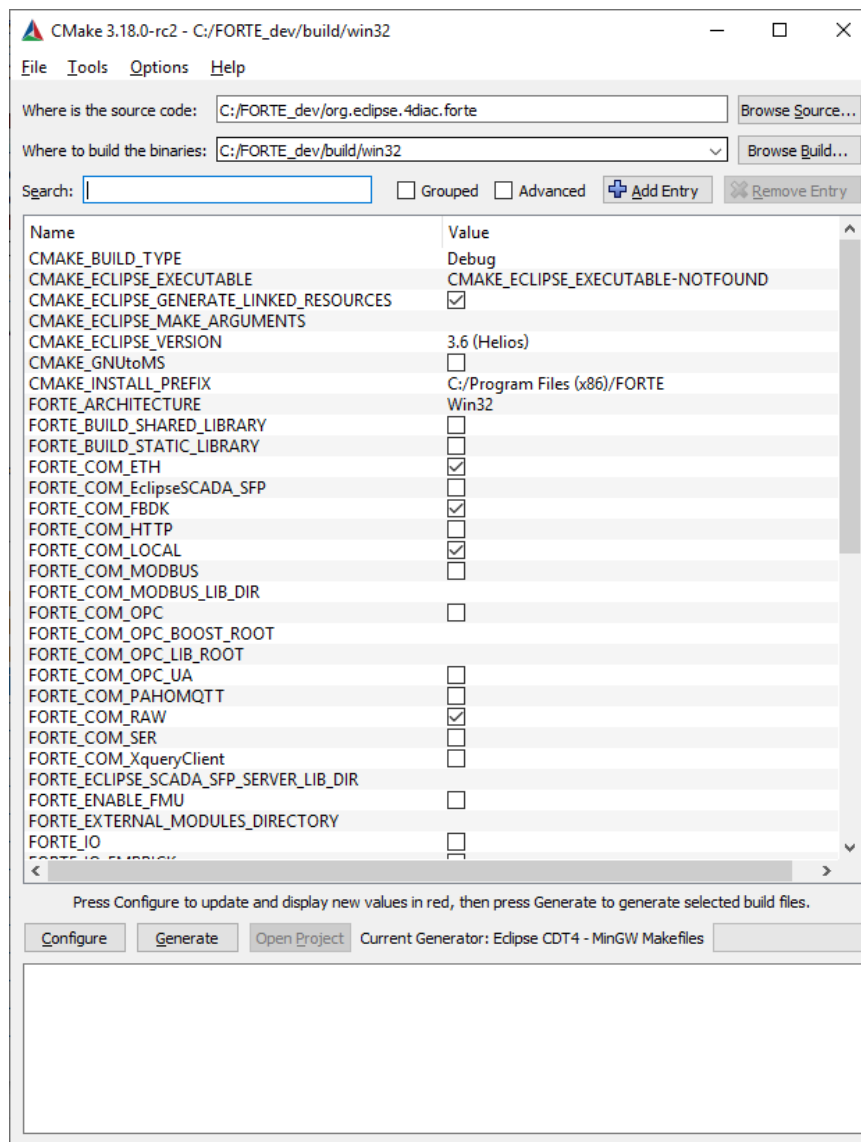
Если вы используете Linux, также выберите «Makefile для Unix». Для других настроек обратитесь к соответствующей странице установки. На этом этапе вы выбираете кросс-компилятор, если компилируете для другой платформы (например, для Raspberry Pi или ПЛК с компьютера под управлением Windows или Linux).



4. После выбора типа проекта и компилятора нажмите «Сгенерировать».

Вы можете получить сообщение об ошибке «Не выбрана допустимая архитектура», нажмите «ОК». В главном окне CMake теперь отображается список отмеченных красным параметров. Красный цвет обозначает не ошибки, а то, что эти строки были затронуты последней конфигурацией, поэтому их нужно проверить. Эти параметры позволяют настроить сборку 4diac FORTE. Вам нужно задать как минимум следующие параметры:

- выберите целевую архитектуру с помощью параметра FORTE_ARCHITECTURE:
 - Win32 для Windows с MinGW-w64
 - Posix для Linux
 - Для других настроек обратитесь к соответствующей странице установки.
- выберите модули с библиотеками функциональных блоков, которые вы хотите включить (в базовой конфигурации используются FORTE_MODULE_CONVERT, FORTE_MODULE_IEC61131 и FORTE_MODULE_UTILS)
- выберите CMAKE_BUILD_TYPE, добавив значение «Отладка» или «Выпуск»



5. После изменения некоторых параметров в CMake необходимо снова нажать «Настроить», пока не исчезнет красная строка.
6. Наконец, нажмите кнопку «Создать», чтобы создать файлы проекта в OUTPUT_FOLDER.

[Вернуться](#)

Скомпилируйте проект

Последний шаг — компиляция сгенерированного проекта. Он генерирует исполняемый файл 4diac.FORTE. Этот шаг зависит от того, какой результат вы выбрали на предыдущем шаге. Для Windows с MinGW-w64 откройте командную строку (cmd). Команда `cd` переходит в определенный каталог. Поэтому вам нужно выбрать OUTPUT_FOLDER, которую вы указали в CMake. Если вы следовали рекомендуемой структуре, то это C:\FORTE_dev\build\win32.

+ Введите:

```
cd C:\FORTE_dev\build\win32 make
```

Для Linux откройте терминал и перейдите в папку OUTPUT_FOLDER:

```
cd FORTE_FOLDER_ROOT/bin/posix make
```

Для разработки с помощью 4diac.FORTE полезно понимать общую структуру файлов. Таким образом, перечислены основные компоненты, а также файлы Makefile, которые важны для настройки и компиляции 4diac.FORTE:

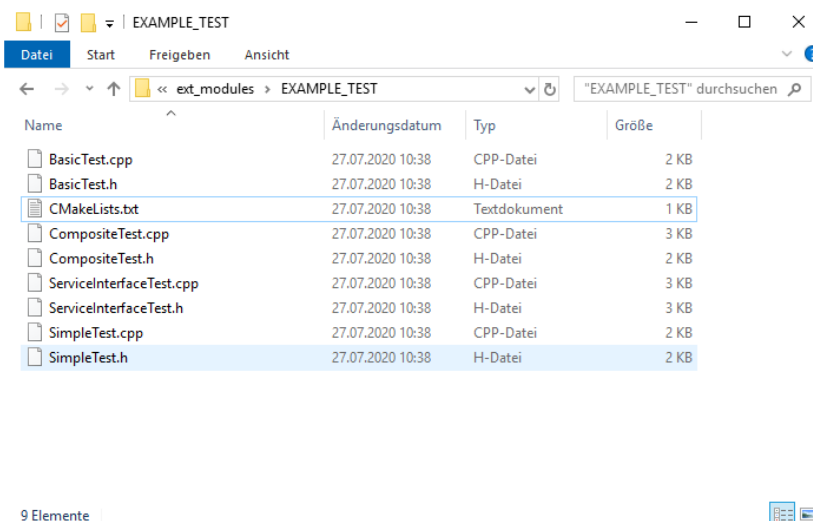
- `src/modules`: эта папка содержит исходный код (cpp, h) всех функциональных блоков, доступных для 4diac.FORTE
- `bin/[ВАША_СИСТЕМА]/src`: содержит исполняемый файл 4diac.FORTE после компиляции
- `bin/[ВАША_СИСТЕМА]/src_gen`: содержит объектные файлы, созданные во время компиляции

[назад](#)

Добавление внешних модулей

Давайте посмотрим, как добавить свои собственные функциональные блоки в 4diac.FORTE. Когда вы создаете и экспортируете свои собственные типы, вам необходимо добавить их в свою компиляцию. Экспорт описан в [share 4 руководства](#). В качестве примера мы покажем, как добавить четыре модуля FBs: BasicTest, CompositeTest и ServiceTest, SimpleTest из этого руководства. Название модуля будет EXAMPLE_TEST. Следуйте инструкциям:

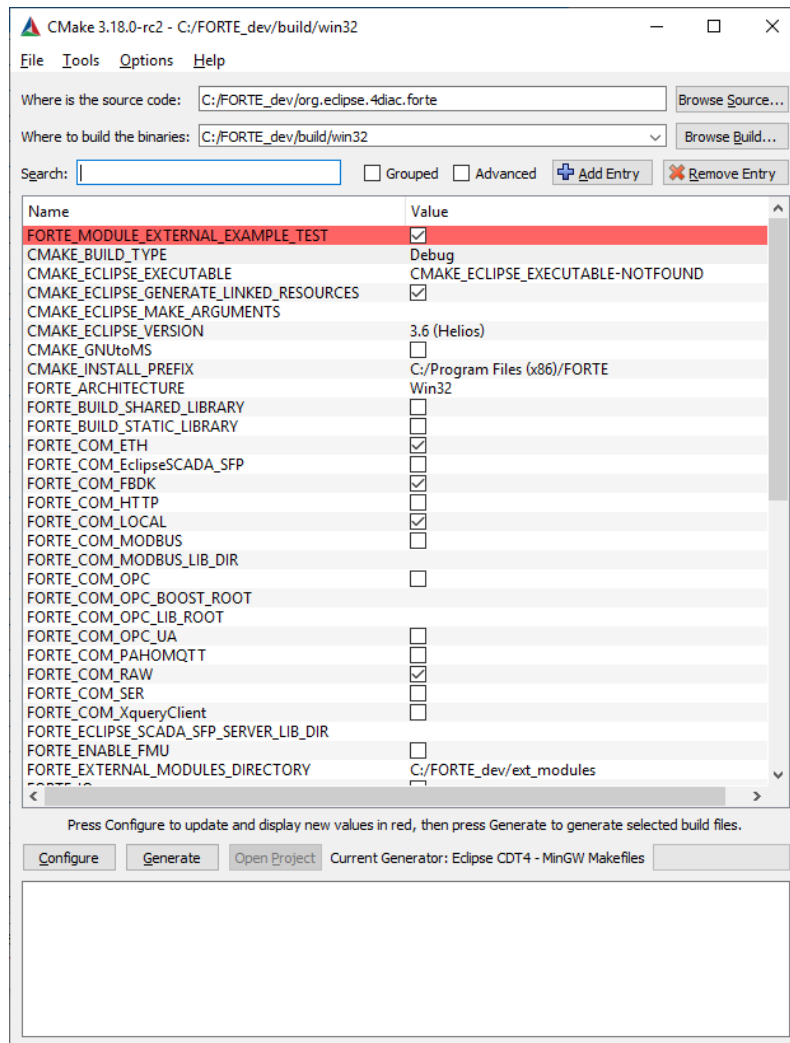
1. На шаге 3 мы создали в вашей системе папку, в которой будут храниться все будущие модули. Мы назвали её `ext_modules`. Создайте подпапку с названием `EXAMPLE_TEST` (название не обязательно должно совпадать с названием модуля, но это поможет в организации) и получите к ней доступ.
2. Поместите все шесть файлов в `EXAMPLE_TEST` (по одному файлу `.cpp` и `.h` для каждого FB).
3. Создайте файл с названием `CMakeLists.txt` (это точное название обязательно).



4. Содержимое этого файла можно создать разными способами.
 - С помощью опции `Export CMakeLists.txt` в 4diac. При этом для создания имени модуля будет использоваться имя каталога. Переменная `CMake` в этом примере будет называться `FORTE_MODULE_EXTERNAL_EXAMPLE_TEST`
 - Комментарий будет содержать список файлов, включенных с помощью этой переменной.
 - Вручную вставив следующий код в файл `CMakeLists.txt`

```
forte_add_module(EXTERNAL_EXAMPLE_TEST "Example test module")
forte_add_sourcefile_hcpp(BasicTest)
forte_add_sourcefile_hcpp(CompositeTest)
forte_add_sourcefile_hcpp(ServiceTest)
forte_add_sourcefile_hcpp(SimpleTest)
```

 и сохраните.
 Первый параметр `forte_add_module` — это имя модуля, второй — комментарий.
 Команда `forte_add_sourcefile_hcpp` сообщает `CMake`, какие файлы `.cpp` и `.h` нужно скомпилировать.
 Вы можете использовать `forte_add_subdirectory`, чтобы включить подкаталоги с собственными файлами `CMakeLists.txt`.
5. В `CMake` убедитесь, что папка с исходным кодом по-прежнему называется `FORTE_FOLDER_ROOT`. Она не была переименована в `ext_modules`. Нажмите «Настроить». Есть переменная с названием `FORTE_EXTERNAL_MODULES_DIRECTORY`. Задайте для неё значение папки `ext_modules`.
6. При повторном нажатии кнопки «Настроить» появится переменная `FORTE_MODULE_EXTERNAL_EXAMPLE_TEST`. Если установить флажок, модуль будет включен в компиляцию.



7. После установки всех необходимых переменных сделайте то же, что и раньше: сгенерируйте, а затем скомпилируйте.

[назад](#)

Сборка 4diac FORTE для других платформ

- [Настройка 4diac FORTE для RaspberryPi](#)
- [Настройка 4diac FORTE для Wago PFCs SPS](#)
- [Настройка 4diac FORTE для Lego Mindstorms EV3](#)
- [Настройка 4diac FORTE для freeRTOS + LwIP](#)

[Запустите 4diac FORTE после загрузки в системах на базе Linux](#)

Использование Eclipse 4diac с Lego Mindstorm EV3

Эта страница предназначена для компиляции 4diac FORTE для Lego Mindstorm EV3. Для получения информации об используемых параметрах перейдите на страницу [конфигурации EV3 IO](#).

О Lego Mindstorm EV3 и ev3dev

Lego Mindstorm — это серия наборов Lego, гибкость которых позволяет создавать различные системы, которыми пользователь может управлять и которые он может контролировать с помощью входов и выходов. В этом документе рассказывается о том, как реализовать 4diac FORTE на конкретной модели Lego Mindstorm EV3 (LMSEV3). Чтобы узнать больше о LMSEV3, вы можете посетить его [страницу](#) в Википедии.

LMSEV3 поставляется с собственным программным обеспечением, но была проделана работа по внедрению дистрибутива Linux, который может на нём работать. ev3dev — это операционная система на базе Debian Linux, которая может работать на LMSEV3. Более подробную информацию о возможностях ev3dev можно найти на его [сайте](#). Процедура запуска ev3dev в LMSEV3 аналогична запуску операционной системы Linux на Raspberry Pi. Лучший и самый быстрый способ запустить ev3dev в LMSEV3 — следовать инструкциям на странице «[Начало работы](#)». По сути, нужно скачать образ ev3dev и записать его на мини-SD-карту (да, без мини-SD-карты вы не сможете его запустить), вставить её в LMSEV3 и загрузить.

Итак, что же вы найдёте в этой специальной версии Linux для LMSEV3? Во-первых, на экране LMSEV3 вы найдёте меню, доступ к которому можно получить с помощью кнопок. Во-вторых, вы сможете подключиться по SSH так же, как на Raspberry Pi, BeagleBone Black или любой другой операционной системе на базе Linux, и работать с системой папок ev3dev, подготовленной для LMSEV3. Чтобы получить доступ ко всем портам ввода-вывода LMSEV3 (включая кнопки, светодиод и экран), на странице [руководств ev3dev](#) есть специальные пояснения, но 4diac FORTE обеспечивает доступ к ним, и вы, как пользователь, будете работать только с обычными функциональными блоками ввода-вывода, такими как IX, QX, IW, QW и так далее.

Чтобы получить доступ к LMSEV3 через SSH, сначала нужно настроить сетевое подключение. Самый простой способ — использовать адаптер Wi-Fi, который можно подключить к USB-порту LMSEV3. [Здесь](#) перечислены несколько совместимых адаптеров Wi-Fi. Если у вас есть адаптер Wi-Fi, вы можете подключиться к сети Wi-Fi через «Беспроводную связь и сети» в меню «Пуск». Включите Wi-Fi, найдите доступные сети и подключитесь к ним. Если вы успешно подключились к сети Wi-Fi, IP-адрес LMSEV3 отобразится в левом верхнем углу экрана. Чтобы упростить себе жизнь, настройте маршрутизатор на использование статических IP-адресов. Теперь вы можете настроить SSH-соединение на своём компьютере. Как настроить соединение с помощью PuTTY или WinSCP, описано [здесь](#).

После того как LMSEV3 запустит ev3dev и станет доступен через SSH, вам нужно будет установить и запустить 4diac FORTE. Как и в случае с другими платформами, у вас есть два варианта. Первый вариант — передать исходный код на ev3dev (используя [WinSCP](#) в Windows или команду «scp» в Linux), установить компилятор и другие необходимые инструменты (в основном компилятор C++ и CMake) и скомпилировать его непосредственно в LMSEV3. Второй вариант — использовать кросс-компилятор. Это означает, что вы будете писать код и компилировать его на своём компьютере, а затем отправлять исполняемый файл в LMSEV3 и запускать его там. Первый вариант проще в реализации, но медленнее компилируется. Второй вариант намного быстрее, но вам придётся настроить среду на своём компьютере. С помощью Docker второй вариант реализовать намного проще. Первый вариант следует использовать только в том случае, если установка Docker невозможна. Особенно если вы будете компилировать несколько раз, вам стоит попробовать второй вариант. Настроить кросс-компиляцию на компьютере для ev3dev непросто, но [здесь](#) представлено полное пошаговое руководство.

Чтобы настроить кросс-компилятор, необходимо выполнить следующие действия:

Примечание! Все команды, начинающиеся с docker, выполняются в командной строке вашего основного компьютера, все остальные команды выполняются внутри вашего контейнера Docker (при кросс-компиляции) или внутри вашего EV3!

1. Установите [Docker](#). Подробное руководство по установке можно найти [здесь](#). Docker будет использоваться для запуска образа ev3dev на вашем компьютере. Это позволит вам создать 4diac FORTE для EV3 с помощью ресурсов вашего компьютера.
2. Следуйте инструкциям [здесь](#), чтобы загрузить образ кросс-компилятора ev3dev.

3. После настройки образа запустите контейнер Docker с помощью следующей команды:

```
docker run --rm -it -v $(pwd) :/src -w /src ev3cc
```

Давайте разберёмся:

- `run` запускает новый контейнер
 - `--rm` гарантирует, что контейнер будет удалён после завершения работы.
 - `-it` это сделает командную строку доступной внутри вашего контейнера.
 - `-v <путь-на-хосте>:<путь-в-контейнере>` это сделает каталог вашего хост-компьютера доступным из контейнера. Не забудьте указать путь на хосте в соответствии с вашей операционной системой. На хостах Windows у вас могут возникнуть проблемы с путями к файлам, например, при использовании `git bash` в Windows вам нужно использовать двойную обратную косую черту в качестве разделителя файлов.
 - `-w <путь-в-контейнере>` это укажет рабочий каталог внутри вашего контейнера.
 - ``ev3cc`` — это название вашего образа Docker.
4. Теперь, когда контейнер запущен, вам нужно установить необходимые инструменты для компиляции 4diac FORTE в вашем контейнере. Выполните следующие команды:
- ```
sudo apt-get update sudo apt-get install build-essential sudo apt-get install cmake
```
- Чтобы сэкономить время в следующий раз, вы можете создать новый образ Docker на основе текущего контейнера. Это сохранит все изменения, и вам не придётся делать это снова. Используйте следующую команду:
- ```
docker commit [CONTAINER_ID] [new_image_name]
```
- Чтобы увидеть все запущенные в данный момент контейнеры (и их идентификаторы), используйте:
- ```
docker ps -a
```
- а чтобы проверить, сработало ли это и вы сохранили новый образ, используйте:
- ```
docker images
```
5. Если вы ещё этого не сделали, перенесите папку 4diac FORTE в папку, к которой вы обращаетесь из своего контейнера, или перенесите её напрямую в EV3 с помощью WinSCP, если вы компилируете там.
6. Затем перейдите в перенесённую папку 4diac FORTE (в командной строке вашего контейнера) и выполните:
- ```
. setup_lmsEv3.sh
```
- Это настроит всё в папке `bin/ev3dev`. Далее вам следует зайти в папку `bin/ev3dev` и просто выполнить команду `"make"`, чтобы начать компиляцию.
7. Теперь вы скомпилировали 4diac FORTE, который может работать внутри вашего brick! Если вы уже настроили сетевое подключение к своему brick, вы можете отправить forte на свой brick с помощью WinSCP. Проверьте, включены ли права на выполнение для forte.

Если вы хотите выполнить компиляцию непосредственно в EV3, то начните с шага 4 и выполняйте все действия в командной строке EV3 (чтобы получить командную строку в EV3, подключитесь к нему с помощью PuTTY).

Помните, что при создании собственных FB вы должны добавить их в папку 4diac FORTE, изменить CMakefiles.txt и выполнить повторную компиляцию. В [разделе с примерами](#) объясняется, как это сделать.

### Устранение неполадок

Если у вас ограниченные права на хост-компьютере (например, ваша компания управляет некоторыми настройками) и поэтому вы не можете использовать Docker, вы можете установить дистрибутив Linux на виртуальную машину и запустить там контейнер Docker. Если у вас возникли проблемы с настройкой виртуальной машины на компьютере с Windows, попробуйте отключить «Hyper-V». Вы можете следовать инструкциям [здесь](#)

Если у вас возникли проблемы с подключением к EV3, проверьте, работает ли сетевое подключение, и пропируйте EV3. Если ваш EV3 подключен, в верхней части экрана должен отображаться IP-адрес.

```
ping [IP ADDRESS]
```

Если у вас возникли проблемы с подключением внешних модулей, добавьте это в файл `setup_lmsEv3.sh` и запустите его снова.

```
-DFORTE_EXTERNAL_MODULES_DIRECTORY="/src/forte/ext_mod"
-DFORTE_MODULE_EV3=ON
```

В папке forte должна быть папка ext\_mod. Затем ваши модули добавляются в качестве подпапок в ext\_mod. В данном случае добавляется модуль под названием «EV3».

## Настройка MinGW-w64

### Установите последнюю версию MinGW-w64 через MSYS2

Поскольку оригинальный установщик mingGW не всегда работает, это лучший способ установить последнюю версию MingGW-64. Скачайте последнюю версию [установщика MSYS2](#) и следуйте инструкциям мастера установки. После установки MSYS2 откройте терминал «MSYS2 MINGW64» и загрузите набор инструментов Mingw-w64 с помощью менеджера пакетов:

```
pacman -S --needed base-devel mingw-w64-x86_64-toolchain
```

### Установите MinGW-w64

MinGW-w64 включает в себя инструменты разработки для компиляции и отладки. Скачайте установщик [Mingw-w64](#) отсюда и следуйте инструкциям мастера установки. Важно, чтобы путь установки не содержал пробелов. Поэтому вы не можете установить MinGW-w64 в папку Program Files. Мы рекомендуем создать папку mingw-w64 на вашем системном диске (например, C:\mingw). Установите текущую версию и при запросе укажите win32 в качестве потока. Кроме того, выберите архитектуру x86\_64.

### Задайте переменную PATH

После установки вам нужно будет отредактировать переменную PATH. Вы можете открыть Центр управления системой, нажав клавишу Windows + Pause. В окне «Система» нажмите «Дополнительные параметры системы» → «Дополнительно» (вкладка) → «Переменные среды». Для Windows 10 быстрый доступ можно получить, введя «Изменить системные переменные среды» в поиске Windows и нажав кнопку «Переменные среды». Измените переменную PATH (дважды щёлкнув по ней или выбрав и отредактировав) и добавьте путь, по которому установлен MinGW-w64, например, C:\mingw\mingw64\bin или C:\msys64\mingw64\bin. В этой папке должно быть несколько файлов .exe, которые вы можете увидеть в проводнике.

### Протестируйте MinGW-w64

Откройте командную строку Windows либо через меню «Программы», либо введя cmd в поиске Windows. Введите:

```
g++ --version
```

Если отображается информация о версии компилятора, значит, всё прошло успешно, и вы можете перейти к следующему разделу.

Если вы получаете сообщение об ошибке, например «Команда g++ не найдена», значит, что-то пошло не так. В этом случае убедитесь, что переменная PATH успешно задана и указывает на правильный путь установки. Кроме того, откройте новую командную строку и попробуйте выполнить команду ещё раз. Если вы пытаетесь выполнить переустановку, не забудьте снова перезапустить командную строку (она не обновляется автоматически). Если это не помогло, пожалуйста, обратитесь за советом на [форум](#).

### Измените имя исполняемого файла make

В папке установки, в каталоге bin, вы должны найти файл с именем mingw32-make.exe. Создайте копию этого файла в том же каталоге с именем make.exe. Это связано с тем, что этот файл вы будете использовать для компиляции 4diac FORTE, а «make» — это команда, используемая для этого во всей документации.

### Следующие шаги

Теперь, когда вы успешно установили компилятор, вы можете перейти к следующему шагу руководства по [установке](#). Вся информация в руководстве по установке основана на настройке с помощью MinGW-w64. Для пользователей, имеющих опыт сборки 4diac FORTE, приведены следующие сведения:

- При создании файлов для компиляции в CMake необходимо выбрать инструмент «Unix Makefiles».
- Также в CMake необходимо задать параметр архитектуры: правильная архитектура FORTE\_ARCHITECTURE — Win32.
- При компиляции откройте командную строку Windows и перейдите в папку, в которой CMake создал двоичные файлы. Для компиляции выполните команду make.

## Настройка 4diac FORTE для Posix PikeOS 5.x

Эта инструкция предполагает, что у вас есть система Linux с установленной PikeOS 5.x и установленная CODEO, соответствующая установленной версии PikeOS.

(CODEO — это интегрированная среда разработки на основе Eclipse для разработки проектов PikeOS, поставляемая с продуктом PikeOS.) Также установлены CMAKE и CMAKE-GUI.

Предполагается, что вы знакомы с основами PikeOS, описанными в «pikeos-tutorial.pdf».

Предполагается, что у вас есть рабочее пространство CODEO с 4 проектами:

- /home/user/workspace + рабочее пространство CODEO
  -  pool  
"НАСТРОЕННЫЙ ПУЛ" для всех проектов PikeOS
  -  forte-incubation  
Исходный код 4diac FORTE  
Импортированный проект с помощью мастера импорта CODEO (Eclipse) (Файл → Импорт → Общие → Проекты из папки или архива). После импорта проект был переименован в "forte-incubation".  
Источником импорта может быть клонированный репозиторий git (git://git.eclipse.org/gitroot/4diac/org.eclipse.4diac.forte.git) или ZIP-файл «4diac FORTE Source», загруженный с eclipse.org/4diac.
  -  lwip.app  
Проект приложения PikeOS 5.x типа Posix Project / lwip-build.
  -  forteserver.int  
Интеграционный проект PikeOS 5.x типа «Интеграционный проект / devel-posix».

### Исправление «lwip.app».

Проект приложения lwip.app создаст библиотеку (iblwip.a), которая требуется для 4diac FORTE. Поэтому мы будем использовать этот проект для создания 4diac FORTE (двоичное имя: forte) вместе с iblwip.a.

Примените патч forte-incubation/src/arch/pikeos\_posix/lwip.patch к lwip.app.

Это позволит адаптировать Makefile и файл lwip.app/lwipopts.h в соответствии с требованиями для сборки 4diac FORTE. Патч lwip.app/lwipopts.h включает многоадресную рассылку, которую 4diac FORTE использует при взаимодействии нескольких экземпляров друг с другом. Кроме того, создается папка lwip.app/4diac-forte. Эта папка будет использоваться проектом forte-incubation для хранения необходимых исходных и Make-файлов для создания двоичного файла forte.

### Запуск CMAKE для "forte-incubation"

Для цели "PikeOS Posix" проекта forte-incubation требуются переменные среды, экспортируемые lwip.app/PikeOS.sh.

Откройте оболочку (например, выбрав lwip.app в CODEO и нажав [.kbd] ⬆ + [.kbd] Alt + [.kbd] W и выберите «Терминал») и выполните:

```
`console source /home/user/workspace/lwip.app/PikeOS.sh cd ../forte-incubation cmake-gui`
```

В качестве альтернативы вы можете:

1. Щелкнуть правой кнопкой мыши по проекту «forte-incubation» и выбрать «Свойства».
2. Выберите «C/C++-Сборка» и нажмите «Управление конфигурациями...»
3. Переименуйте «RELEASE» в «CMAKE», удалите все остальные существующие конфигурации и нажмите кнопку «ОК».
4. Снимите флажок «Использовать команду сборки по умолчанию» и установите его на:  
bash -c "source \${workspace\_loc:/lwip.app/PikeOS.sh}; cd  
\${workspace\_loc:/forte-incubation}; cmake-gui
5. Снимите флажок «Автоматически генерировать Makefile» и установите «Каталог сборки» на:  
\${workspace\_loc:/forte-incubation}/
6. Теперь вы можете запустить CMAKE-GUI, щелкнув правой кнопкой мыши по проекту «forte-incubation» и выбрав «Собрать проект».

CMAKE-GUI показывает диалоговое окно. Сначала установите forte-incubation в качестве исходной папки и двоичную папку lwip.app:

Where is the source code:

Where up build the binaries:

Нажмите «Настроить» и выберите «Makefile Unix» и «Укажите параметры для кросс-компиляции»:

Unix Makefiles

☐ Use default native compilers  
☐ Specify native compilers  
☐ Specify toolchain file for cross-compiling  
☒ Specify options for cross-compiling

Нажмите «Далее», выберите «Операционную систему» как «Универсальную» и оставьте все остальные поля пустыми.

Target

Operating system:

После этого нажмите «Готово», появится ожидаемое диалоговое окно с ошибкой. Закройте диалоговое окно и установите параметр FORTE\_ARCHITECTURE на PikeOS\_Posix.

Where is the source code:

Where up build the binaries:

CMAKE\_SYSTEM\_NAME      Generic  
 FORTE\_ARCHITECTURE      PikeOS\_Posix

Дважды нажмите «Настроить». Теперь ошибок быть не должно. Это означает, что ни один параметр больше не выделен светло-красным цветом.

## Сборка двоичного файла «forte» для PikeOS

В проекте lwip.app команда «all» создаст lwip.a вместе с двоичным файлом «forte». Команда «install» дополнительно скопирует «forte» в \$PIKEOS\_CUSTOM\_POOL\_DIR/posix.

Проект интеграции PikeOS "forteserver.int"

Выберите posix-devel/Application/Posix Process и измените файл Host на CUSTOM\_POOL/posix/forte. Теперь нажмите кнопку «Добавить...» (под заголовком «Структура проекта»), добавьте шаблон posix / lwIP Ethernet-интерфейс в Posix Process и настройте его:

| Ключ             | Значение                                | Пример для QEMU с пользовательским NW | Пример для QEMU с TUN/TAP NW |
|------------------|-----------------------------------------|---------------------------------------|------------------------------|
| Адрес интерфейса | например, целевой IP-адрес MUXA +1      | 10.0.2.16                             | 192.168.0.3                  |
| Маска подсети    | 255.x.x.x                               | 255.255.255.0                         | 255.255.255.0                |
| Адрес шлюза      | Шлюз в целевой сети для доступа к хосту | 10.0.2.2                              | 192.168.0.1                  |

Проект выглядит так:

- posix-devel

☐ Приложение

▣ Раздел POSIX

▣ Процесс POSIX

... lwIP Ethernet ...

☐ По умолчанию

☐ Инструменты разработки

☐ qemu-...

Предположим, что мы используем QEMU с сетью пользовательского режима. Это позволит 4diac FORTE работать на 10.0.2.16:61499, так как 61499 — это порт 4diac FORTE по умолчанию.

Примечание! Если вы хотите использовать другой порт, вам нужно добавить шаблон posix / переменные среды в процесс Posix (аналогично добавлению интерфейса Ethernet posix / lwIP). Параметр -с IP:PORT запустит 4diac FORTE на другом порту.

Щелкните правой кнопкой мыши по разделу POSIX и выберите «Копировать». Щелкните правой кнопкой мыши по приложению и выберите «Вставить». Переименуйте «имя раздела» и «идентификатор раздела», например, в «posix2» и «3», так как они должны быть уникальными.

Повторите эту процедуру для процесса POSIX.

- Щелкните по разделу POSIX (2) и назначьте процесс POSIX (2).
- Нажмите на POSIX Process (2) / ... lwIP Ethernet ... и измените DEPEND: LWIPDEVICE\_IF на ...-vchan2.
- Настройте POSIX Process (2) / ... lwIP Ethernet ...:

| Ключ              | Значение                | Пример для QEMU с пользовательским NW | Пример для QEMU с TUN/TAP NW |         |
|-------------------|-------------------------|---------------------------------------|------------------------------|---------|
| Interface Address | e.g. "POSIX Process" +1 | 10.0.2.17                             | 192.168.0.4                  | Netmask |

| Ключ      | Значение      | Пример для QEMU с пользовательским NW | Пример для QEMU с TUN/TAP NW |                                            |
|-----------|---------------|---------------------------------------|------------------------------|--------------------------------------------|
| 255.x.x.x | 255.255.255.0 | 255.255.255.0                         | Маска подсети                | Шлюз в целевой подсети для доступа к хосту |

Теперь проект выглядит так:

posix-devel

- ☐ Приложение
  - ☐ Раздел POSIX
  - ☒ Процесс POSIX
  - ☐ Раздел POSIX (2)
  - ☒ Процесс POSIX (2)
- ☐ По умолчанию
- ☐ Инструменты разработки
  - ☒ миха
    - каналы
- ☐ qemu-...
  - ☐ Драйверы ядра монитора
  - ☐ ... Драйвер пользовательского уровня Serial
  - ☐ ... Драйвер пользовательского уровня Ethernet
    - ☐ ...-net-device
    - ☐ ...-net-vchan0
    - ☐ ...-net-vchan1
    - ☐ ...-net-vchan2
    - ☐ ...-net-vchan3

Откройте «Инструменты разработки» / миха / каналы и удалите назначение «Процесс POSIX (2)».

Переименуйте канал 4, например, в «posix2-stdio». Назначьте «POSIX-процесс (2)» этому каналу.

Перейдите в qemu-... / ... Драйвер пользовательского уровня Ethernet и установите для ...-net-device, ...-net-vchan0 и ...-net-vchan1 значение «Включить многоадресную передачу» равным true.

Теперь вы можете протестировать разделы, запустив QEMU и выполнив [пошаговое руководство 4DIAC](#).

При использовании сети пользовательского режима вам нужно запустить QEMU, выбрав «Запустить QEMU с пользовательской командной строкой» и указав переадресацию портов:

```
-net user,hostfwd=udp::21500-:1500,hostfwd=tcp:127.0.0.1:61499-
10.0.2.16:61499,hostfwd=tcp 127.0.0.1:61500-
10.0.2.17:61499,host=10.0.2.2,net=10.0.2.0/255.255.255.0
```

«Запустить QEMU с пользовательской командной строкой» не перезаписывает IP-адрес и порт «muxa.xml», как это было бы при выборе «Запустить QEMU с сетью пользовательского режима». Этот шаблон Make-target позволяет адаптировать сгенерированный «muxa.xml» к сети пользовательского режима:

```
make boot: all
 $(eval HOSTIP := 127.0.0.1)
 $(eval TARGETPORT := 21500)
 @sed -ie "s/^. *TargetIP.*$$/ <Parameter Name=\"TargetIP\" \"
Value=\"$(HOSTIP)\"/>\>/" muxa.xml
 @sed -ie "s/^. *HostIP.*$$/ <Parameter Name=\"HostIP\" \"
Value=\"$(HOSTIP)\"/>\>/" muxa.xml
 @sed -ie "s/^. *TargetPort.*$$/ <Parameter Name=\"TargetPort\" \"
Value=\"$(TARGETPORT)\"/>\>/" muxa.xml
```

# Сборка 4diac FORTE на Raspberry Pi

## Введение

Это руководство по сборке FORTE для Raspberry Pi. Информацию об используемых параметрах см. в разделе «[Параметры](#)».

[Raspberry Pi \(RPI\)](#) — одна из самых известных недорогих встраиваемых систем. Raspberry Pi 3, третье поколение, было выпущено в феврале 2016 года. Он оснащен 64-разрядным четырехъядерным процессором ARMv8 с тактовой частотой 1,2 ГГц, 1 ГБ оперативной памяти и несколькими вариантами ввода-вывода, в том числе 40-контактным разъемом. Операционная система загружается с SD-карты, поэтому может поддерживать множество ОС. Наиболее популярным из них является Raspbian на базе Debian. 4diac FORTE ранее использовал библиотеку WiringPi, но теперь она использует виртуальную файловую систему sysFs. В этом руководстве сначала объясняется, как скомпилировать 4diac FORTE в RPI или даже выполнить кросс-компиляцию, а также как легко использовать блоки ввода-вывода в 4diac FORTE для управления 40-контактным разъёмом.

## Подготовка RPI

В этом руководстве не рассматриваются подробности того, как запустить RPI, но если вы никогда раньше не пользовались им, то должны знать, что помимо платы RPI вам понадобится источник питания, подключённый к порту mini-USB, и карта памяти mini-SD. RPI можно подключить к экрану, мыши и клавиатуре через порты HDMI и USB, но вы также можете подключиться к нему без экрана, то есть через ssh с помощью Ethernet или USB-разъёма для Wi-Fi. Существует множество руководств по доступу к RPI, поэтому в этом руководстве предполагается, что у вас есть доступ к командной строке RPI через ssh или непосредственно на нём с помощью графического интерфейса, а также что у RPI есть доступ в Интернет.

## Создание 4diac FORTE на RPI

### Подготовка

Установите дополнительное программное обеспечение, необходимое для вашего RPI. \* Git: 4diac FORTE хранится в виде репозитория Git на сервере Eclipse. \* Инструменты сборки: cmake, make, gcc, g++

```
$ sudo apt-get install git cmake make gcc g++
```

### Сборка

1. Скачайте 4diac FORTE из репозитория Git в вашем домашнем каталоге.

```
$ cd ~ $ git clone
```

<https://git.eclipse.org/r/4diac/org.eclipse.4diac.forte>

2. Подготовьте структуру проекта 4diac FORTE для архитектуры Posix.

```
$ cd org.eclipse.4diac.forte $ chmod +x setup_posix.sh
```

Вам нужно будет добавить модуль SysFs в компиляцию. Для этого вам нужно будет изменить файл setup\_posix.sh с помощью предпочитаемого вами текстового редактора. В файле вы найдёте строку, которая начинается с cmake -G "Unix Makefiles" -DFORTE\_ARCHITECTURE=Posix. В конце строки добавьте -DFORTE\_MODULE\_SysFs=ON. Это позволит подключить 40-контактный разъём RPI. Сохраните файл, закройте редактор и выполните:

```
$./setup_posix.sh
```

Если ошибок не возникает, команда создаёт папку в bin/posix, в которой хранятся все файлы для компиляции 4diac FORTE. Если папки ещё нет, создайте её.

3. Выполните команду «make», которая запустит компиляцию.

```
$ make
```

## Кросс-компиляция для RPI

RPI — очень мощный компьютер, но при разработке проекта иногда приходится много раз компилировать 4diac FORTE. Поскольку Raspberry Pi работает медленно по сравнению с мощностью настольного компьютера, полезна кросс-компиляция. Она позволяет скомпилировать программу на настольном компьютере, а затем отправить исполняемый файл на Raspberry Pi.

Вам понадобится исходный код 4diac FORTE и те же дополнительные инструменты, что и при подготовке, кроме CMake, потому что в этом случае понадобится CMake-GUI.

## Кросс-компиляция с использованием Linux

### 1. Установите необходимые дополнительные инструменты:

- o CMake-GUI

```
$ sudo apt-get install cmake-gui
```

- o Набор инструментов для кросс-компиляции ARM GNU. Вы можете скачать его с [веб-сайта разработчиков ARM](#). Затем вам нужно распаковать этот файл

```
$ tar -xvf gcc-linaro-4.8-2015.06-x86_64_arm-linux-gnueabi.tar.xz
```

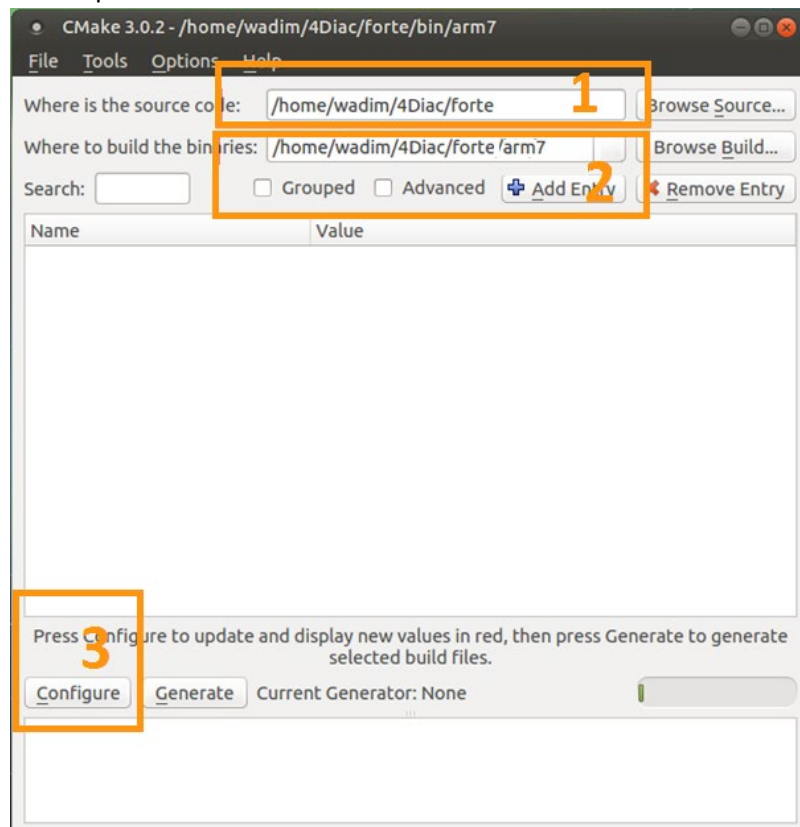
Вы можете не скачивать и не распаковывать файл, используя команду

```
$ sudo apt-get install gcc-arm-linux-gnueabi g++-arm-linux-gnueabi
```

но в некоторых случаях этот метод приводил к ошибкам.

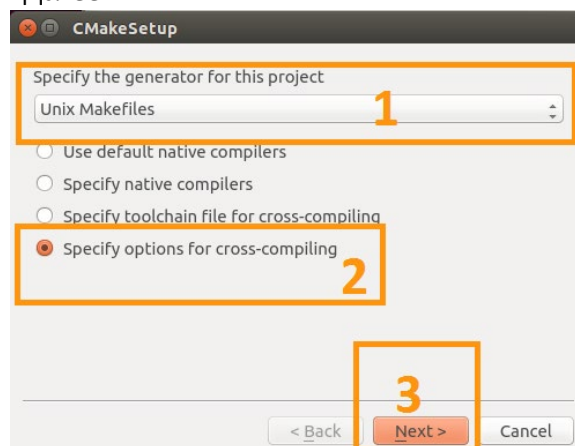
### 2. Откройте CMake-GUI и выполните действия, как показано на изображении

- a. Задайте путь к исходному коду 4diac FORTE, куда вы клонировали репозиторий Git.
- b. Задайте путь к двоичным файлам, где вы хотите создать исполняемый файл. Обычно используется bin/raspPi.
- c. Нажмите «Настроить»



### 3. Настройка CMake

- i. Выберите инструмент, который вы обычно используете для компиляции своих программ. В этом примере используется список Makefile для UNIX.
- ii. Выберите «Указать инструменты для кросс-компиляции»
- iii. Нажмите «Далее»

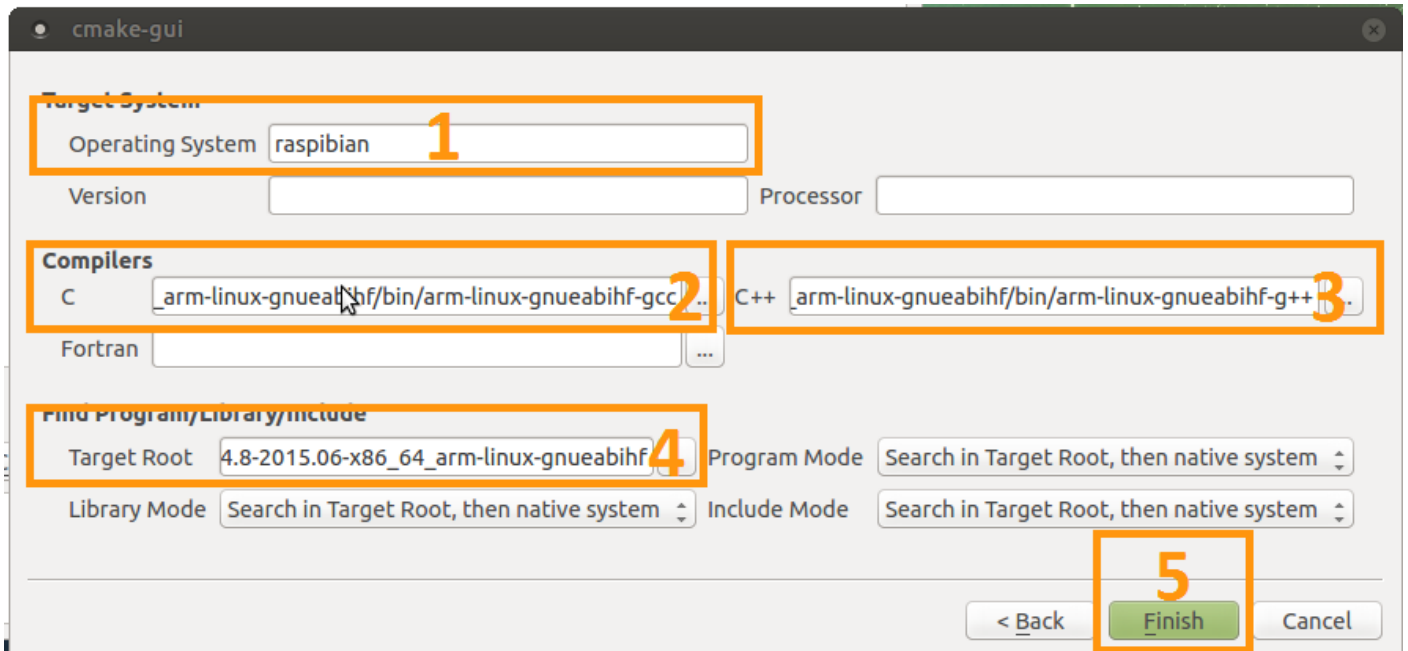


### 4. Настройка CMake

- i. Напишите название ОС (обычно Raspbian, это не повлияет на компиляцию).
- ii. Выберите путь к кросс-компилятору C под названием arm-linux-gnueabi-gcc в папке bin загруженной папки инструментов. Если он не найден, вы можете выполнить команду:  

```
$ which arm-linux-gnueabi-gcc
```
- iii. Выберите путь к кросс-компилятору C++ под названием arm-linux-gnueabi-g++ в папке bin загруженной папки инструментов. Если он не найден, вы можете выполнить команду:  

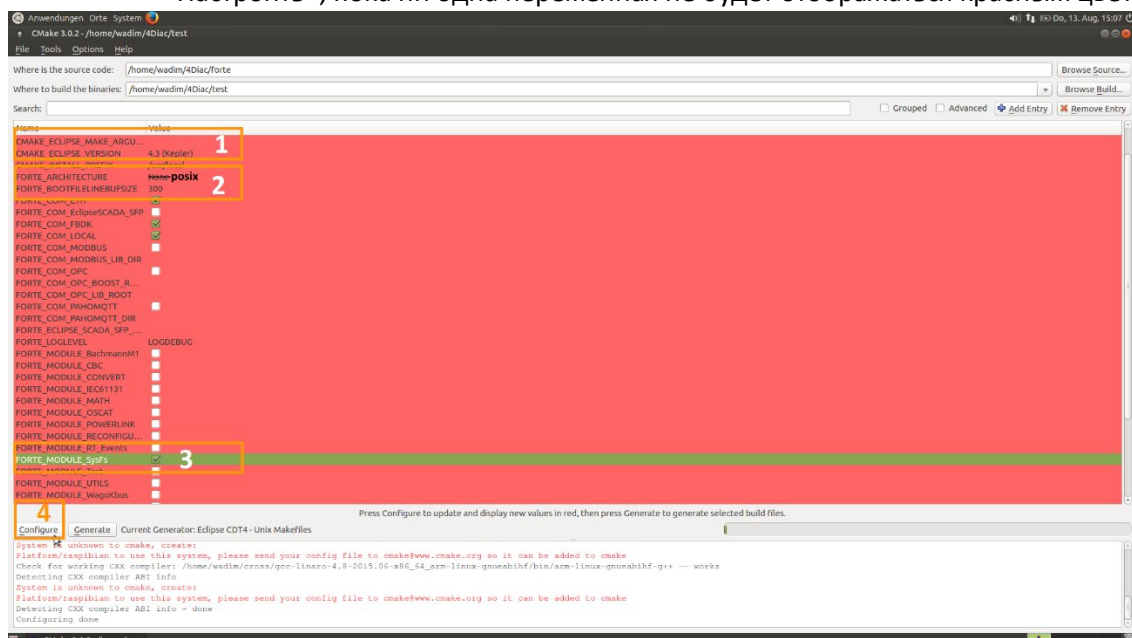
```
$ which arm-linux-gnueabi-g++
```
- iv. Поле целевого корня можно оставить пустым.
- v. Нажмите «Готово»



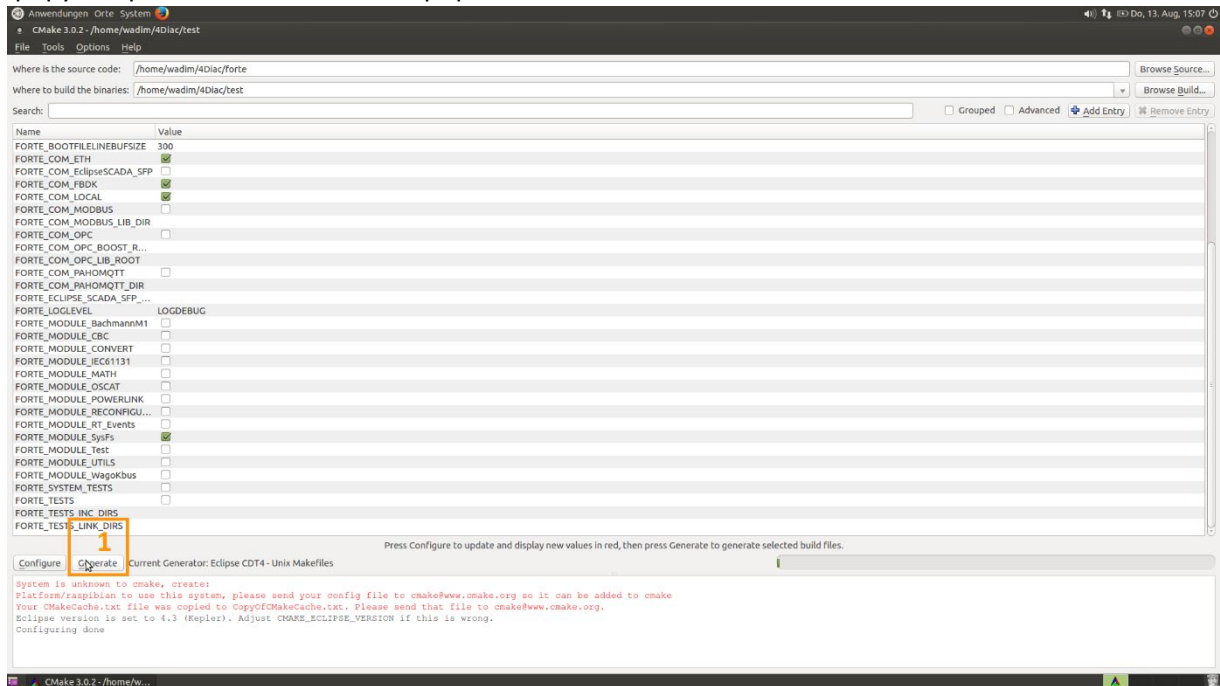
## 5. Настройте компиляцию

В CMake должен отображаться список всех переменных 4diac FORTE, выделенных красным, как показано на рисунке ниже.

- i. Выберите свою версию Eclipse, если вы выбрали её на шаге 3.1. В противном случае можно оставить всё как есть.
- ii. Задайте для переменной FORTE\_ARCHITECTURE значение Posix
- iii. Включите FORTE\_MODULE\_SysFs и все остальные модули, которые вам нужны
- iv. Нажмите «Настроить», и переменные, которые нужно изменить, снова отобразятся красным цветом, а остальные — белым. Проверьте эти переменные и нажимайте «Настроить», пока ни одна переменная не будет отображаться красным цветом.



## 6. Сгенерируйте файлы: нажмите «Сгенерировать»



## 7. Скомпилируйте 4diac FORTE: перейдите в недавно созданную папку и выполните команду make (или соберите его как обычно)

```
$ cd bin/raspPi $ make
```

## Кросс-компиляция с использованием Windows

1. Установите необходимые дополнительные инструменты:
  - CMake со страницы [загрузки](#)
  - Инструмент для кросс-компиляции. Вы можете скачать его по этой [ссылке](#).
2. Следуйте инструкциям по кросс-компиляции на платформе [Linux](#) с шага 2, принимая во внимание следующее:
  - На этапах 4.2 и 4.3 кросс-компиляторы C и C++ находятся в папке bin, куда вы установили инструмент, обычно C:\SysGCC\Raspberry\bin

## Компиляция и отладка 4diac FORTE в MS Visual Studio

Ниже описано, как скомпилировать 4diac FORTE с архитектурой win32 в Visual Studio. В руководстве по установке ([в конце статьи](#)) вы найдёте более подробную информацию о создании собственного 4diac FORTE. Кроме того, там описаны другие инструменты.

Важно! Если вы используете версию Visual Studio старше 2010, вам может потребоваться добавить файл `stdint.h`. См., например, [здесь](#).

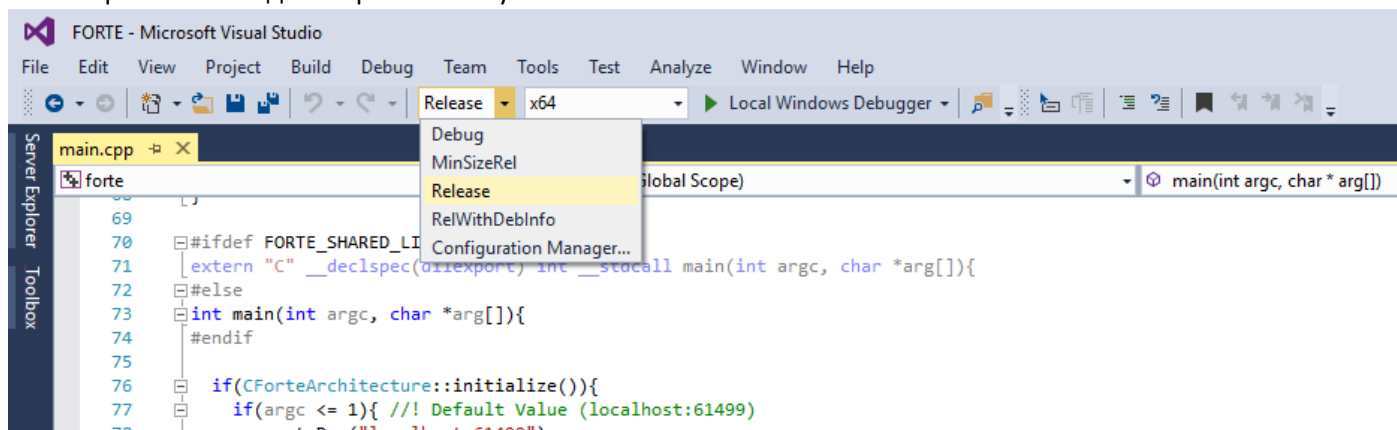
Создание проекта MS Visual Studio для 4diac FORTE с помощью CMake

[В руководстве по установке](#) подробно описаны шаги по созданию проекта с помощью CMake. Однако для использования Visual Studio необходимо выбрать инструмент MS Visual Studio. Правильная архитектура — Win32.

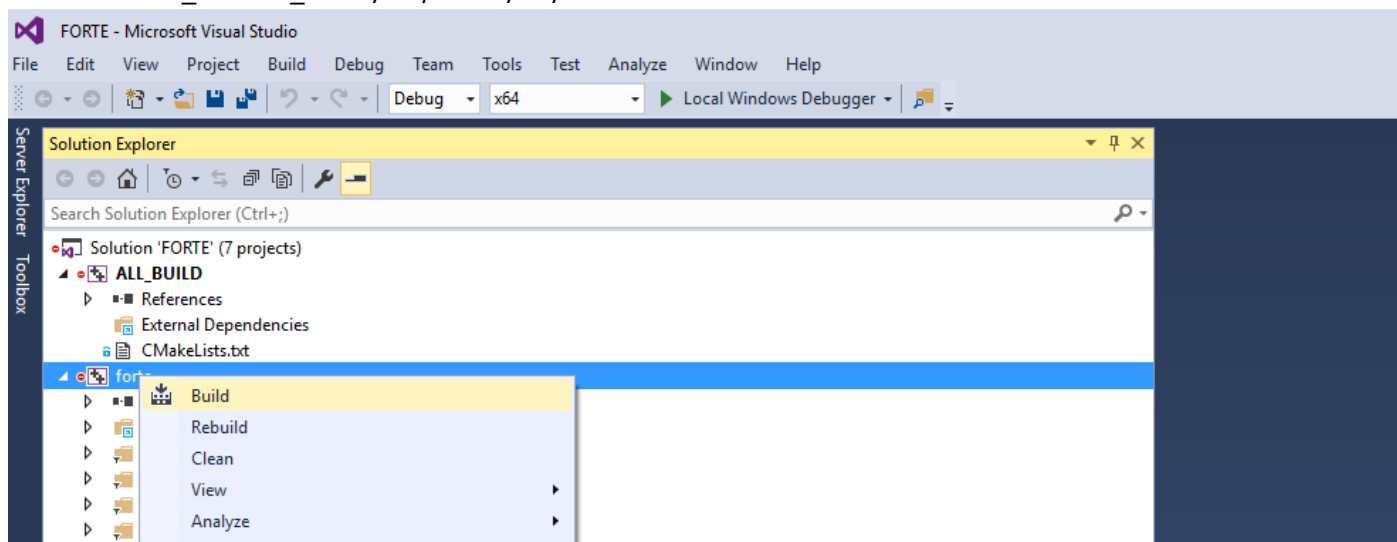
1. Откройте CMake-GUI.
2. Выберите исходную папку `FORTE_FOLDER_ROOT`.
3. Выберите нужную папку назначения. Мы рекомендуем использовать `FORTE_FOLDER_ROOT/bin/win32`.
4. Нажмите кнопку «Настроить» и выберите используемую версию Visual Studio и компиляторы по умолчанию. После этого нажмите кнопку «Готово».
5. Для параметра `FORTE_ARCHITECTURE` выберите Win32. Другие модули и параметры конфигурации можно настроить, следуя инструкциям в разделе «[Создание проекта с помощью CMake](#)».
6. Нажмите кнопку «Настроить», проверьте красные строки и повторяйте это до тех пор, пока все строки не станут красными. Затем нажмите кнопку «Сгенерировать».

## Скомпилируйте 4diac FORTE в Visual Studio

1. Откройте сгенерированный проект 4diac FORTE (.sln-файл) в Visual Studio в папке `FORTE_FOLDER_ROOT/bin/win32`. Если вы не хотите выполнять отладку, вы можете переключиться с режима отладки на режим выпуска.

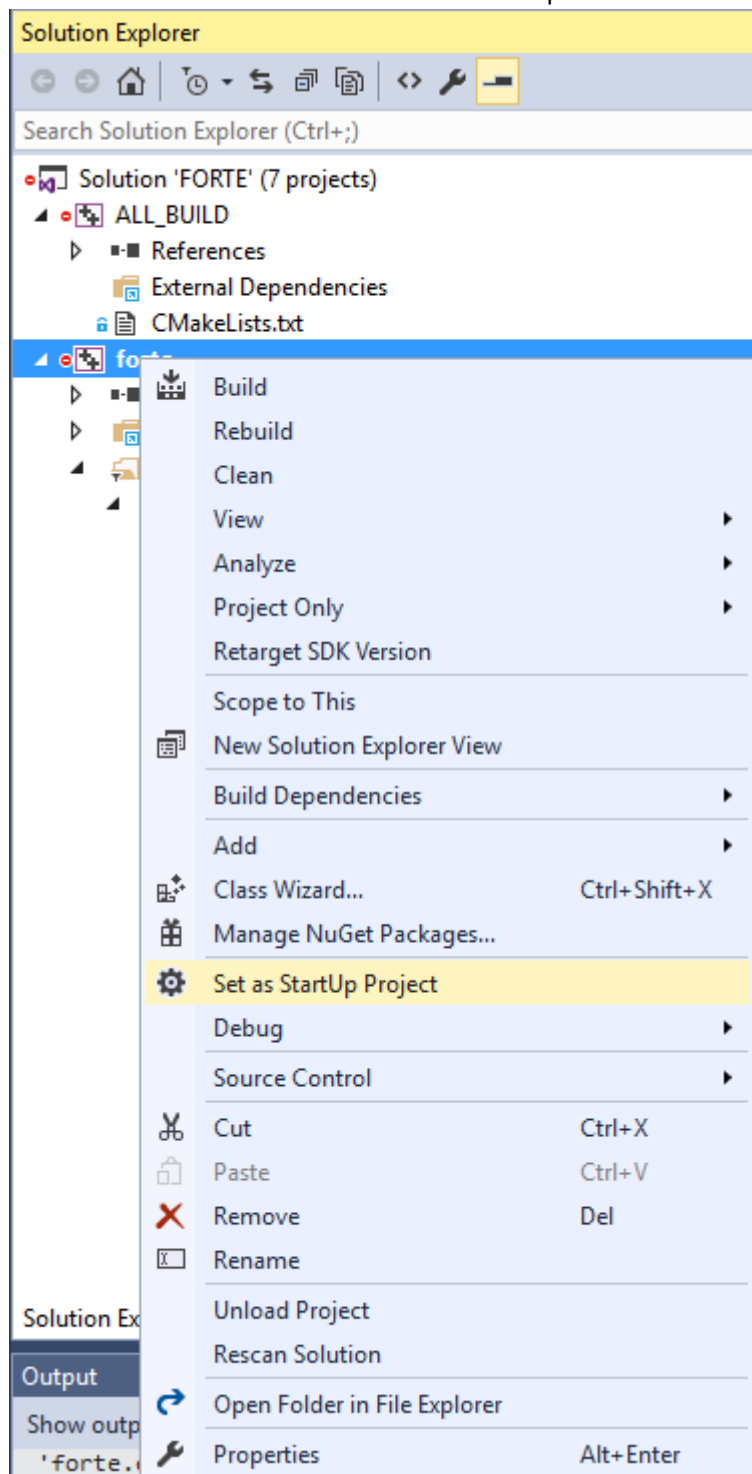


2. Скомпилируйте 4diac FORTE, щелкнув правой кнопкой мыши по forte в окне обозревателя решений и выбрав «Скомпилировать». Файл forte.exe должен быть создан в папке `FORTE_FOLDER_ROOT/bin/win32/src/Release`.



## Отладка

1. Если вы хотите выполнить отладку с помощью Visual Studio, выберите режим отладки и установите 4diac FORTE в качестве основного проекта.



2. Установите 4diac FORTE в качестве основного проекта в Visual Studio

Наконец, нажмите F5 или выберите «Отладка» → «Начать отладку»

## Компиляция и отладка 4diac FORTE с помощью MS Visual Studio Code

Ниже описано, как скомпилировать 4diac FORTE с архитектурой win32 с помощью Visual Studio Code. В руководстве по установке (в [конце](#)) вы найдёте более подробную информацию о создании собственного 4diac FORTE. Кроме того, там описано, как использовать Visual Studio Code.

### Создание проекта MS Visual Studio Code для 4diac FORTE с помощью CMake

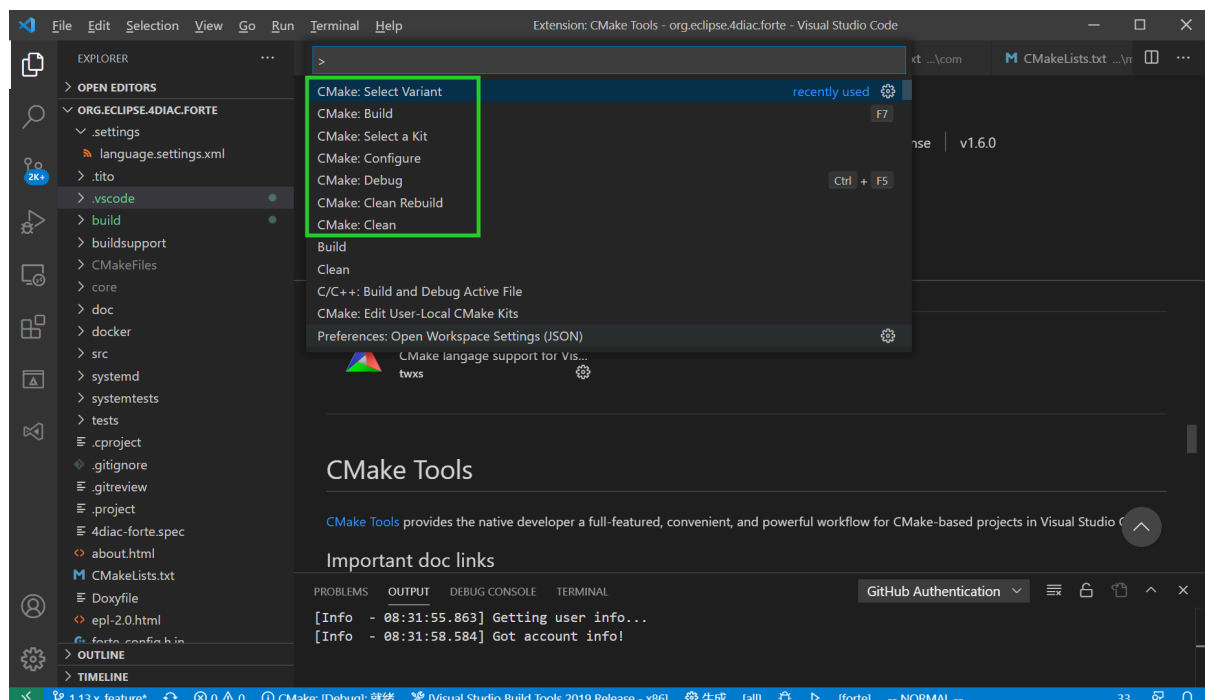
В [руководстве](#) по установке подробно описаны шаги по созданию проекта с помощью CMake. Чтобы использовать Visual Studio Code с CMake, вам нужно указать параметры (все параметры отображаются в CMake-GUI при выборе исходной папки FORTE\_FOLDER\_ROOT, см. раздел «[Создание проекта с помощью CMake](#)») для CMake в файле settings.json для проекта в Visual Studio Code. Правильная архитектура — Win32. Ниже приведен пример файла settings.json для Forte с OPC DA.

```
{ "cmake.configureOnOpen": false, "cmake.configureSettings": { "FORTE_ARCHITECTURE:STRING": "Win32", //Enable 4diac FORTE Com FBDBK "FORTE_COM_FBDBK:BOOL": "ON", //Enable 4diac FORTE Com Ethernet "FORTE_COM_ETH:BOOL": "ON", //Enable 4diac FORTE local communication "FORTE_COM_LOCAL:BOOL": "ON", //IEC61131-3 Function blocks "FORTE_MODULE_IEC61131:BOOL": "ON", "FORTE_MODULE_UTILS:BOOL": "ON", "FORTE_MODULE_CONVERT:BOOL": "ON", //Loglevel to use "FORTE_LOGLEVEL:STRING": "LOGDEBUG", "CMAKE_BUILD_TYPE:STRING": "debug", "FORTE_COM_OPC": "ON", "FORTE_COM_OPC_LIB_ROOT": "D:/4diac/OPC-Client-X64/", "FORTE_COM_OPC_BOOST_ROOT": "D:/4diac/boost_1_84_0/", }}
```

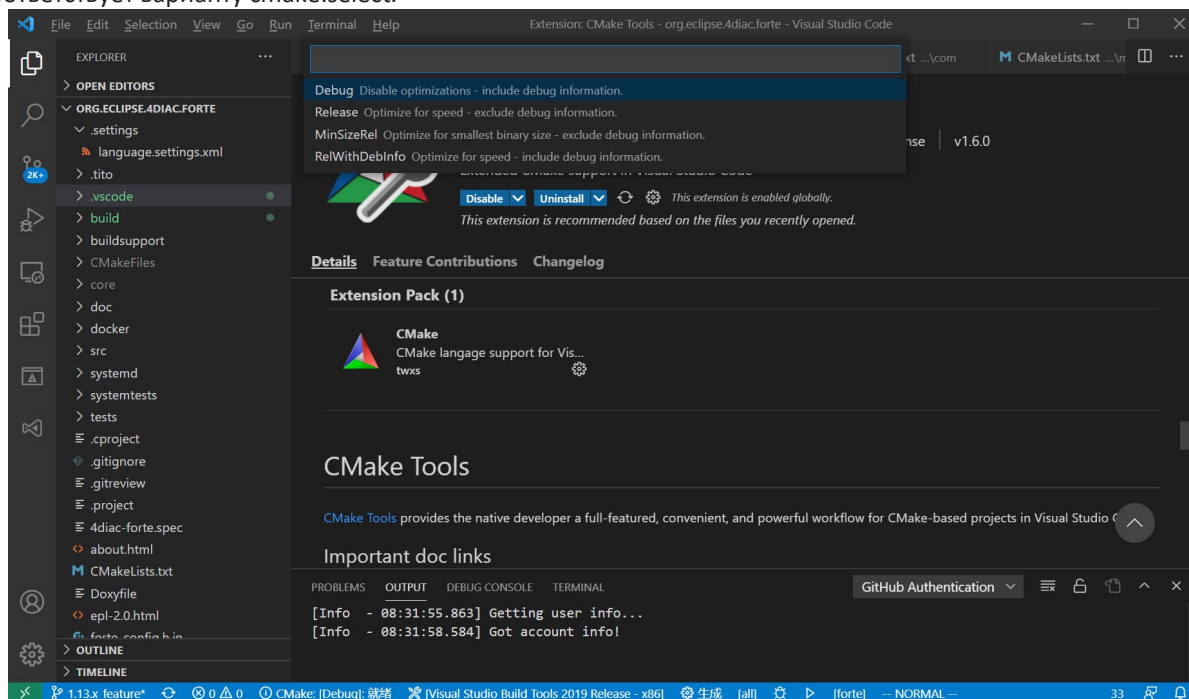
### Скомпилируйте и отладьте 4diac FORTE с помощью Visual Studio Code

1. Установите компилятор. Например, если вы хотите скомпилировать OPC DA, установите компоненты в соответствии с [ms buildtools](#). Или, если вы предпочитаете mingw-w64, следуйте инструкциям для [mingw-w64](#).
2. Установите плагин «CMake tools» в Visual Studio Code.
3. Откройте командную палитру (Ctrl + Shift + P), скомпилируйте или отладьте с помощью следующей команды:

```
cmake:select a kit; //choose MINGW64, msbuildtools or gcc
cmake:select variant; //debug or release
cmake:configure;
cmake:build //compile
cmake:debug
```



4. При выборе режима отладки или выпуска убедитесь, что параметр CMAKE\_BUILD\_TYPE в файле settings.json соответствует варианту cmake:select.



# Создание 4diac FORTE для Wago

## Введение

Серия Wago PFC 200 — это ПЛК, предназначенный для работы на малых и средних предприятиях для мелкомасштабной автоматизации. Он работает на контроллере ARM cortex A8 с тактовой частотой 600 МГц и Linux 3.6 в реальном времени (с патчем RT preemption). Это руководство поможет вам настроить 4diac FORTE на встроенной платформе WAGO. Для выполнения этого руководства вам понадобятся:

- Хост-система [Ubuntu 14.04 LTS](#) с минимальным объемом памяти на жестком диске 25 ГБ (например, в виде виртуальной машины [http://www.virtualbox.org/\[VirtualBox\]](http://www.virtualbox.org/[VirtualBox]))
- В Ubuntu загрузите :
  - пакет поддержки плат [WAGO-PFC-BSP \(BSP\)](#) для настройки и создания пользовательских приложений на платах Wago PFC.
  - Зависимости для Wago BSP
    - [OSELAS](#) — кроссплатформенная цепочка инструментов 2012.12.1 для плат Cortex A8
    - [PTXDist-2013.03.0](#) — система сборки под лицензией GPL для создания целевого встроенного ядра Linux и корневой системы из исходных кодов.
  - [Последняя версия исходного кода 4diac FORTE](#), которая содержит интерфейс процесса для Wago в `forte/src/modules/wagokbus`. `ProcessInterface` упрощает использование аналоговых и цифровых входов и выходов Wago с помощью универсальных блоков IX, QX и IW, QW из 4diac IDE.

Важно! Это руководство основано на WAGO-PFC-BSP-2014.10.3. Приведенная здесь процедура также применима для последних пакетов поддержки плат от Wago. Набор инструментов и версия PTXDist также могут отличаться в зависимости от версии BSP.

## Установка Wago BSP на хост-компьютер Ubuntu

НЕ устанавливайте пакеты в корневую папку! Установка может занять более часа, поэтому наберитесь терпения и дождитесь завершения всего процесса.

1. Создайте папку, например, `~/Wago`
2. Распакуйте пакет поддержки платы WAGO PFC, загруженный ранее, в папку `~/Wago`. Следуйте инструкциям по установке, приведенным в HOWTo\_Install\_WAGO\_PFC\_BSP-версии, до шага «Скомпилировать все пакеты прошивки PFC200».

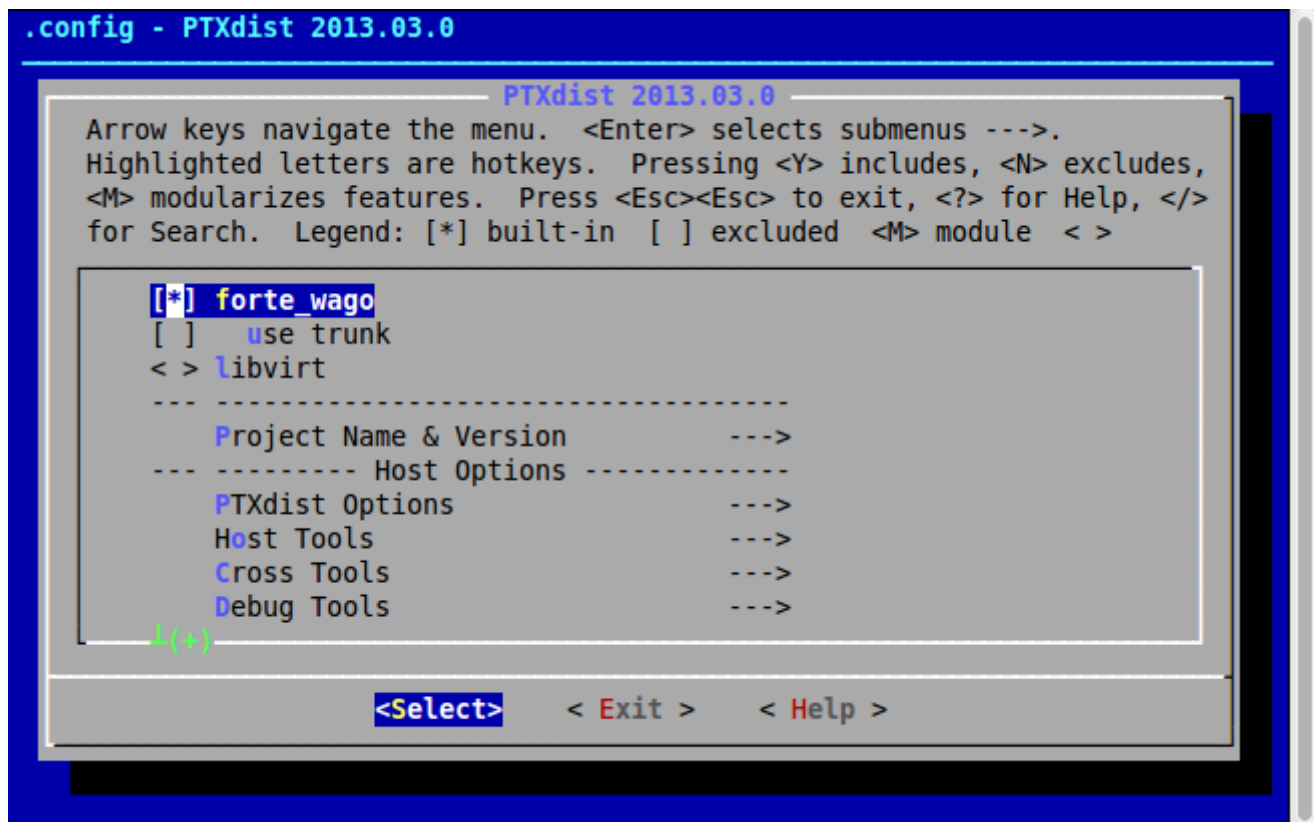
## Сборка 4diac FORTE

4diac FORTE обычно собирается на кроссплатформенных устройствах с помощью CMake. Здесь, для устройства wago, весь процесс сборки контролируется ptxdist. Для процесса сборки 4diac FORTE важно знать о нескольких каталогах в среде разработки wago. Все пакеты ptxdist управляются так называемыми файлами правил. Эти файлы находятся в каталоге `wago/ptxproj-2.2.20/rules`. При создании нового пакета в каталоге `rules` создаются файлы правил `packagename.make` и `packagename.in`. Существует два типа исходных каталогов: `wago/src` для пакетов системного уровня и `wago/local_src` для новых пользовательских пакетов. Каталог сборки для целевой системы (т. е. wago) находится в каталоге `platformwago-wago-pfc200/build-target`.

1. Создайте проект `forte_wago` в PTXDist: из папки `FORTE/buildsupport/wago_pfc200` скопируйте файлы правил `forte_wago.in` и `forte_wago.make` в папку `wago/ptxproj-2.2.20/rules` (создаст новый пакет). В скопированном файле правил `../ptxproj-2.2.20/forte_wago`. Чтобы изменить строку `FORTE_WAGO_URL: file:///<путь к основному каталогу 4diac FORTE>` на абсолютный путь к основному каталогу 4diac FORTE. Чтобы (де)активировать определенные модули (части библиотеки) 4diac FORTE, измените `../ptxproj-2.2.20/wago`. строку `FORTE_WAGO_CONF_OPT := ...`
2. Сохраните пакет в PTXDist: чтобы выполнить стандартную сборку PTXDist, пакет необходимо сохранить в среде сборки PTXDist. Выберите `forte_wago` с помощью клавиши пробела (это будет отображаться как `[*]`) и нажмите «Выход», чтобы завершить работу. Когда появится запрос о сохранении пакета, нажмите «Да».

Важно! PTXDist — это утилита командной строки, которая работает только в определенной среде проекта, например, ВАЖНО: `~wago/ptxproj-2.2.20` ВАЖНО: каталог. Команды ptxdist будут работать только в этом каталоге.

```
> cd wago/ptxproj-2.2.20 > ptxdist menuconfig
```



3. Создайте проект: перейдите в папку ptxproj-2.2.20 и выполните следующие инструкции по созданию 4diac FORTE для целевого устройства Wago PFC.

```
> cd wago/ptxproj-2.2.20 > ptxdist targetinstall forte wago
```

```
dst=/usr/bin/forte
owner=0
group=0
permissions=0755

Packaged contents of /home/milan/wago/ptxproj-2.2.20/platform-wago-pfc200/packages/forte-wago.tmp/ipkg into /home/milan/wago/ptxproj-2.2.20/platform-wago-pfc200/packages/forte-wago_1.6.2_arm.ipk
done.
finished target forte_wago.targetinstall

target: forte_wago.targetinstall.post

finished target forte_wago.targetinstall.post
```

Двоичный файл можно найти в каталоге .../ptxproj-2.2.20/platform-wago-pfc200/buid-target/forte\_wago-1.6.2-build/src. Двоичный файл можно либо скопировать в каталог /usr/bin целевого устройства, либо создать собственный образ Linux, выполнив команду <ptxdist go>; в каталоге ptxproj-2.2.20 и загрузившись с собственным образом прошивки.

4. Перестройка проекта: Для перестройки проекта необходимо очистить проект и выполнить повторную установку targetinstall.

```
> cd wago/ptxproj-2.2.20 > ptxdist clean forte_wago
```

Эта инструкция по очистке удаляет временные метки сборки forte\_wago в среде разработки wago. Теперь при выполнении targetinstall проект будет перестроен заново

```
> ptxdist targetinstall forte_wago
```

Если вы внесли какие-либо изменения в корневую папку 4diac FORTE, они будут отражены здесь, и вы увидите то же окно, что и раньше.

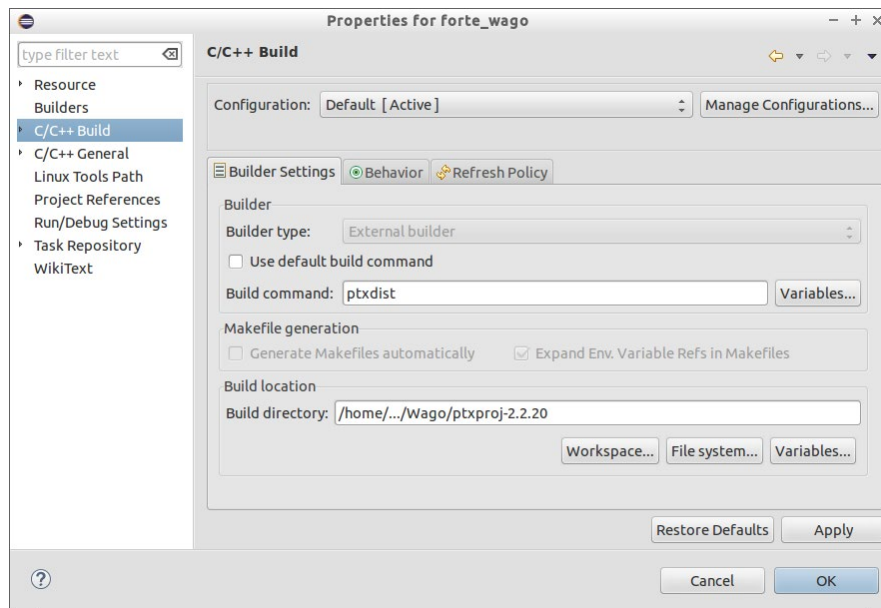
## Сборка и отладка 4diac FORTE удаленно на Wago PFC с помощью Eclipse

### Сборка 4diac FORTE с помощью Eclipse

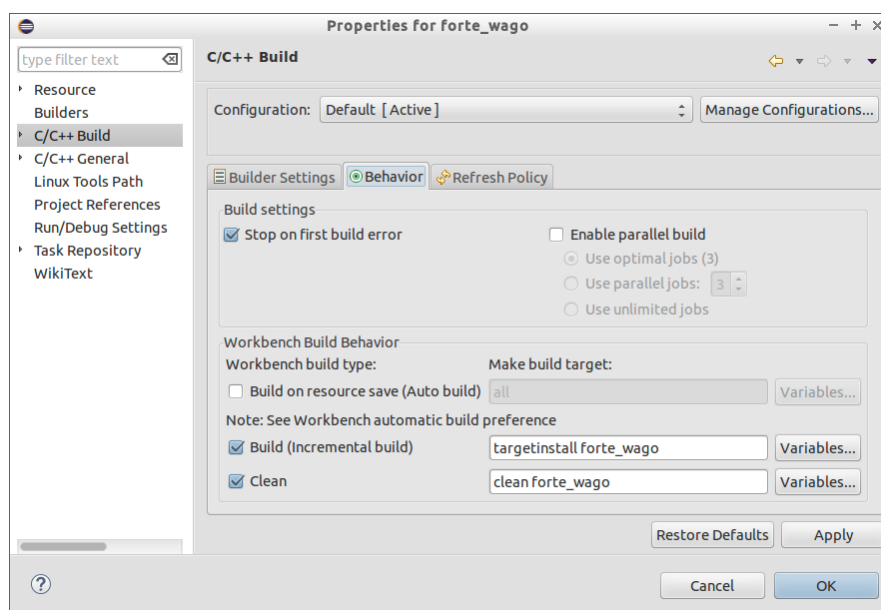
4diac FORTE можно отлаживать удаленно с вашего хост-компьютера Ubuntu с помощью Eclipse CDT. Для получения более подробной информации обратитесь к Wago по вопросу использования EclipseCDT. Используя ту же процедуру, 4diac FORTE можно отлаживать удаленно. Рекомендуемая структура папок для сборки 4diac FORTE в Eclipse:

- eclipseCDT
  - рабочее пространство
    - forte (содержит исходный код 4diac FORTE)
    - forte-wago (содержит проект для доступа к Wago PFC)

В папке forte-wago создайте новый проект на C, выбрав «Файл» → «Создать» → «Проект», введите название проекта, например, forte-wago, отключите расположение по умолчанию и укажите расположение вашего источника 4diac FORTE, например, forte. Выберите «Проект с Makefile»/«Пустой проект с Makefile» и «Другой набор инструментов» и нажмите «Готово». Задайте свойства для новых проектов. В категории «Сборка C/C++» установите команду сборки на ptxdist, а каталог сборки — на место установки ptxdist.



На вкладке «Поведение» в разделе «Сборка» выберите «Установить целевую версию forte\_wago», а в разделе «Очистка» — «Очистить целевую версию forte\_wago».



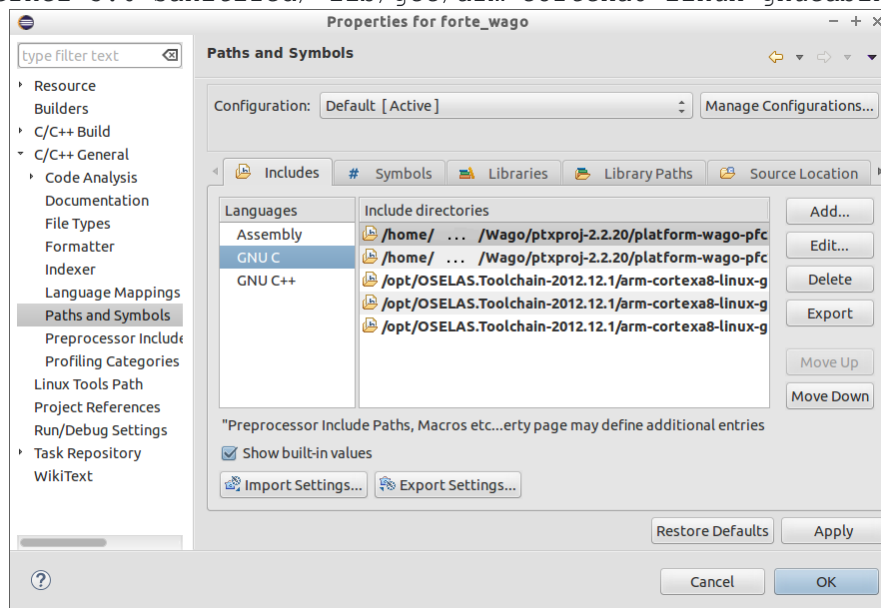
Добавьте необходимые каталоги включения в категорию C/C++ «Общие» / «Пути и символы». Необходимые каталоги включения:

```
.../ptxproj-2.2.20/platform-wago-pfc200/sysroot-target/include
```

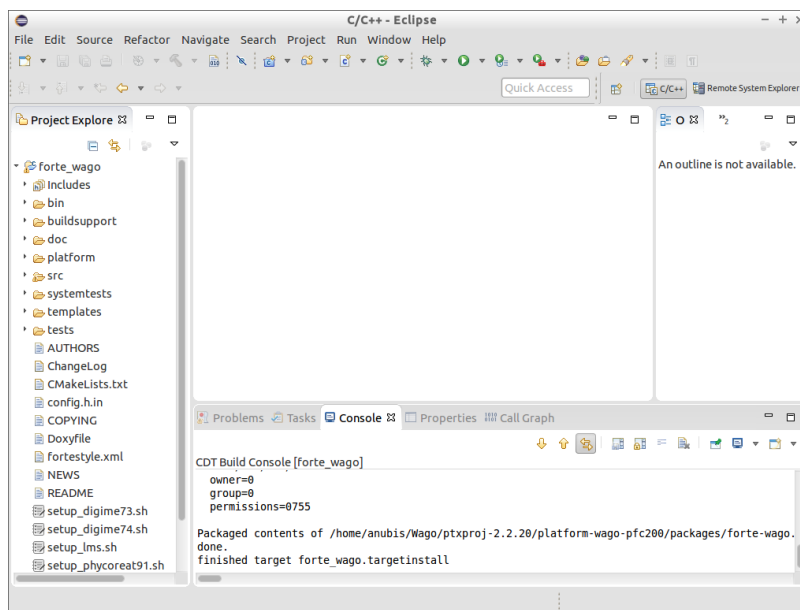
```

.../ptxproj-2.2.20/platform-wago-pfc200/sysroot-target/usr/include
.../OSELAS.Toolchain-2012.12.1/arm-cortexa8-linux-gnueabi/f/gcc-4.7.3-glibc-2.16.0-
binutils-2.22-kernel-3.6-sanitized/include
.../OSELAS.Toolchain-2012.12.1/arm-cortexa8-linux-gnueabi/f/gcc-4.7.3-glibc-2.16.0-
binutils-2.22-kernel-3.6-sanitized/sysroot-arm-cortexa8-linux-gnueabi/f/usr/include
.../OSELAS.Toolchain-2012.12.1/arm-cortexa8-linux-gnueabi/f/gcc-4.7.3-glibc-2.16.0-
binutils-2.22-kernel-3.6-sanitized/ lib/gcc/arm-cortexa8-linux-gnueabi/f/4.7.3/include

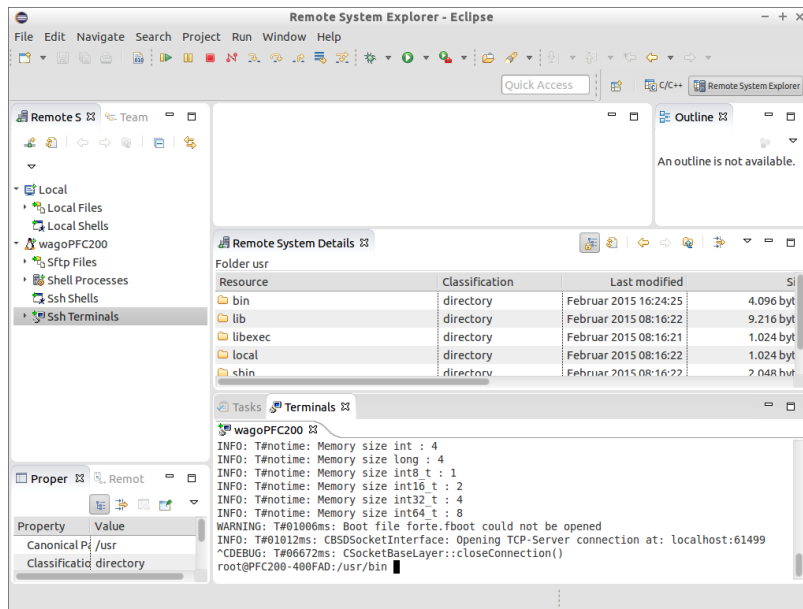
```



В среде C/C++ щёлкните правой кнопкой мыши по своему проекту и выберите «Построить проект» или «Очистить проект». После построения проекта вы должны увидеть вывод в консоли, как показано ниже.



В окне «Обозреватель удалённых систем» создайте новое подключение, нажав кнопку «Новое подключение». Выберите Linux и нажмите «Далее». Введите IP-адрес нашего Wago PFC в качестве имени хоста и введите любое имя подключения, например wagoPFC200, и нажмите «Готово». В окне «Удалённые системы» вы можете войти в Wago PFC, щёлкнув правой кнопкой мыши по терминалу SSH и введя root в качестве пользователя и wago в качестве пароля. Должно открыться окно терминала и окно «Сведения об удалённой системе».



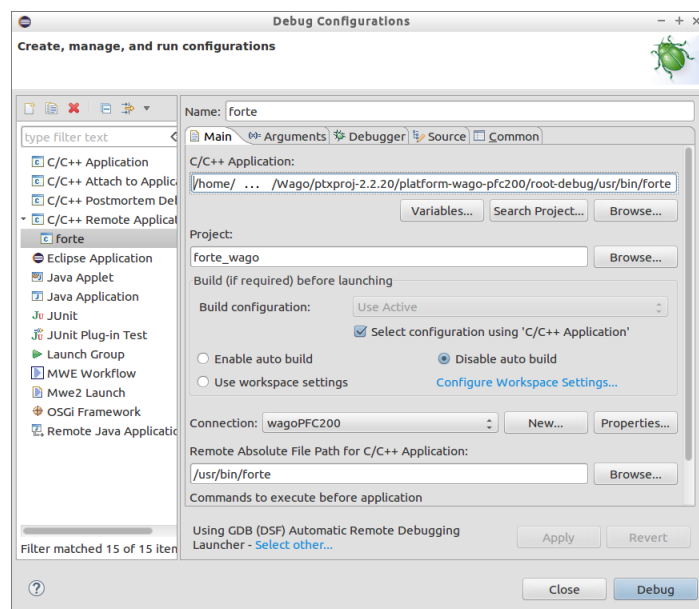
Скопируйте исполняемый файл 4diac FORTE в папку /usr/bin на Wago PFC. Исполняемый файл можно скопировать на Wago PFC, вставив его в папку /usr/bin в разделе «Сведения о удалённой системе». Если вы хотите использовать цифровые входы/выходы Wago PFC, остановите запущенный экземпляр CoDeSys (kill plclinux\_rt) и запустите 4diac FORTE с помощью:

```
> cd /usr/bin > ./forte
```

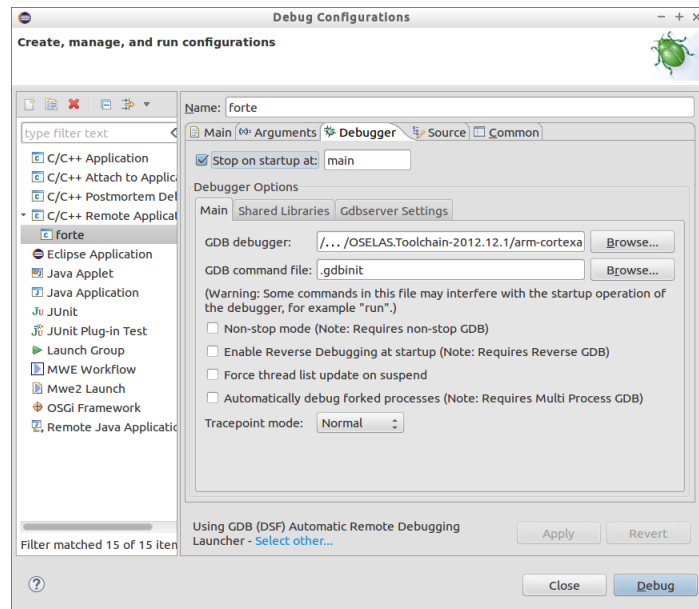
### Отладка 4diac FORTE в Eclipse

Чтобы отладить 4diac FORTE на Wago PFC, нажмите на маленький треугольник рядом с пунктом «Отладка»

и выберите «Конфигурации отладки». Затем выберите «Удаленное приложение C/C++» и задайте любое имя (например, 'forte'). На вкладке [.tab4diac]#main# укажите путь к исполняемому файлу 4diac FORTE в разделе [.specificText]#C/C-приложение, укажите название проекта forte\_wago, установите флажок «Отключить автоматическую сборку», укажите название подключения, например wagoPFC200, и укажите удалённый абсолютный путь к исполняемому файлу 4diac FORTE на Wago /usr/bin/forte.



На вкладке Отладчик установите для отладчика GDB значение /.../OSELAS.Toolchain-2012.12.1/arm-cortexa8-linux-gnueabi/gcc-4.7.3-glibc-2.16.0-binutils-2.22-kernel-3.6-sanitized/bin/arm-cortexa8-linux-gnueabi-gdb.



Отладьте 4diac FORTE с помощью Eclipse, как обычно.

## ВВЕДЕНИЕ

### Проект Eclipse 4diac

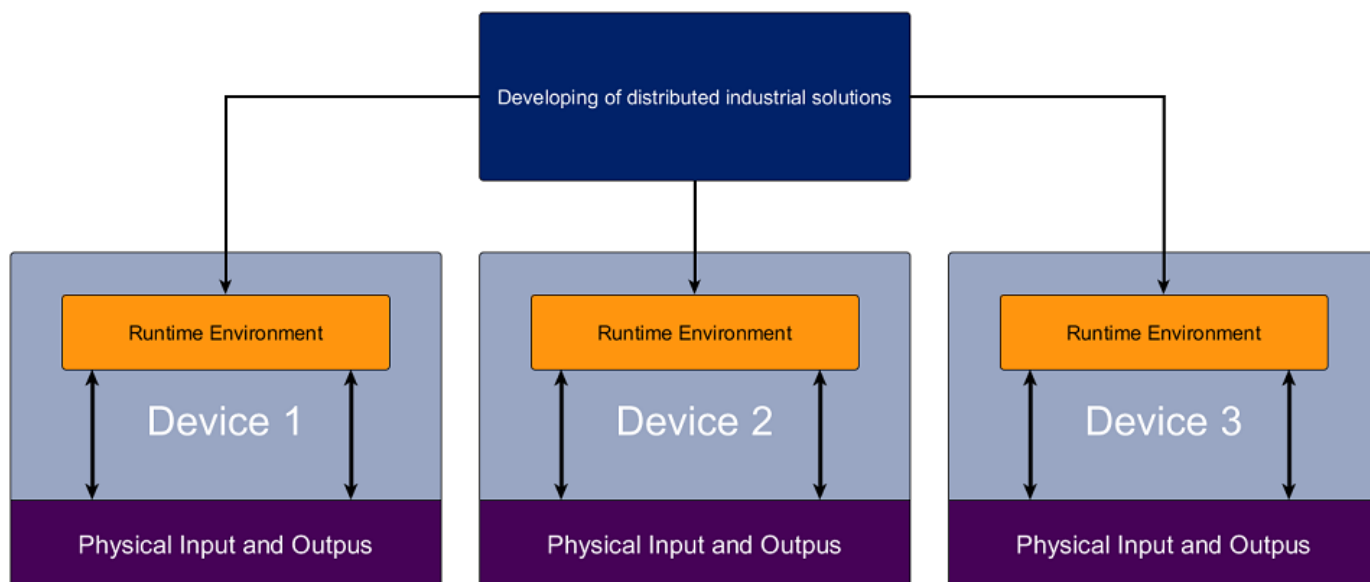
Чтобы понять, что будет дальше, вам нужно знать IEC 61499. Если вы ещё не читали следующую страницу, перейдите по ссылке: [IEC 61499 101](#)

### Обзор проекта Eclipse 4diac

В этом разделе представлена общая картина от проектирования до управления физическими устройствами.

Как показано в описании IEC 61499, приложение, разработанное с использованием этого стандарта, использует функциональные блоки (FB). Существуют способы создания и определения FB, но в какой-то момент в процессе работы, начиная с FB и заканчивая фактическим управлением чем-то физическим, необходимо реализовать логику, лежащую в основе FB и сетей FB. Именно здесь на помощь приходит среда выполнения. Это программное обеспечение загружает сеть FB, а затем выполняет события и следует правилам стандарта, как показано [здесь](#).

Итак, где же среда выполнения в стандарте? Непосредственно нигде. Для выполнения приложений распределённого управления, смоделированных с помощью IEC 61499, стандарт определяет модель устройства, содержащую ресурсы, модель выполнения FB и модель управления, позволяющую настраивать устройства с помощью инструментов. Это то, что обеспечивает среда выполнения. На следующем изображении представлена идея разделения разработки и выполнения приложений.



Прямоугольник сверху представляет собой систему в соответствии со стандартом IEC 61499. Здесь вам нужен инструмент, позволяющий создавать и подключать FB-файлы. Кроме того, он должен представлять устройства вашей системы и каким-то образом показывать часть вашего приложения, которая будет развернута на каждом устройстве. Обычно это делается на обычном компьютере.

Большие квадраты во втором ряду обозначают реальные устройства, такие как ПЛК, управляющее оборудование или Raspberry Pi. На этих устройствах должна выполняться упомянутая среда выполнения. Он получает информацию из верхнего прямоугольника для создания сети FB, выполнения FB, отправки событий между FB и так далее. Устройства обычно имеют входы и выходы, к которым обращается среда выполнения.

### Компоненты Eclipse 4diac

Eclipse 4diac предоставляет два основных компонента для разработки и выполнения распределённых систем управления, соответствующих стандарту IEC 61499:

- 4diac FORTE — это небольшая переносимая реализация среды выполнения IEC 61499 на языке C++, которая поддерживает выполнение сетей IEC 61499 FB на небольших встроенных устройствах. 4diac FORTE обычно работает поверх операционной системы (реального времени). 4diac FORTE — это многопоточная среда выполнения с низким потреблением памяти. Он был протестирован в

нескольких различных операционных системах, например, Windows, Linux (i386, amd64, ppc, xScale, arm), NetOS, eCos, rcX от Hilscher, vxWorks и freeRTOS.

- 4diac IDE: интегрированная среда разработки, написанная на Java и основанная на фреймворке Eclipse. Она предоставляет расширяемую среду разработки для моделирования приложений распределённого управления, соответствующих стандарту IEC 61499. Вы используете 4diac IDE для создания FB, приложений, настройки устройств и выполнения других задач, связанных со стандартом IEC 61499. В 4diac IDE эти результаты также можно использовать на устройствах, работающих под управлением 4diac FORTE или других совместимых сред выполнения.

Поскольку разработка систем в соответствии со стандартом IEC 61499 и среда выполнения не связаны между собой, необходимо учитывать некоторые факторы. Как описано выше, вы можете создавать FB в 4diac IDE. Однако на этом этапе среда выполнения не знает, что FB существует, и не знает, как его выполнить. Поэтому в 4diac IDE у вас есть возможность экспортировать созданный FB в код 4diac FORTE (т. е. в файлы C). Затем вам нужно добавить экспортированный код в исходный код 4diac FORTE и скомпилировать его, как описано в разделе [xref:../installation/index.adoc#FORTE\[Компиляция 4diac FORTE\]](#) документации по установке. Это возможно только для базовых и составных функциональных блоков (BFB и CFB), поскольку оба определения содержатся в стандарте. Однако для функциональных блоков сервисных интерфейсов (SIFB) определяется только интерфейс. Внутреннюю реализацию SIFB необходимо кодировать вручную на языке C, на котором написан 4diac FORTE.

### Человеко-машинный интерфейс

4diac FORTE предназначен для работы в ПЛК, поэтому графический пользовательский интерфейс (GUI) в качестве человеко-машинного интерфейса (HMI) не поддерживается. Таким образом, для создания человеко-машинного интерфейса для ваших решений по управлению, разработанных в Eclipse 4diac, вам понадобятся внешние компоненты. 4diac FORTE поддерживает набор различных протоколов связи, которые позволяют взаимодействовать с такими внешними компонентами человеко-машинного интерфейса. В настоящее время наиболее часто используемыми протоколами являются MQTT или OPC UA.

## IEC 61499 101

### Программирование ПЛК в соответствии со стандартом IEC 61131-3

ПЛК — это, по сути, небольшой компьютер, используемый в промышленности. У него есть входы и выходы для управления различными системами. Как правило, кнопки и датчики генерируют входы, а выходы управляют двигателями. Если у вас есть опыт работы в сфере информационных технологий, вы можете представить себе ПЛК как [Raspberry Pi](#), <https://www.arduino.cc/> [Arduino], [Beagle Bone Black](#) или аналогичную встроенную плату с входами и выходами. Однако ПЛК специально разработан для использования в промышленности. ПЛК программируется в соответствии с требуемым приложением. Как и компьютеры, ПЛК можно приобрести у множества разных производителей. ПЛК эффективно используются для решения узконаправленных задач, в отличие от обычного ПК, который является компьютером общего назначения, то есть его можно использовать для решения любых задач. Поэтому каждый производитель ПЛК разрабатывает не только аппаратное обеспечение, но и базовое программное обеспечение (прошивку), работающее на ПЛК. Прошивка сравнима с операционной системой обычных компьютеров (Linux, Windows, Mac OS).

Чтобы запрограммировать ПЛК, вам понадобится IDE (интегрированная среда разработки — это программное обеспечение, с помощью которого вы можете выполнять все задачи, связанные с определённой темой, в данном случае — с используемым ПЛК) от поставщика ПЛК. IDE работает на обычном компьютере (Linux, Windows, Mac OS) и позволяет программировать приложение, а затем загружать его на ПЛК. Обратите внимание на различные термины, которые мы используем для обозначения схожих понятий:

- Приложение: программное обеспечение, которое программируется пользователем и работает на ПЛК. Когда вы «программируете ПЛК», вы создаёте приложение.
- IDE: программное обеспечение, предоставляемое поставщиком ПЛК, которое позволяет кодировать приложение и загружать его в ПЛК, отлаживать его, ...
- Прошивка: базовое программное обеспечение, которое уже загружено в ПЛК поставщиком. IDE взаимодействует с прошивкой, и пользователь обычно может только обновлять её, но не изменять.

Проще говоря, чтобы ПЛК мог чем-то управлять, необходимо выполнить следующие шаги:

1. Загрузите IDE с веб-страницы поставщика и узнайте, как ее использовать.
2. Ознакомьтесь с возможностями ПЛК и с тем, как ваше приложение может получать доступ к входам и выходам.
3. Закодируйте свое приложение в IDE.
4. Подключите компьютер к ПЛК, как правило, с помощью кабеля Ethernet, и загрузите приложение из IDE на ПЛК. (Некоторые производители могут предложить несколько небольших инструментов вместо IDE)
5. Теперь ваше приложение запускается на ПЛК при каждом его включении.

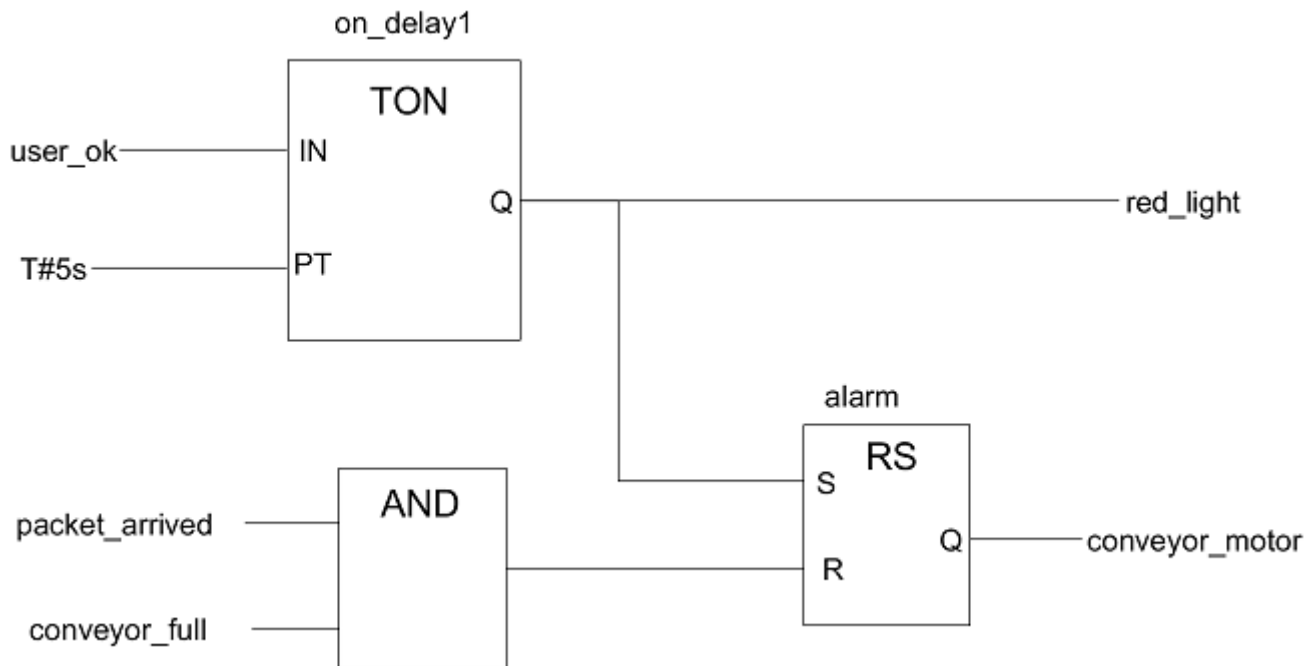
### Программирование ПЛК

Несмотря на то, что каждый производитель ПЛК предлагает собственную интегрированную среду разработки, языки программирования для ПЛК были стандартизированы в [IEC 61131-3](#). В стандарт входят пять различных языков программирования:

1. Лестничная диаграмма (LD), графическая
2. Диаграмма функциональных блоков (FBD), графическая
3. Структурированный текст (ST), текстовый
4. Список инструкций (IL), текстовый (устаревший)
5. Последовательная функциональная схема (SFC), графическая.

Эти языки существенно отличаются от "обычных" языков программирования, таких как C, C++, Java или Python. Для упрощения программирования ПЛК языки, упомянутые в стандарте, имеют высокий уровень абстракции, особенно графические. Кроме того, некоторые ПЛК могут быть запрограммированы с помощью обычных языков программирования.

Мы сосредоточимся на языке программирования FBD. На следующем рисунке показан базовый пример FBD:



На этом изображении мы видим 3 функциональных блока (**on\_delay1**, **AND**, **alarm**), 4 входа (**user\_ok**, **T#5s**, **packet\_arrived**, **conveyor\_full**), 2 выхода (**red\_light**, **conveyor\_motor**) и соединения между функциональными блоками (FB). Давайте сосредоточимся на крайнем правом FB «alarm». Его тип — «RS», а имя экземпляра — «alarm». Можно создать множество экземпляров одного типа. Все эти экземпляры ведут себя одинаково, но у каждого экземпляра есть собственное внутреннее состояние. У всех FB входы расположены слева, а выходы — справа от блока. Если у вас есть опыт работы с обычным языком программирования, вы можете интерпретировать FB как функцию, которую можно вызвать. Входы соответствуют параметрам функции, а выходы — возвращаемому значению. У функционального блока может быть несколько выходов. Для RS FB выход Q становится истинным, если вход S истинен, а вход R ложен; и Q становится ложным, когда R истинен.

После «программирования»/написания этого приложения в IDE вам нужно скомпилировать его, а затем загрузить в ПЛК. Чтобы увидеть поведение в реальной системе, физические входы и выходы ПЛК должны быть подключены к реальным датчикам и двигателю. В каждой IDE есть свой способ представления физических входов/выходов во время программирования.

Хорошо! Это всё, что вам нужно знать о ПЛК на данный момент. В этом разделе мы рассмотрели стандарт IEC 61131. В следующем разделе, где мы обсудим более новый стандарт, мы подробнее рассмотрим Eclipse 4diac.

## IEC 61499

Промышленность переходит от централизованных систем к более распределённой парадигме. Крупные системы с центральным интеллектом, контролирующим всё, превращаются в распределённые системы. В распределённой системе отдельные части обладают интеллектом и могут беспрепятственно взаимодействовать друг с другом, поэтому система работает как единое целое. Стандарт IEC 61499 определяет специализированный язык моделирования для разработки распределённых решений для промышленного управления. Стандарт IEC 61499 расширяет стандарт IEC 61131-3, улучшая инкапсуляцию программных компонентов для повышения их повторного использования, предоставляя независимый от поставщика формат и упрощая поддержку взаимодействия между контроллерами. Его функции распределения и встроенная поддержка динамической реконфигурации обеспечивают необходимую инфраструктуру для «Индустрии 4.0» и промышленных IoT-приложений.

При использовании IEC 61131-3 возникают две большие проблемы. Одна из них связана с обратными связями в приложении. В функциональной блок-схеме, подобной той, что показана выше, но с дополнительной обратной связью, поведение системы зависит от реализации базового программного обеспечения. Во-вторых, в случае объединения ПЛК разных производителей обмен данными между ними не стандартизирован и, как правило, затруднен.

Другими словами, IEC 61499 определяет язык моделирования, аналогичный старым FB, показанным в предыдущем разделе. Однако этот новый язык ориентирован на распределённые системы. Поэтому он позволяет моделировать всю систему, даже если она состоит из более мелких частей (например, отдельных ПЛК). FB используются для полной инкапсуляции функциональности, поскольку глобальные переменные не допускаются. Затем приложение создаётся путём соединения отдельных FB. Кроме того, стандарт определяет модель для представления устройств в системе и их соединений. Если приложение разделено на несколько устройств, все FB в приложении могут быть сопоставлены с соответствующими устройствами.

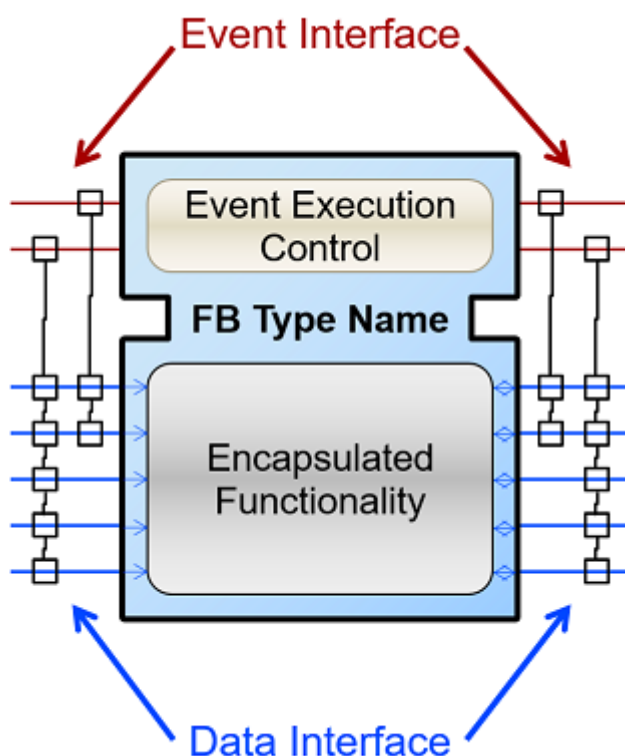
Итак, не углубляясь в концепции, давайте рассмотрим несколько изображений, чтобы понять возможности и новые функции, представленные в IEC 61499.

## Основные части IEC 61499

### Интерфейс FB

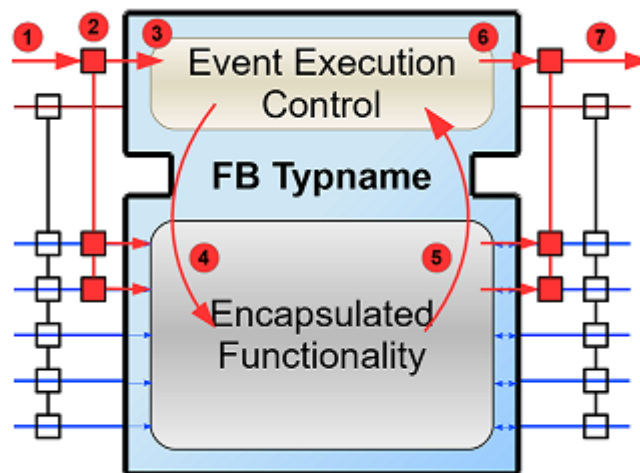
На изображении ниже показан новый вид FB в стандарте IEC 61499. Функциональный блок содержит требуемую функциональность. Входы по-прежнему находятся слева, а выходы — справа, но теперь интерфейс различает события и данные. События в верхней части FB показаны красными линиями, а данные в нижней части — синими линиями. События запускают функциональность FB, которые затем используют данные, доступные на входах данных. События и соединения данных несовместимы, то есть вы не можете их как-либо соединить. Допускается разветвление соединений данных. Разветвление — это выход, который соединён с несколькими входами следующего этапа. Это означает, что несколько входов данных следующего этапа могут быть соединены с выходом. Напротив, объединение соединений данных не допускается. Объединение — это соединение нескольких выходов предыдущего этапа с входом. Таким образом, FB должен будет выбрать входное значение из подключений данных. Для подключений событий возможны как входы, так и выходы.

На графике каждый вход события соединён с несколькими входами данных с помощью линий и маленьких квадратов. Аналогично, каждый выход события соединён с несколькими выходами данных. Это соединение определяет, какие входы/выходы данных обновляются при возникновении события на входе/выходе. Поведение функционального блока зависит от его управления выполнением событий (ECC). ECC — это конечный автомат, который получает входные события. В зависимости от текущего состояния ECC выполняет определённую часть инкапсулированной функциональности.



## Внутренняя последовательность действий FB

На следующем рисунке показано, как запускается FB для выполнения своей функциональности. При поступлении события на FB выполняется последовательность шагов.



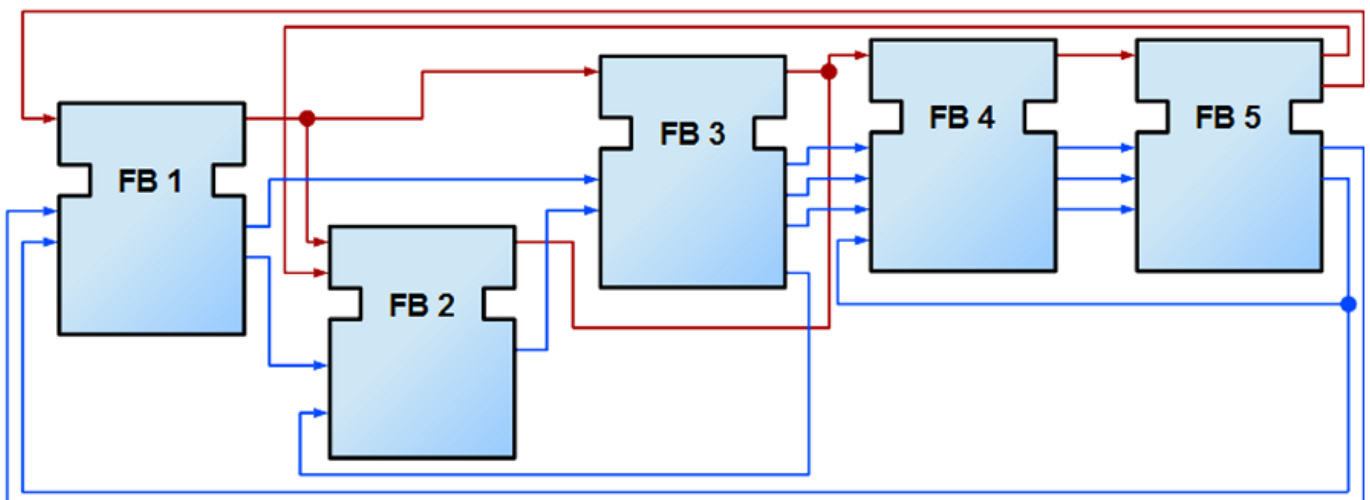
### Последовательность активации функционального блока

1. Входные события поступают на FB.
2. Вводимые данные, относящиеся к входящему событию, обновляются.
3. Событие передается в ECC.
4. В зависимости от типа FB и управления выполнением запускаются внутренние функциональные возможности для выполнения.
5. Внутренняя функция завершает выполнение и предоставляет новые выходные данные.
6. Выходные данные, связанные с выходным событием, обновляются.
7. Отправляется выходное событие.

Шаги с 4 по 7 могут повторяться несколько раз. Хотя выходное событие может быть запущено, это не обязательно.

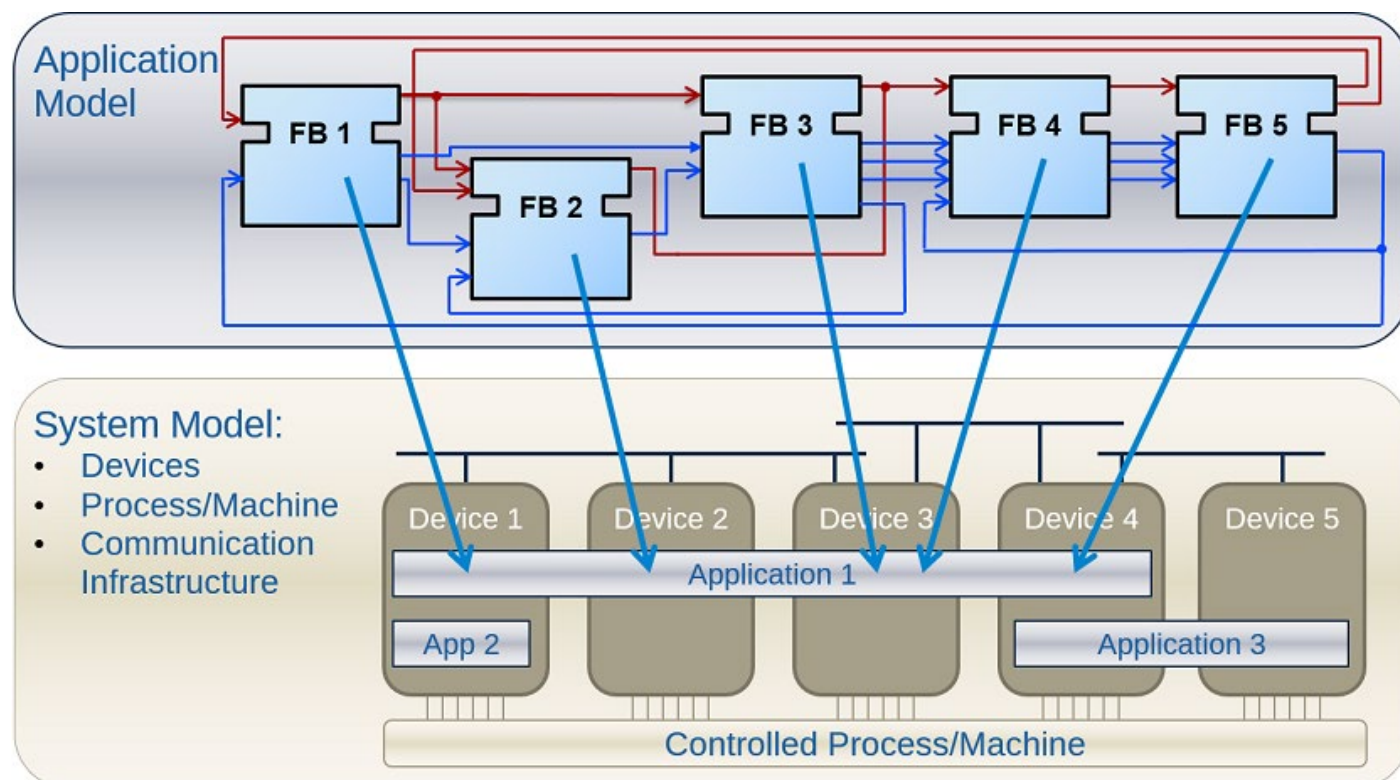
## Распределенное приложение

Теперь мы определили FB и их поведение. На следующем изображении показан пример приложения, созданного в соответствии со стандартом IEC 61499. Вы можете видеть, что данные и события не связаны друг с другом.



Поскольку стандарт позволяет моделировать распределённые системы, приложение не обязательно должно работать только на одном устройстве. Вместо этого его можно разделить и развернуть на нескольких устройствах (ПЛК). Вы даже можете создать несколько приложений, которые затем будут распределены по нескольким устройствам. Представление «Системная модель» помогает пользователю спроектировать такое

распределение. Это показано на следующем рисунке. Устройство может состоять из нескольких ресурсов, которые можно представить как потоки внутри устройства. Если быть точным, то FB на самом деле загружаются в ресурс, а не в само устройство.

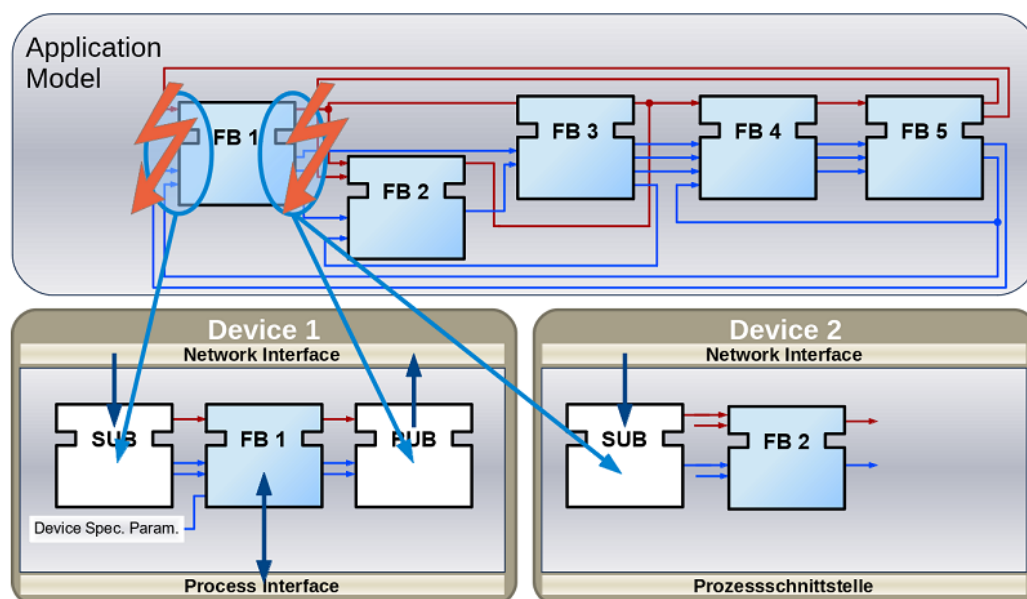


На изображении показано, что не все FB приложения работают на одном устройстве. Кроме того, на одном устройстве может одновременно работать множество приложений или их частей.

Важно! FB нельзя разделить на несколько устройств.

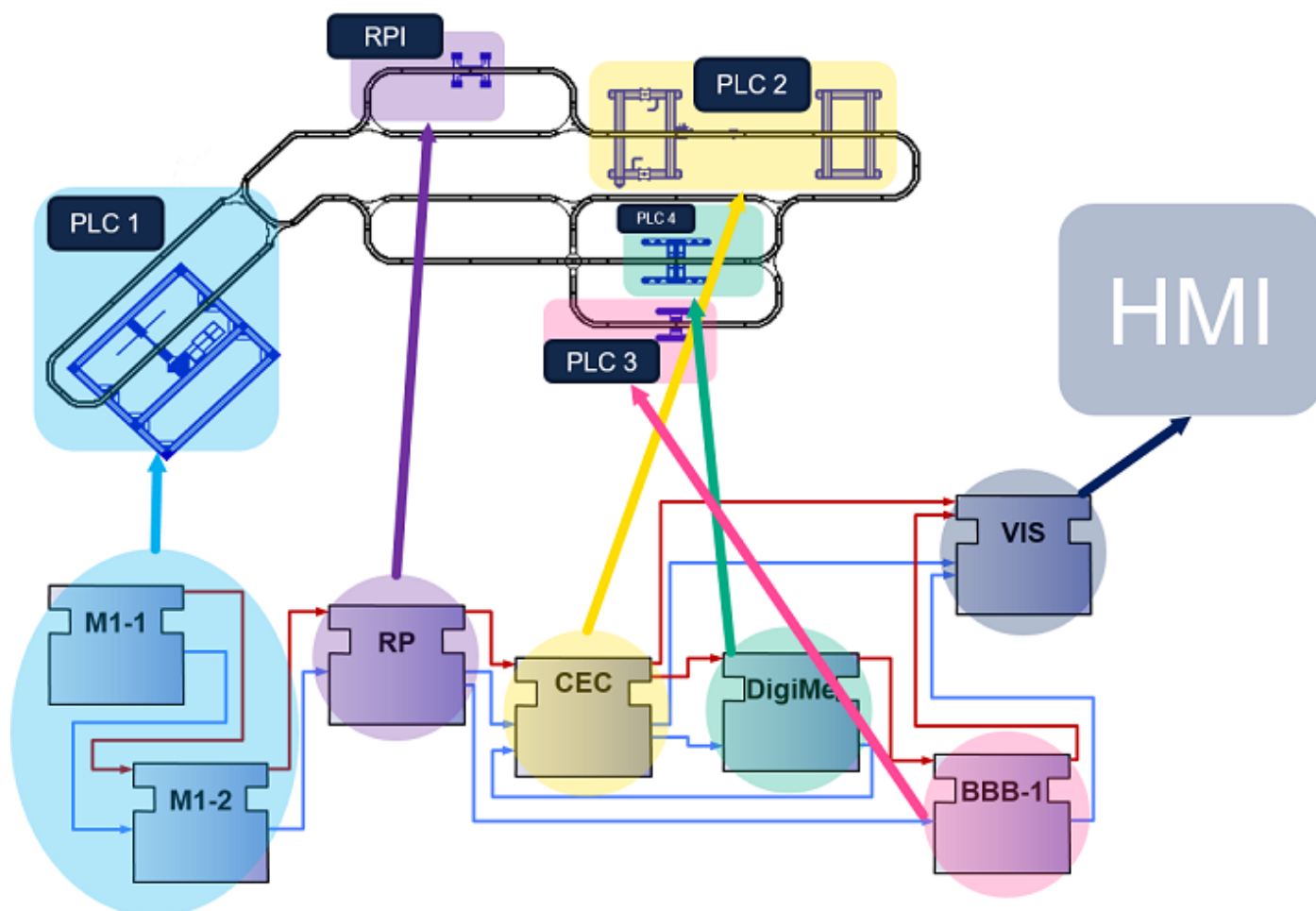
### Нарушенные соединения

Хотя хорошо, что приложение можно разделить на несколько устройств, данные и события необходимо отправлять на следующее устройство, чтобы поддерживать их поток. Да! Именно поэтому вы фиксируете эти соединения с помощью специальных FB для публикации информации и подписки на неё, как показано на изображении ниже. Поскольку эти новые FB не являются частью полного приложения, они видны только в представлении ресурсов.



Теперь изображение ниже имеет смысл. Различные ПЛК соединены между собой, и функциональные блоки распределены между ними. HMI — это человеко-машинный интерфейс. Конечно, нам нужно изучить, как на

самом деле реализованы разорванные соединения или как осуществляется доступ к реальным аппаратным входам/выходам. Мы рассмотрим это позже, когда будем рассматривать различные типы функциональных блоков.



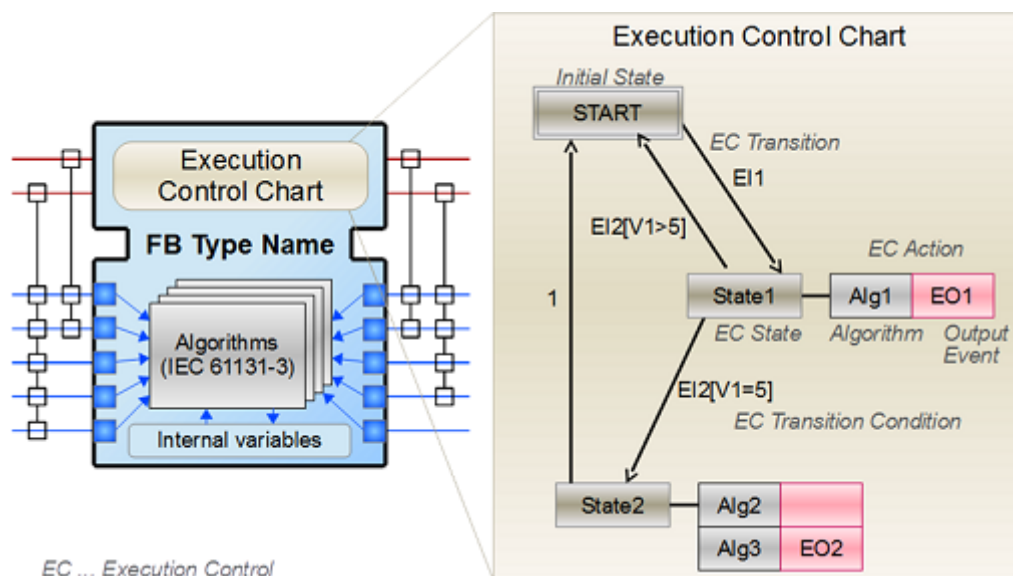
### Типы функциональных блоков

IEC 61499 определяет три типа функциональных блоков, которые вы можете использовать при разработке своего приложения:

Базовый функциональный блок (БФБ):

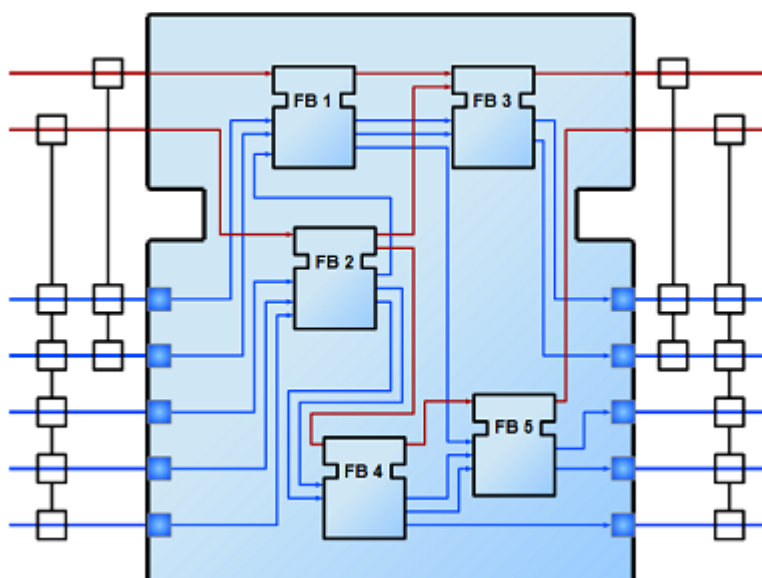
1. В БФБ можно определить конечный автомат с помощью диаграммы управления выполнением (ДУВ). ДУВ определяет, какой алгоритм будет выполняться в зависимости от его состояния и входных событий. На графике ниже показан БФБ с примером ДУВ. Серые блоки (Alg1-Alg3) в ДУВ — это алгоритмы, заключенные в функциональные блоки. Эти алгоритмы записываются пользователем, например, с помощью структурированного текста, который является одним из языков программирования, определенных в стандарте IEC 61131. Розовые прямоугольники (EO1, EO2) — это выходные события, которые запускаются при переходе в состояние. Переходы между состояниями (коричневые прямоугольники) показаны стрелками. Они связаны с определённым событием, если только переход не назван «1», в этом случае событие не требуется и управление переходит непосредственно в следующее состояние. Например, если FB находится в состоянии START и поступает событие EI1, FB переходит в состояние 1, выполняет алгоритм 1 и выводит событие EO1

Некоторые переходы включают выражение в квадратных скобках. Эти выражения представляют собой условия, которые должны быть выполнены при поступлении события. Только при выполнении условий управление переходит к следующему состоянию. Событие используется только один раз. Если переход из состояния 1 обратно в состояние START снова происходит в E1, бесконечный цикл не создаётся. Только при поступлении нового события E1 состояние возвращается в START. Такие циклы возникают при переходе с «1», поскольку используются только события.



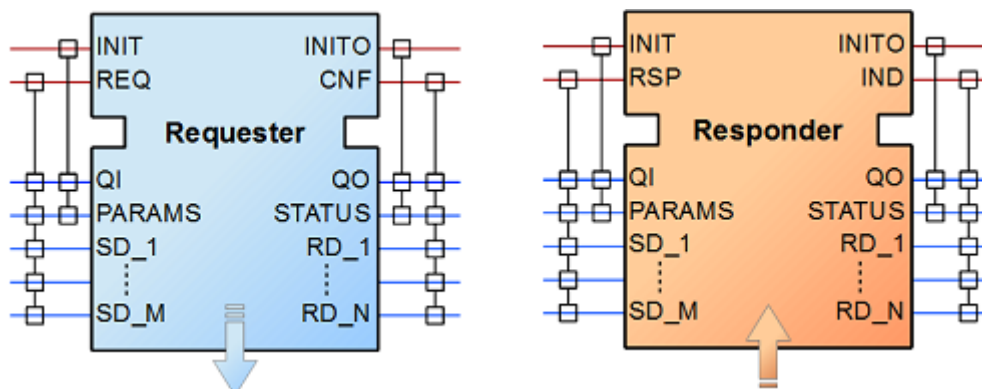
## 2. Составной функциональный блок (CFB):

CFB просто имеет внутреннюю сеть из других функциональных блоков.



## 3. Функциональный блок интерфейса сервиса (SIFB):

SFB — это функциональные блоки, необходимые для доступа к определённым частям оборудования. Как видно из приведённого выше примера, одно и то же приложение может быть развёрнуто на нескольких устройствах. Затем приложению потребуется доступ к входам или выходам, а также к определённому оборудованию для связи. Для этого нужны SFB. Они используются для всего, что требует доступа к платформе, чего не могут сделать BFB или CFB. Эти типы FB активируются не только входящим событием, но и оборудованием. Таким образом, FB-ответчик может запускать событие при каждом поступлении пакета.



## Профили соответствия требованиям

Поскольку стандарт является абстрактным, многие «вещи», необходимые для правильной работы системы, такие как связь, в него не включены. Но в стандарте указано, как должны быть описаны эти «вещи». Такие описания называются профилями соответствия требованиям. Примером профиля соответствия требованиям является профиль [IEC 61499](#) для [демонстрационных проектов](#), предоставленный компанией [Holobloc Inc.](#) Профиль соответствия требованиям можно рассматривать как способ восполнения пробелов, вызванных абстрактностью стандарта.

# Конфигурирование IO

## Lego Mindstorms EV3

В этом разделе представлен список параметров, которые можно использовать в 4diac FORTE для доступа к вводу-выводу LMSEV3. Если вы внимательно прочитаете этот раздел, но не будете использовать эти параметры, то это может показаться скучным и даже бесполезным, потому что детали будут забыты, если их не использовать. Мы рекомендуем быстро просмотреть этот раздел, чтобы знать, к чему можно получить доступ с помощью 4diac FORTE. В примере представлено управление двигателем с помощью ПИД-регулятора.

### Стандартный доступ

#### Прозрачный доступ

Стандартный доступ — это доступ через чётко определённые блоки ввода-вывода, которые обеспечивают прозрачность доступа к вводу-выводу на разных платформах. Каждый блок поддерживает различные параметры на входе PARAM, которые определяют ввод-вывод системы, к которой будет осуществляться доступ. Блоки, используемые для управления вводом-выводом, обозначаются двумя заглавными буквами.

1. Первая буква указывает направление:
  - Q: вывод
  - I: ввод
2. Вторая буква указывает размер:
  - X: логическое значение
  - W: слово (16 бит)
  - D: двойное (32 бита)

Таким образом, например, логическое значение на входе считывается с помощью функционального блока IX. Выходное слово, например значение ШИМ-сигнала двигателя, управляется с помощью блока QW. Не все размеры реализованы на всех платформах.

### Параметры для Lego Mindstorms EV3

Параметры Lego Mindstorms EV3 были разработаны для общего использования, и их легко расширить. Формат параметров соответствует двум важным правилам:

1. Разрешены только заглавные буквы A, B, C, D, если они используются для определения выходных портов EV3 (outA, outB, outC, outD).
2. Параметры приведены в трех частях, разделенных точками:
  - port.typeOfIO.specific
  - 1. порт

Первая часть параметра указывает порт ввода или вывода. Для EV3 возможными значениями являются [in1 | in2 | in3 | in4 | outA | outB | outC | outD | emb]. Первые восемь значений напрямую связаны с реальными портами EV3. Порт «emb» относится ко всему, чем можно управлять в EV3, но не в портах ввода-вывода. До сих пор это были светодиоды и кнопки, но в будущем экран и динамик также будут относиться к порту «emb».

#### 2. typeOfIO

Вторая часть определяет, что именно подключено к указанному порту. Возможные значения: [датчик | датчикw | кнопка | светодиод | двигатель]. Разница между «датчиком» и «датчикомw» заключается в том, что «датчик» используется для логических датчиков, а «датчикомw» — для датчиков, значения которых являются словами (16 бит). «Светодиод» и «Кнопка» используются для управления встроенными светодиодами и кнопками EV3. «Двигатель» используется для подключения двигателей к любому из выходов.

Следует понимать, что не все типы можно использовать с любым портом. Например, светодиод является только выходом, поэтому при использовании с входом, например IX, он не инициализируется. Кроме того, при использовании для управления в качестве слова QW он также не инициализируется.

#### 3. конкретный

Как следует из названия, этот третий параметр не имеет общей формы и может состоять из нескольких других параметров, разделённых точками (как в случае со светодиодом). В зависимости от второго параметра допустимые значения для этого параметра различаются, и они будут рассмотрены в следующем разделе.

### Конкретные параметры

Для каждого возможного типа ввода-вывода представлен список возможных конкретных параметров с соответствующими ограничениями и примерами.

1. led (только вывод):

- правый.красный
- правый.зеленый
- левый.красный
- левый.зеленый

Каждый конкретный параметр используется для управления одним из четырех доступных светодиодов. Оранжевого светодиода не существует, но он представляет собой сумму красного и зелёного, поэтому, чтобы включить, например, оранжевый светодиод слева, нужно включить и зелёный, и красный светодиоды слева.

Ограничение: 'led' должен использоваться с QX и с портом 'emb'.

Пример:

- emb.led.right.red
- 2. кнопка (только ввод):
- вверх
- вниз
- влево
- вправо
- ввод
- возврат

Считывает данные с кнопок EV3.

Ограничение: 'button' необходимо использовать с IX и с портом 'emb'.

Пример:

emb.button.ввод

3. sensorw:
- [0-N] (только ввод)
  - режим (ввод или вывод)

Датчик w используется с датчиками, которые считывают числовое значение вместо логического и обычно являются более сложными, чем обычные сенсорные датчики (логические). Каждый датчик определяет, как он получает данные, сколько значений и в каком формате. Характеристики поддерживаемых датчиков ev3dev перечислены [здесь](#).

Значения каждого из датчиков хранятся в отдельном файле в EV3. Конкретный [0-N] определяет, в каком файле они хранятся. Например, датчик гироскопа может показывать угол и скорость вращения. Значения хранятся в файлах value0 и value1, поэтому для доступа только к скорости вращения необходимо использовать конкретное значение "1". Действительное значение необходимо проверить по ссылке выше.

Параметр "mode" используется для изменения режима работы датчика в соответствии с его спецификацией. В качестве входных данных mode будет отображать режим, в котором работает датчик, а в качестве выходных данных режим будет изменен. Режимы отображаются зеркально с использованием индекса 0 в соответствии со спецификацией датчика.

Ограничение: датчик не может использоваться с логическими блоками (QX или IX). Он должен использоваться с портом in[1-4], но не ограничивается блоками ввода или вывода при использовании конкретного «режима».

Ограничение: конкретный [0-N] может использоваться только в качестве входного сигнала, и число не должно превышать допустимое для датчика в режиме, который работает в момент инициализации.

Примеры:

Давайте рассмотрим пример использования датчика GYRO, указанного здесь. По умолчанию GYRO работает в режиме GYRO-ANG, показывая только одно значение, тогда единственный возможный параметр — это:

- in1.sensorw.0 (IW)

Если бы использовался 1 вместо 0, инициализация не удалась бы.

Если необходимо получить режим работы датчика, следует использовать параметр «mode» с входным сигналом IW.

- in1.режим работы датчика (IW)

Поскольку датчик гироскопа по умолчанию находится в GYRO-ANG, а GYRO-ANG является первым в списке по ссылке выше, FB считывает значение 0 (0-индекс из списка по ссылке).

Если затем изменить режим датчика, используя QW с указанием "mode"

- in1.sensorw.mode (QW)

При записи значения 3 (режим с индексом 0) режим датчика изменится на GYRO-G&A, который показывает 2 значения: угол и скорость вращения. После изменения режима два блока IW с параметрами

- in1.sensorw.0 (IW)
- in1.sensorw.1 (IW)

будут получать оба значения.

ВНИМАНИЕ: для получения скорости вращения блок с параметром «in1.sensorw.1» необходимо инициализировать ПОСЛЕ изменения режима, так как до изменения режима файл скорости вращения не существует в системе и не может быть прочитан.

- 4. датчик (только ввод):
  - [0-N]

Используется для управления сенсорным датчиком (двоичный). Спецификация датчика является подмножеством спецификации sensorw. Он не поддерживает режим, и, несмотря на то, что значение всегда сохраняется в «0», необходимо использовать «0» по двум основным причинам: 1. Чтобы иметь стандартизированный способ отображения параметров и 2. Чтобы избежать проблем в будущем, когда новый датчик будет храниться в «1» вместо «0».

+ Ограничения: «датчик» должен использоваться с QX и с портом in[1-4].

+ Пример:

in2.датчик.0

- 5. двигатель
  - включить (только логический выход)
  - сброс (только логический выход)
  - остановка (только логический выход)
  - ШИМ (только логический вход или выход)
  - положение (только логический вход или выход)
  - скорость (только логический вход)
  - вращение (только логический вход)

Включение управляет состоянием двигателя в соответствии с логическим входом QX. Значение «ИСТИНА» включает двигатель, а значение «ЛОЖЬ» выключает его. Его необходимо использовать с QX. Функция «сброс» обнуляет значения положения, ШИМ и скорости и выключает двигатель при значении «ИСТИНА» на выходе OUT. Значение «ЛОЖЬ» на выходе OUT не имеет эффекта. Его необходимо использовать с QX.

Специфику «остановки» не следует путать с остановкой двигателя, поскольку она определяет поведение двигателя при нулевом значении ШИМ в соответствии со значением, записанным на выходе OUT. Его необходимо использовать с QW

- 0: [простой ход]: отключает питание двигателя. Двигатель будет свободно двигаться до остановки.
- 1: [тормоз]: отключает питание двигателя и создает пассивную электрическую нагрузку. Обычно это делается путем короткого замыкания клемм двигателя. Эта нагрузка поглощает энергию вращения двигателя и заставляет его останавливаться быстрее, чем при простом ходе.
- 3: [удержание]: заставляет двигатель активно удерживать текущее положение. Если внешняя сила попытается повернуть двигатель, он «оттолкнётся», чтобы сохранить своё положение.

Функция «pwm» считывает текущий уровень pwm при использовании с IW двигателя или записывает желаемый уровень pwm для двигателя при использовании с QW.

Функция «position» считывает текущее положение двигателя. Количество шагов, на которые делится вращение, можно узнать с помощью функции «rot». Её необходимо использовать с ID. Положение можно записать, что позволяет установить желаемое значение.

«Скорость» считывает текущую скорость в положениях в секунду. Она должна использоваться с IW.

«Поворот» считывает из спецификации двигателя, сколько шагов приходится на один оборот двигателя. Эту информацию можно получить из спецификации двигателя, и нет необходимости добавлять блок в приложение только для считывания этой информации. Она должна использоваться с IW.

Ограничения: несмотря на то, что у каждого конкретного блока есть свои ограничения в зависимости от типа блока, тип ввода-вывода «двигатель» должен использоваться с одним из выходов EV3 out[A-B]

Примеры:

- outB.motor.enable (QX)
- outB.motor.pwm (QW записывает желаемый pwm, IW считывает фактический pwm)
- outB.motor.position (ID)

## Таблица ввода-вывода

В этом разделе нет новой информации, но в нём кратко изложено, какие параметры и FB следует использовать с каждым устройством ввода-вывода.

1. Сенсорный датчик, подключённый к входному порту. Его следует использовать с IX
  - in[X].sensor.[Y]

где X — номер входа в EV3 (1-4), а Y — номер значения, с которого следует считывать данные в соответствии с датчиком. Для уверенности ознакомьтесь со спецификацией датчика. Если информация недоступна, начните с 0.
2. Одна из 6 кнопок EV3. Она должна использоваться с IX
  - emb.button.[вверх | вниз | влево | вправо | ввод | удаление]

Для каждого IX можно выбрать только один из входов.
3. Светодиод EV3. Он должен использоваться с QX.
  - emb.led.[вправо | влево].[красный | зеленый]

Оранжевого светодиода не существует, но он представляет собой сумму зелёного и красного.

4. Двигатель, подключённый к выходному порту. Используемый FB зависит от параметра двигателя, с которым необходимо работать.

- Включение/выключение двигателя. Необходимо использовать с QX.
  - `out[X].motor.enable`

Где X — заглавная буква выходного порта EV3 (A-D, заглавные буквы). Значение «ИСТИНА» на входе OUT включает двигатель, а значение «ЛОЖЬ» выключает его.

- Сбросьте переменные двигателя и остановите его. Это необходимо использовать с QX.
  - `out[X].motor.reset`

Где X — заглавная буква вывода EV3 (A-D, в верхнем регистре). Значение «ИСТИНА» на входе OUT сбрасывает все параметры двигателя до значений по умолчанию. Это также приведет к остановке двигателя. ЛОЖНОЕ значение не оказывает никакого влияния.

- Считывание текущего рабочего цикла (ШИМ) с двигателя. Должно использоваться с IW.
  - `out[X].motor.pwm`

Где X — буква вывода EV3 (A-D, в верхнем регистре). Считываемое значение находится в диапазоне от -100 до 100. Может показаться странным считывание входного сигнала с выходного, но, несмотря на то, что сигнал поступает с выходного устройства, он является входным для системы.

- Считайте текущую скорость двигателя. Это необходимо использовать с IW.
  - `out[X].motor.speed`

Где X — выходная буква EV3 (A-D, заглавные буквы). Считывается скорость в шагах в секунду. Количество шагов в одном обороте двигателя можно считать с помощью параметра "rot" (см. далее). Если у двигателя 360 шагов на один оборот, а скорость составляет 720, то скорость вращения составляет 2 оборота в секунду.

- Считайте количество шагов на один оборот. Это значение должно использоваться с IW.
  - `out[X].motor.rot`

Где X — выходная буква EV3 (A-D, заглавные буквы). Это значение фиксировано в двигателе и указано в характеристиках двигателя. Обычно его не следует использовать в приложении.

- Считайте текущее положение двигателя. Его необходимо использовать с ID.
  - `out[X].motor.position`

Где X — выходная буква EV3 (A-D, в верхнем регистре). Если двигатель находился в положении 0, а параметр «rot» возвращает 360, то он делает 3 оборота, и положение будет 1080.

- Запишите положение двигателя. Его необходимо использовать с QD.
  - `out[X].двигатель.положение`

Где X - выходная буква EV3 (A-D, в верхнем регистре). Устанавливает текущее положение на определенное значение.

- Задаёт желаемый рабочий цикл (pwm) для двигателя. Он должен использоваться с QW.
  - `out[X].motor.pwm`

Где X - это выходная буква EV3 (A-D, в верхнем регистре). Диапазон значений должен быть от -100 до 100.

- Укажите желаемое поведение при значении pwm, равном 0. Это значение должно использоваться с QW.
  - `out[X].motor.stop`

Где X — буква вывода EV3 (A-D, заглавные буквы). Возможные значения: 0, 1 и 2:

- 0: [выключение]: отключает питание двигателя. Двигатель будет свободно двигаться по инерции до полной остановки.
  - 1: [тормоз]: отключает питание двигателя и создаёт пассивную электрическую нагрузку. Обычно это делается путём короткого замыкания клемм двигателя. Эта нагрузка поглощает энергию вращения двигателя и заставляет его останавливаться быстрее, чем при движении по инерции.
  - 3: [удержание]: заставляет двигатель активно удерживать текущее положение. Если внешняя сила попытается повернуть двигатель, он будет сопротивляться, чтобы сохранить своё положение.
5. датчик со значениями 16 бит (не сенсорный датчик)
- Считайте значение датчика. Он должен использоваться с IW.
    - `in[X].sensorw.[Y]`

где X — номер входа в EV3 (1-4), а Y — номер значения, с которого нужно считать в соответствии с датчиком. Для уверенности ознакомьтесь со спецификацией датчика. Если информация недоступна, начните с 0.
  - Считайте режим работы датчика. Он должен использоваться с IW.
    - `in[X].sensorw.mode`

где X — номер входа в EV3 (1-4). Считываемое значение — это номер индекса в соответствии со списком режимов работы датчика.
  - Запишите режим работы датчика. Он должен использоваться с QW.
    - `in[X].sensorw.mode`

где X - номер входного сигнала в EV3 (1-4). Записываемое значение должно быть индексным номером в соответствии со списком режимов работы датчика.

### Пользовательский доступ

Пользовательский доступ позволяет пользователю получить доступ к части системы, которая не реализована в стандартном доступе. Необходимо соблюдать осторожность, поскольку может быть получен доступ к критически важной части системы. Для каждого блока приведены спецификации новых входов и выходов.

1. `fileWriter`: блок для записи в любой файл в системе. Специальные входные данные:
  - `FILE_NAME`: абсолютный путь к файлу.

Пример: `/sys/class/tacho-motor/motor1/command`

  - `S1`: строка для записи в файл

Пример: `run-direct`
2. `fileReader`: блок для чтения из любого файла в системе. Специальные входные и выходные данные:
  - `ИМЯ_ФАЙЛА [ввод]`: абсолютный путь к файлу.

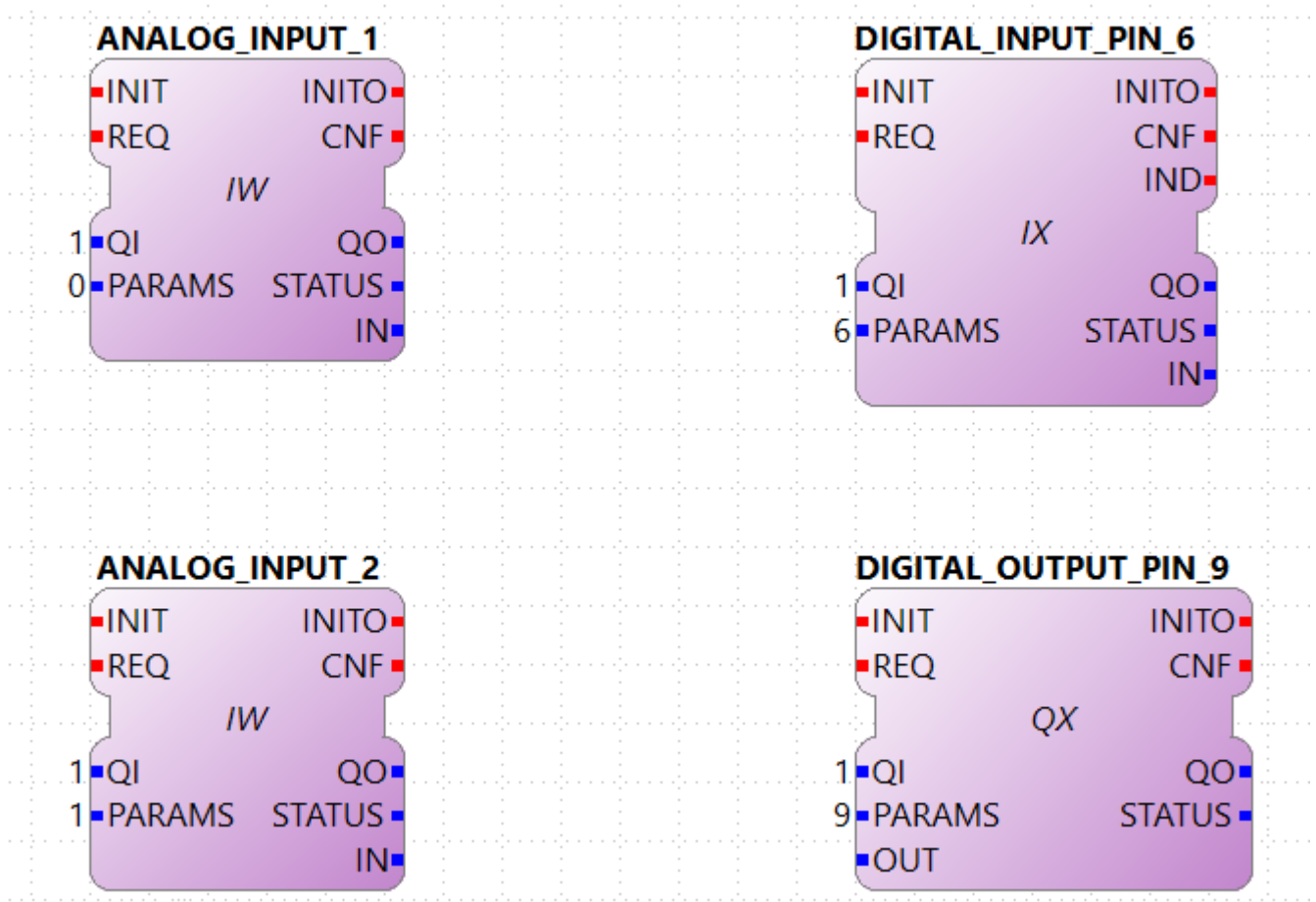
Пример: `/sys/class/tacho-motor/motor1/duty_cycle`

  - `S1 [вывод]`: строка, считанная из файла

Пример: 58

## Odroid

[Odroid](#) — это встраиваемая плата, которая, можно сказать, является более мощным аналогом Raspberry Pi и имеет аналоговые входы. Для использования ввода-вывода этой платы. Она работает так же, как Raspberry Pi, поэтому вы можете использовать QX и IX с номером, соответствующим номеру контакта, а для доступа к аналоговым входам следует использовать IW. 0 и 1 должны быть установлены в PARAMS IW для доступа к первому и второму аналоговым входам соответственно.



## PiFace Digital 2

[PiFace 2](#) — это плата для Raspberry Pi с цифровыми входами и выходами. Вы можете управлять входами и выходами от 0 до 7, просто используя нужный номер, как на Raspberry Pi.

Важно! PiFace не тестировалась, поэтому, если вы обнаружите проблему или захотите добавить поддержку других функций платы, пожалуйста, отправьте [запрос](#).

## Revolution Pi

Revolution Pi от KUNBUS GmbH реализован на основе концепции ввода-вывода IO\_CONFIG. Revolution Pi — это модульный ПЛК, в котором всегда есть как минимум один основной модуль и, возможно, один или несколько дополнительных модулей ввода-вывода.

### Настройте оборудование

Объедините несколько модулей, подключив их через PiBridge. Также подключите каждый из ваших модулей к +24 В и 0 В через основной блок питания X4 и дополнительный (если он есть у вашего модуля) блок питания X2. Все подключения к блоку питания должны быть подключены, чтобы устройства работали правильно и все светодиоды были зелёными.

При первом использовании основного модуля Revolution Pi его необходимо настроить с помощью [Picturey](#).

Чтобы подключить любой аналоговый вход/выход, ознакомьтесь с соответствующим [руководством](#).

Настройки Picturey зависят от характеристик вашего аналогового датчика. Если вы получаете слишком низкие значения, возможно, вам придется увеличить частоту АЦП в настройках AIO в Picturey.

## Скомпилируйте 4diac для Revolution Pi

Для компиляции 4diac FORTE для Revolution Pi лучше всего использовать [среду сборки 4diac FORTE](#). Следуйте инструкциям по установке и настройке среды сборки 4diac FORTE. Ваш файл конфигурации для Revolution Pi для этой среды сборки может выглядеть следующим образом:

```
ARCH=arm-linux-musleabihf

DEPS=tinyxml

CMAKE_BUILD_TYPE=Debug
FORTE_LOGLEVEL=LOGDEBUG

COM=ETH,FBDK,LOCAL

//RT_Events,,RECONFIGURATION
MODULE=CONVERT,IEC61131,UTILS,UTILS_ADDON,CUSTOM_FBS

// address sanitizer doesn't work with static linking
TOOLCHAIN_EXTRA_C_FLAGS=-fno-sanitize=address -fno-lto
TOOLCHAIN_EXTRA_LINKER_FLAGS=-fno-sanitize=address -fno-lto

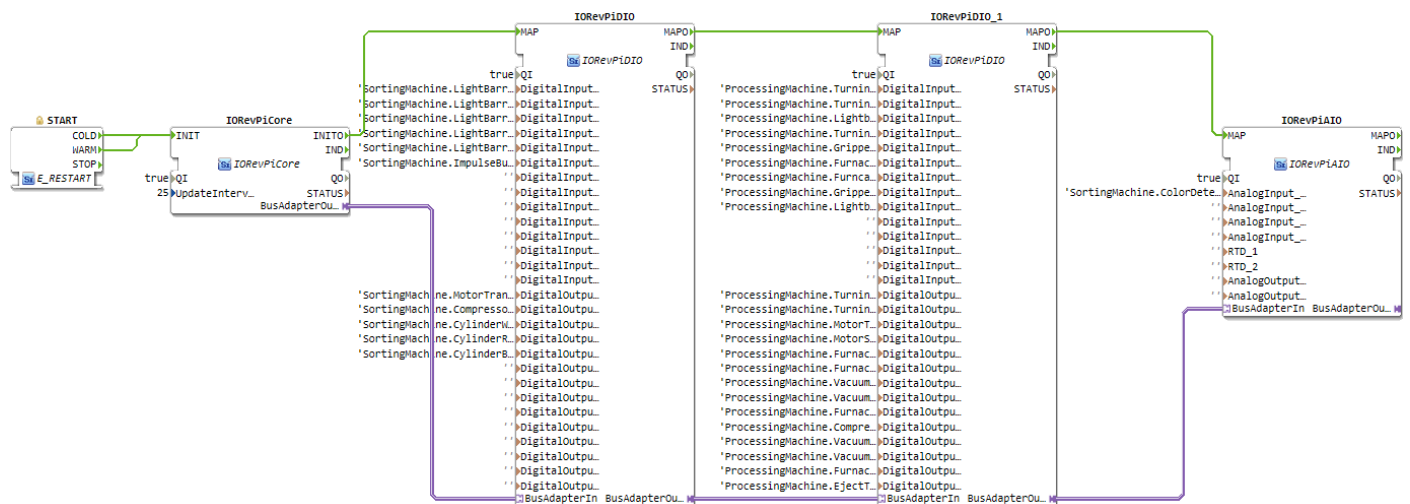
FORTE_IO=ON
IO=REVOLUTIONPI

// no local additions
FORTE_EXTERNAL_MODULES_DIRECTORY=/nonexistent
```

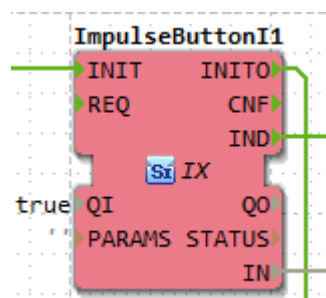
После успешной компиляции исполняемый файл можно найти в папке `build/yourConfigurationFileName/output/bin` в среде сборки 4diac FORTE. Скопируйте этот исполняемый файл в модуль ядра Revolution Pi, например, с помощью [WinSCP](#), возможно, в папку `/home/pi/forte`. Чтобы запустить скопированный исполняемый файл, вам нужно установить права на его выполнение. Войдите в модуль ядра Revolution Pi, например, с помощью [Putty](#), перейдите в папку, куда вы поместили исполняемый файл, и запустите его с помощью `./forte`. Теперь вы можете использовать нужные ресурсы IEC 61499 на своём Revolution Pi.

## Настройте своё оборудование в 4diac

В 4diac IDE создайте новое устройство IEC 61499 с одним ресурсом IEC 61499. Возможно, вы назовёте его IO\_CONFIG\_RES. Этот IO\_CONFIG\_RES содержит настройки вашего оборудования в виде функциональных блоков. Если у вас есть один основной модуль Revolution Pi, два модуля DIO и один модуль AIO, это может выглядеть так, как показано на следующем изображении.



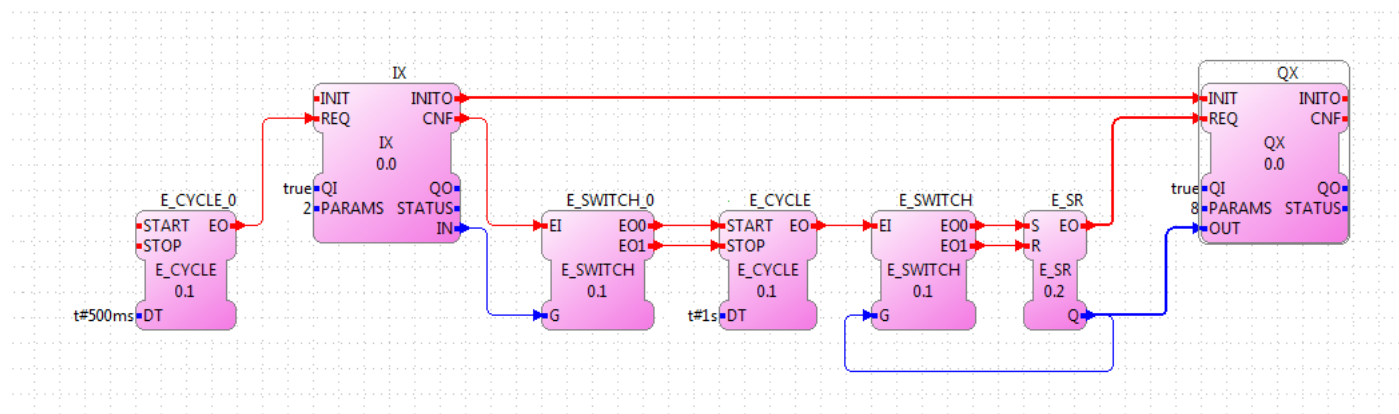
В вашем приложении IEC 61499 может быть функциональный блок, как показано на следующем изображении. Возьмите название приложения IEC 61499 и его экземпляр, разделенные точкой, и добавьте их в качестве входных данных для соответствующего функционального блока IO\_CONFIG.



Когда все входы и выходы подключены и названы в соответствии с сетью функционального блока IO\_CONFIG, приложение IEC 61499 готово к развёртыванию и доступу к оборудованию.

## SysFs (Raspberry Pi, Beagle Bone Black и аналогичные платы)

На следующем рисунке показан пример приложения из [руководства по миганию](#), которое дополнено функциональными блоками IX и QX. В расширенном приложении периодически включается вывод с номером 8. Его можно запускать и останавливать с помощью контакта с номером 2. Для получения мигающего сигнала можно подключить светодиод.



Параметры, вводимые в IX и QX FB, определяют, какой вывод следует использовать. Приложение также содержит функциональные блоки E\_CYCLE и E\_SWITCH (которых нет в руководстве по миганию). Первый из них генерирует событие каждые 500 мс. Таким образом, блок IX будет проверять аппаратный ввод каждые полсекунды, изменился ли сигнал. E\_SWITCH\_0 запускает или останавливает действие в зависимости от входных данных, выводимых функциональным блоком IX.

Помните, что если у вас возникла какая-либо проблема и вы не можете найти решение в документации, посетите [форум](#).

## Конфигурация ввода-вывода для различных платформ

Использование QX, IX и аналогичных функций позволяет получить доступ к входам и выходам поддерживаемого оборудования. Конечно, в случае обычного компьютера это неприменимо, поскольку у него нет ввода-вывода, которым можно управлять напрямую, как в ПЛК (обычно логическом). В настоящее время существует две концепции интеграции ввода-вывода: на основе параметров и на основе IO\_CONFIG.

### Ввод-вывод на основе параметров

Поддержка ввода-вывода на основе параметров использует ввод данных PARAMS в функциональных блоках IX, QX и т. д. Чтобы получить доступ к вводу-выводу аппаратного обеспечения, необходимо установить для ввода данных PARAMS соответствующее значение, чтобы выбрать, к какому вводу-выводу получить доступ. 4diac FORTE должен быть скомпилирован с соответствующим модулем, который можно выбрать в CMake. Здесь вы найдёте значения, которые необходимо установить для PARAMS для следующих поддерживаемых платформ:

1. [SysFs \(Raspberry Pi, Beagle Bone Black и аналогичные платы\)](#)
2. [Odroid](#)
3. [PiFace Digital 2](#)
4. [µMIC.200](#)
5. [ПЛК Bosch Rexroth \(MLPI\)](#)
6. [Lego Mindstorms EV3](#)

## Ввод-вывод на основе IO\_CONFIG

Ввод-вывод на основе IO\_CONFIG использует ресурс IEC 61499, который можно назвать IO\_CONFIG\_RES, но вы можете назвать его по-своему. Этот ресурс IEC 61499 содержит функциональный блок или сеть функциональных блоков, представляющих ваше оборудование. Если это только один функциональный блок или сеть функциональных блоков, зависит от вашего оборудования. Например, модульные ПЛК представлены в виде сети функциональных блоков, где один модуль представляет основной модуль, а остальные — модули расширения. Функциональные блоки для поддерживаемых аппаратных модулей находятся в папке io библиотеки типов. Любой ввод-вывод аппаратного модуля представлен вводом функционального блока типа STRING или WSTRING. Затем для входных данных STRING или WSTRING задаётся иерархическое имя экземпляра функционального блока IX, QX, ..., который вы использовали в своём приложении. Каждый уровень иерархии разделён точкой '.'. Иерархия начинается с имени приложения IEC 61499, за которым следует точка, возможные имена подприложений IEC 61499 или составных функциональных блоков, также разделённые точками, и, наконец, имя экземпляра функционального блока IX, QX, .... Эта концепция поддержки ввода-вывода позволяет вашему приложению IEC 61499 быть полностью независимым от аппаратного обеспечения. Информацию о поддерживаемых платформах можно найти в следующих главах:

1. [PLC01A1](#)
2. [Revolution Pi](#)

## ПЛК Bosch Rexroth (MLPI)

Интерфейс MLPI был протестирован на ПЛК [IndraControl XM22](#) от Bosch Rexroth. Для того чтобы 4diac FORTE мог получить доступ к вводу-выводу, в ПЛК уже должна выполняться программа ПЛК. 4diac FORTE подключится к этой программе и будет отправлять/получать информацию в/из нее.

Таким образом, параметры, которые необходимо использовать в PARAMS, зависят от названий, которые вы присвоили вводу-выводу в программе ПЛК. Это означает, что вам нужно создать свою программу и присвоить им имена. Был протестирован пример, в котором цифровые входы назывались di01, di01, di02 и т.д., и аналогично с цифровыми выходами (do00, do01, do02 и т.д.). Чтобы получить к ним доступ из 4diac FORTE, параметры были следующими:

- Приложение.UserVarGlobal.di00 → Цифровой вход 00
- Приложение.UserVarGlobal.di01 → цифровой вход 01
- Application.UserVarGlobal.do00 → цифровой выход 00
- ... и так далее

## Используйте модуль PLC01A1 от STM32

Это руководство, которое вам нужно, чтобы модуль PLC01A1 работал с платой stm32mp157\*. [Модуль](#) выглядит так.

1. [Запустите Linux на плате](#)
2. [Запустите 4diac FORTE на плате](#)
3. [Включите SPI на плате](#)
4. [Полезная информация](#)

### Запустите Linux на плате

Следуйте приведённому [здесь](#) руководству, чтобы запустить Linux на плате и ознакомиться с платой, подключив её к последовательному порту. Чтобы подключить плату расширения NUCLEO PLC01A1, следуйте [этому](#) руководству. Будьте осторожны, так как плате расширения требуется напряжение 24 В, которое может

повредить плату STM32. Внимательно прочитайте документацию, чтобы узнать, где используется напряжение 24 В.

### Запустите 4diac FORTE на плате

1. Следуйте инструкциям [здесь](#), чтобы загрузить SDK на компьютер с Ubuntu и установить кросс-компиляторы по умолчанию для сеанса терминала.
2. Клонировать репозиторий 4diac FORTE и скомпилируйте как обычно.
3. Измените IP-адрес платы, выполнив команду:

```
ip addr add XXX.XXX.XXX.XXX/MM dev eth0
```

которая установит IP-адрес в соответствии с тем, что вы используете в XXX и YY, где YY — это двузначное число, представляющее маску.

4. Протестируйте с помощью простого приложения без ввода-вывода и отслеживайте его работу в 4diac IDE.

### Включите SPI на плате

По какой-то причине модуль PLC01A1 нельзя использовать напрямую с платой. Вам нужно изменить дерево устройств Linux. Это долгий процесс, но вам нужно будет сделать это только один раз. Результатом будет файл .dtb, который нужно обновить на вашей плате. Возможно, в будущем STM32 предоставит такую возможность.

1. Проверьте файл, который вызывается при загрузке платы. Для этого подключитесь к ней с помощью USB-кабеля для последовательной связи с платой stm32, используя скорость 115200. Установите соединение и перезагрузите плату. Вы увидите что-то вроде:

```
Found /mmc0_stm32mp157a-dk1_extlinux/extlinux.conf Retrieving file:
/mmc0_stm32mp157a-dk1_extlinux/extlinux.conf
```

В этом случае вы видите, что плата — stm32mp157a-dk1, а используемый файл конфигурации — /mmc0\_stm32mp157a-dk1\_extlinux/extlinux.conf или что-то подобное.

2. Следуйте инструкциям [здесь](#) и скомпилируйте образ Linux. Это важный шаг.
3. Как только вы сможете скомпилировать его, вам нужно будет изменить некоторые параметры в зависимости от вашей платы. Перейдите в папку arch/arm/boot/dts исходного кода Linux, и вы найдете несколько файлов stm32mp157\* с расширением .dts. Вам нужно отредактировать тот файл, который вы видели на своей плате при загрузке. (некоторые пользователи жаловались, что загрузочный файл не соответствует реальной плате, которая у них была, поэтому безопаснее проверять во время загрузки). В файле вы увидите несколько блоков в фигурных скобках {}. В первом случае перед открытием используется косая черта /. После всего этого блока (он заканчивается на };) вам нужно добавить следующий текст в новой строке:

```
&spi4 {
 pinctrl-names = "default";
 pinctrl-0 = <&spi4_pins_a>;
 status = "okay";
 cs-gpios = <&gpioh 6 GPIO_ACTIVE_LOW>, <&gpioe 11 GPIO_ACTIVE_LOW>; /* Usage of
CS1 and CS2 */

 CLT01-38SQ7@0 { /* INPUTS */
 compatible = "spidev";
 reg = <0>; /* CS1 */
 spi-max-frequency = <6250000>;
 };

 VNI8200XP@1 { /* OUTPUTS */
 compatible = "spidev";
 reg = <1>;
 spi-max-frequency = <5000000>;
 };
};
```

4. Перекомпилируйте образ Linux, и вы получите новый файл .dtb для вашей платы.

- Поместите этот файл на плату в папку /boot. Для этого можно использовать клиент scp. Используйте другое имя, чтобы не перезаписать исходное, на всякий случай.
- Затем вам нужно отредактировать файл загрузки, который выбирает загружаемый файл .dtb. Поэтому вам нужно изменить файл mmc0\_stm32mp157a-dk1\_extlinux/extlinux.conf или mmc0\_stm32mp157c-dk2\_extlinux/extlinux.conf в папке /boot в зависимости от того, что вы видели ранее
- Там вам нужно добавить новую метку в конце и её конфигурацию. Вы можете скопировать её из первой и изменить соответствующим образом. Вам нужно изменить метку на желаемое имя (например, FORTE\_SPI) и имя FDT, которое должно указывать на только что скопированный файл .dtb. Это должно выглядеть примерно так (обратите внимание на поле FDT, которое должно совпадать с тем, которое вы используете):

```

LABEL FORTE_SPI
 KERNEL /uImage
 FDT /stm32mp157a-dk1-SPI4.dtb
 APPEND root=/dev/mmcblk0p6 rootwait rw console=ttySTM0,115200

```

или

```

LABEL FORTE_SPI
 KERNEL /uImage
 FDT /stm32mp157c-dk2-SPI4.dtb
 APPEND root=/dev/mmcblk0p6 rootwait rw console=ttySTM0,115200

```

- В том же файле есть строка DEFAULT с последующим текстом. Если её нет, добавьте её перед первой меткой. Измените этот текст на FORTE\_SPI (или на любое другое имя, которое вы использовали в метке). Убедитесь, что после текста нет пробелов. Строка должна выглядеть примерно так:

```

DEFAULT FORTE_SPI

```

- Перезагрузите плату.
- Затем вам нужно включить вывод напрямую, что следует делать при каждой загрузке платы, поэтому вы можете поместить следующую команду в какой-нибудь скрипт (лучше всего поместить её в тот же скрипт, который запускает Forte при загрузке)

```

gpiochip4 9=1

```

Вот и всё. Вы можете проверить, что включили SPI, выполнив

```

ls -l /sys/class/spidev

```

которая должна показать два файла.

- Чтобы получить доступ к вводам-выводам, вам нужно использовать специальный FB под названием PLC01A1 в 4diac IDE.

### Некоторая полезная информация

- Запускайте 4diac FORTE при загрузке: убедитесь, что вы можете запустить 4diac FORTE вручную, а затем создайте файл сценария, который запускает его, и поместите его в папку /usr/local/weston-start-at-startup и предоставьте этому файлу права на выполнение. Пример сценария для запуска 4diac FORTE, при условии, что файл 4diac FORTE находится в /home/root/

```

#!/bin/bash
FOLDER=/home/root/
sleep 5
Enable output
gpiochip4 9=1
Execute forte
cd ${FOLDER}
if [-f ./currentLog.txt];
then
 mv ./currentLog.txt ./lastLog.txt
fi
./forte > ./currentLog.txt 2>&1 &

```

это сохранит текущий журнал и предыдущий на случай перезагрузки платы.

## μMIC.200

Этот модуль использует библиотеку, предоставляемую для [μMIC.200](#). Вам нужны заголовки `umic_dio.h`, `umic_relay.h` и `umic_led.h`, а также библиотека `umic.so`, которая должна присутствовать в вашей системе и быть доступной для компилятора.

Параметры должны быть представлены в виде типа и его номера, разделённых точкой:

`TYPE.NUMBER`

НОМЕР зависит от ТИПА, где ТИПОМ может быть (все буквы строчные):

1. `led`: управляет светодиодами 1-4 на μMIC.200 (с `led` можно использовать только QX). Возможные значения НОМЕРА от 0 до 7 в соответствии с таблицей ниже:

- 0 → LED1\_ЗЕЛЕНый
- 1 → LED2\_ЗЕЛЕНый
- 2 → LED3\_ЗЕЛЕНый
- 3 → LED4\_ЗЕЛЕНый
- 4 → LED1\_КРАСНый
- 5 → LED2\_RED
- 6 → LED3\_RED
- 7 → LED4\_RED

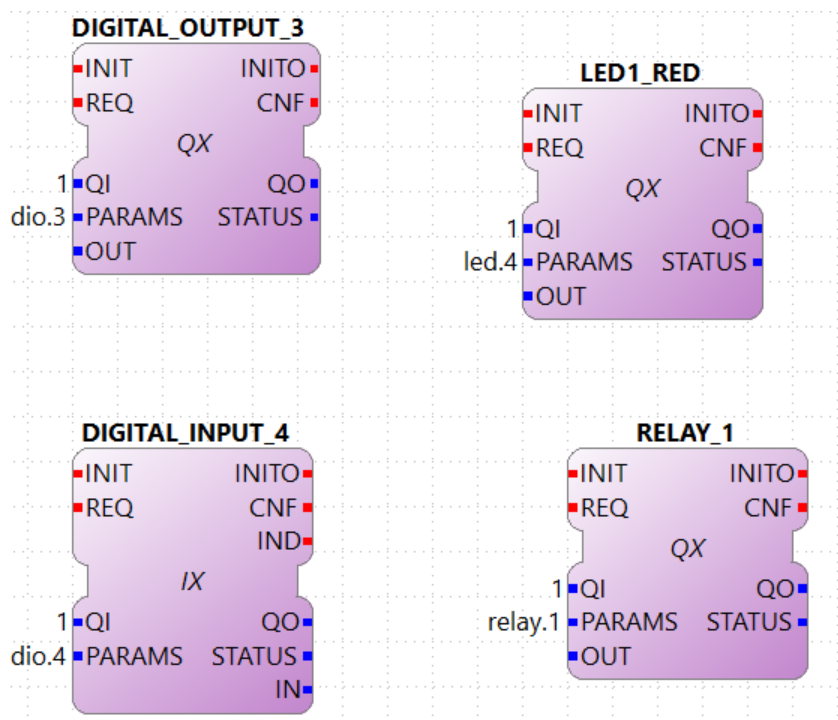
Пример: `led.4` → управляет светодиодом 1 красного цвета

2. `dio`: управляет 8 цифровыми входами/выходами на выводах 5-12. Номер может быть от 1 до 8. Используйте IX для ввода и QX для вывода.

Пример: `dio.3` → доступ к цифровому входу/выходу 3 на выводе 7 μMIC.200

3. `реле`: управляет реле. Возможные номера: 1 и 2. 1 управляет реле NO, а 2 — реле NC. С реле можно использовать только QX.

Пример: `реле.1` → управляет реле NC на выводах 14–15



Важно! Использование реле:

Чтобы использовать реле и светодиод, в некоторых случаях вам может потребоваться изменить некоторые заголовочные файлы в вашем μMIC.200. Вам нужно найти файлы `/usr/include/umic_led.h` и `/usr/include/umic_relay.h` и добавить две вещи:

- В начале, сразу после первого #define, добавьте:

```
//-----//
// take precautions if compiled with C++ compiler #ifdef __cplusplus
extern "C" \{ #endif
//-----//
```

- И в конце, прямо перед последним #endif, добавьте

```
+
//-----//
#ifdef __cplusplus } #endif // end of C++ compiler wrapper
//-----//
```

+ Если вы не хотите использовать реле и светодиод или не хотите изменять файлы заголовков, перейдите в [~/org.eclipse.4diac.org/src/modules/umic/processinterface.h](http://org.eclipse.4diac.org/src/modules/umic/processinterface.h) и установите для UMIC\_LED\_ENABLED и UMIC\_RELAY\_ENABLED значение 0.